



Desktop Assist

Code Guide

Morse Micro · 8 October 2026

How the app fits together, and the actual code of every function and method with an explanation. Each part starts on a new page; links within the guide jump to the right place, and each code block's caption links to its lines on GitHub.

Contents

1. [Overview](#)
2. [Shared code and the preload bridge](#)
3. [Main process: startup, IPC and app plumbing](#)
4. [JumpCloud sign-in](#)
5. [Talking to Claude](#)
6. [The bubble: windows, layout, bounce and the state machine](#)
7. [The draft, screenshots and safe files](#)
8. [The pages: entry points, styles, hooks and views](#)
9. [The UI components](#)

Desktop Assist: Code Guide

A walk through the whole project: what every file is for, and the actual code of every function and method with an explanation of what it does and why. Covers Milestones 0 (Foundation), 1 (Chat with Claude), 2.1 (bubble fixes) and 3 (JumpCloud sign-in).

This page is the overview. The code itself is in the [code walkthrough](#), one page per area of the app.

Contents:

1. [Background: how an Electron app is put together](#)
 2. [How the pieces work together](#)
 3. [Repository root](#)
 4. [Tenants \(branding\)](#)
 5. [Desktop app configuration](#)
 6. [Code walkthrough](#)
 7. [Tests \(tests\)](#)
-

1. Background: how an Electron app is put together

An Electron app is a Chromium browser and Node.js bundled together. It runs as several processes, and the code is split the same way:

- **Main process** (`apps/desktop/src/main`): Node.js, one per app. Creates the windows, reads and writes files, owns the tray icon and screen capture, and talks to JumpCloud and Claude.
- **Renderer** (`apps/desktop/src/renderer`): Chromium, one per window. Draws the UI with HTML, CSS and React. It **cannot** touch files, the network or the operating system.
- **Preload** (`apps/desktop/src/preload`): runs inside each renderer before the page loads, and hands the page a small, safe set of functions (`window.assist`).
- **Shared** (`apps/desktop/src/shared`): bundled into all three. Types and constants every side must agree on.

The renderer and the main process talk over **IPC** (inter-process communication): the renderer sends a named message (a *channel*), and the main process handles it and optionally replies. The main process can also push messages to the renderer, for example each chunk of a streaming Claude reply. Every channel is listed in `src/shared/ipc.ts`.

Why two windows? The bubble and the panel are separate, transparent, always-on-top windows. Both load the same web page; the URL's `?view=bubble` or `?view=panel` decides which UI it shows. Keeping the bubble in its own tiny window means it never resizes (no flicker when the panel opens), and only a 72×72 window moves around during Bounce.

Click-through. The panel window is a large rectangle, but most of it is transparent. So that you can still click whatever is underneath those empty areas, the panel window starts out *ignoring the mouse*. Windows still forwards pointer movement to the page, and while the pointer is over a real piece of UI (anything marked `data-hit`), the page asks the main process to switch mouse input back on. Each time the panel is shown, this is reset from the

pointer's actual position. The bubble window is **not** click-through: it's barely bigger than the bubble, and depending on Windows' mouse forwarding there made clicks occasionally fall through the bubble to the window behind it.

Where Claude is called, and how you're signed in. Only the main process talks to JumpCloud and to Claude. You sign in with JumpCloud in your browser; the app keeps the resulting *refresh token* (a long-lived "stay signed in" pass) encrypted on disk. To call Claude it swaps a JumpCloud *ID token* (a short-lived, signed statement of who you are) for a short-lived Claude token, using Anthropic's Workload Identity Federation. There is no API key. The pages only ever learn whether you're signed in and your name, never a token.

2. How the pieces work together

Starting up (`src/main/index.ts` → `start()`)

1. Validate `tenant.json`, create the bubble and panel windows (both hidden), load the saved draft from disk, and load preferences.
2. Create the `BubbleController` (the bubble's state machine), the `ScreenshotService`, the Claude backend, the `ChatSession` and the `AuthManager`, then the IPC handlers and tray.
3. **Renew the saved sign-in** (`auth.init()`): if there isn't one, or JumpCloud refuses it, the chat box shows **Sign in with JumpCloud** instead of the chat. If JumpCloud can't be reached, the sign-in is kept and a banner offers Retry.
4. Once both pages have painted, `controller.start()` shows the bubble in its saved corner (bottom-right of the main display the first time).

Signing in

1. **Sign in with JumpCloud** (`SignInPanel`) → `window.assist.auth.signIn()` → IPC `assist:auth-sign-in` → `AuthManager.signIn()`. The status becomes `signing-in` and the panel says to finish in the browser.
2. `listenForRedirect()` starts a tiny web server on `127.0.0.1:47621`, only on this PC. `createOidc().begin()` builds the JumpCloud sign-in address, with a fresh PKCE code, `state` and `nonce`, and the app opens it in your default browser.
3. You sign in at JumpCloud (or you already are), and JumpCloud sends the browser back to `http://127.0.0.1:47621/callback?code=...`. The little server answers "You can close this tab" and shuts down.
4. `finish()` swaps the code (plus the PKCE secret that only this app knows) for tokens, and checks the ID token. The refresh token and your name are saved encrypted (`SecretStore`, Windows DPAPI). The status becomes `signed-in` and the chat box reopens.

Getting Claude access (happens inside the Anthropic SDK, the first time you send a message and then roughly hourly)

1. The SDK needs a Claude token, so it asks `AnthropicBackend`'s identity-token function, which calls `AuthManager.freshIdToken()`.
2. That returns the unused ID token from signing in or starting up, or, if it has been used, refreshes with JumpCloud for a new one (saving JumpCloud's replacement refresh token).
3. The SDK posts the ID token to Anthropic's `/v1/oauth/token`. Anthropic checks it against the federation rule set up in the Claude Console and returns a short-lived Claude token, which the SDK uses for chat requests until shortly before it expires.

Sending a message

1. Enter in the text box → `Panel.send()` → IPC `assist:chat-send` with the text.
2. The main process collects the draft's attached screenshots, shrinks each one for Claude (`ScreenshotService.forClaude()`), and calls `ChatSession.send()`, then clears the draft.
3. `ChatSession` appends your message to the conversation history and starts a reply: `AnthropicBackend.reply()` streams it from the Claude API.
4. As text arrives, `ChatSession` updates the reply and broadcasts it (`assist:chat-message`, batched to at most one update every 50 ms). The page re-renders it as Markdown.
5. When the reply finishes, the whole reply (including Claude's hidden thinking blocks) is appended to the history unchanged, ready to be sent back with the next message.

Stop, errors and Retry

- **Stop** aborts the request. Text that already arrived stays on screen and is added to the history as a plain-text reply (the history is only ever added to, never edited; see `ChatSession`).
- **A temporary error** (offline, overloaded) before any text arrived shows the reason and a **Retry** button, which sends the same conversation again.
- **The sign-in has expired** (JumpCloud refuses the refresh): `AuthManager` signs out, and the chat box shows Sign in with JumpCloud with an explanation. The conversation is kept, and the failed message offers Retry once you're signed back in.
- **Anthropic refuses the swap** (the federation rule doesn't accept you): the message explains that IT needs to set you up.

Typing in the text box

`Composer` textarea → `Panel.changeDraft()` → `window.assist.notes.setText()` → `NotesStore.setText()`. The store keeps the draft in memory and writes `notes.json` about half a second after you stop typing, so an unsent message survives a restart.

Taking a screenshot

Camera icon → `Panel.runAction('screenshot')` → IPC `assist:invoke-action` → the `screenshot` handler in `src/main/actions.ts` → `BubbleController.hide()` hides both windows, waits for Windows to repaint, then `ScreenshotService.capture()` saves the PNG. The windows come back and the page shows a "Screenshot saved" toast. **Attach latest screenshot** then adds it to the draft as a chip, and it's sent with your next message.

Bounce

Bounce icon → `BubbleController.startBounce()` hides the panel and runs a 60 fps timer that calls `bounce.step()` and moves the bubble window within its display. Clicking the bubble calls `glideHome()`, which animates it back to the corner with `bounce.glidePosition()`.

Dragging the bubble

1. `BubbleView` sees the mouse pressed on the bubble and moved more than 5 pixels → `window.assist.bubbleDragStart()` → `BubbleController.startDrag()`.
2. The controller notes where the bubble was grabbed and, 60 times a second, moves the bubble window to follow the mouse pointer (it reads the real pointer from the main process, which keeps it smooth across monitors with different scaling).
3. On release → `bubbleDragEnd()` → `endDrag()`: the display under the bubble is found, then the nearest of its four corners (`nearestCorner()`), and the bubble glides there.

4. The new anchor (display + corner) is saved to `preferences.json` and broadcast to the panel, which re-lays itself out to open toward the middle of the screen: in a left corner the chat opens to the right, in a top corner the icons drop below the bubble and the chat grows downward.

3. Repository root

- `package.json`: The npm *workspaces* root. `apps/*` are the packages (only `apps/desktop` for now; a gateway service can be added beside it later). Its scripts (`dev`, `test`, `lint`, `typecheck`, `dist`) forward to the desktop app; `format` runs Prettier and `docs` refreshes the code walkthrough.
- `package-lock.json`: Exact versions of every installed package, so every machine installs the same thing. Generated by npm; don't edit by hand.
- `.gitignore`: Keeps `node_modules/`, build output (`out/`, `dist/`) and logs out of git.
- `.gitattributes`: Keeps line endings as LF in git and on disk, matching Prettier.
- `.prettierrc.json`: Code formatting rules (no semicolons, single quotes, 100-character lines). Run `npm run format` to apply. Code inside the docs' Markdown is left alone, because it's copied from the source.
- `.prettierignore`: Files Prettier must not touch (build output, the lock file).
- `README.md`: What the app does and how to run, test and build it.
- `docs/PLAN.md`: The plan: tech stack, architecture, milestones, decisions and open questions.
- `docs/code-guide/`: The Code Guide: this overview (`CODE_GUIDE.md`) and the code walkthrough pages (see section 6).
- `docs/CODE_GUIDE.pdf`: The whole Code Guide as one PDF, built from these pages by `npm run docs:pdf`.
- `scripts/code-guide.mjs`: Copies the real code into the walkthrough pages (`npm run docs`), and checks they're up to date (`npm run docs:check`, also run by `npm test`).
- `scripts/code-guide-pdf.mjs`: Builds `docs/CODE_GUIDE.pdf` from the pages (`npm run docs:pdf`).
- `docs/JUMPCLOUD_SETUP.md`: For IT: how to create the JumpCloud app and the Claude Console federation, and which IDs to put in `tenant.json`.

4. Tenants (branding)

A *tenant* is one business's branding. The build includes exactly one, chosen by the `TENANT` environment variable (default `morse-micro`). This is what makes the app easy to re-brand later.

- `tenants/README.md`: How tenants work, and how to change the logo.
- `tenants/morse-micro/tenant.json`: `id` (must match the folder name), `companyName`, `appName` (also the name of the screenshots folder), `accentColor` (the highlight colour), `actions` (which icons appear above the bubble, from the bubble upward), optionally `systemPrompt` (extra instructions for Claude), `signIn` (JumpCloud's address, the app's JumpCloud client ID and the sign-in port) and `claudeAccess` (the Claude Console organization, federation rule, service account and optional workspace IDs). None of these are secret. See `docs/JUMPCLOUD_SETUP.md`.
- `tenants/morse-micro/logo.png`: The logo: the Morse Micro "Mμ" mark cut to a circle (512×512, transparent corners). It's the bubble, the tray icon and the `.exe` icon. A tenant can use `logo.svg` for the bubble instead (if both exist, the PNG wins), but then the tray falls back to a plain circle in the accent colour and the `.exe` gets Electron's default icon.

5. Desktop app configuration

All in `apps/desktop/`.

- `package.json`: The desktop app's own package. `productName` ("Desktop Assist") becomes the app's name and its `%APPDATA%` folder. `main` points Electron at the built main process. Scripts: `dev`, `build`, `typecheck`, `lint`, `test`, `dist` (build the installer). Every library is a `devDependency` because the build bundles them; the installer ships no `node_modules`. `electron` is pinned to an exact version because the installer builder requires it.
- `electron.vite.config.ts`: Build config for electron-vite, which compiles the three parts (main, preload, renderer). Defines the import shortcuts `@shared` → `src/shared` and `@tenant` → `tenants/<TENANT>`, turns on React and Tailwind for the renderer, and minifies the renderer.
- `electron-builder.cjs`: Installer config: app ID, product name, include only the built `out/` folder, use the tenant's `logo.png` as the `.exe` icon, and build a per-user one-click NSIS installer (`Desktop Assist-Setup-<version>.exe`) into `dist/`. It's JavaScript rather than YAML so the icon can follow `TENANT`.
- `tsconfig.json`: Points TypeScript at the two configs below.
- `tsconfig.node.json`: TypeScript settings for code that runs in Node: main, preload, shared, tests and config files. Strict mode on.
- `tsconfig.web.json`: TypeScript settings for the renderer (browser code with React/JSX).
- `vitest.config.ts`: Test runner config: the same `@shared/@tenant` shortcuts, run `tests/**/*.test.ts` in Node.
- `eslint.config.mjs`: Lint rules: recommended JavaScript and TypeScript rules everywhere, React Hooks rules for the renderer, and Prettier compatibility.

6. Code walkthrough

Each page goes through its files in order. For every function, method, class and type it shows the real code, with a link to the exact lines on GitHub, then explains what it does and why, with notes on the lines that need them. Read them in order the first time; after that, jump to the area you're working on.

1. **Shared code and the preload bridge**: `src/shared` (sizes and layout constants, the action list, data types, the IPC contract, chip labels) and `src/preload` (the `window.assist` bridge between the pages and the main process).
2. **Main process: startup, IPC and app plumbing**: `index.ts` (startup and shutdown), `ipc.ts` (handling requests from the pages), `actions.ts`, `tenant.ts`, `settings.ts`, the tray icon.
3. **JumpCloud sign-in**: `auth/` (`AuthManager`, OpenID Connect with JumpCloud, the loopback listener) and `storage/SecretStore.ts`.
4. **Talking to Claude**: `claude/` (the model settings, `AnthropicBackend`, error handling, `ChatSession`).
5. **The bubble**: `bubble/` (`BubbleController`, the layout maths, bounce physics, creating the windows).
6. **The draft, screenshots and safe files**: `notes/NotesStore.ts`, `screenshots/`, `storage/jsonFile.ts`.
7. **The pages: entry points, styles, hooks and views**: `src/renderer`: `index.html`, `main.tsx`, `styles.css`, `lib/`, `hooks/`, `BubbleView`, `PanelView`.
8. **The UI components**: `src/renderer/src/components`: the chat box, messages, Markdown, sign-in panel, settings menu, action icons, screenshot chips.

Keeping it current. The code on these pages is not typed by hand: each block is a marker like `` that `npm run docs` fills from the source. After changing code, run `npm run docs`; `npm test` fails if a page shows code that no longer matches. When you add a new function, add a marker and a short explanation for it on the right page. The explanations are written by hand, so check they still hold when behaviour changes. Then `npm run docs:pdf` rebuilds `docs/CODE_GUIDE.pdf`, the whole guide as one PDF.

7. Tests (`tests`)

Run with `npm test`. Each file tests code that doesn't need a real window or a real API.

- `layout.test.ts`: Home positions in all four corners (including displays not at the origin), nearest-corner snapping, window sizes, and that the panel window opens into the screen and fits on it in every corner.
- `bounce.test.ts`: Launch direction (away from each corner) and speed, bouncing off every edge, never leaving the screen, and the glide.
- `bubbleController.test.ts`: The bubble state machine with fake windows, a fake clock and two fake displays: open/close, blur, the saved corner, dragging (follows the mouse, snaps to the nearest corner, onto another display, panel re-placed), bouncing on the current display, screenshots, and display changes (including unplugging the bubble's display).
- `notesStore.test.ts`: Saving the draft, reloading, overtaken writes, damaged files, attachments.
- `screenshotFiles.test.ts`: Screenshot names, never overwriting, finding the newest, the inside-the-folder check.
- `format.test.ts`: Chip labels.
- `tenants.test.ts`: Every tenant folder has a valid `tenant.json` and a logo; the list of missing sign-in settings.
- `trayIcon.test.ts`: The tray circle.
- `chatSession.test.ts`: The conversation with a scripted fake backend: streaming, images before text, replaying replies unchanged (thinking included), busy/empty/signed-out, Stop with and without text, Retry, an expired sign-in, refusals and length limits, the tool loop, and New conversation ignoring a late reply.
- `authManager.test.ts`: The sign-in with a fake JumpCloud (whose refresh tokens work once, like the real one): not set up, nothing saved, renewing at startup and saving the replacement token, offline and Retry, a refused sign-in, browser sign-in, Cancel, no refresh token, a port in use, handing out a different ID token each time, never refreshing twice at once, skipping an almost-expired ID token, Log out (revoking), and logging out during a renewal.
- `loopback.test.ts`: The browser-return listener: catches `/callback` and stops, 404 for anything else, shows JumpCloud's error safely escaped, Cancel, timeout, and a port in use.
- `secretStore.test.ts`: Encrypted save and load, unreadable files, refusing to save without encryption, clear.
- `claudeHelpers.test.ts`: Error classification (including sign-in errors wrapped by the SDK and failed swaps), what can be retried, the system prompt, and image scaling.
- `codeGuide.test.ts`: The code shown in the walkthrough pages matches the source (runs `npm run docs:check`).

Shared code and the preload bridge

The app runs as two kinds of process: the **main process** (Node.js, which owns the windows, files, sign-in and Claude) and a **renderer** for each window (Chromium, which draws the React UI). They talk over **IPC** (inter-process communication): one side sends a message on a named *channel* and the other side listens on it. `src/shared` holds the TypeScript both sides must agree on (window sizes, action names, data shapes and the list of channels), imported as `@shared/...` and bundled into each part. `src/preload/index.ts` is the small script that turns that list of channels into `window.assist`, the only way a page can reach the main process.

`src/shared/geometry.ts`: sizes and positions

Every size and offset the bubble and the panel use, plus a few helpers built on them. The main process uses these numbers to size and place the two windows (`src/main/index.ts` and `src/main/bubble/layout.ts`). The renderer uses the same numbers to place elements inside those windows (`ActionStack.tsx`, `ChatBox.tsx`, `SettingsMenu.tsx`). Because both read one file, a window can never be the wrong size for what is drawn in it.

All values are **DIPs** (device-independent pixels): pixels before Windows display scaling. On a display scaled to 150%, one DIP is 1.5 real pixels. Electron's window and screen functions and the page's CSS pixels both work in DIPs, so one number means the same thing on both sides.

The key idea is that the panel's layout is measured from the corner the bubble sits in, not from the top-left of the window. That way one set of numbers works in all four screen corners.

Point, Size and Rect

`src/shared/geometry.ts`, lines 1–14

```
// Geometry shared by the main process (which sizes and moves windows) and the renderer (which
// positions elements inside them). All values are DIPs (device-independent pixels).

export interface Point {
  x: number
  y: number
}

export interface Size {
  width: number
  height: number
}

export interface Rect extends Point, Size {}
```

Plain shapes for positions and sizes, used all over the main process and the renderer.

- `export interface Rect extends Point, Size {}`: an interface can extend several others. `Rect` adds nothing of its own; it is `x`, `y`, `width` and `height` together, the same shape Electron uses for window bounds, so a `Rect` can be passed straight to `win.setBounds()` (see `src/main/bubble/windows.ts`).

Corner, isLeftCorner() and isTopCorner()

src/shared/geometry.ts, lines 16–20

```
/** The screen corner the bubble rests in. The panel always opens toward the screen's middle. */
export type Corner = 'top-left' | 'top-right' | 'bottom-left' | 'bottom-right'

export const isLeftCorner = (corner: Corner): boolean => corner.endsWith('left')

export const isTopCorner = (corner: Corner): boolean => corner.startsWith('top')
```

`Corner` is a *union of string literals*: a value of this type can only be one of those four strings, and TypeScript rejects anything else at compile time. The bubble rests in one corner of a display's work area (the screen minus the taskbar), and everything else opens toward the middle of the screen from there. The two helpers answer "which side is the bubble on?", which is all the layout code needs in order to mirror itself.

- `corner.endsWith('left')`: every name is "vertical-horizontal", so the horizontal side is the end of the string and the vertical side is the start.
- Callers: `homePosition()` and `panelWindowBounds()` in `src/main/bubble/layout.ts`, `launch()` in `src/main/bubble/bounce.ts` (a bounce sets off away from the corner), and `anchored()` in `src/renderer/src/lib/anchor.ts`, which turns an offset into CSS `left/right` and `top/bottom`.

UI

src/shared/geometry.ts, lines 22–40

```
export const UI = {
  /** Gap between the bubble and the edges of the work area. */
  edgeMargin: 16,
  bubbleSize: 56,
  /** Transparent margin around the bubble inside its window (room for the shadow and ring). */
  bubblePad: 8,
  /** Space between the bubble and the chat card / action stack. */
  gap: 12,
  /** The chat card. It starts as just the text box and grows away from the bubble to this height. */
  panelWidth: 400,
  panelMaxHeight: 520,
  /** Room beyond the chat card for a toast message, even when the card is at full height. */
  toastRoom: 40,
  actionSize: 40,
  actionGap: 8,
  /** Room for drop shadows along the panel window's far edges. */
  shadowPad: 16,
  settingsMenuWidth: 248,
} as const
```

Every fixed measurement of the UI in one object, in DIPs.

- `as const`: makes the object read-only and keeps each value's exact type (`56` rather than `number`). Nothing can change these numbers at runtime by accident.
- `edgeMargin: 16`: how far the resting bubble sits from the work area's edges (`homePosition()` in `src/main/bubble/layout.ts`).
- `bubblePad: 8`: the bubble's window is a little bigger than the bubble itself, so the drop shadow and the ring around the bubble aren't clipped by the window edge.
- `panelWidth: 400` and `panelMaxHeight: 520`: the chat card's width and its tallest height. `ChatBox.tsx` sets them as the card's CSS `width` and `maxHeight`; the card starts as just the text box and grows as the

conversation fills it.

- `toastRoom: 40`: toasts ("Screenshot saved") appear just outside the card, on the side away from the screen edge. The window keeps this much room for them even when the card is full height.
- `actionSize: 40` and `actionGap: 8`: the round action icons and the space between them (`ActionStack.tsx`).
- `shadowPad: 16`: transparent room along the panel window's far edges so shadows there aren't cut off.

BUBBLE_BOX

`src/shared/geometry.ts`, lines 42–43

```
/** The bubble window is the bubble plus its padding on every side. */  
export const BUBBLE_BOX = UI.bubbleSize + UI.bubblePad * 2
```

The bubble window's width and height: $56 + 8 + 8 = 72$. `start()` in `src/main/index.ts` creates the bubble window at this size, and `bubbleWindowBounds()` in `src/main/bubble/layout.ts` places it. The bubble window only ever moves; it never changes size.

PANEL_LAYOUT

`src/shared/geometry.ts`, lines 45–56

```
/**  
 * Where things sit inside the panel window. The panel window shares one corner with the bubble  
 * window (the corner of the screen the bubble is in), and these offsets are measured from that  
 * corner: `X` across, `Y` away from the screen edge. So the same numbers work in every corner.  
 */  
export const PANEL_LAYOUT = {  
  actionsX: UI.bubblePad + (UI.bubbleSize - UI.actionSize) / 2,  
  actionsY: UI.bubblePad + UI.bubbleSize + UI.gap,  
  chatX: UI.bubblePad + UI.bubbleSize + UI.gap,  
  chatY: UI.bubblePad,  
  settingsX: UI.bubblePad + (UI.bubbleSize + UI.actionSize) / 2 + UI.actionGap,  
} as const
```

Where the three parts of the panel (the action icons, the chat card and the settings menu) sit inside the panel window. The panel window and the bubble window share one corner: in the bottom-right of the screen, their bottom-right corners are in the same place (`panelWindowBounds()` in `src/main/bubble/layout.ts`). Every offset here is measured from that shared corner: `X` across, `Y` away from the screen edge (upward in a bottom corner, downward in a top corner). In the renderer, `anchored()` (`src/renderer/src/lib/anchor.ts`) turns an `(X, Y)` pair into CSS `right/bottom`, `left/top` and so on, so the same numbers mirror into every corner.

Here is the bottom-right case with four icons (not to scale; the other corners are mirror images):

```
+-----+  
| +-----+ |  
| | chat card | [ icon 4 ] |  
| | (grows upward, | [ icon 3 ] |  
| | at most 520 tall) | [ icon 2 ] |  
| | [ text box ] | [ icon 1 ] |  
| +-----+ | ( bubble ) |  
+-----+
```

- `actionsX`: `UI.bubblePad + (UI.bubbleSize - UI.actionSize) / 2`: $8 + 8 = 16$, which centres the 40-DIP icons on the 56-DIP bubble.
- `actionsY`: `UI.bubblePad + UI.bubbleSize + UI.gap`: 76. The first icon starts one gap beyond the bubble.
- `chatX`: `UI.bubblePad + UI.bubbleSize + UI.gap`: also 76. The chat card starts one gap to the side of the bubble, toward the middle of the screen.
- `chatY`: `UI.bubblePad`: 8, so the card's near edge is level with the bubble's near edge.
- `settingsX`: $8 + 48 + 8 = 64$. The first two terms reach the far side of the icon column; one more `actionGap` puts the menu just beside the gear icon. Its other coordinate depends on where the gear is in the stack, so it comes from `actionOffset()` instead.

`actionStackHeight()` and `actionOffset()`

`src/shared/geometry.ts`, lines 58–65

```
export function actionStackHeight(actionCount: number): number {
  return actionCount > 0 ? actionCount * UI.actionSize + (actionCount - 1) * UI.actionGap : 0
}

/** Distance from the bubble's edge of the panel window to the near side of the action at `index`. */
export function actionOffset(index: number): number {
  return PANEL_LAYOUT.actionsY + index * (UI.actionSize + UI.actionGap)
}
```

Two small sums over the icon stack.

- `actionCount > 0 ? n` icons have $n - 1$ gaps between them. With no icons the formula would give -8, so zero is handled separately. Only `panelWindowSize()` below uses this.
- `PANEL_LAYOUT.actionsY + index * (UI.actionSize + UI.actionGap)`: icon `index` (0 is nearest the bubble) starts this far from the bubble's edge of the window. `PanelView.tsx` passes `actionOffset(settingsIndex)` to `SettingsMenu` as its `offset`, so the menu lines up with the gear wherever the tenant put the gear in its list.

`panelWindowSize()`

`src/shared/geometry.ts`, lines 67–74

```
export function panelWindowSize(actionCount: number): Size {
  const width = PANEL_LAYOUT.chatX + UI.panelWidth + UI.shadowPad
  const contentHeight = Math.max(
    PANEL_LAYOUT.chatY + UI.panelMaxHeight + UI.toastRoom,
    PANEL_LAYOUT.actionsY + actionStackHeight(actionCount),
  )
  return { width, height: contentHeight + UI.shadowPad }
}
```

How big the panel window must be to hold everything at its largest. The window is created at this size (`start()` in `src/main/index.ts`) and never resizes when the chat card grows, so there is no flicker. The unused parts are transparent and let clicks through to whatever is underneath (the *click-through* handled by `useClickThrough()` in the renderer).

- `PANEL_LAYOUT.chatX + UI.panelWidth + UI.shadowPad`: $76 + 400 + 16 = 492$ wide.
- `Math.max()`: the height is whichever reaches further from the screen edge: the chat card at full height plus room for a toast ($8 + 520 + 40 = 568$), or the icon stack. With the usual four icons the stack ends at $76 + 184 = 260$, so the card decides and the window is 584 tall. The stack only matters with 11 or more icons.

src/shared/actions.ts: the action registry

The icons that can appear in the stack beside the bubble (the *actions*), each with a tooltip label and a *kind*. A tenant's `tenant.json` has an `actions` list that picks which ones appear and in what order, nearest the bubble first. There are two kinds:

- `command` actions run in the main process: screenshot, bounce and close (handlers in `createActionHandlers()`, `src/main/actions.ts`).
- `popover` actions open some UI inside the panel and never leave the renderer. Settings is the only one; `PanelView.runAction()` handles it.

The ids are written once, as a real array, and the types are derived from it. So adding an action is one edit here, and then the compiler points at the rest: `ICONS` in `ActionStack.tsx` is a `Record<ActionId, ...>` (it must have an icon for every action), and `ActionHandlers` in `src/main/actions.ts` is a `Record<CommandActionId, ...>` (every command needs a handler).

ACTION_IDS and ActionId

src/shared/actions.ts, lines 1–9

```
// The action registry: every icon that can appear in the stack above the bubble. A tenant's
// 'actions' list chooses which of these show and in what order (from the bubble upward).
//
// - 'command' actions run in the main process (see src/main/actions.ts).
// - 'popover' actions open UI inside the panel and never reach the main process.

export const ACTION_IDS = ['screenshot', 'settings', 'bounce', 'close'] as const

export type ActionId = (typeof ACTION_IDS)[number]
```

The list of every action that exists, and the type of one of its entries.

- `as const`: without it the array's type would be `string[]`. With it, the type is a read-only list of these four exact strings, which is what lets the next line work.
- `(typeof ACTION_IDS)[number]`: "the type of whatever you get by indexing the array with a number", which is the union `'screenshot' | 'settings' | 'bounce' | 'close'`. The type follows the list automatically.
- The array also exists at runtime, which a type doesn't: `TenantSchema` in `src/main/tenant.ts` uses `z.enum(ACTION_IDS)` to reject a `tenant.json` that lists an unknown action. **zod** is a validation library: you describe a shape (a *schema*) and zod checks, while the app runs, that untrusted data really has that shape. TypeScript types are erased when the code is compiled, so they can't check anything at runtime.

ACTIONS

src/shared/actions.ts, lines 11–16

```
export const ACTIONS = {
  screenshot: { label: 'Take screenshot', kind: 'command' },
  settings: { label: 'Settings', kind: 'popover' },
  bounce: { label: 'Bounce', kind: 'command' },
  close: { label: 'Close Desktop Assist', kind: 'command' },
} as const satisfies Record<ActionId, { label: string; kind: 'command' | 'popover' }>
```

The label and kind of each action. `ActionStack.tsx` uses `label` as each icon's tooltip and screen-reader name, and gives `popover` actions an `aria-expanded` attribute so screen readers announce whether the menu is open.

- `satisfies Record<ActionId, { label: string; kind: 'command' | 'popover' }>`: checks that there is exactly one entry per action id, each with a label and a valid kind. A missing, extra or misspelt action is a compile error. Unlike a normal type annotation, `satisfies` only checks: it doesn't change the object's own type.
- `as const`: keeps each `kind` as its exact string (`'command'`), not just `string`. `CommandActionId` below depends on that; a plain `: Record<...>` annotation would have widened every `kind` to `'command' | 'popover'` and lost which is which.

CommandActionId

`src/shared/actions.ts`, lines 18–20

```
export type CommandActionId = {
  [K in ActionId]: (typeof ACTIONS)[K]['kind'] extends 'command' ? K : never
}[ActionId]
```

The type of just the command actions: `'screenshot' | 'bounce' | 'close'`. It's worked out from `ACTIONS`, so it can't drift out of step with it.

- `[K in ActionId]:` a *mapped type*. It builds an object type with one property per action id. Each property's type is the id itself if that action's kind is `'command'`, or `never` (the "impossible" type) if not, giving `{ screenshot: 'screenshot'; settings: never; bounce: 'bounce'; close: 'close' }`.
- `}[ActionId]`: looking up every key at once gives the union of the property types. `never` disappears from a union, leaving the three command ids.
- `AssistApi.invokeAction()` in `src/shared/ipc.ts` only accepts these, so a page can't even try to send `settings` to the main process. In `PanelView.runAction()`, the early return for `'settings'` narrows the id to exactly this type, which is why it can then be passed on.

COMMAND_ACTION_IDS

`src/shared/actions.ts`, lines 22–24

```
export const COMMAND_ACTION_IDS = ACTION_IDS.filter(
  (id): id is CommandActionId => ACTIONS[id].kind === 'command',
)
```

The same set as a real array, for checks at runtime. `registerIpc()` in `src/main/ipc.ts` uses `z.enum(COMMAND_ACTION_IDS)` to check the id that arrives with `assist:invoke-action` before looking up its handler. The main process treats the pages as untrusted, so a compile-time type isn't enough.

- `(id): id is CommandActionId =>`: a *type guard*. It tells TypeScript that when this function returns true, `id` is a `CommandActionId`, so the filtered array is typed `CommandActionId[]` rather than `ActionId[]`.

src/shared/types.ts: data shapes

The shapes of the data that travels between the main process and the pages: the bubble's mode, the draft, settings, branding, sign-in status, chat messages and the answers to requests. The file holds only types, so nothing in it exists at runtime; every file imports it with `import type`.

Two ideas run through it:

- **Plain data only.** Everything is strings, numbers, booleans, arrays and plain objects. Values sent over IPC are copied (Electron serialises them with the browser's *structured clone* algorithm, which drops functions and class methods), and some are also saved as JSON files. That is why timestamps are ISO strings such as `"2026-10-02T14:03:11.000Z"` rather than `Date` objects.
- **Tagged unions.** Several types are "one of these shapes", told apart by a tag field (`state` or `ok`). Checking the tag tells TypeScript which other fields exist: after `if (result.ok)`, it knows `result.notes` is there; otherwise it knows `result.reason` is.

Mode

src/shared/types.ts, lines 4–10

```
/**
 * What the bubble is doing. The main process owns this; renderers only display it.
 * - `dragging`: following the mouse while the user drags it.
 * - `returning`: gliding into its corner after a bounce or a drag.
 * - `capturing`: windows are hidden while a screenshot is taken.
 */
export type Mode = 'collapsed' | 'expanded' | 'dragging' | 'bouncing' | 'returning' | 'capturing'
```

What the bubble is doing right now. Only `BubbleController` (`src/main/bubble/BubbleController.ts`) changes it, through its private `setMode()`, and each change is broadcast to both windows on `assist:mode-changed`. `collapsed` is just the bubble; `expanded` means the panel is open; `bouncing` is the Bounce animation. The panel shows its contents only while the mode is `expanded`, and `BubbleView` styles the bubble differently while `dragging` and `expanded`.

Attachment and Notes

src/shared/types.ts, lines 12–27

```
export interface Attachment {
  id: string
  /** Absolute path of the screenshot file. */
  path: string
  fileName: string
  /** ISO timestamp. */
  addedAt: string
}

/** The unsent draft in the text box. */
export interface Notes {
  text: string
  attachments: Attachment[]
  /** ISO timestamp of the last change, or null if nothing has been typed yet. */
  updatedAt: string | null
}
```

A screenshot attached to the draft (or to a message already sent), and the draft itself. The code calls the draft "notes" throughout (`NotesStore`, `window.assist.notes`). `NotesStore` (`src/main/notes/NotesStore.ts`) owns it and saves it to `notes.json`, so an unsent message survives a restart.

- `id: string`: a random UUID made by `NotesStore.addAttachment()`. The page uses it to remove one attachment (`notes.removeAttachment(id)`).
- `path: string`: where the PNG is on disk. The main process never trusts a path that comes back from a page: `ScreenshotService` checks it is inside the screenshots folder before reading it.
- `fileName: string`: just the file's name, which `screenshotLabel()` (below) turns into the chip's label.
- `updatedAt: string | null`: set by `NotesStore` on every change; `null` for a draft nobody has typed in yet.

Effort and Settings

`src/shared/types.ts`, lines 29–38

```
/** How hard Claude thinks before answering (the API's `effort`), shown as Fast/Balanced/Thorough. */
export type Effort = 'low' | 'medium' | 'high'

export interface Settings {
  autoStart: boolean
  /** Starting with Windows only works for an installed build, not `npm run dev`. */
  autoStartAvailable: boolean
  screenshotsDir: string
  effort: Effort
}
```

`Effort` is the response style. It is sent to the Claude API as `output_config.effort` (`src/main/claude/AnthropicBackend.ts`) and shown in the settings menu as Fast, Balanced and Thorough. `Settings` is what the settings menu shows; `SettingsService.get()` in `src/main/settings.ts` builds a fresh one each time.

- `autoStart: boolean`: whether the app starts with Windows. It is read from Windows each time, because the user can also change it in Windows Settings.
- `autoStartAvailable: boolean`: `app.isPackaged`, true only in the installed app. Under `npm run dev` the switch is greyed out with "Only in the installed app".
- `screenshotsDir: string`: the folder screenshots are saved in, opened by "Open screenshots folder".

Branding

`src/shared/types.ts`, lines 40–45

```
export interface Branding {
  companyName: string
  appName: string
  accentColor: string
  actions: ActionId[]
}
```

The parts of the tenant's `tenant.json` that the pages need. `brandingOf()` in `src/main/tenant.ts` copies just these four fields, deliberately leaving out the sign-in settings, the Claude access settings and the system prompt. `accentColor` becomes the page's accent colour (`useAccentColor()` sets the CSS variable `--color-accent`), and `actions` decides which icons `ActionStack` shows, in order.

SignedInUser and AuthStatus

src/shared/types.ts, lines 47–66

```
export interface SignedInUser {
  name?: string
  email?: string
}

/**
 * Whether the user is signed in with JumpCloud.
 * - 'unconfigured': the JumpCloud or Claude access settings in tenant.json aren't filled in yet.
 * - 'checking': renewing a saved sign-in at startup.
 * - 'signing-in': waiting for the user to finish in their browser.
 * - 'offline': a saved sign-in couldn't be renewed because JumpCloud couldn't be reached. It's
 *   kept, and Retry tries again.
 */
export type AuthStatus =
  | { state: 'unconfigured'; missing: string[] }
  | { state: 'checking' }
  | { state: 'signed-out'; message?: string }
  | { state: 'signing-in' }
  | { state: 'signed-in'; user: SignedInUser }
  | { state: 'offline'; user: SignedInUser; message: string }
```

Who is signed in, and where sign-in stands. The user signs in with **JumpCloud**, the company's single sign-on service. Only `AuthManager` (`src/main/auth/AuthManager.ts`) sets the status, and every change is broadcast on `assist:auth-status`. The pages only ever learn the state and the user's name and email; no token ever reaches them.

- `name?: string` and `email?: string`: taken from the `name` and `email` claims in JumpCloud's ID token (its signed statement of who signed in). Both are optional because JumpCloud may leave them out.
- `{ state: 'unconfigured'; missing: string[] }`: `missing` lists the `tenant.json` settings still to fill in, by name (from `missingSettings()` in `src/main/tenant.ts`).
- `{ state: 'signed-out'; message?: string }`: the optional message says why, for example that the sign-in expired.
- `{ state: 'offline'; user: SignedInUser; message: string }`: the saved sign-in is kept and the chat stays usable. `PanelView` treats both `signed-in` and `offline` as signed in, and shows the sign-in panel for `unconfigured`, `signed-out` and `signing-in`.

ChatMessage and SendResult

src/shared/types.ts, lines 68–84

```
/** One message as the chat shows it. */
export interface ChatMessage {
  id: string
  role: 'user' | 'assistant'
  text: string
  /** Screenshots sent with a user message. */
  attachments: Attachment[]
  status: 'streaming' | 'done' | 'stopped' | 'error'
  /** Shown under the message, e.g. why it stopped. */
  notice?: string
  /** A failed reply that can be requested again. */
  retryable?: boolean
}

export type SendResult =
  | { ok: true; notes: Notes }
  | { ok: false; reason: 'busy' | 'empty' | 'signed-out' | 'missing-screenshot' }
```

`ChatMessage` is one message as the chat shows it. `ChatSession` (`src/main/claude/ChatSession.ts`) keeps the list and broadcasts every new or changed message on `assist:chat-message`; the page replaces the message with the same `id` (`upsert()` in `useAssistState.ts`). This is only the display copy: the history sent to Claude, including Claude's hidden thinking, is kept separately inside `ChatSession`.

- `status: 'streaming' | 'done' | 'stopped' | 'error'`: `streaming` while text is still arriving; `stopped` means the user pressed Stop.
- `retryable?: boolean`: set on a failed reply that can be asked for again (a temporary error before any text arrived). `ChatSession.retry()` only acts if the last message has it.

`SendResult` is the answer to `window.assist.chat.send()`. On success, `notes` is the now-empty draft, which the panel shows. The reply itself arrives later, through `onChatMessage`.

- `reason: 'busy' | 'empty' | 'signed-out' | 'missing-screenshot'`: `busy` means a reply is still streaming, and `empty` means there was no text and no screenshot. The first three come from `ChatSession.send()`; `missing-screenshot` (an attached file has gone) comes from the `chat-send` handler in `src/main/ipc.ts`.

AppState

src/shared/types.ts, lines 86–96

```
export interface AppState {
  mode: Mode
  /** The screen corner the bubble rests in; the panel lays itself out to open away from it. */
  corner: Corner
  notes: Notes
  settings: Settings
  branding: Branding
  version: string
  auth: AuthStatus
  chat: ChatMessage[]
}
```

Everything a page needs when it first loads, returned by `window.assist.getState()`. It is put together by the `getState` function that `start()` in `src/main/index.ts` passes to `registerIpc()`. After that first load, `useAssistState()` in the renderer keeps `mode`, `corner`, `auth` and `chat` up to date from the `on...` events. `notes` and `settings` only change through the page's own requests, so the panel takes their new values from the replies.

- `version: string: app.getVersion()`, shown in the settings menu.

ActionResult and AttachResult

`src/shared/types.ts`, lines 98–101

```
export type ActionResult = { ok: true; message?: string } | { ok: false; message: string }

export type AttachResult =
  { ok: true; notes: Notes } | { ok: false; reason: 'no-screenshots' | 'already-attached' }
```

The answers to two more requests.

- `{ ok: true; message?: string } | { ok: false; message: string }`: what a command action returns (`src/main/actions.ts`). `PanelView.runAction()` shows `message` as a toast, styled as an error when `ok` is false. A failure must carry a message; a success may (the screenshot action says "Screenshot saved", Bounce says nothing).
- `reason: 'no-screenshots' | 'already-attached'`: why "Attach latest screenshot" did nothing: the folder has no screenshots, or the newest one is already in the draft (`NotesStore.addAttachment()` refuses duplicates).

src/shared/ipc.ts: the IPC contract

The agreement between the pages and the main process: the name of every IPC channel (`IPC`) and the exact set of functions a page gets as `window.assist` (`AssistApi`). Three places depend on it. `src/preload/index.ts` implements `AssistApi` by sending on these channels; `registerIpc()` in `src/main/ipc.ts` listens on them; and `src/renderer/src/env.d.ts` tells TypeScript that `window.assist` is an `AssistApi`.

Messages come in three styles:

- **Request and reply**: the page calls `ipcRenderer.invoke()`, the main process answers from an `ipcMain.handle()` handler, and the page gets a promise of that answer. If the handler throws (for example because the input failed its zod check), the promise rejects.
- **Fire and forget**: `ipcRenderer.send()` to an `ipcMain.on()` listener, with no answer. Used for frequent or one-way signals such as each keystroke in the text box. In `AssistApi`, the methods that return `void` are these; the ones that return a `Promise` are requests.
- **Pushed by the main process**: `webContents.send()` to a window, which the page receives through one of the `on...` listeners. These are the last six channels.

IPC

src/shared/ipc.ts, lines 16–51

```
/** IPC channel names. The preload maps them onto `window.assist`; src/main/ipc.ts handles them. */
export const IPC = {
  getState: 'assist:get-state',
  bubbleClick: 'assist:bubble-click',
  bubbleDragStart: 'assist:bubble-drag-start',
  bubbleDragEnd: 'assist:bubble-drag-end',
  collapse: 'assist:collapse',
  setInteractive: 'assist:set-interactive',
  invokeAction: 'assist:invoke-action',
  openExternal: 'assist:open-external',
  copyText: 'assist:copy-text',
  notesSetText: 'assist:notes-set-text',
  notesAttachLatest: 'assist:notes-attach-latest',
  notesRemoveAttachment: 'assist:notes-remove-attachment',
  screenshotThumbnail: 'assist:screenshot-thumbnail',
  screenshotOpen: 'assist:screenshot-open',
  screenshotsOpenFolder: 'assist:screenshots-open-folder',
  settingsGet: 'assist:settings-get',
  settingsSetAutoStart: 'assist:settings-set-auto-start',
  settingsSetEffort: 'assist:settings-set-effort',
  authSignIn: 'assist:auth-sign-in',
  authCancel: 'assist:auth-cancel',
  authSignOut: 'assist:auth-sign-out',
  authRetry: 'assist:auth-retry',
  chatSend: 'assist:chat-send',
  chatStop: 'assist:chat-stop',
  chatRetry: 'assist:chat-retry',
  chatNew: 'assist:chat-new',
  // main → renderer
  modeChanged: 'assist:mode-changed',
  cornerChanged: 'assist:corner-changed',
  clickThroughReset: 'assist:click-through-reset',
  authStatus: 'assist:auth-status',
  chatMessage: 'assist:chat-message',
  chatReset: 'assist:chat-reset',
} as const
```

Every channel name in one object, so the preload and the main process can't disagree about a name: a typo is a compile error instead of a message nobody hears.

- 'assist:get-state': every name starts with `assist:`, so it can't clash with a channel that Electron or a library might use.
- `// main → renderer`: the channels from here on go the other way. `broadcast()` in `start()` (`src/main/index.ts`) sends them to both windows, except `clickThroughReset`, which `resetClickThrough()` in `src/main/bubble/windows.ts` sends only to the panel.

AssistApi

src/shared/ipc.ts, lines 53–108

```
/** The API the preload exposes to renderers as `window.assist`. */
export interface AssistApi {
  getState(): Promise<AppState>
  bubbleClick(): void
  /** The bubble was pressed and moved: the main process makes it follow the mouse. */
  bubbleDragStart(): void
```

```

/** Released: the bubble snaps to the nearest corner of the display it's on. */
bubbleDragEnd(): void
collapse(): void
/** Tell the main process whether the pointer is over real UI (true) or a see-through area. */
setInteractive(interactive: boolean): void
invokeAction(id: CommandActionId): Promise<ActionResult>
/** Opens a web (http/https) or email (mailto) link in the default app. */
openExternal(url: string): Promise<void>
copyText(text: string): Promise<void>
notes: {
  setText(text: string): void
  attachLatestScreenshot(): Promise<AttachResult>
  removeAttachment(id: string): Promise<Notes>
}
screenshots: {
  /** A small data-URL preview, or null if the file is missing. */
  thumbnail(path: string): Promise<string | null>
  open(path: string): Promise<boolean>
  openFolder(): Promise<void>
}
settings: {
  get(): Promise<Settings>
  setAutoStart(enabled: boolean): Promise<Settings>
  setEffort(effort: Effort): Promise<Settings>
}
auth: {
  /** Opens JumpCloud in the browser; progress arrives through `onAuthStatus`. */
  signIn(): Promise<void>
  /** Stops waiting for a sign-in started in the browser. */
  cancel(): Promise<void>
  signOut(): Promise<void>
  /** Tries again to renew a saved sign-in that couldn't reach JumpCloud. */
  retry(): Promise<void>
}
chat: {
  /** Sends the draft text plus the draft's attached screenshots. */
  send(text: string): Promise<SendResult>
  stop(): Promise<void>
  retry(): Promise<void>
  newConversation(): Promise<void>
}
onModeChanged(callback: (mode: Mode) => void): () => void
onCornerChanged(callback: (corner: Corner) => void): () => void
/** The panel was just shown with click-through reset; `pointer` is where the mouse is now. */
onClickThroughReset(callback: (pointer: Point) => void): () => void
onAuthStatus(callback: (status: AuthStatus) => void): () => void
/** A message was added or changed (streamed text arrives this way). */
onChatMessage(callback: (message: ChatMessage) => void): () => void
onChatReset(callback: () => void): () => void
}

```

The API a page sees as `window.assist`. Each method maps onto one channel in `IPC`. Related calls are grouped into nested objects (`notes`, `screenshots`, `settings`, `auth`, `chat`), which is only for readability.

- `getState(): Promise<AppState>`: called once when a page loads (`useAssistState()`).
- `bubbleClick(): void`: this and the next three are fire-and-forget calls into `BubbleController` (`clickBubble()`, `startDrag()`, `endDrag()`, `collapse()`).
- `setInteractive(interactive: boolean): void`: the heart of click-through. `useClickThrough()` sends `true` when the pointer moves onto an element marked `data-hit` and `false` when it leaves; the main process then turns mouse input on or off for that window with `setIgnoreMouseEvents()`.

- `invokeAction(id: CommandActionId): Promise<ActionResult>`: runs a command action; only command ids are allowed (see `CommandActionId`).
- `openExternal(url: string): Promise<void>`: opens a link from one of Claude's replies. The main process only accepts web (`http:`, `https:`) and email (`mailto:`) addresses (`ExternalUrl` in `src/main/ipc.ts`).
- `setText(text: string): void`: sent on every keystroke, so it doesn't wait for an answer. `NotesStore` saves the draft to disk about half a second after typing stops.
- `thumbnail(path: string): Promise<string | null>`: returns the preview as a *data URL*, the image encoded into a `data:image/...` string. The page can't load files from disk itself; its Content-Security-Policy (`index.html`) only allows images from the app or from data URLs.
- `signIn(): Promise<void>`: resolves straight away, because signing in happens in the browser and can take minutes. Progress arrives through `onAuthStatus`.
- `send(text: string): Promise<SendResult>`: sends the draft. The promise only says whether it was accepted; Claude's reply streams in through `onChatMessage`.
- `onModeChanged(callback: (mode: Mode) => void): () => void`: each `on...` method starts listening and returns a function that stops. That suits React's `useEffect`, whose cleanup step can simply call it (see `useAssistState()`).
- `onClickThroughReset(callback: (pointer: Point) => void): () => void`: `pointer` is the mouse position relative to the panel window, so the page can decide at once whether the pointer is over real UI.

`src/shared/format.ts`: screenshot labels

One small text helper. `AttachmentChip.tsx` uses it for the label on each screenshot chip in the text box. It has no Electron or React code in it, and `tests/format.test.ts` tests it directly. The file is short, so here it is whole.

`screenshotLabel()`

`src/shared/format.ts`, lines 1–16

```
const SCREENSHOT_NAME =
  /^Screenshot (\d{4})-(\d{2})-(\d{2}) (\d{2})(\d{2})(\d{2})(?: \(\d+\d+\))?.png$/
const MONTHS = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

/**
 * Short chip label for a screenshot file: "Screenshot 14:03" if it was taken today, otherwise
 * "Screenshot 2 Oct 14:03". Unrecognised names are shown as-is.
 */
export function screenshotLabel(fileName: string, now: Date = new Date()): string {
  const match = SCREENSHOT_NAME.exec(fileName)
  if (!match) return fileName
  const [, year, month, day, hour, minute] = match.map(Number) as number[]
  const time = `${String(hour).padStart(2, '0')}:${String(minute).padStart(2, '0')}`
  const today = year === now.getFullYear() && month === now.getMonth() + 1 && day === now.getDate()
  return today ? `Screenshot ${time}` : `Screenshot ${day} ${MONTHS[(month ?? 1) - 1]} ${time}`
}
```

Turns a screenshot's file name into a short label: just the time for today's screenshots, the date and time for older ones.

- `SCREENSHOT_NAME` =: matches the names that `screenshotFileName()` in `src/main/screenshots/files.ts` creates, such as `Screenshot 2026-10-02 140311.png`. The parentheses capture year, month, day, hour, minute

and second. `(?: \(\d+\))?` allows an optional `(2)` for a second capture in the same second, without capturing the number.

- `now: Date = new Date()`: a default parameter. The tests pass a fixed date so the result doesn't depend on the day they run.
- `if (!match) return fileName`: any other name is shown as it is.
- `const [, year, month, day, hour, minute] = match.map(Number)`: turns the captured text into numbers. The leading comma skips `match[0]`, the whole name; the seconds are captured but not used.
- `month === now.getMonth() + 1`: JavaScript months count from 0, file names from 1. Both the file name and `now` use the local clock, so "today" is the user's today.
- `(month ?? 1) - 1`: the regex guarantees `month` is there, but this project turns on TypeScript's `noUncheckedIndexedAccess` setting, which treats every array element as possibly missing. The `?? 1` fallback satisfies the compiler.

`src/preload/index.ts`: the bridge to the page

A *preload script* runs inside each window (bubble and panel) just before the page's own code. This one builds `window.assist` and is the only route from a page to the main process. To see why it exists, look at how the windows are created (`webPreferences` in `src/main/bubble/windows.ts`):

- `contextIsolation: true`: the preload runs in its own JavaScript "world", with its own globals, separate from the page's. The page can't see or tamper with the preload's variables, including its `ipcRenderer`.
- `nodeIntegration: false`: the page has no Node.js at all: no `require`, no file access.
- `sandbox: true`: even the preload gets only a small part of Electron and Node, and can't load other files at runtime. So electron-vite bundles `@shared/ipc` into the built `out/preload/index.js`, which contains its own copy of `IPC`.

contextBridge is Electron's way across the wall between the two worlds: it copies a chosen object into the page's world. The page therefore gets a fixed list of functions, each sending one specific channel with only the arguments it names. It never gets `ipcRenderer` itself, which would let it send anything on any channel.

`subscribe()`

`src/preload/index.ts`, lines 4–10

```
function subscribe<T>(channel: string, callback: (payload: T) => void): () => void {
  const listener = (_event: IpcRendererEvent, payload: T) => callback(payload)
  ipcRenderer.on(channel, listener)
  return () => {
    ipcRenderer.removeListener(channel, listener)
  }
}
```

Starts listening on a channel the main process pushes to, and returns a function that stops listening. Every `on...` method in `api` uses it.

- `const listener = (_event: IpcRendererEvent, payload: T) => callback(payload)`: Electron calls a listener with an event object first. That object's `sender` is the `ipcRenderer` itself, so passing it on would hand the page the very thing the isolation hides. The wrapper passes only the payload.

- `ipcRenderer.removeListener(channel, listener)`: removing a listener needs the exact function that was added. The returned closure keeps hold of `listener`, so the page never has to hand its callback back to unsubscribe.

api

src/preload/index.ts, lines 12–55

```
const api: AssistApi = {
  getState: () => ipcRenderer.invoke(IPC.getState),
  bubbleClick: () => ipcRenderer.send(IPC.bubbleClick),
  bubbleDragStart: () => ipcRenderer.send(IPC.bubbleDragStart),
  bubbleDragEnd: () => ipcRenderer.send(IPC.bubbleDragEnd),
  collapse: () => ipcRenderer.send(IPC.collapse),
  setInteractive: (interactive) => ipcRenderer.send(IPC.setInteractive, interactive),
  invokeAction: (id) => ipcRenderer.invoke(IPC.invokeAction, id),
  openExternal: (url) => ipcRenderer.invoke(IPC.openExternal, url),
  copyText: (text) => ipcRenderer.invoke(IPC.copyText, text),
  notes: {
    setText: (text) => ipcRenderer.send(IPC.notesSetText, text),
    attachLatestScreenshot: () => ipcRenderer.invoke(IPC.notesAttachLatest),
    removeAttachment: (id) => ipcRenderer.invoke(IPC.notesRemoveAttachment, id),
  },
  screenshots: {
    thumbnail: (path) => ipcRenderer.invoke(IPC.screenshotThumbnail, path),
    open: (path) => ipcRenderer.invoke(IPC.screenshotOpen, path),
    openFolder: () => ipcRenderer.invoke(IPC.screenshotsOpenFolder),
  },
  settings: {
    get: () => ipcRenderer.invoke(IPC.settingsGet),
    setAutoStart: (enabled) => ipcRenderer.invoke(IPC.settingsSetAutoStart, enabled),
    setEffort: (effort) => ipcRenderer.invoke(IPC.settingsSetEffort, effort),
  },
  auth: {
    signIn: () => ipcRenderer.invoke(IPC.authSignIn),
    cancel: () => ipcRenderer.invoke(IPC.authCancel),
    signOut: () => ipcRenderer.invoke(IPC.authSignOut),
    retry: () => ipcRenderer.invoke(IPC.authRetry),
  },
  chat: {
    send: (text) => ipcRenderer.invoke(IPC.chatSend, text),
    stop: () => ipcRenderer.invoke(IPC.chatStop),
    retry: () => ipcRenderer.invoke(IPC.chatRetry),
    newConversation: () => ipcRenderer.invoke(IPC.chatNew),
  },
  onModeChanged: (callback) => subscribe(IPC.modeChanged, callback),
  onCornerChanged: (callback) => subscribe(IPC.cornerChanged, callback),
  onClickThroughReset: (callback) => subscribe(IPC.clickThroughReset, callback),
  onAuthStatus: (callback) => subscribe(IPC.authStatus, callback),
  onChatMessage: (callback) => subscribe(IPC.chatMessage, callback),
  onChatReset: (callback) => subscribe(IPC.chatReset, () => callback()),
}
```

The implementation of `AssistApi`: one line per method, each sending its channel from `IPC`.

- `const api: AssistApi`: the type annotation makes TypeScript check that every method exists with the right parameters, and lets it infer parameter types (for example, `interactive` is a `boolean`). It can't check the answers: `ipcRenderer.invoke()` returns `Promise<any>`, so the main process's handlers must return what `AssistApi` promises by agreement.

- `ipcRenderer.invoke(IPC.getState)`: request and reply. Nothing extra is passed, which matters because the main process checks that no-argument channels really receive no argument (`NoArgs` in `src/main/ipc.ts`).
- `ipcRenderer.send(IPC.bubbleClick)`: fire and forget, as for the other `void` methods.
- `subscribe(IPC.chatReset, () => callback())`: the reset carries no data, so the page's callback is called with no arguments, matching its `() => void` type.

`contextBridge.exposeInMainWorld()`

The last line of the file, `contextBridge.exposeInMainWorld('assist', api)`, publishes `api` to the page as `window.assist` ("main world" is Electron's name for the page's own world). From then on the renderer calls `window.assist.chat.send(...)` and so on, and `src/renderer/src/env.d.ts` declares `window.assist` as an `AssistApi` so those calls are type-checked too.

Main process: startup, IPC and app plumbing

The *main process* is the Node.js side of an Electron app: the single process that owns the windows, the files, the tray icon and the network. This page covers how it starts and how the pages talk to it. `index.ts` creates every service and wires them together, `ipc.ts` answers the pages' requests, and the smaller files hold the tenant config, the user's preferences, the commands behind the action icons and the tray icon. The services themselves (sign-in, Claude, the bubble, the draft and screenshots) each have their own page.

`src/main/index.ts`: startup and shutdown

The main process's entry point. electron-vite builds it into `out/main/index.js`, and the `main` field of `apps/desktop/package.json` tells Electron to run that file. It works in three stages:

1. **As soon as the file loads**, module-level code picks the data folder and makes sure only one copy of the app is running.
2. **Once Electron is ready**, `start()` creates the windows and every service, connects them with callbacks, registers the IPC handlers and the tray icon, and finally shows the bubble.
3. **When the app quits**, a `before-quit` handler saves the draft before letting it close.

The key idea is that the services don't know about each other or about Electron's globals. Each one is handed what it needs (callbacks, small adapter objects) when it's created, which is why the tests can drive them with fakes.

`index.ts` is the only file that knows about all of them.

Module-level code

`src/main/index.ts`, lines 33–47

```
const SHUTDOWN_TIMEOUT_MS = 2000

let controller: BubbleController | null = null

let tray: Tray | null = null
```

This runs the moment Electron loads the file, before the app is ready. The block shows only the three declarations. Two `if` statements sit between them (they can't be shown on their own), so they're quoted in the bullets below, in the order they run.

- `SHUTDOWN_TIMEOUT_MS = 2000`: the longest quitting will wait for the draft to be saved (see [Quitting](#) below).
- `if (!app.isPackaged)`: `app.isPackaged` is true for an installed build and false for `npm run dev`. Dev runs move their `userData` folder (Electron's per-app data folder, normally `%APPDATA%\Desktop Assist`) to `Desktop Assist (Dev)`, so they don't share the draft, the saved sign-in or the single-instance lock with an installed copy.
- `process.env['DESKTOP_ASSIST_DEV_USER_DATA'] || ''`: the automated tests name a folder of their own in this environment variable, so they never touch your dev data. `app.setPath('userData', ...)` has to run before anything reads that path, which is why this is at the very top of the file.
- `let controller`: the bubble controller, kept at module level so the `second-instance` handler below can reach it. It stays `null` until `start()` has created it.
- `let tray`: if JavaScript frees the `Tray` object (garbage collection, which happens once nothing refers to it), its icon can vanish from the notification area. A module-level variable keeps it alive for the whole run.

- `if (!app.requestSingleInstanceLock())`: the *single-instance lock* lets only one copy of the app run per data folder. A second copy fails to get the lock and quits at once (`app.quit()`).
- `app.on('second-instance', () => controller?.expand())`: Electron fires `second-instance` in the copy that's already running when another copy tries to start. So launching the app again (from the Start menu, say) just opens the panel. The `?.` skips the call if `start()` hasn't created the controller yet.
- `app.whenReady().then(start).catch(fail)`: windows, the screen and the tray can only be used once Electron has finished starting up, and `whenReady()` returns a promise for that moment. Then `start()` runs. If anything fails along the way, `fail()` shows the error.
- `.catch(fail)` after `.then(start)`: as the comment says, the order matters. Written as `.then(start, fail)`, `fail` would only run if `whenReady()` itself failed; an error thrown inside `start()` would go unhandled and no error box would appear. A `.catch()` at the end catches a failure from either step.

start()

Builds the whole app once Electron is ready. It's long, so it's shown below in pieces, in source order. Only declarations (`const`, `let`) can be picked out as pieces; the plain statements in between, such as `registerIpc({...})` and the `before-quit` handler, are quoted in the bullets. The link above each piece opens the full source.

The tenant config

src/main/index.ts, line 59

```
const tenant = TenantSchema.parse(tenantConfig)
```

- `TenantSchema.parse(tenantConfig)`: `tenantConfig` is the bundled `tenant.json` (the `@tenant` import shortcut points at `tenants/<TENANT>/`). TypeScript only checks it at build time, so it's checked again here against the schema in `tenant.ts`. A bad config throws and the app doesn't start.
- `const { signIn, claudeAccess } = await withDevOverrides(tenant)` (the next line): the JumpCloud and Claude access settings, possibly replaced in dev runs by `withDevOverrides()`. The rest of `start()` uses these two rather than `tenant.signIn` and `tenant.claudeAccess`.
- `Menu.setApplicationMenu(null)`: removes Electron's default menu (File, Edit, View and so on) and the keyboard shortcuts that come with it, such as reload and developer tools.
- `app.setAppUserModelId('com.morsemicro.desktopassist')`: the `AppUserModelId` is how Windows identifies an app, for example to label its notifications. It matches `appId` in `electron-builder.yml`, the ID the installer registers.

Folders, the draft and preferences

src/main/index.ts, lines 64–73

```
const userData = app.getPath('userData')

const screenshotsDir = join(app.getPath('pictures'), tenant.appName)

const actionCount = tenant.actions.length

const notes = new NotesStore(join(userData, 'notes.json'), {
  onError: (error) => console.error('Saving the text box failed', error),
})

const settings = new SettingsService(join(userData, 'preferences.json'), screenshotsDir)
```

- `app.getPath('pictures')`: the user's Pictures folder. Screenshots go in a subfolder named after the app (`appName` in `tenant.json`).
- `new NotesStore(join(userData, 'notes.json'), ...)`: the unsent draft in the text box. It's followed by `await notes.load()`, which reads the saved draft before anything can ask for it. `onError` only logs: a failed save shouldn't interrupt the user (see [The draft, screenshots and safe files](#)).
- `new SettingsService(join(userData, 'preferences.json'), screenshotsDir)`: the user's preferences, followed by `await settings.init()`, which loads them and, on an installed build's first run, turns on "Start with Windows" (see `SettingsService.init()`).

The two windows

`src/main/index.ts`, lines 76–85

```
// The windows are created after the last `await`: from here to registerIpc() below nothing
// waits, so the pages can't ask for the app state before there's anything to answer them.
const bubbleWindow = createOverlayWindow('bubble', { width: BUBBLE_BOX, height: BUBBLE_BOX })

const panelWindow = createOverlayWindow('panel', panelWindowSize(actionCount))

const windows = [bubbleWindow, panelWindow]

// Listen straight away, so a page that finishes loading quickly isn't missed.
const windowsReady = Promise.all(windows.map((win) => once(win, 'ready-to-show')))

const broadcast = (channel: string, payload?: unknown) => {
  for (const win of windows) if (!win.isDestroyed()) win.webContents.send(channel, payload)
}
```

The windows are created only now, after the last `await`, and the comment above them says why. A page starts loading as soon as its window exists, and its first request is for the app state (`getState`). From here down to `registerIpc()`, `start()` never waits, so JavaScript runs it all in one go and can't handle anything else in between, a page's request included. By the time a page's request is handled, its handler has been registered.

- `createOverlayWindow('bubble', ...)`: creates a hidden, frameless, transparent, always-on-top window and starts loading the page into it (`bubble/windows.ts`, see [The bubble](#)). The bubble window is just big enough for the bubble; the panel window's size depends on how many action icons the tenant has (`panelWindowSize()`).
- `once(win, 'ready-to-show')`: Node's `once()` turns the next firing of an event into a promise. `ready-to-show` fires when a hidden window's page has painted for the first time. The listeners are attached as soon as the windows exist, because an event that fires before anyone is listening is lost and the promise would never resolve. `start()` waits for it at the very end.
- `broadcast`: sends a message to both pages. `webContents.send(channel, payload)` is how the main process pushes a message to a page (the page listens through the preload's `on...` functions, see [Shared code and the preload bridge](#)). `isDestroyed()` skips a window that has already been closed, which happens while quitting.

The bubble controller

src/main/index.ts, lines 87–98

```
const bubble = new BubbleController({
  bubble: bubbleSurface(bubbleWindow),
  panel: panelSurface(panelWindow),
  displays: electronDisplays,
  actionCount,
  initialAnchor: settings.bubbleAnchor,
  onModeChange: (mode) => broadcast(IPC.modeChanged, mode),
  onAnchorChange: (anchor) => {
    broadcast(IPC.cornerChanged, anchor.corner)
    void settings.setBubbleAnchor(anchor) // remembered for next time
  },
})
```

`BubbleController` is the bubble's state machine: it decides when the panel opens, where the bubble sits, and moves and shows the windows to match (see [The bubble](#)). It never touches Electron directly; everything it needs is passed in here.

- `bubbleSurface(bubbleWindow)`, `panelSurface(panelWindow)`: small adapters (in `bubble/windows.ts`) that give the controller just "set bounds, show, hide, focus" for each window.
- `displays: electronDisplays`: the adapter for Electron's `screen` module, defined further down this file (`electronDisplays`).
- `initialAnchor: settings.bubbleAnchor`: the corner and display the bubble was dragged to last time, or `null` the first time.
- `onModeChange`: every mode change (collapsed, expanded, dragging and so on) is broadcast, so the pages can animate to match.
- `onAnchorChange`: when the bubble settles in a new corner, the panel is told (it lays itself out to open away from that corner) and the new anchor is saved. `void` in front of a promise means "start this and don't wait for it"; it also tells the linter the missing `await` is deliberate.
- `controller = bubble` (the next line): makes the controller reachable from the module-level `second-instance` handler.
- `panelWindow.on('blur', () => bubble.panelBlurred())`: `blur` fires when the panel loses keyboard focus, for example when the user clicks another app. The controller then closes the panel.

Screenshots and Claude

src/main/index.ts, lines 102–122

```
const screenshots = new ScreenshotService(screenshotsDir, () =>
  screen.getDisplayMatching(bubbleWindow.getBounds()),
)

// Dev runs may point at a local test server; an installed app always talks to Anthropic.
const devBaseUrl = !app.isPackaged ? process.env['ANTHROPIC_BASE_URL'] : undefined

const backend = new AnthropicBackend({
  baseUrl: devBaseUrl || API_BASE_URL,
  access: claudeAccess,
  identityToken: () => auth.freshIdToken(),
})

const chat = new ChatSession({
  backend,
  canChat: () => auth.canChat,
  getEffort: () => settings.get().effort,
  system: buildSystemPrompt(tenant),
  onMessage: (message) => broadcast(IPC.chatMessage, message),
  onReset: () => broadcast(IPC.chatReset),
  // The details (status codes, request IDs) go to the terminal in dev runs.
  onError: (error) => console.error('Claude request failed:', error),
})
```

- `screen.getDisplayMatching(bubbleWindow.getBounds())`: the screenshot service is given a function rather than a fixed display, so each capture takes the display the bubble is on at that moment.
- `devBaseUrl`: lets a dev run send Claude requests to a local mock server named in `ANTHROPIC_BASE_URL`. An installed build ignores the variable, so it can't be redirected.
- `identityToken: () => auth.freshIdToken()`: how `AnthropicBackend` gets a JumpCloud ID token to swap for Claude access (see [Talking to Claude](#)). `auth` is only declared further down; that's fine because this arrow function only runs later, when the first message is sent, by which time `auth` exists. Calling it any earlier would throw.
- `canChat: () => auth.canChat`, `getEffort: () => settings.get().effort`: functions rather than values, so `ChatSession` reads the current sign-in and the current Fast/Balanced/Thorough choice each time it sends.
- `system: buildSystemPrompt(tenant)`: the system prompt (Claude's standing instructions), built once from the app name, company name and any extra `systemPrompt` in `tenant.json`.
- `onMessage`, `onReset`: each new or changed chat message, and "New conversation", are broadcast to the pages.

Sign-in

src/main/index.ts, lines 124–148

```
const missing = missingSettings(signIn, claudeAccess)

// Plain http is allowed only for a test identity provider on this PC, in dev runs.
const localIssuer = !app.isPackaged && isLoopback(new URL(signIn.issuer))

let authState: AuthStatus['state'] = 'checking'

const auth = new AuthManager({
  oidc: missing.length === 0 ? createOidc(signIn, { allowInsecure: localIssuer }) : null,
  missing,
  store: new SecretStore(join(userData, 'jumpcloud-session.bin'), safeStorageEncryptor),
  redirectPort: signIn.redirectPort,
  listen: (port) => listenForRedirect(port),
  openBrowser: (url) => shell.openExternal(url),
  onStatus: (status) => {
    broadcast(IPC.authStatus, status)
    // Signing in took the user to their browser; bring the chat back once it's done.
    if (status.state === 'signed-in' && authState === 'signing-in') bubble.expand()
    authState = status.state
  },
  onSignedOut: (reason) => {
    backend.reset()
    // Logging out clears the chat. If the sign-in expired, it stays, so after signing back in
    // the user can carry on (Retry resends a message that failed).
    if (reason === 'logout') chat.newConversation()
  },
  onError: (error) => console.error('JumpCloud sign-in failed:', error),
})
```

`AuthManager` owns the JumpCloud sign-in (see [JumpCloud sign-in](#)). JumpCloud is an *OpenID Connect* (OIDC) provider: a standard way for an app to sign users in through the company's identity service in the browser.

- `missingSettings(signIn, claudeAccess)`: the IDs still blank in `tenant.json`. If any are missing, `oidc` is `null` and the chat box lists what it still has to fill in instead of offering to sign in.
- `localIssuer`: OIDC normally requires `https`. A dev run pointed at a test identity provider on this PC (`isLoopback()`) may use plain `http`; `createOidc(..., { allowInsecure })` passes that on to the OIDC library.
- `let authState`: the previous sign-in state. Inside `onStatus`, `auth.status` already holds the new state, so the old one has to be remembered here.
- `new SecretStore(join(userData, 'jumpcloud-session.bin'), safeStorageEncryptor)`: where the sign-in is kept between runs, encrypted with `safeStorageEncryptor`.
- `listen: (port) => listenForRedirect(port)`: starts the small web server on `127.0.0.1` that the browser returns to after sign-in. `openBrowser: (url) => shell.openExternal(url)` opens the JumpCloud page in the user's default browser.
- `if (status.state === 'signed-in' && authState === 'signing-in') bubble.expand()`: signing in sent the user to their browser. When it completes, the panel opens again so they can carry on chatting. Renewing at startup goes from `checking` to `signed-in`, so it doesn't pop the panel open.
- `backend.reset()`: whenever nobody is signed in any more, `AnthropicBackend` drops its Claude client and the Claude token it holds, so nothing from the old sign-in is reused.
- `if (reason === 'logout') chat.newConversation()`: Log out clears the chat. If the sign-in merely expired, the conversation stays so the user can sign back in and press Retry.

Two statements follow:

- `void auth.init()`: loads the saved sign-in and renews it with JumpCloud. It isn't awaited, so the windows appear while that happens; the result reaches the pages through `onStatus`. An `await` here would also let a page's request in before `registerIpc()` has run.
- `void rm(join(userData, 'claude-api-key.bin'), { force: true }).catch(() => {})`: an earlier version saved a Claude API key in this file. Nothing uses it now, so it's deleted. `force: true` means "no error if it isn't there", and `.catch(() => {})` ignores any other failure.

IPC, the tray and display changes

src/main/index.ts, line 182

```
const onDisplayChange = () => bubble.displayChanged()
```

Everything else in this part of `start()` is plain statements:

- `registerIpc({}`: connects the pages' requests to the services (see `registerIpc()`). It's given both windows (only they may call in) and every service.
- `actions: createActionHandlers({ controller: bubble, screenshots, quit: () => app.quit() })`: the commands behind the Screenshot, Bounce and Close icons (`actions.ts`).
- `getState: () => ({}`: builds the snapshot a page asks for when it first loads: the bubble's mode and corner, the draft, the settings, the branding, the app version (`app.getVersion()`, from `package.json`), the sign-in status and the chat so far. After that the page keeps up through the broadcasts above.
- `tray = createTray({}`: adds the icon in the notification area. `onOpen` opens the panel and `onQuit` quits (see `createTray()`).
- `screen.on('display-added', onDisplayChange)`: the same handler also runs for `display-removed` and `display-metrics-changed` (a resolution, scaling or taskbar change). `BubbleController.displayChanged()` moves the bubble back into its corner, or onto the main display if its own display was unplugged.

Quitting

src/main/index.ts, lines 187–188

```
// Save the text box before quitting. before-quit fires again after the second app.quit().  
let shuttingDown = false
```

The `app.on('before-quit', (event) => {` handler that follows makes sure the draft reaches the disk before the app closes. `before-quit` fires when something calls `app.quit()`: the tray's Quit or the Close icon. It doesn't fire when Windows shuts down or logs off, which is what the `session-end` handler at the end is for.

- `if (shuttingDown) return`: the handler ends by calling `app.quit()` again, which fires `before-quit` a second time. This flag lets that second one through.
- `event.preventDefault()`: cancels this first quit, to make time for the steps below.
- `bubble.dispose()`, `chat.stop()`: stops the bubble's animation timers and aborts a reply that is still streaming.
- `tray?.destroy()`: removes the tray icon now. Otherwise it can linger in the tray until the mouse passes over it.
- `Promise.race([notes.flush(), timeout])`: waits for the draft to be written, or `SHUTDOWN_TIMEOUT_MS` (2 seconds), whichever comes first, so a stuck disk can't stop the app from closing. `.finally(() => app.quit())` then quits for real.

- `panelWindow.on('session-end', () => notes.flushSync())`: when Windows logs off or shuts down, it doesn't wait for asynchronous work, so `session-end` (a Windows-only window event) saves the draft with a blocking write instead.

Finally, `await windowsReady` waits until both pages have painted, and `bubble.start()` puts the bubble in its corner and shows it. Waiting avoids showing a window before its page has drawn anything.

safeStorageEncryptor

src/main/index.ts, lines 206–211

```
/** Windows DPAPI through Electron: only this Windows user can decrypt what it encrypts. */
const safeStorageEncryptor: Encryptor = {
  isAvailable: () => safeStorage.isEncryptionAvailable(),
  encrypt: (text) => safeStorage.encryptString(text),
  decrypt: (data) => safeStorage.decryptString(data),
}
```

The encryption `SecretStore` uses for the saved sign-in. `SecretStore` only knows the small `Encryptor` interface (so the tests can give it a fake); this object fills it in with Electron's `safeStorage`.

- `safeStorage`: Electron's API for encrypting strings with the operating system's own protection. On Windows that's *DPAPI*, which ties the encrypted data to the signed-in Windows user, so another user of the same PC can't decrypt it.
- `isAvailable`: `safeStorage` only works once the app is ready. Nothing calls these methods before then, because the `SecretStore` is created inside `start()`.

withDevOverrides()

src/main/index.ts, lines 213–225

```
/**
 * Dev runs only: the sign-in and Claude access settings can be overridden from a JSON file named
 * by DESKTOP_ASSIST_DEV_CONFIG, to test against a local identity provider.
 */
async function withDevOverrides(tenant: Tenant): Promise<Pick<Tenant, 'signIn' | 'claudeAccess'>> {
  const path = !app.isPackaged ? process.env['DESKTOP_ASSIST_DEV_CONFIG'] : undefined
  if (!path) return tenant
  const override = DevOverrideSchema.parse(JSON.parse(await readFile(path, 'utf8')))
  return {
    signIn: SignInSchema.parse({ ...tenant.signIn, ...override.signIn }),
    claudeAccess: { ...tenant.claudeAccess, ...override.claudeAccess },
  }
}
```

Returns the sign-in and Claude access settings to use. In an installed build, or when `DESKTOP_ASSIST_DEV_CONFIG` isn't set, that's simply the ones from `tenant.json`. In a dev run, the JSON file named by that variable can replace some of them, for example to point the app at a test identity provider, without editing `tenant.json`.

- `Promise<Pick<Tenant, 'signIn' | 'claudeAccess'>>`: `Pick` is a TypeScript helper that keeps just the named properties of a type. That's why `return tenant` is allowed: the whole tenant has those two properties.
- `DevOverrideSchema.parse(JSON.parse(await readFile(path, 'utf8')))`: the file is checked against `DevOverrideSchema`, in which every field is optional.

- `SignInSchema.parse({ ...tenant.signIn, ...override.signIn })`: ... copies an object's properties, and later ones win, so the override's fields replace the tenant's. The merged result is checked again, so an override can't produce an invalid sign-in config.

isLoopback()

src/main/index.ts, lines 227–229

```
function isLoopback(url: URL): boolean {
  return ['127.0.0.1', 'localhost', '::1'].includes(url.hostname)
}
```

Whether a URL points at this PC. The three names are the IPv4 address, the usual name and the IPv6 address for "this machine"; the URL parser keeps the square brackets around IPv6 addresses in `hostname`. Used only to decide whether plain `http` is allowed for a dev identity provider.

toArea and electronDisplays

src/main/index.ts, lines 231–245

```
/** Electron's `screen`, in the shape the bubble controller uses. All coordinates are DIPs. */
const toArea = (display: Electron.Display): DisplayArea => ({
  id: display.id,
  workArea: display.workArea,
})

const electronDisplays: Displays = {
  primary: () => toArea(screen.getPrimaryDisplay()),
  byId: (id) => {
    const display = screen.getAllDisplays().find((d) => d.id === id)
    return display && toArea(display)
  },
  nearest: (point) =>
    toArea(screen.getDisplayNearestPoint({ x: Math.round(point.x), y: Math.round(point.y) })),
  cursor: () => screen.getCursorScreenPoint(),
}
```

The adapter between Electron's `screen` module and the `Displays` interface `BubbleController` uses (see [The bubble](#)). Keeping Electron behind this small interface is what lets `tests/bubbleController.test.ts` run the controller with two fake displays.

- `toArea`: keeps only what the controller needs from a display: its ID and its work area (the screen minus the taskbar).
- `DIPs`: device-independent pixels. On a display scaled to 150%, one DIP is 1.5 physical pixels. Electron's window positions and display sizes are all in DIPs, so the controller never has to think about scaling.
- `return display && toArea(display)`: if no display has that ID any more (it was unplugged), this returns `undefined` rather than throwing.
- `Math.round(point.x)`: the bubble's position can be fractional while it animates, but this Electron call takes whole numbers.
- `screen.getCursorScreenPoint()`: where the mouse pointer is, used while dragging the bubble.

These are defined at module level, before the app is ready, but `screen` is only touched when a method is called, which happens after `start()` has begun.

fail()

src/main/index.ts, lines 247–251

```
function fail(error: unknown): void {
  console.error(error)
  dialog.showErrorBox('Desktop Assist could not start', String(error))
  app.exit(1)
}
```

Shows a plain error box and exits. It's the `.catch()` of `app.whenReady().then(start)`, so it runs for any error during startup: an invalid `tenant.json`, an unreadable dev override file or preferences file, and so on. Errors after `start()` has finished don't come here.

- `dialog.showErrorBox(...)`: a simple, blocking message box. Unlike most dialogs, Electron allows it before the app is ready.
- `app.exit(1)`: quits immediately with exit code 1, closing any windows already created and without firing `before-quit`. There's nothing to save yet, and an app that half-started would otherwise keep running with nothing on screen.

src/main/ipc.ts: handling requests from the pages

IPC (inter-process communication) is how the pages, which run in separate renderer processes, ask the main process to do things. The preload script gives each page `window.assist`, whose functions send messages on named channels (listed in `src/shared/ipc.ts`; see [Shared code and the preload bridge](#)). This file registers the main process's answer to every channel.

Two rules apply to every request: it must come from one of the app's own two windows, and its argument must pass a *zod schema*. *zod* is a validation library: a schema describes the shape a value must have (a string of at most so many characters, one of three words, and so on), and `schema.parse(value)` returns the value if it matches or throws if it doesn't. The pages are trusted code, but checking at this boundary means a bug or an injected script in a page can't send the main process something unexpected.

IpcContext

src/main/ipc.ts, lines 22–33

```
export interface IpcContext {
  /** Only these windows' renderers may call in. */
  windows: BrowserWindow[]
  controller: BubbleController
  notes: NotesStore
  screenshots: ScreenshotService
  settings: SettingsService
  auth: AuthManager
  chat: ChatSession
  actions: ActionHandlers
  getState(): AppState
}
```

Everything the handlers need, passed in by `start()` in `index.ts`. The main process's services are created there, so this file only needs their types, not how to build them.

- `windows: BrowserWindow[]`: the bubble and panel windows; requests from any other page are refused.

- `getState(): AppState`: builds the full snapshot a page asks for when it first loads (defined in `start()`).

NoArgs, FilePath and ExternalUrl

src/main/ipc.ts, lines 35–47

```
const NoArgs = z.undefined()

const FilePath = z.string().min(1).max(1024)

/** Only web and email links may be opened from the chat (no file:, javascript: and so on). */
const ExternalUrl = z
  .string()
  .max(4096)
  .refine((value) => {
    try {
      return ['http:', 'https:', 'mailto:'].includes(new URL(value).protocol)
    } catch {
      return false
    }
  })
```

Schemas used by several channels.

- `z.undefined()`: for channels that take no argument. `ipcRenderer.invoke(channel)` with no argument arrives as `undefined`; anything else is rejected.
- `FilePath`: only checks for a sensible string. The real check, that the file is inside the screenshots folder, is done by `ScreenshotService (owns())`.
- `.refine(...)`: adds a custom check to a schema. Here the string must parse as a URL and use `http:`, `https:` (opened in the browser) or `mailto:` (opened in the email app), so a link in a Claude reply can't open a local file (`file:`), run script (`javascript:`) or launch another program through its own URL scheme. `new URL()` throws for text that isn't a URL, hence the `try/catch`.

registerIpc()

src/main/ipc.ts, lines 49–137

```
/** Wires every renderer request to the main process. All arguments are validated with zod. */
export function registerIpc(ctx: IpcContext): void {
  const trusted = (sender: WebContents) => ctx.windows.some((w) => w.webContents.id === sender.id)

  function handle<S extends z.ZodType, R>(
    channel: string,
    schema: S,
    run: (arg: z.output<S>) => R | Promise<R>,
  ): void {
    ipcMain.handle(channel, (event: IpcMainInvokeEvent, raw: unknown) => {
      if (!trusted(event.sender)) throw new Error('Untrusted sender for ${channel}')
      return run(schema.parse(raw))
    })
  }

  function on<S extends z.ZodType>(
    channel: string,
    schema: S,
    run: (arg: z.output<S>, event: IpcMainEvent) => void,
  ): void {
    ipcMain.on(channel, (event: IpcMainEvent, raw: unknown) => {
```

```

    if (!trusted(event.sender)) return
    const parsed = schema.safeParse(raw)
    if (parsed.success) run(parsed.data, event)
  })
}

handle(IPC.getState, NoArgs, () => ctx.getState())

on(IPC.bubbleClick, NoArgs, () => ctx.controller.clickBubble())
on(IPC.bubbleDragStart, NoArgs, () => ctx.controller.startDrag())
on(IPC.bubbleDragEnd, NoArgs, () => ctx.controller.endDrag())
on(IPC.collapse, NoArgs, () => ctx.controller.collapse())
on(IPC.setInteractive, z.boolean(), (interactive, event) => {
  BrowserWindow.fromWebContents(event.sender)?.setIgnoreMouseEvents(!interactive, {
    forward: true,
  })
})

handle(IPC.invokeAction, z.enum(COMMAND_ACTION_IDS), (id) => ctx.actions[id]())
handle(IPC.openExternal, ExternalUrl, (url) => shell.openExternal(url))
handle(IPC.copyText, z.string().max(1_000_000), (text) => clipboard.writeText(text))

on(IPC.notesSetText, z.string().max(MAX_NOTE_LENGTH), (text) => ctx.notes.setText(text))
handle(IPC.notesAttachLatest, NoArgs, async () : Promise<AttachResult> => {
  const latest = await ctx.screenshots.latest()
  if (!latest) return { ok: false, reason: 'no-screenshots' }
  const notes = ctx.notes.addAttachment(latest)
  return notes ? { ok: true, notes } : { ok: false, reason: 'already-attached' }
})
handle(IPC.notesRemoveAttachment, z.string().min(1).max(64), (id) =>
  ctx.notes.removeAttachment(id),
)

handle(IPC.screenshotThumbnail, FilePath, (path) => ctx.screenshots.thumbnail(path))
handle(IPC.screenshotOpen, FilePath, (path) => ctx.screenshots.open(path))
handle(IPC.screenshotsOpenFolder, NoArgs, () => ctx.screenshots.openFolder())

handle(IPC.settingsGet, NoArgs, () => ctx.settings.get())
handle(IPC.settingsSetAutoStart, z.boolean(), (enabled) => ctx.settings.setAutoStart(enabled))
handle(IPC.settingsSetEffort, z.enum(['low', 'medium', 'high']), (effort) =>
  ctx.settings.setEffort(effort),
)

// Sign-in happens in the browser and can take minutes, so these return straight away;
// progress is broadcast on IPC.authStatus. No tokens ever reach the renderer.
handle(IPC.authSignIn, NoArgs, () => void ctx.auth.signIn())
handle(IPC.authCancel, NoArgs, () => ctx.auth.cancelSignIn())
handle(IPC.authSignOut, NoArgs, () => ctx.auth.signOut())
handle(IPC.authRetry, NoArgs, () => void ctx.auth.retry())

// Sends the draft: the text from the box plus the screenshots attached to it.
handle(IPC.chatSend, z.string().max(MAX_NOTE_LENGTH), (text): SendResult => {
  if (ctx.chat.busy) return { ok: false, reason: 'busy' }
  const draft = ctx.notes.get()
  const images: OutgoingImage[] = []
  for (const attachment of draft.attachments) {
    const image = ctx.screenshots.forClaude(attachment.path)
    if (!image) return { ok: false, reason: 'missing-screenshot' }
    images.push(image)
  }
  const status = ctx.chat.send(text, images, draft.attachments)
  if (status !== 'sent') return { ok: false, reason: status }
  return { ok: true, notes: ctx.notes.clear() }
})
handle(IPC.chatStop, NoArgs, () => ctx.chat.stop())

```

```

    handle(IPC.chatRetry, NoArgs, () => ctx.chat.retry())
    handle(IPC.chatNew, NoArgs, () => ctx.chat.newConversation())
  }

```

Registers a handler for every channel, once, at startup. The first part sets up a sender check and two helpers; the rest is one line (or a few) per channel, grouped by area.

The sender check and the two helpers

- `trusted`: whether a message came from one of the app's two windows. `event.sender` is the *WebContents* (Electron's object for the page inside a window) that sent it, and each one has a unique `id`. The windows already refuse to open other pages or navigate away (`bubble/windows.ts`), so this is a second line of defence.
- `function handle<S extends z.ZodType, R>()`: for request-and-reply channels, which the page calls with `ipcRenderer.invoke()` and gets a promise back. `S` and `R` are *type parameters*: `S` is the schema's type and `R` the result's, so each call is type-checked against its schema.
- `z.output<S>`: the TypeScript type a schema produces, so `run` receives a correctly typed argument.
- `throw new Error(...), schema.parse(raw)`: when a handler throws (an untrusted sender, or an argument that fails the schema), Electron rejects the page's promise with the error message.
- `function on<S extends z.ZodType>()`: for one-way messages, which the page sends with `ipcRenderer.send()` and gets nothing back. These are the frequent ones (clicks, every keystroke, the pointer moving). With no reply to carry an error, a bad message is simply dropped (`safeParse` checks without throwing).

The bubble and the windows

- `handle(IPC.getState, ...)`: the snapshot each page loads at startup.
- `IPC.bubbleClick, IPC.bubbleDragStart, IPC.bubbleDragEnd, IPC.collapse`: forwarded to the matching `BubbleController` methods.
- `setIgnoreMouseEvents(!interactive, { forward: true })`: click-through for the panel. While the pointer is over a see-through area, the page asks for mouse input to be ignored, so clicks go to the window underneath; `forward: true` keeps sending pointer movement to the page so it can tell when the pointer comes back over real UI. `BrowserWindow.fromWebContents()` finds the window that sent the message.

Actions, links and the clipboard

- `z.enum(COMMAND_ACTION_IDS)`: only the `command` actions (Screenshot, Bounce, Close) are accepted. `ctx.actions[id]()` runs the matching handler from `actions.ts` and returns its result, which the panel shows as a toast. The Settings icon is a `popover` action that never leaves the page.
- `shell.openExternal(url)`: opens a link from a Claude reply in the default browser, after `ExternalUrl` has checked it (an email link opens the email app instead).
- `z.string().max(1_000_000)`: the text of a chat message whose Copy button was pressed, capped at a million characters (`_` is just a digit separator). `clipboard.writeText` puts it on the Windows clipboard.

The draft

- `on(IPC.notesSetText, ...)`: sent on every keystroke, so it's one-way. The length cap matches what `NotesStore` keeps (`MAX_NOTE_LENGTH`).
- `IPC.notesAttachLatest`: finds the newest screenshot in the folder and attaches it to the draft. It answers `no-screenshots` if there are none and `already-attached` if `addAttachment()` returns `null` because that file is already attached.
- `z.string().min(1).max(64)`: an attachment ID (a UUID, 36 characters).

Screenshots and settings

- `IPC.screenshotThumbnail`, `IPC.screenshotOpen`, `IPC.screenshotsOpenFolder`: a small preview for a chip, opening a screenshot in the default image viewer, and opening the folder. The two that take a path refuse one outside the screenshots folder.
- `IPC.settingsGet`, `IPC.settingsSetAutoStart`, `IPC.settingsSetEffort`: the Settings menu. Each returns the full, current settings, so the menu always shows what actually took effect.

Sign-in

- `() => void ctx.auth.signIn()`: signing in happens in the browser and can take minutes. `void` discards the promise, so the page's call returns at once; progress arrives as `IPC.authStatus` broadcasts. Retry works the same way.
- `ctx.auth.cancelSignIn()`: stops waiting for the browser (it closes the local listener).
- No handler returns a token: the pages only ever see the status and the user's name.

Chat

- `handle(IPC.chatSend, ...)`: sends the text plus the screenshots attached to the draft.
- `if (ctx.chat.busy) return { ok: false, reason: 'busy' }`: checked first so the screenshots aren't processed for nothing while a reply is still streaming.
- `ctx.screenshots.forClaude(attachment.path)`: shrinks each attached screenshot and encodes it as JPEG for Claude. If any file has gone missing, nothing is sent and the panel shows a toast.
- `ctx.chat.send(text, images, draft.attachments)`: starts the reply and returns straight away (it streams in through `IPC.chatMessage`). Anything but 'sent' (`busy`, `empty`, `signed-out`) is passed back as the reason.
- `{ ok: true, notes: ctx.notes.clear() }`: on success the draft is emptied, and the empty draft is returned so the text box clears.
- `IPC.chatStop`, `IPC.chatRetry`, `IPC.chatNew`: Stop, Retry and New conversation, passed straight to `ChatSession`.

`src/main/actions.ts`: the action icons' commands

The action icons above the bubble come from a registry in `src/shared/actions.ts`. Each action is either a `popover` (opens UI inside the panel, handled entirely by the page) or a `command` (runs in the main process). This file has the main-process side of the commands.

ActionHandlers and createActionHandlers()

src/main/actions.ts, lines 6–33

```
export type ActionHandlers = Record<CommandActionId, () => Promise<ActionResult>>

/** Main-process handlers for the `command` actions in the registry (src/shared/actions.ts). */
export function createActionHandlers(deps: {
  controller: BubbleController
  screenshots: ScreenshotService
  quit: () => void
}): ActionHandlers {
  return {
    async screenshot() {
      try {
        await deps.controller.whileHidden(() => deps.screenshots.capture())
        return { ok: true, message: 'Screenshot saved' }
      } catch (error) {
        console.error('Screenshot failed', error)
        return { ok: false, message: "Couldn't take a screenshot" }
      }
    },
    async bounce() {
      deps.controller.startBounce()
      return { ok: true }
    },
    async close() {
      deps.quit()
      return { ok: true }
    },
  }
}
```

`createActionHandlers()` returns one async function per command action. `start()` in `index.ts` creates them, and `registerIpc()` runs one when the page sends `IPC.invokeAction`. The `{ ok, message }` each returns is shown by the panel as a toast.

- `Record<CommandActionId, () => Promise<ActionResult>>`: `Record<K, V>` is a TypeScript type for an object with a `V` under every key in `K`. `CommandActionId` is the list of command actions (`'screenshot' | 'bounce' | 'close'`), so if a new command is added to the registry, this file won't compile until it has a handler.
- `deps.controller.whileHidden(() => deps.screenshots.capture())`: hides both windows, waits for Windows to repaint the area under them, takes the screenshot, then brings the panel back. This way the app isn't in its own screenshot.
- `catch (error)`: any failure (including `whileHidden()` refusing because the panel isn't open) becomes a friendly message; the details go to the log.
- `async bounce()`: starts the bubble bouncing around the screen. It's `async` only so that it fits the type: every handler returns a promise.
- `deps.quit()`: `app.quit()`, which goes through the `before-quit` handler in `index.ts`, so the draft is saved first.

src/main/tenant.ts: the tenant config

A *tenant* is one company's branding and settings, from `tenants/<id>/tenant.json`. This file defines the shape of that file as zod schemas, plus two small helpers. `start()` in `index.ts` checks the bundled config with `TenantSchema.parse()`, and `tests/tenants.test.ts` checks every tenant folder the same way, so a mistake is

caught by the tests before it reaches a build.

None of the values here are secret. The client ID and the Claude IDs identify the app; on their own they don't grant access to anything.

SignInSchema

src/main/tenant.ts, lines 5–14

```
/**
 * How users sign in: the company's OpenID Connect app (JumpCloud for Morse Micro). The client ID
 * isn't secret; it identifies the app to JumpCloud. Left empty until IT creates the app.
 */
export const SignInSchema = z.object({
  issuer: z.url(),
  clientId: z.string(),
  /** The browser comes back to http://127.0.0.1:<port>/callback after sign-in. */
  redirectPort: z.number().int().min(1024).max(65535),
})
```

How the app signs users in with JumpCloud (see [JumpCloud sign-in](#)).

- `issuer: z.url()`: the identity provider's address. The OIDC library reads the provider's endpoints from it.
- `clientId: z.string()`: allowed to be empty, so a tenant can ship before IT has created the JumpCloud app; `missingSettings()` reports it instead.
- `redirectPort`: the port of the local web server the browser returns to after sign-in. It must match the redirect address registered in JumpCloud. Ports below 1024 are reserved for system services, hence the minimum.

ClaudeAccessSchema

src/main/tenant.ts, lines 16–27

```
/**
 * How a signed-in user reaches Claude: Anthropic Workload Identity Federation swaps the user's
 * JumpCloud ID token for a short-lived Claude API token, acting as a service account. None of
 * these IDs are secret. Left empty until set up in the Claude Console.
 */
export const ClaudeAccessSchema = z.object({
  organizationId: z.string(),
  federationRuleId: z.string(),
  serviceAccountId: z.string(),
  /** Only needed when the federation rule covers more than one workspace. */
  workspaceId: z.string().optional(),
})
```

How a signed-in user reaches Claude. There's no API key: Anthropic's *Workload Identity Federation* accepts the user's JumpCloud ID token and returns a short-lived Claude token for the company's service account, if the federation rule set up in the Claude Console allows it (see [Talking to Claude](#) and `docs/JUMPCLOUD_SETUP.md`). These are the IDs that exchange needs.

SignInConfig and ClaudeAccessConfig

src/main/tenant.ts, lines 29–30

```
export type SignInConfig = z.infer<typeof SignInSchema>

export type ClaudeAccessConfig = z.infer<typeof ClaudeAccessSchema>
```

The TypeScript types of the two settings blocks. `z.infer<typeof SignInSchema>` derives the type from the schema, so the compile-time type and the run-time check can't drift apart. `oidc.ts` and `AnthropicBackend.ts` use these types.

TenantSchema and Tenant

src/main/tenant.ts, lines 32–47

```
export const TenantSchema = z.object({
  id: z.string().regex(/^[a-z0-9-]+$/),
  companyName: z.string().min(1),
  appName: z.string().min(1),
  accentColor: z.string().regex(/^#[0-9a-fA-F]{6}$/, 'must be a #RRGGBB colour'),
  actions: z
    .array(z.enum(ACTION_IDS))
    .min(1)
    .refine((ids) => new Set(ids).size === ids.length, 'actions must not repeat'),
  /** Extra instructions for Claude, added to the built-in system prompt. */
  systemPrompt: z.string().max(8000).optional(),
  signIn: SignInSchema,
  claudeAccess: ClaudeAccessSchema,
})

export type Tenant = z.infer<typeof TenantSchema>
```

The whole `tenant.json`.

- `id: z.string().regex(/^[a-z0-9-]+$/)`: lowercase letters, digits and hyphens only. It must also match the tenant's folder name; the tenants test checks that.
- `accentColor: a #RRGGBB colour`. The second argument of `regex()` is the error message.
- `z.array(z.enum(ACTION_IDS)).min(1)`: which icons appear above the bubble, in order. Each must be a known action, and there must be at least one.
- `.refine((ids) => new Set(ids).size === ids.length, ...)`: a `Set` drops duplicates, so if it's smaller than the list, an action is listed twice.
- `systemPrompt`: optional extra instructions for Claude, added after the built-in ones by `buildSystemPrompt()`.

brandingOf()

src/main/tenant.ts, lines 49–52

```
export function brandingOf(tenant: Tenant): Branding {
  const { companyName, appName, accentColor, actions } = tenant
  return { companyName, appName, accentColor, actions }
}
```

The part of the tenant the pages need: names, the accent colour and the action list. It goes into the state snapshot from `getState`. The object is rebuilt field by field rather than passed as is, so the sign-in and Claude settings never go to the pages.

missingSettings()

src/main/tenant.ts, lines 54–65

```
/** The tenant.json settings still to be filled in before anyone can sign in, by name. */
export function missingSettings(signIn: SignInConfig, access: ClaudeAccessConfig): string[] {
  const required: Record<string, string> = {
    'signIn.clientId': signIn.clientId,
    'claudeAccess.organizationId': access.organizationId,
    'claudeAccess.federationRuleId': access.federationRuleId,
    'claudeAccess.serviceAccountId': access.serviceAccountId,
  }
  return Object.entries(required)
    .filter(([, value]) => !value.trim())
    .map(([name]) => name)
}
```

Lists the required IDs that are still blank, by their name in `tenant.json` (for example `signIn.clientId`). `start()` passes the list to `AuthManager`; if it isn't empty, the status is `unconfigured` and the chat box shows the list instead of a sign-in button.

- `required`: only the IDs a tenant can leave blank. `issuer` and `redirectPort` aren't here because the schema already requires them, and `workspaceId` is optional.
- `!value.trim()`: a value of only spaces counts as blank.
- `.filter(([, value]) => ...)`, `.map(([name]) => name)`: `Object.entries` gives `[name, value]` pairs; the filter keeps the blank ones and the map keeps just their names.

DevOverrideSchema

src/main/tenant.ts, lines 67–75

```
/**
 * Dev runs only: a JSON file (path in DESKTOP_ASSIST_DEV_CONFIG) can override `signIn` and
 * `claudeAccess`, so the app can be pointed at a test identity provider without editing
 * tenant.json. Installed builds never read it.
 */
export const DevOverrideSchema = z.object({
  signIn: SignInSchema.partial().optional(),
  claudeAccess: ClaudeAccessSchema.partial().optional(),
})
```

The shape of the dev-only override file read by `withDevOverrides()`. `.partial()` makes every field of a schema optional, so the file only needs the settings it changes.

src/main/settings.ts: the user's preferences

`SettingsService` keeps the user's preferences in `preferences.json` in the `userData` folder: the reply effort (Fast, Balanced, Thorough), where the bubble was dragged to, and whether "Start with Windows" has been set up. It also reports the settings the Settings menu shows. Created in `start()`; the Settings menu reaches it through the

settings... IPC channels.

DEFAULT Effort, PreferencesSchema and Preferences

src/main/settings.ts, lines 7–21

```
/** Fast answers by default; Balanced and Thorough think longer. */
export const DEFAULT_EFFORT: Effort = 'low'

const PreferencesSchema = z.object({
  autoStartInitialized: z.boolean().optional(),
  effort: z.enum(['low', 'medium', 'high']).optional(),
  /** Where the bubble was dragged to: a corner of a particular display. */
  bubbleAnchor: z
    .object({
      displayId: z.number(),
      corner: z.enum(['top-left', 'top-right', 'bottom-left', 'bottom-right']),
    })
    .optional(),
})

type Preferences = z.infer<typeof PreferencesSchema>
```

- `DEFAULT_EFFORT`: the effort until the user picks one. `low` is shown as Fast.
- `PreferencesSchema`: what the file may contain. Every field is optional, so a missing field just means "not set yet", and an older file still loads after a new preference is added.
- `bubbleAnchor`: a display ID and one of the four corners (`BubbleAnchor` in `BubbleController.ts`).
- `type Preferences = z.infer<typeof PreferencesSchema>`: the type derived from the schema.

SettingsService

src/main/settings.ts, lines 24–30

```
private prefs: Preferences = {}

private saving: Promise<unknown> = Promise.resolve()

constructor(
  private readonly preferencesPath: string,
  private readonly screenshotsDir: string,
) {}
```

- `private prefs: Preferences = {}`: the preferences in memory. Reads come from here; every change is also written to the file.
- `private saving: Promise<unknown> = Promise.resolve()`: the most recent write to the file, so the next one can wait for it (see `save()`). It starts as an already-finished promise, so the first write doesn't wait for anything.
- `private readonly preferencesPath: string`: a TypeScript shorthand. A parameter marked `private` (or `public`, `readonly`) in a constructor is also stored as a field of the same name, which is why the constructor body is empty.
- `screenshotsDir`: not a preference; it's included in the settings so the menu can show where screenshots go.

SettingsService.init()

src/main/settings.ts, lines 32–40

```
/** Loads preferences. The first time an installed build runs, turns on "Start with Windows". */
async init(): Promise<void> {
  const result = await readJsonFile(this.preferencesPath, PreferencesSchema)
  this.prefs = result.status === 'ok' ? result.value : {}
  if (app.isPackaged && !this.prefs.autoStartInitialized) {
    this.setAutoStart(true)
    await this.save({ autoStartInitialized: true })
  }
}
```

Loads the file once at startup (called from `start()`).

- `readJsonFile(this.preferencesPath, PreferencesSchema)`: reads and checks the file (see `storage/jsonFile.ts` in [The draft, screenshots and safe files](#)). If it's missing or damaged, the result isn't `ok` and the defaults (`{}`) are used; the damaged file is replaced at the next save. A file that exists but can't be read at all (a permissions error) throws, and startup ends with the error box from `fail()`.
- `if (app.isPackaged && !this.prefs.autoStartInitialized)`: the first time an installed build runs, "Start with Windows" is switched on and `autoStartInitialized` is saved. Because of that flag it happens only once: if the user turns it off later, it stays off.

SettingsService.get()

src/main/settings.ts, lines 42–51

```
get(): Settings {
  const available = app.isPackaged
  return {
    // Read from Windows each time: the user can also change it in Settings > Apps > Startup.
    autoStart: available && app.getLoginItemSettings().openAtLogin,
    autoStartAvailable: available,
    screenshotsDir: this.screenshotsDir,
    effort: this.prefs.effort ?? DEFAULT_EFFORT,
  }
}
```

The current settings, as the Settings menu shows them (`Settings` in `src/shared/types.ts`).

- `const available = app.isPackaged`: starting with Windows only makes sense for an installed build. A dev run would register the development copy of Electron.
- `app.getLoginItemSettings().openAtLogin`: whether Windows will start the app at sign-in. It's read from Windows every time rather than remembered, because the user can also change it in Windows' Settings > Apps > Startup.
- `this.prefs.effort ?? DEFAULT_EFFORT`: `??` uses the right-hand value when the left is `null` or `undefined`.

SettingsService.setAutoStart()

src/main/settings.ts, lines 53–56

```
setAutoStart(enabled: boolean): Settings {  
  if (app.isPackaged) app.setLoginItemSettings({ openAtLogin: enabled })  
  return this.get()  
}
```

Turns "Start with Windows" on or off. `app.setLoginItemSettings()` adds or removes the app's entry in the user's startup list (a "Run" entry in the user's part of the Windows registry). Nothing is saved in `preferences.json`, since Windows itself is the record. Returns the new settings, so the menu shows what actually happened; in a dev run that's still "off".

SettingsService.setEffort()

src/main/settings.ts, lines 58–61

```
async setEffort(effort: Effort): Promise<Settings> {  
  await this.save({ effort })  
  return this.get()  
}
```

Saves the reply effort. `ChatSession` reads it through `getEffort()` each time it sends, so the change applies to the next message.

SettingsService.bubbleAnchor and SettingsService.setBubbleAnchor()

src/main/settings.ts, lines 63–69

```
get bubbleAnchor(): BubbleAnchor | null {  
  return this.prefs.bubbleAnchor ?? null  
}  
  
async setBubbleAnchor(anchor: BubbleAnchor): Promise<void> {  
  await this.save({ bubbleAnchor: anchor })  
}
```

Where the bubble rests. `get bubbleAnchor()` is a *getter*: it's read like a property (`settings.bubbleAnchor`, in `start()`), but runs this code. It returns `null` if the bubble has never been moved, and `BubbleController.start()` then uses the bottom-right corner of the main display. `setBubbleAnchor()` is called by `onAnchorChange` in `start()` each time the bubble settles in a new corner.

SettingsService.save()

src/main/settings.ts, lines 71–78

```
private async save(changes: Preferences): Promise<void> {  
  this.prefs = { ...this.prefs, ...changes }  
  // One write at a time, in order, so an older save can't finish last and overwrite a newer one.  
  const snapshot = this.prefs  
  const write = this.saving.then(() => writeJsonFile(this.preferencesPath, snapshot))  
  this.saving = write.catch(() => {})  
  await write  
}
```

Merges the changes into the preferences in memory, then writes the whole object to the file. Writes are queued: each one starts only after the previous one has finished.

- `{ ...this.prefs, ...changes }`: later properties win, so only the changed fields are replaced.
- `const snapshot = this.prefs`: the preferences as they are now, including this change. A queued write may start a little later, but it writes exactly this.
- `this.saving.then(() => writeJsonFile(this.preferencesPath, snapshot))`: waits for the previous write, then writes this one. Without the queue, two saves close together could finish in the wrong order and leave the older preferences on disk.
- `writeJsonFile(...)`: writes to a temporary file and then renames it over the old one, so a crash mid-write can't leave a half-written file.
- `this.saving = write.catch(() => {})`: the next save waits for this one, but a failed write mustn't stop all later saves, so the queue's copy ignores the error.
- `await write`: the caller still gets this write's own result, so a failure is reported to whoever asked for the save.

src/main/tray.ts: the tray icon

The icon in the Windows notification area (by the clock). Clicking it opens the panel, and its right-click menu has Open and Quit. It matters because the app has no taskbar button or window frame: this is the one place that always shows the app is running and offers a way to quit. The icon is the tenant's logo, the same image as the bubble, or a plain circle in the accent colour if the tenant has no `logo.png`.

Logos, logoPath, TRAY_SIZE and SCALE_FACTORS

src/main/tray.ts, lines 4–15

```
// The tenant's logo, the same image as the bubble. Only a PNG can be used: Windows can't draw a
// tray icon from SVG. `?asset` makes the build copy the file and gives its path at runtime.
const logos = import.meta.glob<string>('@tenant/logo.png', {
  query: '?asset',
  import: 'default',
  eager: true,
})

const logoPath = Object.values(logos)[0]

/** The tray icon is 16px at 100% display scaling; Windows picks the size for each scaling. */
const TRAY_SIZE = 16

const SCALE_FACTORS = [1, 1.25, 1.5, 2]
```

- `import.meta.glob<string>('@tenant/logo.png', {`: a Vite build feature. At build time it finds every file matching the pattern and puts them in an object, keyed by path. Unlike a plain `import`, it doesn't break the build when nothing matches: a tenant with only a `logo.svg` just gets an empty object.
- `query: '?asset'`: an electron-vite feature for the main process. The build copies the file into its output and the import gives the file's path at run time, rather than its contents.
- `import: 'default', eager: true`: take each match's default export (the path) straight away, rather than a function that loads it later.
- `Object.values(logos)[0]`: the logo's path, or `undefined` if the tenant has no `logo.png`.

- `TRAY_SIZE = 16`: tray icons are 16×16 at 100% display scaling. `SCALE_FACTORS` are the common Windows scalings (100%, 125%, 150%, 200%) that get an image of their own.

createTray()

src/main/tray.ts, lines 17–34

```
export function createTray(options: {
  appName: string
  accentColor: string
  onOpen: () => void
  onQuit: () => void
}): Tray {
  const tray = new Tray(trayImage(options.accentColor))
  tray.setToolTip(options.appName)
  tray.setContextMenu(
    Menu.buildFromTemplate([
      { label: `Open ${options.appName}`, click: options.onOpen },
      { type: 'separator' },
      { label: 'Quit', click: options.onQuit },
    ]),
  )
  tray.on('click', options.onOpen)
  return tray
}
```

Called once from `start()`, which keeps the returned `Tray` in the module-level `tray` variable.

- `new Tray(trayImage(options.accentColor))`: adds the icon, using the image from `trayImage()`.
- `tray.setToolTip(options.appName)`: the text shown when the mouse rests on the icon.
- `Menu.buildFromTemplate([...])`: builds the right-click menu from a list of items: a label and a `click` function, or a separator.
- `tray.on('click', options.onOpen)`: a left-click opens the panel too.

trayImage()

src/main/tray.ts, lines 36–61

```
/**
 * The logo, resized smoothly for each display scaling (letting Windows shrink the 512px image
 * itself looks jagged). A tenant without logo.png gets a plain circle in its accent colour.
 */
function trayImage(accentColor: string): NativeImage {
  const logo = logoPath ? nativeImage.createFromPath(logoPath) : null
  if (!logo || logo.isEmpty()) {
    const size = TRAY_SIZE * 2
    return nativeImage.createFromBitmap(circleBitmap(size, accentColor), {
      width: size,
      height: size,
      scaleFactor: 2,
    })
  }
  const image = nativeImage.createEmpty()
  for (const scaleFactor of SCALE_FACTORS) {
    const size = Math.round(TRAY_SIZE * scaleFactor)
    image.addRepresentation({
      scaleFactor,
      width: size,
      height: size,
      buffer: logo.resize({ width: size, height: size, quality: 'best' }).toPNG(),
    })
  }
  return image
}
```

Builds the icon image. A *NativeImage* is Electron's image object, and one *NativeImage* can hold several versions of the same picture (*representations*), one per display scaling.

- `nativeImage.createFromPath(logoPath)`: loads the PNG. If the file can't be read or decoded, the image is empty (`isEmpty()`), and the fallback is used as if there were no logo.
- `nativeImage.createFromBitmap(circleBitmap(size, accentColor), {`: the fallback, made from raw pixels drawn by `circleBitmap()` at 32×32. `scaleFactor: 2` says that bitmap is meant for 200% scaling, so the image counts as 16×16 and is scaled down on a 100% display.
- `nativeImage.createEmpty(), image.addRepresentation({`: starts with an empty image and adds the logo at 16, 20, 24 and 32 pixels, one per scaling. The icon on each display then uses the version made for its scaling.
- `logo.resize({ width: size, height: size, quality: 'best' }).toPNG()`: the logo file is 512×512. Shrinking it here with the best-quality filter looks smooth, while leaving Windows to shrink it that far looks jagged (as the comment says).

src/main/trayIcon.ts: drawing the fallback tray icon

Draws a tray icon in code: a filled circle in the tenant's accent colour. `trayImage()` uses it only for a tenant that has no `logo.png`. It has no Electron imports, so `tests/trayIcon.test.ts` can check the pixels directly.

circleBitmap()

src/main/trayIcon.ts, lines 1–22

```
/**
 * A filled, anti-aliased circle as raw premultiplied BGRA pixels: the format
 * nativeImage.createFromBitmap expects on Windows. The tray icon for a tenant without logo.png.
 */
export function circleBitmap(size: number, hexColor: string): Buffer {
  const [r, g, b] = parseHexColor(hexColor)
  const pixels = Buffer.alloc(size * size * 4)
  const centre = size / 2
  const radius = size / 2 - 0.5
  for (let y = 0; y < size; y++) {
    for (let x = 0; x < size; x++) {
      const distance = Math.hypot(x + 0.5 - centre, y + 0.5 - centre)
      const alpha = Math.min(1, Math.max(0, radius - distance + 0.5))
      const i = (y * size + x) * 4
      pixels[i] = Math.round(b * alpha)
      pixels[i + 1] = Math.round(g * alpha)
      pixels[i + 2] = Math.round(r * alpha)
      pixels[i + 3] = Math.round(255 * alpha)
    }
  }
  return pixels
}
```

Returns the raw pixels of a `size × size` image: a circle filling the square, with a smooth edge.

- `Buffer.alloc(size * size * 4)`: four bytes per pixel, all zero to begin with (fully transparent). Pixels go row by row, so pixel (x, y) starts at byte `(y * size + x) * 4`.
- `Math.hypot(x + 0.5 - centre, y + 0.5 - centre)`: the distance from the middle of this pixel to the centre of the circle.
- `Math.min(1, Math.max(0, radius - distance + 0.5))`: how much of the pixel the circle covers, roughly. 1 when the pixel's middle is at least half a pixel inside the edge, 0 when it's at least half a pixel outside, and in between on the edge. This smooth edge is called *anti-aliasing*; without it the circle would look jagged.
- `radius = size / 2 - 0.5`: leaves half a pixel of room so the soft edge fits inside the square.
- `pixels[i] = Math.round(b * alpha)`: the bytes go blue, green, red, alpha (*BGRA*), the order Windows uses. Each colour is multiplied by the coverage (*premultiplied* alpha), which is the form `nativeImage.createFromBitmap` expects.

parseHexColor()

src/main/trayIcon.ts, lines 24–28

```
export function parseHexColor(hex: string): [number, number, number] {
  const match = /^#([0-9a-f]{2})([0-9a-f]{2})([0-9a-f]{2})$/i.exec(hex)
  if (!match) throw new Error(`Not a #RRGGBB colour: ${hex}`)
  return [parseInt(match[1]!, 16), parseInt(match[2]!, 16), parseInt(match[3]!, 16)]
}
```

Turns `#RRGGBB` into three numbers from 0 to 255.

- `/^#([0-9a-f]{2})([0-9a-f]{2})([0-9a-f]{2})$/i`: exactly a `#` and three pairs of hex digits; the `i` flag allows upper or lower case. The short form `#fff` is rejected.

- `parseInt(match[1]!, 16)`: reads one pair as a base-16 number. The `!` tells TypeScript the group is definitely there (the regex matched), because this project's settings make it treat any array element as possibly missing.

`src/main/env.d.ts`: types for Vite's build features

A `.d.ts` file holds only type information and produces no code. This one lets TypeScript understand the build-time features `tray.ts` uses, which plain TypeScript doesn't know about.

`src/main/env.d.ts`, lines 1–3

```
// Vite's additions to TypeScript for the main process, e.g. import.meta.glob and `?asset` imports.
/// <reference types="vite/client" />
/// <reference types="electron-vite/node" />
```

- `/// <reference types="vite/client" />`: a *triple-slash directive*, which pulls in another package's type declarations. Vite's declarations add `import.meta.glob` and Vite's other `import.meta` extras.
- `/// <reference types="electron-vite/node" />`: electron-vite's declarations describe its special imports for the main process; for example, a `?asset` import is a file path (a `string`).

JumpCloud sign-in

This part covers how Desktop Assist knows who you are, and how that turns into access to Claude without an API key. You sign in once with JumpCloud in your normal browser. The app keeps a long-lived *refresh token*, encrypted on disk, and uses it to get short-lived *ID tokens*, which Anthropic swaps for Claude tokens. All of it runs in the main process: the pages only ever learn the sign-in status and your name and email (`AuthStatus` in `src/shared/types.ts`), never a token.

Signing in

1. The user clicks **Sign in with JumpCloud**. IPC `assist:auth-sign-in` calls `AuthManager.signIn()`, which sets the status to `signing-in`.
2. `signIn()` starts a tiny web server on `127.0.0.1` with `listenForRedirect()` (`loopback.ts`).
3. `oidc.begin()` (made by `createOidc()` in `oidc.ts`) builds the JumpCloud sign-in address with fresh PKCE, `state` and `nonce` values, and the app opens it in the default browser.
4. The user signs in at JumpCloud (often it finishes by itself). JumpCloud sends the browser back to `http://127.0.0.1:47621/callback?code=...&state=...`. The little server answers "you can close this tab", stops, and hands the address to `signIn()`.
5. `PendingSignIn.finish()` checks `state`, swaps the code (plus the PKCE secret) for tokens at JumpCloud, and has openid-client check the ID token.
6. `signIn()` insists on a refresh token, saves it with your name and email (`startSession()` → `SecretStore.save()`), keeps the unused ID token as a *spare*, and sets `signed-in`. `index.ts` then reopens the panel.

Starting up

1. `start()` in `index.ts` calls `AuthManager.init()`, which reads the saved sign-in with `loadSession()` → `SecretStore.load()`.
2. If there is one, `renew()` → `refresh()` → `Oidc.refresh()` swaps the refresh token for new tokens, saves JumpCloud's replacement refresh token (if it sent one), and keeps the new ID token as the spare. If JumpCloud can't be reached the status becomes `offline`; if it refuses, `endSession('expired')` signs the user out.

Getting Claude access

1. On the first chat request, the Anthropic SDK inside `AnthropicBackend` needs a Claude token. Its `oidcFederationProvider` calls the `identityToken()` option, which is `AuthManager.freshIdToken()`.
2. `freshIdToken()` returns the spare ID token if it has more than a minute left, otherwise gets a new one with `refresh()`.
3. The SDK posts the ID token to Anthropic's `/v1/oauth/token`. Anthropic checks it against the federation rule and returns a short-lived Claude token. The SDK keeps that token and only repeats steps 1 to 3 when it's about to expire, or once if Anthropic rejects it.

Signing out

`AuthManager.signOut()` stops any sign-in in progress, `endSession('logout')` deletes the saved file, and `Oidc.revoke()` asks JumpCloud to cancel the refresh token. Through `onSignedOut`, `index.ts` drops the SDK's cached Claude token (`backend.reset()`) and clears the chat.

src/main/auth/AuthManager.ts: owning the sign-in

`AuthManager` decides whether someone is signed in, keeps the refresh token, and hands out ID tokens for Claude. `start()` in `src/main/index.ts` creates the one instance. The IPC handlers in `src/main/ipc.ts` call `signIn()`, `cancelSignIn()`, `signOut()` and `retry()`; `ChatSession` asks `canChat` before sending; `AnthropicBackend` calls `freshIdToken()`. It never touches JumpCloud, the disk or the browser itself: those arrive through `AuthManagerDeps`, so `tests/authManager.test.ts` can drive it with fakes.

The ideas it is built around:

- **OpenID Connect (OIDC)** is the standard way for an app to ask an identity provider (here JumpCloud) "who is this user?". The user signs in on the provider's own web page, so the app never sees a password, and the app receives tokens.
- **ID token:** a short-lived JWT (a signed, base64-encoded JSON document) from JumpCloud saying who the user is (`sub`, `email`, `name`), which app it was made for (`aud`, the app's client ID) and when it expires (`exp`). It's what Anthropic accepts as proof of identity.
- **Refresh token:** a long-lived, opaque string that lets the app get new tokens from JumpCloud without the user signing in again. It's the only thing saved to disk. Its lifetime is set in JumpCloud (up to 90 days; see `docs/JUMPCLOUD_SETUP.md`).
- **Refresh token rotation:** JumpCloud may answer each refresh with a new refresh token and stop accepting the old one. So the newest must always be saved, and two refreshes must never run at once with the same token: the second would present a token that was just replaced, JumpCloud would refuse it, and a refusal ends the sign-in. That's why refreshes go through a queue.
- **Workload Identity Federation (WIF)** is how Anthropic accepts another identity provider's tokens instead of an API key. IT sets up a *federation rule* in the Claude Console: trust tokens from this issuer (JumpCloud), made for this audience (the app's client ID), whose claims match a condition (an `@morsemicro.com` email). The SDK sends an ID token; Anthropic checks its signature against JumpCloud's published keys and checks the rule, then returns a short-lived Claude token for the company's *service account*.
- **Each ID token works once.** Every ID token has a unique ID in its `jti` claim, and Anthropic refuses a token whose `jti` it has already accepted (replay protection: a copied token can't be swapped again). So every swap needs a new ID token, and `freshIdToken()` never hands out the same one twice.

SignInRequiredError, SignInUnavailableError

`src/main/auth/AuthManager.ts`, lines 13–25

```
/** Claude was asked for while nobody is signed in (or the sign-in just expired). */
export class SignInRequiredError extends Error {
  constructor() {
    super('Sign in with JumpCloud to use Claude')
  }
}

/** A saved sign-in couldn't be renewed because JumpCloud couldn't be reached. */
export class SignInUnavailableError extends Error {
  constructor(cause: unknown) {
    super("Couldn't reach JumpCloud", { cause })
  }
}
```

The two ways getting an ID token can fail. `refresh()` throws them, and they travel through the Anthropic SDK (which wraps them in its own error, with the original as `cause`) to `classifyError()` in `src/main/claude/errors.ts`. That turns them into the `signed-out` message ("Sign in with JumpCloud to keep chatting.") and the `sign-in-unreachable` message, both of which offer Retry.

- `extends Error`: a custom error class, so other code can tell the cases apart with `instanceof`.
- `{ cause }`: the standard way to attach the original error (for example a network failure) to a new one, so the log still shows what really happened.

SessionSchema, Session

`src/main/auth/AuthManager.ts`, lines 27–32

```
/** What's kept on disk (encrypted) between runs. */
const SessionSchema = z.object({
  refreshToken: z.string().min(1),
  user: z.object({ name: z.string().optional(), email: z.string().optional() }),
})

type Session = z.infer<typeof SessionSchema>
```

The sign-in as it's kept in memory and saved (as JSON, encrypted) in `jumpcloud-session.bin`: the refresh token, and the user's name and email so the app can show "Signed in as ..." while offline.

- `z.object({`: a zod schema, a description of the shape that can check data at runtime. `loadSession()` uses it to reject a saved file that doesn't fit.
- `z.infer<typeof SessionSchema>`: derives the TypeScript type from the schema, so the type and the runtime check can't drift apart.

AuthManagerDeps

`src/main/auth/AuthManager.ts`, lines 34–51

```
export interface AuthManagerDeps {
  /** Null when tenant.json isn't filled in yet ('missing' says what's needed). */
  oidc: Oidc | null
  missing: string[]
  store: Pick<SecretStore, 'load' | 'save' | 'clear'>
  redirectPort: number
  listen(port: number): Promise<RedirectListener>
  openBrowser(url: string): Promise<void>
  onStatus(status: AuthStatus): void
  /**
   * Nobody is signed in any more: the user logged out, or JumpCloud ended their sign-in (they'll
   * usually sign straight back in, so the conversation can be kept for them).
   */
  onSignedOut(reason: 'logout' | 'expired'): void
  /** Signing in or renewing failed; gets the full error, for the log. */
  onError?(error: unknown): void
  now?: () => number
}
```

Everything `AuthManager` needs from outside, passed to its constructor. `start()` in `index.ts` supplies the real ones: `createOidc()`, a `SecretStore` using Windows DPAPI, `listenForRedirect()` and Electron's `shell.openExternal()`.

- `oidc: Oidc | null: null` when `missingSettings()` in `src/main/tenant.ts` says `tenant.json` isn't filled in; `missing` then lists what's needed, for the panel to show.
- `Pick<SecretStore, 'load' | 'save' | 'clear'>`: a TypeScript utility type meaning "any object with these three methods of `SecretStore`", so tests can pass a simple in-memory store.
- `onStatus(status: AuthStatus): void`: called on every status change. `index.ts` broadcasts it to both pages on IPC `assist:auth-status`, and reopens the panel when `signing-in` turns into `signed-in`.
- `onSignedOut(reason: 'logout' | 'expired'): void`: `index.ts` calls `backend.reset()` (forgetting the cached Claude token) for both reasons, and clears the chat only for `'logout'`. After `'expired'` the conversation stays, so Retry can resend the failed message once the user signs back in.
- `now?: () => number`: an optional clock; tests use it to make ID tokens look old.

MIN_ID_TOKEN_LIFE_S

`src/main/auth/AuthManager.ts`, lines 53–54

```
/** An ID token is only handed out if it's valid for at least this much longer. */
const MIN_ID_TOKEN_LIFE_S = 60
```

`takeIdToken()` only hands out the spare ID token if it has more than 60 seconds left. A token that expires on its way to Anthropic, or that a slightly fast clock at Anthropic already sees as expired, would make the swap fail.

class AuthManager

The class, with the comment that describes it:

`src/main/auth/AuthManager.ts`, lines 56–67

```
/**
 * Owns the JumpCloud sign-in.
 *
 * Signing in opens JumpCloud in the browser (where the user is usually already signed in, so it
 * completes on its own). What's kept is a refresh token, saved encrypted, so the user stays
 * signed in across restarts; `init()` renews it at every start.
 *
 * Claude access is granted per ID token, and each ID token can only be exchanged once, so
 * `freshIdToken()` hands out a new one every time: the unused one from signing in first, then one
 * from a refresh. JumpCloud may replace the refresh token on every refresh; the newest is saved.
 */
export class AuthManager {
  // ...
}
```

This block shows only the class's own description (its doc comment) and its first line; the body is left out (`// ...`), and its fields and methods follow one by one below. The description sums up the design: sign in through the browser, keep the refresh token so the user stays signed in, renew it at every start, and hand out a new ID token for every swap with Anthropic.

AuthManager fields

src/main/auth/AuthManager.ts, lines 68–78

```
private state: AuthStatus = { state: 'checking' }

private session: Session | null = null

/** An ID token not yet exchanged for Claude access. */
private spare: { token: string; exp: number } | null = null

private listener: RedirectListener | null = null

/** Cancel was pressed during the current sign-in (possibly before the browser opened). */
private signInCancelled = false

/** Refreshes run one at a time, since each may replace the refresh token. */
private queue: Promise<unknown> = Promise.resolve()

/** Bumped by sign-in and sign-out, so a slow refresh can't bring back a session that ended. */
private generation = 0
```

The class keeps all its state in these private fields.

- `state`: the current `AuthStatus`. It starts as `checking` because `init()` hasn't run yet. It's only ever changed through `setStatus()`, so the pages always hear about it.
- `session`: who is signed in, and their refresh token. `null` means nobody is.
- `spare`: an ID token that JumpCloud handed over (when signing in, or when renewing at startup) and that hasn't been swapped yet. `exp` is its expiry in Unix seconds, straight from the token.
- `listener`: the loopback listener while a browser sign-in is waiting, so `cancelSignIn()` can close it.
- `signInCancelled`: set by `cancelSignIn()` during a sign-in. Closing the listener only stops the wait for the browser; this flag also covers a Cancel pressed before the browser has opened.
- `queue`: a chain of promises. Each refresh is attached to the end, so refreshes run strictly one after another (see rotation above).
- `generation`: a counter that `signIn()` and `endSession()` increase. Slow work notes the value before it starts waiting and compares afterwards; if it changed, the user signed in or out meanwhile and the result is thrown away. This stops, for example, a refresh that finishes after Log out from saving the refresh token again.

AuthManager.constructor, status, canChat

src/main/auth/AuthManager.ts, lines 80–89

```
constructor(private readonly deps: AuthManagerDeps) {}

get status(): AuthStatus {
  return this.state
}

/** Whether chat requests can be made (they may still need JumpCloud to renew the sign-in). */
get canChat(): boolean {
  return this.session !== null && this.state.state !== 'signing-in'
}
```

- `private readonly deps: AuthManagerDeps`: TypeScript shorthand that declares a private, read-only field `deps` and stores the argument in it, so the constructor body is empty.

- `get status()`: a getter, read like a field (`auth.status`). `getState()` in `index.ts` uses it to give a freshly loaded page the current status.
- `get canChat()`: whether `ChatSession` may start a request. It's true whenever a sign-in exists, including `checking` and `offline`: the request itself calls `freshIdToken()`, which tries JumpCloud again. It's false while a browser sign-in is in progress.

AuthManager.init()

src/main/auth/AuthManager.ts, lines 91–101

```
/** Loads the saved sign-in and renews it. Called once at startup. */
async init(): Promise<void> {
  if (!this.deps.oidc)
    return this.setStatus({ state: 'unconfigured', missing: this.deps.missing })
  const generation = this.generation
  const session = await this.loadSession()
  if (generation !== this.generation) return // signed in or out meanwhile
  this.session = session
  if (!session) return this.setStatus({ state: 'signed-out' })
  await this.renew()
}
```

Runs once at startup (`start()` in `index.ts` calls it without waiting). It loads the saved sign-in and renews it straight away, before any chat. That shows an expired sign-in at once rather than on the first message, saves JumpCloud's newest refresh token, and leaves a spare ID token ready for the first message.

- `if (!this.deps.oidc)`: `tenant.json` isn't filled in, so the status becomes `unconfigured` with the list of missing settings, and JumpCloud is never contacted. `return this.setStatus(...)` is just a short way to set the status and stop.
- `if (generation !== this.generation) return`: reading the file takes a moment. If the user signed in or out in the meantime, what was read is out of date and must not replace the newer state.
- `if (!session) return this.setStatus({ state: 'signed-out' })`: nothing saved (or the file couldn't be read or decrypted), so the panel offers Sign in.

AuthManager.retry()

src/main/auth/AuthManager.ts, lines 103–107

```
/** Tries again after `init()` couldn't reach JumpCloud. */
async retry(): Promise<void> {
  if (this.state.state !== 'offline' || !this.session) return
  await this.renew()
}
```

The **Retry** button on the offline banner (IPC `assist:auth-retry`). It renews again, but only if the app really is offline with a saved sign-in, so a stray click in any other state does nothing.

AuthManager.signIn()

src/main/auth/AuthManager.ts, lines 109–143

```
/** Signs in through the browser. Resolves when it finishes, fails or is cancelled. */
async signIn(): Promise<void> {
  const oidc = this.deps.oidc
  if (!oidc || this.state.state === 'signing-in') return
  const generation = ++this.generation
  this.signInCancelled = false
  this.setStatus({ state: 'signing-in' })

  let listener: RedirectListener | null = null
  try {
    listener = await this.deps.listen(this.deps.redirectPort)
    this.listener = listener
    const pending = await oidc.begin(listener.redirectUri)
    // Cancel may have been pressed while JumpCloud's settings were being fetched.
    if (this.signInCancelled) throw new SignInCancelledError()
    await this.deps.openBrowser(pending.url)
    const tokens = await pending.finish(await listener.result)
    if (!tokens.refreshToken) {
      throw new Error(
        "JumpCloud didn't allow staying signed in. Ask IT to turn on the Refresh Token grant for this
app.",
      )
    }
    if (generation !== this.generation) return
    await this.startSession(tokens.refreshToken, userOf(tokens.claims))
    this.spare = { token: tokens.idToken, exp: tokens.claims.exp }
    this.setStatus({ state: 'signed-in', user: userOf(tokens.claims) })
  } catch (error) {
    if (generation !== this.generation) return
    if (!(error instanceof SignInCancelledError)) this.deps.onError?.(error)
    this.setStatus({ state: 'signed-out', message: signInFailure(error) })
  } finally {
    listener?.close()
    if (this.listener === listener) this.listener = null
  }
}
```

The whole browser sign-in, from opening JumpCloud to saving the result. The IPC handler starts it and returns at once, because it can take minutes; progress reaches the panel through the status.

- `if (!oidc || this.state.state === 'signing-in') return`: not set up, or already waiting for the browser (a second click does nothing).
- `const generation = ++this.generation`: increases the counter first, so a refresh still running from before (for example the startup renewal) can't overwrite this sign-in when it finishes.
- `this.signInCancelled = false`: a fresh sign-in starts uncanceled.
- `listener = await this.deps.listen(this.deps.redirectPort)`: the loopback server is started before the browser opens, so it's ready when JumpCloud sends the browser back. This is also where "port in use" fails, before the browser opens.
- `const pending = await oidc.begin(listener.redirectUri)`: makes the sign-in address. `pending` keeps the secret PKCE, state and nonce values needed by `finish()`.
- `if (this.signInCancelled) throw new SignInCancelledError()`: `begin()` may have spent a few seconds fetching JumpCloud's settings (the first time only), and the user may have pressed Cancel meanwhile. Without

this check the browser would still open JumpCloud, with nothing listening for its return. Throwing lands in the `catch` below, just like a cancel while waiting for the browser.

- `await this.deps.openBrowser(pending.url)`: opens the default browser.
- `const tokens = await pending.finish(await listener.result)`: waits (up to 5 minutes) for the browser to come back, then swaps the code for tokens.
- `if (!tokens.refreshToken)`: without a refresh token the app couldn't keep the user signed in, or get a second ID token, so it stops with a message for IT.
- `if (generation !== this.generation) return`: the user signed out while this was running, so the tokens are thrown away.
- `this.spare = { token: tokens.idToken, exp: tokens.claims.exp }`: the ID token from signing in hasn't been swapped yet, so it's kept for the first chat message, saving a refresh.
- `if (!(error instanceof SignInCancelledError)) this.deps.onError?.(error)`: a cancel is the user's choice, so only real failures are logged. Either way the status goes back to `signed-out`, with the message from `signInFailure()` (none for a cancel).
- `finally`: always closes the listener (closing an already finished listener does nothing), and clears the `listener` field only if it still belongs to this sign-in.

AuthManager.cancelSignIn()

src/main/auth/AuthManager.ts, lines 145–149

```
/** Stops waiting for a sign-in started in the browser. */
cancelSignIn(): void {
  if (this.state.state === 'signing-in') this.signInCancelled = true
  this.listener?.close()
}
```

The **Cancel** button while waiting for the browser (IPC `assist:auth-cancel`), and the first step of `signOut()`. Either way `signIn()` ends with `signed-out` and no message.

- `if (this.state.state === 'signing-in') this.signInCancelled = true`: records the cancel, for the case where Cancel is pressed before the browser has opened (while `begin()` is still running). `signIn()` checks the flag after `begin()` and stops there. Outside a sign-in there's nothing to cancel, so the flag is left alone.
- `this.listener?.close()`: once the listener exists, closing it makes its `result` fail with `SignInCancelledError`, which ends a sign-in that is waiting for the browser.

AuthManager.signOut()

src/main/auth/AuthManager.ts, lines 151–160

```
/** Forgets the sign-in here, and tells JumpCloud to revoke it. */
async signOut(): Promise<void> {
  const refreshToken = this.session?.refreshToken
  this.cancelSignIn()
  await this.endSession('logout')
  if (refreshToken) {
    // Best effort: the saved copy is already gone, so failing to reach JumpCloud is harmless.
    await this.deps.oidc?.revoke(refreshToken).catch(() => {})
  }
}
```

Log out in Settings (IPC `assist:auth-sign-out`). It forgets the sign-in on this PC first, then asks JumpCloud to revoke the refresh token.

- `const refreshToken = this.session?.refreshToken`: taken first, because `endSession()` clears `session`.
- `await this.endSession('logout')`: deletes the saved file and sets `signed-out`; `onSignedOut('logout')` makes `index.ts` reset the Claude client and clear the chat.
- `revoke(refreshToken).catch(() => {})`: revoking means JumpCloud stops accepting that refresh token, so a leftover copy (in a backup, say) is useless. If JumpCloud can't be reached the error is ignored: the user is signed out on this PC either way.

`AuthManager.freshIdToken()`

`src/main/auth/AuthManager.ts`, lines 162–171

```
/**
 * A new, unused ID token for one exchange with Claude. Throws SignInRequiredError when nobody is
 * signed in or the sign-in has expired, and SignInUnavailableError when JumpCloud can't be
 * reached.
 */
freshIdToken(): Promise<string> {
  const next = this.queue.then(() => this.takeIdToken())
  this.queue = next.catch(() => {})
  return next
}
```

Gives the Anthropic SDK one new, never-swapped ID token. `AnthropicBackend` passes it to the SDK as `identityToken`, and the SDK calls it whenever it needs a new Claude token.

- `this.queue.then(() => this.takeIdToken())`: runs after every refresh already queued, so two requests at once never refresh with the same refresh token in parallel.
- `this.queue = next.catch(() => {})`: the queue continues from this step even if it fails. Without the `catch`, one failed refresh would leave a rejected promise at the end of the queue, and every later `.then` would be skipped and fail too.
- `return next`: the caller still gets the real outcome, error included (`SignInRequiredError` or `SignInUnavailableError`).

`AuthManager.takeIdToken()`

`src/main/auth/AuthManager.ts`, lines 173–178

```
private async takeIdToken(): Promise<string> {
  const spare = this.spare
  this.spare = null
  if (spare && spare.exp - this.nowSeconds() > MIN_ID_TOKEN_LIFE_S) return spare.token
  return (await this.refresh()).idToken
}
```

Runs inside the queue: uses the spare ID token if there's a good one, otherwise refreshes for a new one.

- `this.spare = null`: the spare is removed before it's checked, so it can be used at most once. A spare that's too old is simply dropped.
- `spare.exp - this.nowSeconds() > MIN_ID_TOKEN_LIFE_S`: more than a minute left, so it's safe to send.

AuthManager.renew()

src/main/auth/AuthManager.ts, lines 180–196

```
/** Renews the saved sign-in at startup (or on Retry), keeping its ID token for the first chat. */
private async renew(): Promise<void> {
  this.setStatus({ state: 'checking' })
  try {
    const tokens = await this.queueRefresh()
    this.spare = { token: tokens.idToken, exp: tokens.claims.exp }
  } catch (error) {
    if (error instanceof SignInUnavailableError && this.session) {
      this.setStatus({
        state: 'offline',
        user: this.session.user,
        message: "Couldn't reach JumpCloud to renew your sign-in. Check your connection.",
      })
    }
    // SignInRequiredError: refresh() has already signed the user out.
  }
}
```

The startup renewal (from `init()`) and the offline Retry. Unlike a refresh made for a chat request, it sets the status along the way and never throws.

- `this.setStatus({ state: 'checking' })`: the panel shows "Checking your JumpCloud sign-in...". `refresh()` sets `signed-in` when it succeeds.
- `this.spare = { token: tokens.idToken, exp: tokens.claims.exp }`: the new ID token is kept for the first chat message.
- `error instanceof SignInUnavailableError && this.session`: JumpCloud couldn't be reached, so the sign-in is kept and the status becomes `offline`, with a banner offering Retry. Chat stays usable (`canChat` is still true) and each send tries JumpCloud again. The `this.session` check also tells TypeScript that `this.session.user` on the next lines is safe.
- `// SignInRequiredError: refresh() has already signed the user out.`: so there's nothing left to do here. (The same error also comes back when the user signed in or out while this was running, and then the newer state stands.)

AuthManager.queueRefresh()

src/main/auth/AuthManager.ts, lines 198–202

```
private queueRefresh(): Promise<OidcTokens> {
  const next = this.queue.then(() => this.refresh())
  this.queue = next.catch(() => {})
  return next
}
```

The same queueing as `freshIdToken()`, but for `renew()`, which needs the whole `OidcTokens` (including the ID token's expiry) rather than just the token.

AuthManager.refresh()

src/main/auth/AuthManager.ts, lines 204–237

```
/** Swaps the refresh token for new tokens and saves the replacement refresh token. */
private async refresh(): Promise<OidcTokens> {
  const session = this.session
  if (!session || !this.deps.oidc) throw new SignInRequiredError()
  const generation = this.generation

  let tokens: OidcTokens
  try {
    tokens = await this.deps.oidc.refresh(session.refreshToken)
  } catch (error) {
    if (generation !== this.generation) throw new SignInRequiredError()
    this.deps.onError?.(error)
    if (error instanceof SessionExpiredError) {
      await this.endSession(
        'expired',
        'Your JumpCloud sign-in has expired. Sign in again to keep chatting.',
      )
      throw new SignInRequiredError()
    }
    throw new SignInUnavailableError(error)
  }
  if (generation !== this.generation) throw new SignInRequiredError()

  const user = userOf(tokens.claims, session.user)
  if (tokens.refreshToken && tokens.refreshToken !== session.refreshToken) {
    await this.startSession(tokens.refreshToken, user)
  } else {
    this.session = { ...session, user }
  }
  if (this.state.state !== 'signed-in' || !sameUser(this.state.user, user)) {
    this.setStatus({ state: 'signed-in', user })
  }
  return tokens
}
```

Swaps the refresh token with JumpCloud for a new ID token (and possibly a new refresh token), and decides what a failure means. It's always called through the queue.

- `if (!session || !this.deps.oidc) throw new SignInRequiredError()`: nobody is signed in, so there is nothing to refresh.
- `tokens = await this.deps.oidc.refresh(session.refreshToken)`: the refresh-token request to JumpCloud (`Oidc.refresh()` in `oidc.ts`).
- `if (generation !== this.generation) throw new SignInRequiredError()` (in the `catch`): the user signed in or out while waiting, so this failure no longer matters and isn't logged or acted on.
- `if (error instanceof SessionExpiredError)`: JumpCloud answered and said no (expired, revoked, or the user lost access). The sign-in is over, so `endSession('expired', ...)` signs out with an explanation.
- `throw new SignInUnavailableError(error)`: anything else (offline, a timeout, a JumpCloud server error, an unexpected response) counts as "couldn't reach JumpCloud". The sign-in is kept, because it may well work next time.
- `if (generation !== this.generation) throw new SignInRequiredError()` (after success): the user logged out while the request was on its way. Saving the new refresh token now would bring back a sign-in they just ended.

- `userOf(tokens.claims, session.user)`: the name and email from the new ID token, falling back to the saved ones for anything it leaves out.
- `if (tokens.refreshToken && tokens.refreshToken !== session.refreshToken)`: JumpCloud rotated the refresh token, so the new one is saved at once (the old one may already be dead). Otherwise only the in-memory user is updated, and the saved file is left alone.
- `if (this.state.state !== 'signed-in' || !sameUser(this.state.user, user))`: tells the pages only when something changed, not on every refresh. This is also how `checking` (after `renew()`) and `offline` turn into `signed-in`.

AuthManager.startSession()

src/main/auth/AuthManager.ts, lines 239–247

```
private async startSession(refreshToken: string, user: SignedInUser): Promise<void> {
  this.session = { refreshToken, user }
  try {
    await this.deps.store.save(JSON.stringify(this.session))
  } catch (error) {
    // Still signed in for this run; the user will just be asked again next start.
    console.error('Saving the sign-in failed', error)
  }
}
```

Records a new sign-in, or a rotated refresh token, in memory and on disk (`SecretStore.save()`, as JSON).

- `console.error('Saving the sign-in failed', error)`: saving can fail, for example if Windows encryption isn't available. The user stays signed in for this run; at the next start the old file (or none) is found and they may have to sign in again. That's better than failing a sign-in that worked.

AuthManager.endSession()

src/main/auth/AuthManager.ts, lines 249–258

```
private async endSession(reason: 'logout' | 'expired', message?: string): Promise<void> {
  this.generation++
  this.session = null
  this.spare = null
  await this.deps.store.clear().catch((error: unknown) => {
    console.error('Removing the saved sign-in failed', error)
  })
  this.setStatus(message ? { state: 'signed-out', message } : { state: 'signed-out' })
  this.deps.onSignedOut(reason)
}
```

Ends the sign-in, for Log out (`'logout'`) or because JumpCloud refused it (`'expired'`).

- `this.generation++`: happens first, before anything that waits, so every refresh or sign-in still in flight sees that its result is out of date.
- `this.spare = null`: an unused ID token belongs to the old sign-in and mustn't be swapped any more.
- `this.deps.store.clear()`: if deleting the file fails, the user is still signed out for this run; the error is only logged.
- `this.deps.onSignedOut(reason)`: `index.ts` forgets the cached Claude token, and for Log out clears the chat.

AuthManager.loadSession()

src/main/auth/AuthManager.ts, lines 260–269

```
private async loadSession(): Promise<Session | null> {
  try {
    const saved = await this.deps.store.load()
    if (!saved) return null
    const parsed = SessionSchema.safeParse(JSON.parse(saved))
    return parsed.success ? parsed.data : null
  } catch {
    return null
  }
}
```

Reads the saved sign-in for `init()`. Any problem means "nothing saved", so the worst case is that the user signs in again.

- `SessionSchema.safeParse(JSON.parse(saved))`: `safeParse` returns success or failure instead of throwing, so a file in an old or unexpected shape is ignored.
- `catch {}`: covers a file that can't be read and text that isn't JSON. (`SecretStore.load()` already returns `null` for a missing file or one that can't be decrypted.)

AuthManager.setStatus(), AuthManager.nowSeconds()

src/main/auth/AuthManager.ts, lines 271–278

```
private setStatus(status: AuthStatus): void {
  this.state = status
  this.deps.onStatus(status)
}

private nowSeconds(): number {
  return (this.deps.now?.() ?? Date.now()) / 1000
}
```

- `setStatus`: the one place the status changes, so the stored value and what the pages are told can't differ.
- `nowSeconds`: the current time in seconds, because a token's `exp` is in Unix seconds while `Date.now()` is in milliseconds. `this.deps.now?.()` lets tests supply the clock.

userOf(), sameUser(), signInFailure()

src/main/auth/AuthManager.ts, lines 281–294

```
function userOf(claims: IdTokenClaims, fallback: SignedInUser = {}): SignedInUser {
  return { name: claims.name ?? fallback.name, email: claims.email ?? fallback.email }
}

function sameUser(a: SignedInUser, b: SignedInUser): boolean {
  return a.name === b.name && a.email === b.email
}

/** What to tell the user when signing in didn't work (nothing if they cancelled). */
function signInFailure(error: unknown): string | undefined {
  if (error instanceof SignInCancelledError) return undefined
  if (error instanceof SignInTimeoutError) return 'Sign-in timed out. Try again.'
  return `Sign-in didn't work. ${describeSignInError(error)}`
}
```

Small helpers outside the class.

- `claims.name ?? fallback.name`: an ID token may leave out `name` or `email` (they're optional claims), so the previously known value is kept. `??` uses the right side only when the left is `null` or `undefined`.
- `sameUser`: lets `refresh()` skip a status update when nothing changed.
- `signInFailure`: the message under the Sign in button. A cancel shows nothing, the 5-minute timeout has its own message, and anything else is explained by `describeSignInError()` in `oidc.ts`.

src/main/auth/oidc.ts: OpenID Connect with JumpCloud

The only file that speaks OIDC. It wraps the `openid-client` library (version 6) behind a small `Oidc` interface with three calls (`begin`, `refresh`, `revoke`), so `AuthManager` and its tests don't depend on the library. `start()` in `index.ts` creates it with `createOidc(signIn, { allowInsecure: localIssuer })` only when `tenant.json` is complete.

The ideas behind it:

- **Authorization Code flow**: the app sends the browser to JumpCloud's *authorization endpoint*. After the user signs in, JumpCloud sends the browser back to the app's *redirect URI* with a one-time `code`. The app then posts that code straight to JumpCloud's *token endpoint* and gets the tokens in the reply, so tokens never appear in the browser.
- **Public client with PKCE** (Proof Key for Code Exchange): a desktop app can't keep a secret, since anyone can unpack the installer, so it has no client secret (JumpCloud's "Public (None PKCE)" setting). Instead, each sign-in makes a random *code verifier* and puts only its SHA-256 hash (the *code challenge*) in the browser address. When swapping the code, the app sends the verifier itself, and JumpCloud checks that it matches. A code intercepted on its way back is useless without the verifier, which never left the app.
- **state**: a random value sent out with the browser and expected back unchanged. It proves the callback belongs to the sign-in this app started, so a forged link can't slip in someone else's code (cross-site request forgery).
- **nonce**: a random value JumpCloud copies into the ID token. Checking it proves the ID token was made for this sign-in and isn't an old one replayed.
- **Discovery**: an OIDC provider publishes its endpoints, keys and features at `<issuer>/well-known/openid-configuration`. `openid-client` reads that, so only the issuer address and the client ID need to be configured.

What openid-client checks on an ID token: the issuer, the audience (must be this app's client ID), the expiry and issue time, that `sub` is present, the signing algorithm, and (when signing in) the nonce. As the comment above `createOidc` says, it doesn't verify the signature (openid-client 6 only does that when `enableNonRepudiationChecks` is turned on, and this code doesn't turn it on). OIDC allows that: the token came straight from JumpCloud's token endpoint over HTTPS, so TLS already proves who sent it. Anthropic does verify the signature when the token is swapped.

SCOPES

src/main/auth/oidc.ts, lines 4–5

```
/** `offline_access` asks for a refresh token, so the user stays signed in across restarts. */  
export const SCOPES = 'openid email profile offline_access'
```

What the app asks JumpCloud for: `openid` (this is an OIDC sign-in, so send an ID token), `email` and `profile` (put the email and name in it), and `offline_access` (also send a refresh token).

IdTokenClaims, OidcTokens

src/main/auth/oidc.ts, lines 7–21

```
export interface IdTokenClaims {  
  sub: string  
  email?: string  
  name?: string  
  /** Expiry, Unix seconds. */  
  exp: number  
}  
  
export interface OidcTokens {  
  /** A signed JWT identifying the user. Exchanged for Claude access (one exchange per token). */  
  idToken: string  
  /** JumpCloud may rotate this on every refresh; always keep the newest. */  
  refreshToken?: string  
  claims: IdTokenClaims  
}
```

The parts of JumpCloud's answer the app uses. `toTokens()` builds them.

- `sub`: the user's permanent, unique ID at JumpCloud ("subject").
- `exp`: `number`: when the ID token expires, in Unix seconds. `AuthManager` uses it to decide whether a spare is still fresh enough.
- `idToken`: `string`: the raw JWT, exactly as JumpCloud signed it, because that's what Anthropic needs.
- `refreshToken`: `string`: optional, because a refresh answer may leave it out when JumpCloud doesn't rotate it.

PendingSignIn, Oidc

src/main/auth/oidc.ts, lines 23–35

```
/** A sign-in started in the browser. */
export interface PendingSignIn {
  url: string
  /** Completes the sign-in from the URL the browser was sent back to. */
  finish(callbackUrl: string): Promise<OidcTokens>
}

/** The identity-provider calls the app needs. The real one wraps openid-client; tests use fakes. */
export interface Oidc {
  begin(redirectUri: string): Promise<PendingSignIn>
  refresh(refreshToken: string): Promise<OidcTokens>
  revoke(refreshToken: string): Promise<void>
}
```

The interface `AuthManager` sees.

- `PendingSignIn`: a sign-in in progress. `url` is the address to open in the browser. `finish(callbackUrl)` completes it with the address the browser came back to; it remembers the secret values from `begin()`, so they never pass through `AuthManager`.
- `begin(redirectUri: string)`: starts a sign-in that will come back to `redirectUri` (the loopback listener's address).
- `refresh(refreshToken: string)`: new tokens without the browser. Throws `SessionExpiredError` if JumpCloud refuses.
- `revoke(refreshToken: string)`: asks JumpCloud to stop accepting a refresh token (Log out).

SessionExpiredError

src/main/auth/oidc.ts, lines 37–42

```
/** JumpCloud refused a saved sign-in (expired, revoked, or the user lost access). */
export class SessionExpiredError extends Error {
  constructor(cause?: unknown) {
    super('Your JumpCloud sign-in has expired', { cause })
  }
}
```

Thrown by `refresh()` when JumpCloud answered "no" (`isRefusal()`), as opposed to not answering. `AuthManager.refresh()` treats it as the end of the sign-in; any other error keeps the sign-in.

createOidc()

src/main/auth/oidc.ts, lines 44–113

```
/**
 * OpenID Connect with JumpCloud, using openid-client: Authorization Code flow with PKCE as a
 * public client (no client secret; the code verifier proves the app that started the sign-in is
 * the one finishing it), plus state and nonce checks. openid-client checks the ID token's issuer,
 * audience, expiry and nonce. It doesn't check the signature: the token comes straight from
 * JumpCloud over HTTPS, which OpenID Connect allows, and Anthropic verifies the signature when
 * the token is swapped for Claude access.
 */
export function createOidc(config: SignInConfig, options: { allowInsecure?: boolean } = {}): Oidc {
  // Discovery reads the issuer's endpoints once; a failed attempt is retried next time.
```

```

let discovered: Promise<client.Configuration> | null = null
const configuration = (): Promise<client.Configuration> => {
  discovered ??= client
    .discovery(new URL(config.issuer), config.clientId, undefined, client.None(), {
      // Plain http is only ever allowed for a local test issuer in dev runs.
      execute: options.allowInsecure ? [client.allowInsecureRequests] : [],
      timeout: 15,
    })
    .catch((error: unknown) => {
      discovered = null
      throw error
    })
  return discovered
}

return {
  async begin(redirectUri) {
    const configured = await configuration()
    const codeVerifier = client.randomPKCECodeVerifier()
    const state = client.randomState()
    const nonce = client.randomNonce()
    const url = client.buildAuthorizationUrl(configured, {
      redirect_uri: redirectUri,
      scope: SCOPES,
      code_challenge: await client.calculatePKCECodeChallenge(codeVerifier),
      code_challenge_method: 'S256',
      state,
      nonce,
    })
    return {
      url: url.href,
      finish: async (callbackUrl) =>
        toTokens(
          await client.authorizationCodeGrant(configured, new URL(callbackUrl), {
            pkceCodeVerifier: codeVerifier,
            expectedState: state,
            expectedNonce: nonce,
            idTokenExpected: true,
          })
        ),
    }
  },

  async refresh(refreshToken) {
    const configured = await configuration()
    try {
      return toTokens(await client.refreshTokenGrant(configured, refreshToken))
    } catch (error) {
      if (isRefusal(error)) throw new SessionExpiredError(error)
      throw error
    }
  },

  async revoke(refreshToken) {
    const configured = await configuration()
    if (!configured.serverMetadata().revocation_endpoint) return
    await client.tokenRevocation(configured, refreshToken, { token_type_hint: 'refresh_token' })
  },
}
}

```

The real `Oidc`, built on `openid-client`. It returns an object with the three methods; they share the discovery result through the closure (the local variables they can see).

Discovery (discovered and configuration):

- `let discovered: Promise<client.Configuration> | null = null`: the cached discovery. It caches the promise, not the result, so callers that arrive while discovery is still running wait for the same request instead of starting another.
- `discovered ??= client`: `??=` assigns only when the variable is `null`, so discovery runs on first use (not at startup) and only once.
- `config.clientId`, `undefined`, `client.None()`: the client ID, no extra client settings, and no client authentication. `None()` is how a public client is set up: token requests send just the client ID, with no secret.
- `execute: options.allowInsecure ? [client.allowInsecureRequests] : []`: openid-client refuses plain `http` unless told otherwise. `index.ts` only sets `allowInsecure` in dev runs whose issuer is on this PC (a test sign-in server).
- `timeout: 15`: seconds. openid-client also uses it for every later request made with this configuration, so a hung JumpCloud fails after 15 seconds instead of the default 30.
- `discovered = null`: a failed discovery (say, offline at startup) isn't kept, so the next call tries again rather than failing forever.

begin(redirectUri):

- `client.randomPKCECodeVerifier()`, `client.randomState()`, `client.randomNonce()`: new random values for every sign-in. They live only in this function's variables.
- `client.buildAuthorizationUrl(configured, {`: JumpCloud's authorization endpoint (from discovery) with these settings added as query parameters. openid-client also adds `client_id` and `response_type=code`.
- `code_challenge: await client.calculatePKCECodeChallenge(codeVerifier)`: the SHA-256 hash of the verifier, which is all the browser sees. `code_challenge_method: 'S256'` tells JumpCloud which hash was used.
- `finish: async (callbackUrl) =>`: an arrow function that still has the verifier, state and nonce from this call.
- `client.authorizationCodeGrant(configured, new URL(callbackUrl), {`: does the second half. It throws `AuthorizationResponseError` if the callback carries an `error` instead of a code, checks `state`, posts the code with the verifier to the token endpoint, and checks the ID token, including the nonce. The `redirect_uri` it sends is the callback address without its query, which must match the registered one exactly; see `listenForRedirect()`.
- `idTokenExpected: true`: an answer without an ID token is an error.

refresh(refreshToken):

- `client.refreshTokenGrant(configured, refreshToken)`: the refresh-token request. The new ID token gets the same checks as at sign-in, except the nonce.
- `if (isRefusal(error)) throw new SessionExpiredError(error)`: JumpCloud said no, so the sign-in is over. Every other error is passed on unchanged, and `AuthManager.refresh()` treats it as "couldn't reach JumpCloud".

revoke(refreshToken):

- `if (!configured.serverMetadata().revocation_endpoint) return`: not every provider offers revocation (discovery says whether it does), so without one there is nothing to do.
- `token_type_hint: 'refresh_token'`: tells JumpCloud what kind of token it's being sent, which helps it find it.

TokenResponse, toTokens()

src/main/auth/oidc.ts, lines 115–130

```
type TokenResponse = Awaited<ReturnType<typeof client.refreshTokenGrant>>

function toTokens(response: TokenResponse): OidcTokens {
  const claims = response.claims()
  if (!response.id_token || !claims) throw new Error('JumpCloud did not return an ID token')
  return {
    idToken: response.id_token,
    refreshToken: response.refresh_token,
    claims: {
      sub: claims.sub,
      email: typeof claims.email === 'string' ? claims.email : undefined,
      name: typeof claims.name === 'string' ? claims.name : undefined,
      exp: claims.exp,
    },
  }
}
```

Turns openid-client's token answer into the app's `OidcTokens`, keeping only what's used. JumpCloud's access token is ignored, because the app never calls a JumpCloud API.

- `Awaited<ReturnType<typeof client.refreshTokenGrant>>`: the type of what openid-client's token functions resolve to, taken from the library rather than written out by hand. `ReturnType` gives the function's return type (a promise) and `Awaited` unwraps the promise.
- `response.claims()`: the ID token's contents, already checked by openid-client; `undefined` when there's no ID token.
- `if (!response.id_token || !claims)`: without an ID token there's nothing to give Anthropic. During a refresh this error is treated as "couldn't reach JumpCloud".
- `typeof claims.email === 'string' ? claims.email : undefined`: a JWT's claims can hold any JSON, so only real strings are accepted.

describeSignInError()

src/main/auth/oidc.ts, lines 132–143

```
/** Why a browser sign-in failed, in words the user can act on. */
export function describeSignInError(error: unknown): string {
  if (error instanceof client.AuthorizationResponseError) {
    return error.error_description ?? `JumpCloud said "${error.error}"`.
  }
  // fetch failures are TypeErrors; openid-client reports its timeouts with code OAUTH_TIMEOUT.
  const timedOut = error instanceof client.ClientError && error.code === 'OAUTH_TIMEOUT'
  if (error instanceof TypeError || timedOut) {
    return "Couldn't reach JumpCloud. Check your connection."
  }
  return error instanceof Error ? error.message : String(error)
}
```

Explains a failed browser sign-in in words the user can act on. Used only by `signInFailure()` in `AuthManager.ts`.

- `error instanceof client.AuthorizationResponseError`: JumpCloud sent the browser back with an error instead of a code, most often because the user isn't in a group assigned to the app. `error_description` is JumpCloud's own explanation; if there isn't one, the short error code is shown.

- `error instanceof client.ClientError && error.code === 'OAUTH_TIMEOUT'`: a request to JumpCloud that took longer than the 15-second `timeout` set in `createOidc()`. `openid-client` doesn't pass on the `TimeoutError` that `fetch` raises: it wraps it in its own `ClientError` with the code `OAUTH_TIMEOUT`, so that's what is checked.
- `error instanceof TypeError || timedOut: TypeError` is what Node's `fetch` throws ("fetch failed") when it can't connect at all, for example offline or a DNS failure. Both cases get "Couldn't reach JumpCloud. Check your connection."
- `error instanceof Error ? error.message : String(error)`: anything else shows its own message, for example the "Port ... is in use" error from `loopback.ts`.

isRefusal()

src/main/auth/oidc.ts, lines 145–153

```
/** JumpCloud answered, and said no (as opposed to not being reachable). */
function isRefusal(error: unknown): boolean {
  return (
    error instanceof client.ResponseBodyError &&
    ['invalid_grant', 'invalid_client', 'unauthorized_client', 'access_denied'].includes(
      error.error,
    )
  )
}
```

Tells "JumpCloud said no" apart from "JumpCloud didn't answer".

- `error instanceof client.ResponseBodyError`: JumpCloud's token endpoint answered with a standard OAuth error (`{"error": "..."}`), so it was reachable.
- `'invalid_grant'`: the refresh token is expired, revoked or already replaced. `'invalid_client'` and `'unauthorized_client'`: the app's JumpCloud registration was removed or changed. `'access_denied'`: the user lost access. In every case only signing in again can help.

Other error codes, such as `server_error` or `temporarily_unavailable`, aren't refusals: the sign-in is kept and tried again later.

src/main/auth/loopback.ts: catching the browser's return

After the user signs in, JumpCloud has to send the browser somewhere the app can see the `code`. A desktop app has no website, so it uses a **loopback redirect** (RFC 8252, the standard for native apps): for each sign-in, the app runs a tiny web server on `127.0.0.1`, the PC's address for itself, which other computers can't reach. JumpCloud is registered with `http://127.0.0.1:47621/callback` as the redirect URI. Plain `http` is fine because the traffic never leaves the PC, and PKCE protects the code even if another program on the PC sees it. `index.ts` passes `listenForRedirect` to `AuthManager` as `listen`, and `signIn()` starts one listener per sign-in. It uses Node's built-in `http` module, which works in Electron's main process.

CALLBACK_PATH, RedirectListener

src/main/auth/loopback.ts, lines 3–13

```
export const CALLBACK_PATH = '/callback'

/** Waiting for the browser to come back from the sign-in page. */
export interface RedirectListener {
  /** What JumpCloud must send the browser back to (registered in the JumpCloud app). */
  redirectUri: string
  /** The full URL the browser came back to, including 'code' and 'state'. */
  result: Promise<string>
  /** Stops waiting (cancel); 'result' then rejects with SignInCancelledError. */
  close(): void
}
```

- `CALLBACK_PATH`: the only path the server answers.
- `redirectUri`: the full address to give JumpCloud. `oidc.ts` puts it in the sign-in address, and JumpCloud only accepts it if it exactly matches the one registered in the JumpCloud app.
- `result: Promise<string>`: settles once: with the address the browser came back to, or with an error for cancel or timeout.
- `close()`: cancels the wait. `AuthManager` calls it from `cancelSignIn()` and always when a sign-in ends.

SignInCancelledError, SignInTimeoutError

src/main/auth/loopback.ts, lines 15–25

```
export class SignInCancelledError extends Error {
  constructor() {
    super('Sign-in was cancelled')
  }
}

export class SignInTimeoutError extends Error {
  constructor() {
    super('Sign-in took too long')
  }
}
```

The two ways `result` can fail. `signInFailure()` in `AuthManager.ts` shows nothing for a cancel and "Sign-in timed out. Try again." for a timeout.

listenForRedirect()

src/main/auth/loopback.ts, lines 27–81

```
/**
 * Starts a one-shot web server on 127.0.0.1 that catches the browser's redirect after sign-in
 * (the "loopback" method for desktop apps, RFC 8252). It only listens on the local machine,
 * answers the browser with a short page saying it can close the tab, and shuts down after the
 * first callback, a cancel, or the timeout.
 */
export function listenForRedirect(port: number, timeoutMs = 5 * 60_000): Promise<RedirectListener> {
  return new Promise((resolveListener, rejectListener) => {
    let settle: { resolve: (url: string) => void; reject: (error: Error) => void }
    const result = new Promise<string>((resolve, reject) => (settle = { resolve, reject }))
    result.catch(() => {}) // a rejection nobody awaited yet (cancel before the caller waits)
```

```

let timer: ReturnType<typeof setTimeout> | undefined
const server: Server = createServer((req, res) => {
  const url = new URL(req.url ?? '/', `http://127.0.0.1:${port}`)
  if (req.method !== 'GET' || url.pathname !== CALLBACK_PATH) {
    res.writeHead(404).end()
    return
  }
  const error = url.searchParams.get('error_description') ?? url.searchParams.get('error')
  res.writeHead(200, {
    'content-type': 'text/html; charset=utf-8',
    'cache-control': 'no-store',
  })
  // Shut down only once the page has been sent.
  res.end(callbackPage(error), () => finish(() => settle.resolve(url.href)))
})

let done = false
const finish = (then: () => void) => {
  if (done) return
  done = true
  clearTimeout(timer)
  server.close()
  server.closeAllConnections()
  then()
}

server.once('error', (error: NodeJS.ErrnoException) => {
  rejectListener(
    error.code === 'EADDRINUSE'
      ? new Error(`Port ${port} is in use by another program, so sign-in can't finish.`)
      : error,
  )
})

server.listen(port, '127.0.0.1', () => {
  timer = setTimeout(() => finish(() => settle.reject(new SignInTimeoutError()))), timeoutMs)
  resolveListener({
    redirectUri: `http://127.0.0.1:${port}${CALLBACK_PATH}`,
    result,
    close: () => finish(() => settle.reject(new SignInCancelledError())),
  })
})
})
}

```

Starts the one-shot server and returns once it's actually listening. That lets `signIn()` be sure the port is open before it opens the browser.

- `return new Promise((resolveListener, rejectListener) => {`: the outer promise resolves with the `RedirectListener` once the server is listening, or rejects if it can't start.
- `const result = new Promise<string>((resolve, reject) => {settle = { resolve, reject }})`: creates `result` and keeps its `resolve` and `reject` functions in `settle`, so the request handler, the timer and `close()` can settle it later. (The function given to `new Promise` runs immediately, so `settle` is set straight away.)
- `result.catch(() => {})`: if `close()` is called before anyone waits on `result` (a cancel during `begin()`, say), Node would report an unhandled rejection. This empty handler prevents that; anyone who awaits `result` later still gets the error.
- `new URL(req.url ?? '/', `http://127.0.0.1:${port}`)`: `req.url` is only the path and query (`/callback?code=...`), so the base is added to get the full address. It's built with exactly the registered host and port because `openid-client` takes the `redirect_uri` it sends to JumpCloud from this address.

- `if (req.method !== 'GET' || url.pathname !== CALLBACK_PATH)`: anything else, such as the browser asking for `/favicon.ico`, gets a 404 and doesn't end the wait.
- `url.searchParams.get('error_description') ?? url.searchParams.get('error')`: if JumpCloud sent back an error, the page says so. The address is still handed on, and `finish()` in `oidc.ts` turns it into the proper error.
- `'cache-control': 'no-store'`: the browser shouldn't keep a copy of a page whose address holds a one-time code.
- `res.end(callbackPage(error), () => finish(() => settle.resolve(url.href)))`: the callback runs once the page has been sent, so the server isn't shut down halfway through the reply.
- `if (done) return`: only the first of the callback, the timeout and `close()` counts; later ones do nothing. That's why `AuthManager` can call `close()` after a sign-in has already finished.
- `server.close()`: stops accepting new connections. `server.closeAllConnections()` also drops connections the browser is keeping open for reuse, which would otherwise keep the server alive.
- `error.code === 'EADDRINUSE'`: another program already uses the port, so starting fails with an explanation (IT can pick another port; see `docs/JUMPCLOUD_SETUP.md`).
- `server.listen(port, '127.0.0.1', () => {})`: listening on `127.0.0.1` only (not all network interfaces) keeps the server invisible to other computers. The callback runs once it's listening: it starts the timeout (5 minutes by default) and hands back the listener.

`callbackPage(), escapeHtml()`

`src/main/auth/loopback.ts`, lines 83–94

```
function callbackPage(error: string | null): string {
  const message = error
    ? `Sign-in didn't complete: ${escapeHtml(error)}`
    : 'You're signed in to Desktop Assist. You can close this tab.'
  return `<!doctype html><html><head><meta charset="utf-8"><title>Desktop Assist</title>
<style>body{font-family:'Segoe UI',system-ui,sans-serif;display:grid;place-
items:center;height:100vh;margin:0;color:#18181b}
p{font-size:16px}</style></head><body><p>${message}</p><script>setTimeout(()=>window.close(),1500)
</script></body></html>`
}

function escapeHtml(text: string): string {
  return text.replace(/<|>|'"/g, (c) => `&#${c.charCodeAt(0)};`)
}
```

The small page the browser shows after the redirect.

- `escapeHtml(error)`: the error text comes from the address, which anyone could make up, so it's escaped before going into the HTML. Otherwise a crafted link could run its own script on a page served from this PC.
- `setTimeout(()=>window.close(),1500)`: tries to close the tab after 1.5 seconds. Browsers usually only let a script close a tab that a script opened, so the tab may well stay open; the text tells the user they can close it.
- `text.replace(/<|>|'"/g, (c) => `&#${c.charCodeAt(0)};`)`: replaces each character that means something in HTML with its numeric code, for example `<` with `<`.

src/main/storage/SecretStore.ts: keeping the sign-in safe

Keeps one secret in one file, encrypted so only the same Windows user on the same PC can read it. The app has exactly one secret: the JumpCloud sign-in (`AuthManager` saves the refresh token and user as JSON in `%APPDATA%\Desktop Assist\jumpcloud-session.bin`). The encryption itself is passed in as an `Encryptor`: `index.ts` gives it Electron's `safeStorage`, which on Windows uses **DPAPI** (the Windows Data Protection API, which encrypts with a key tied to the user's Windows login). Tests use a fake.

Encryptor

src/main/storage/SecretStore.ts, lines 4–9

```
/** Encrypts secrets at rest. In the app this is Electron's safeStorage (Windows DPAPI). */
export interface Encryptor {
  isAvailable(): boolean
  encrypt(text: string): Buffer
  decrypt(data: Buffer): string
}
```

What `SecretStore` needs to encrypt. `safeStorageEncryptor` in `index.ts` maps the three methods onto `safeStorage.isEncryptionAvailable()`, `encryptString()` and `decryptString()`.

- `encrypt(text: string): Buffer`: the result is raw bytes (a Node `Buffer`), not text.
- `decrypt(data: Buffer): string`: throws if the bytes can't be decrypted, for example a file encrypted by another Windows user.

SecretStore

The class, with the comment that describes it:

src/main/storage/SecretStore.ts, lines 11–15

```
/**
 * Keeps one secret in a file, encrypted so only the signed-in Windows user can read it. Used for
 * the JumpCloud sign-in (its refresh token), which never leaves the main process.
 */
export class SecretStore {
  // ...
}
```

As with `AuthManager`, this block shows only the class's description and first line; the members follow below, starting with the constructor, which declares the class's fields:

src/main/storage/SecretStore.ts, lines 16–19

```
constructor(
  private readonly filePath: string,
  private readonly encryptor: Encryptor,
) {}
```

The class's two fields, declared with the same constructor shorthand as `AuthManager`: the file to use (`jumpcloud-session.bin` in the app's data folder) and the `Encryptor`.

SecretStore.load()

src/main/storage/SecretStore.ts, lines 21–35

```
/** The saved secret, or null if there is none or it can't be decrypted (e.g. another user's). */
async load(): Promise<string | null> {
  let data: Buffer
  try {
    data = await fs.readFile(this.filePath)
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return null
    throw err
  }
  try {
    return this.encryptor.decrypt(data) || null
  } catch {
    return null
  }
}
```

Reads and decrypts the file. `AuthManager.loadSession()` calls it at startup.

- `fs.readFile(this.filePath)`: with no text encoding given, it returns the raw bytes.
- `isErrno(err, 'ENOENT')`: "no such file", meaning nothing is saved. Any other read error (for example access denied) is thrown, and `loadSession()` treats that as nothing saved too.
- `this.encryptor.decrypt(data) || null`: an empty secret counts as none.
- `catch {}`: a file that can't be decrypted (copied from another user or PC, or damaged) is treated as no sign-in. The user signs in again, and the file is overwritten.

SecretStore.save(), SecretStore.clear()

src/main/storage/SecretStore.ts, lines 37–46

```
async save(secret: string): Promise<void> {
  if (!this.encryptor.isAvailable()) {
    throw new Error("Windows can't encrypt data on this PC, so the sign-in wasn't saved.")
  }
  await writeFileAtomic(this.filePath, this.encryptor.encrypt(secret))
}

async clear(): Promise<void> {
  await fs.rm(this.filePath, { force: true })
}
```

- `if (!this.encryptor.isAvailable())`: refuses to save rather than write the refresh token in plain text. `AuthManager.startSession()` logs the error, and the user stays signed in for this run only.
- `writeFileAtomic(this.filePath, this.encryptor.encrypt(secret))`: from `src/main/storage/jsonFile.ts`. It writes a temporary file and then renames it over the real one (retrying briefly if antivirus is holding the file), so a crash halfway through can't leave a half-written file that won't decrypt. It also creates the folder if needed.
- `fs.rm(this.filePath, { force: true })`: deletes the file; `force` means a file that's already gone isn't an error. Called by `AuthManager.endSession()`.

Talking to Claude

Everything that talks to the Claude API lives in four files in `src/main/claude`, all in the main process. `model.ts` holds the settings every request uses, `AnthropicBackend.ts` sends one request through Anthropic's SDK, `errors.ts` turns failures into plain English, and `ChatSession.ts` keeps the conversation and runs each reply. There is no API key: the SDK swaps the user's JumpCloud sign-in for a short-lived Claude token (see `AnthropicBackend.getClient()`).

What happens when you send a message

1. The panel sends IPC `assist:chat-send`. Its handler in `ipc.ts` turns the draft's screenshots into base64 JPEGs (`ScreenshotService.forClaude()`) and calls `ChatSession.send()`.
2. `send()` appends your turn to the conversation `history`, adds your message and an empty "streaming" reply to the chat, and calls `startTurn()`, which runs `runTurn()` in the background. `send()` returns `'sent'` straight away, and the IPC handler clears the draft.
3. `runTurn()` calls `AnthropicBackend.reply()` with the whole history, the system prompt from `buildSystemPrompt()` and the current effort setting.
4. `reply()` gets the SDK client from `getClient()` and calls `client.beta.messages.stream()`. Before the request goes out, the SDK needs a Claude token. The first time, and again shortly before each token expires, it calls `AuthManager.freshIdToken()` and swaps that JumpCloud ID token for a Claude token.
5. As the reply streams in, `onText` adds each piece of text to the reply, and `scheduleEmit()` sends it to the page (IPC `assist:chat-message`) at most once every 50 ms.
6. When the stream ends, `runTurn()` appends the complete reply to the history: every content block, thinking included, unchanged, unless another model took over partway through, in which case `replayContent()` first drops what the API says must not be sent back. If Claude asked for tools, `runTools()` runs them and the loop sends another request. Otherwise `finish()` marks the reply done, with a notice from `stopNotice()` if it was refused or cut short.
7. If anything fails, `classifyError()`, `errorMessage()` and `isRetryable()` decide what the chat shows and whether it offers Retry.

`src/main/claude/model.ts`: what every request uses

The settings that shape every request, kept in one file so the model or the request can be changed in one place. `AnthropicBackend.ts` reads the model, the fallback beta and the token limit; `index.ts` uses `API_BASE_URL` and `buildSystemPrompt()`; `ChatSession.ts` uses `MAX_TOOL_ROUNDS`; and `errors.ts` puts `CLAUDE_MODEL_NAME` in one of its messages.

`CLAUDE_MODEL`, `CLAUDE_MODEL_NAME`

`src/main/claude/model.ts`, lines 1–4

```
// How Desktop Assist calls Claude. Kept in one place so the model and request shape are easy to change.

export const CLAUDE_MODEL = 'claude-opus-5-5'

export const CLAUDE_MODEL_NAME = 'Claude Opus 5.5'
```

The model's ID, which goes in every request, and its human name, which only appears in the "can't use this model" error message (`errors.ts`).

- `'claude-opus-5-5'`: Claude Opus 5.5. On this model thinking is always on and can't be switched off; effort (see `AnthropicBackend.reply()`) is the only control over how much it thinks.

FALLBACK_BETA

`src/main/claude/model.ts`, lines 6–10

```
/**
 * Server-side refusal fallback, "default" mode: if Claude's safety classifiers decline a request,
 * the API re-runs it on Anthropic's recommended fallback model instead of returning a refusal.
 */
export const FALLBACK_BETA = 'server-side-fallback-2026-07-01'
```

Turns on the **server-side refusal fallback**. Claude Opus 5.5 runs safety classifiers alongside each request. If they decline a request, the reply normally just ends with `stop_reason: 'refusal'` and no proper answer. With the fallback on, the API instead re-runs the request on another model inside the same call, so the user still gets an answer. A `fallback` content block in the reply marks where the other model took over: first in the reply if the decline came before any output, or partway through if it came mid-reply (see `replayContent()` in `ChatSession.ts` for what that means for the history). After a fallback, later requests in the same conversation may go straight to the fallback model for about an hour.

`AnthropicBackend.reply()` turns it on with two request fields: `betas: [FALLBACK_BETA]`, which the SDK sends as an `anthropic-beta` header to unlock the feature (the date is the feature's version), and `fallbacks: 'default'`, which lets Anthropic pick the fallback model by the kind of refusal instead of the app naming one. For kinds of refusal with no recommended fallback model, or if the fallback model declines too, the reply still ends with `refusal`, and `stopNotice()` in `ChatSession.ts` shows "Claude declined to answer this."

MAX_TOKENS

`src/main/claude/model.ts`, lines 12–13

```
/** Replies stream, so a high ceiling costs nothing unless Claude actually writes that much. */
export const MAX_TOKENS = 64_000
```

The most Claude may write in one reply, its thinking included. A token is a short piece of text, a few characters long.

Replies are **streamed**: instead of one response at the end, the API sends the reply in small pieces as it is written. The chat can show text as it arrives, and a long reply never runs into an HTTP timeout. Only the tokens actually written are billed, so a high ceiling costs nothing by itself. A reply that does reach it ends with `stop_reason: 'max_tokens'`, which `stopNotice()` reports as cut short.

API_BASE_URL, MAX_TOOL_ROUNDS

`src/main/claude/model.ts`, lines 15–18

```
export const API_BASE_URL = 'https://api.anthropic.com'

/** Never resend a turn more than this many times in one tool-use loop. */
export const MAX_TOOL_ROUNDS = 10
```

- `API_BASE_URL`: Where requests go. `index.ts` passes it to `AnthropicBackend`, except that a dev run may replace it with the `ANTHROPIC_BASE_URL` environment variable to use a local test server. The token exchange uses the same address.
- `MAX_TOOL_ROUNDS`: A safety limit for `ChatSession.runTurn()`. When Claude uses tools, one reply takes several requests (see `ChatTool`); this stops a reply that never settles from looping forever.

buildSystemPrompt()

src/main/claude/model.ts, lines 20–36

```
/**
 * The system prompt, fixed for the whole conversation (changing it mid-conversation would throw
 * away the prompt cache). Tenants can add their own instructions in tenant.json.
 */
export function buildSystemPrompt(tenant: {
  appName: string
  companyName: string
  systemPrompt?: string
}): string {
  const base = [
    `You are ${tenant.appName}, a desktop assistant for people at ${tenant.companyName}.`,
    "You appear as a small chat panel in the corner of the user's Windows screen, so keep answers",
    'concise and easy to scan: short paragraphs, lists and code blocks where they help.',
    'The user can attach screenshots of their screen; when they do, use what you can see in them.',
  ].join(' ')
  return tenant.systemPrompt ? `${base}\n\n${tenant.systemPrompt}` : base
}
```

Builds the **system prompt**: standing instructions sent at the top of every request, separate from the conversation. It tells Claude who it is, that it lives in a small panel (so answers should be short and easy to scan), and that screenshots may be attached. `index.ts` calls it once at startup and gives the result to `ChatSession`, which sends the same text with every request.

- `systemPrompt?: string`: Optional extra instructions from `tenant.json` (up to 8,000 characters, checked by `TenantSchema`), added after the built-in text with a blank line between them. The parameter's type lists only the three fields used; `index.ts` passes the whole tenant object.
- `.join(' ')`: The four lines are one paragraph; they are split in the source only to keep lines short.

Why the prompt must never change during a conversation: **prompt caching**. The API remembers nothing between requests, so every request sends the whole conversation again. Prompt caching lets Anthropic keep the start of a recent request for a few minutes. When the next request begins with exactly the same content, that part is read from the cache, which is faster and much cheaper. The match is on the exact beginning of the request (tools, then the system prompt, then the messages), so changing even one character of the system prompt would make everything after it miss. That's why the prompt is built once and contains nothing that changes, such as the date or time.

src/main/claude/AnthropicBackend.ts: sending one request

The only file that calls the Anthropic SDK (`@anthropic-ai/sdk`). Its job is narrow: send one request and stream back the reply. It doesn't keep the conversation (that's `ChatSession`) and knows nothing about the UI. `index.ts` creates one `AnthropicBackend` at startup, gives it to `ChatSession`, and calls `reset()` when the user is signed out.

`ChatSession` only sees the `ChatBackend` interface, so the tests replace the backend with a scripted fake. There is no API key: requests are made as the signed-in user (see the class comment below and `getClient()`).

ReplyRequest, ChatBackend

src/main/claude/AnthropicBackend.ts, lines 7–23

```
export interface ReplyRequest {
  system: string
  messages: Anthropic.Beta.BetaMessageParam[]
  tools: Anthropic.Beta.BetaTool[]
  effort: Effort
  signal: AbortSignal
  /** Called with each piece of reply text as it streams in. */
  onText: (delta: string) => void
}

/** Where chat requests go. Tests swap in a fake. */
export interface ChatBackend {
  /** Streams one reply and resolves with the complete message (all content blocks). */
  reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage>
  /** Forgets the current Claude access, e.g. after signing out. */
  reset(): void
}
```

`ReplyRequest` is everything one request needs. `ChatBackend` is the interface `ChatSession` talks to, which `AnthropicBackend` implements.

Content blocks. In the Claude API a message's `content` is a list of blocks, each with a `type`. A user turn here holds `image` blocks and a `text` block. A reply can hold `thinking` blocks (Claude's reasoning before it answers), `text` blocks (the answer), `tool_use` blocks (requests to run a tool) and, after a refusal fallback, a `fallback` block. The types come from the SDK's `Beta` namespace because the request uses a beta feature (`FALLBACK_BETA`).

- `messages: Anthropic.Beta.BetaMessageParam[]`: The whole conversation so far, one entry per turn. The API keeps nothing between requests, so every request sends all of it again.
- `tools: Anthropic.Beta.BetaTool[]`: The definitions of the tools Claude may call. Always empty for now.
- `signal: AbortSignal`: Lets `ChatSession` cancel the request (Stop, New conversation, quitting).
- `onText: (delta: string) => void`: Called with each new piece of answer text. Thinking is never passed here.
- `reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage>`: Resolves only once the reply is complete, with every content block, because `ChatSession` sends them back next time. Rejects if the request fails or is cancelled.

AnthropicBackendOptions

src/main/claude/AnthropicBackend.ts, lines 25–30

```
export interface AnthropicBackendOptions {
  baseUrl: string
  access: ClaudeAccessConfig
  /** A new, unused JumpCloud ID token (each can only be exchanged once). */
  identityToken(): Promise<string>
}
```

What `index.ts` passes when it creates the backend.

- `baseUrl`: `string`: `API_BASE_URL`, or a local test server in dev runs.
- `access`: `ClaudeAccessConfig`: The `claudeAccess` section of `tenant.json`: the Claude Console organization, federation rule and service account IDs, and an optional workspace ID. None of these is secret; they say which federation rule should check the user's ID token.
- `identityToken()`: `Promise<string>`: In the app this is `() => auth.freshIdToken()`. In `index.ts`, `auth` is declared a few lines after the backend; that works because the function only reads `auth` when the SDK calls it, long after startup has finished.

AnthropicBackend

The class, with the comment that describes it:

`src/main/claude/AnthropicBackend.ts`, lines 32–38

```
/**
 * Calls Claude as the signed-in user, with no API key. Anthropic's Workload Identity Federation
 * exchanges the user's JumpCloud ID token for a short-lived access token for the company's
 * service account. The SDK does the exchange, keeps the token, gets a new one shortly before it
 * expires, and retries once with a new one if Anthropic rejects it.
 */
export class AnthropicBackend implements ChatBackend {
  // ...
}
```

In plain terms: a JumpCloud ID token is a short-lived, signed statement of who the user is. Anthropic accepts it in place of an API key because IT has set up a federation rule and a service account in the Claude Console (see `docs/JUMPCLOUD_SETUP.md`). `getClient()` below shows how the SDK is told to do this, and the steps it follows.

- `implements ChatBackend`: TypeScript checks that the class has everything the interface lists (`reply()` and `reset()`), so it can be passed wherever a `ChatBackend` is expected.

The fields:

`src/main/claude/AnthropicBackend.ts`, lines 39–41

```
private client: Anthropic | null = null

constructor(private readonly options: AnthropicBackendOptions) {}
```

- `private client: Anthropic | null = null`: The SDK client, created on first use by `getClient()`. The client also holds the current Claude token, so dropping it drops the token.
- `private readonly options`: TypeScript shorthand. `private readonly` on a constructor parameter declares a field and stores the argument in it, so the constructor body can stay empty.

AnthropicBackend.reply()

src/main/claude/AnthropicBackend.ts, lines 43–61

```
reply(request: ReplyRequest): Promise<Anthropic.Beta.BetaMessage> {
  const stream = this.getClient().beta.messages.stream(
    {
      model: CLAUDE_MODEL,
      max_tokens: MAX_TOKENS,
      betas: [FALLBACK_BETA],
      fallbacks: 'default',
      // Thinking is always on for this model; effort sets how much it does.
      output_config: { effort: request.effort },
      system: request.system,
      cache_control: { type: 'ephemeral' }, // cache the conversation so far
      messages: request.messages,
      ...(request.tools.length > 0 ? { tools: request.tools } : {}),
    },
    { signal: request.signal },
  )
  stream.on('text', (delta) => request.onText(delta))
  return stream.finalMessage()
}
```

Sends one request and streams back the reply. `ChatSession.runTurn()` calls it once per round.

- `this.getClient().beta.messages.stream()`: `beta.messages` is the Messages API with beta features allowed. `stream()` starts a streaming request and returns a stream object straight away; the reply arrives as events on it.
- `betas: [FALLBACK_BETA]`: Sent as the `anthropic-beta` header; together with `fallbacks: 'default'` it turns on the refusal fallback (see `FALLBACK_BETA`).
- `output_config: { effort: request.effort }`: **Effort** tells the model how much work to put in: how long it thinks and how thorough its answer is. The user picks it in the settings menu: Fast (`low`, the default in `settings.ts`), Balanced (`medium`) or Thorough (`high`). There is no `thinking` field: on this model thinking is always on, and Claude decides how much it needs. Changing effort mid-conversation is allowed, but the next request can't reuse the cached conversation.
- `cache_control: { type: 'ephemeral' }`: Prompt caching (see `buildSystemPrompt()`) for the whole request. At the top level of a request, it makes the API put a cache point on the last block, so everything up to there is kept. The next request repeats that content and adds one new turn, so everything before the new turn is read from the cache. An `'ephemeral'` entry lasts 5 minutes, and each use restarts the clock. Very short requests aren't cached at all (no error, just no saving).
- `...(request.tools.length > 0 ? { tools: request.tools } : {})`: Adds the `tools` field only when there are tools. None are registered yet, so today's requests have no `tools` field.
- `{ signal: request.signal }`: The second argument holds options for the HTTP request rather than the API. Aborting the signal cancels the request, and the stream then fails with the SDK's `APIUserAbortError`.
- `stream.on('text', (delta) => request.onText(delta))`: The `'text'` event fires for each new piece of answer text. Thinking arrives as separate `'thinking'` events, which nothing listens to.
- `return stream.finalMessage()`: A promise for the complete reply once the stream ends: all its content blocks, the `stop_reason` and the token usage. It rejects if the request failed, including when getting the Claude token failed.

Thinking blocks. Because thinking is on, a reply usually starts with one or more `thinking` blocks before its `text`. On Claude Opus 5.5 their text is empty by default (the reasoning isn't shown), but each carries a `signature` that ties it to this model and to the exact conversation it was produced in. They must be sent back unchanged in later

requests, which is why `ChatSession` keeps whole replies and never edits its history. (After a refusal fallback, the declining model's thinking blocks are the exception; see `replayContent()`.)

`AnthropicBackend.reset()`

`src/main/claude/AnthropicBackend.ts`, lines 63–65

```
reset(): void {
  this.client = null
}
```

Forgets the SDK client, and with it the Claude token it holds. `index.ts` calls it from `AuthManager`'s `onSignedOut`, so it runs on Log out and when JumpCloud ends the sign-in. Nothing keeps using access that belonged to a sign-in that has ended; the next `reply()` builds a new client, which asks for a new ID token.

`AnthropicBackend.getClient()`

`src/main/claude/AnthropicBackend.ts`, lines 67–85

```
private getClient(): Anthropic {
  const { baseUrl, access, identityToken } = this.options
  this.client ??= new Anthropic({
    // null stops an API key or token in the environment from being used instead.
    apiKey: null,
    authToken: null,
    baseUrl,
    credentials: oidcFederationProvider({
      identityTokenProvider: identityToken,
      organizationId: access.organizationId,
      federationRuleId: access.federationRuleId,
      serviceAccountId: access.serviceAccountId,
      workspaceId: access.workspaceId || undefined,
      baseUrl,
      fetch,
    }),
  })
  return this.client
}
```

Creates the SDK client the first time it's needed and returns the same one after that.

- `this.client ??= new Anthropic({: ??=` assigns only when `this.client` is `null` (or `undefined`). So the client is built on the first message, not at startup, when nobody may be signed in yet.
- `apiKey: null`, : Left out, the SDK would read `ANTHROPIC_API_KEY` (and `authToken` would read `ANTHROPIC_AUTH_TOKEN`) from the environment, and either would win over `credentials`. `null` means "none", so the app always calls Claude as the signed-in user, even on a PC where a developer has a key set.
- `credentials: oidcFederationProvider({`: The SDK's Workload Identity Federation helper. It returns a function that, each time it is called, gets an ID token from `identityTokenProvider` and swaps it for a Claude access token. The SDK client wraps that function in its token cache.
- `identityTokenProvider: identityToken, : AuthManager.freshIdToken()` in the app. A swap uses up the ID token and doesn't return a refresh token, so every new Claude token needs a new ID token.
- `workspaceId: access.workspaceId || undefined`, : An empty `workspaceId` in `tenant.json` becomes `undefined`, so it isn't sent at all, and Anthropic uses the federation rule's only workspace. It is needed only when the rule covers several workspaces.

- `fetch`: The provider makes its own HTTP call to the token endpoint, so it needs the address (`baseURL` again) and a `fetch` function; this is the one built into Node.js.

How a JumpCloud sign-in becomes a Claude token (all inside the SDK, during the request):

1. Before each request, the client asks its token cache for a token.
2. If it has none (the first message, or after `reset()`), the cache calls the provider and the request waits.
3. The provider calls `AuthManager.freshIdToken()`. That hands out the unused ID token kept from signing in or from the startup renewal, or gets a new one from JumpCloud with the saved refresh token. If nobody is signed in, or JumpCloud ends the sign-in, it throws `SignInRequiredError`; if JumpCloud can't be reached, `SignInUnavailableError`.
4. The provider posts the ID token to `<baseURL>/v1/oauth/token` with the organization, federation rule, service account and (if set) workspace IDs. It refuses to send it over plain `http` unless the address is this PC (`localhost`, `127.0.0.1`, `::1`). Anthropic checks the ID token against the federation rule set up in the Claude Console and returns a short-lived access token and how long it lasts. A refusal or failure throws the SDK's `WorkloadIdentityError`.
5. The client sends the access token as `Authorization: Bearer ...` with the chat request.
6. Later requests reuse it. With 30 seconds to 2 minutes left, the cache hands out the old token and fetches a new one in the background; with under 30 seconds left, the request waits for a new one. Requests that need a token at the same moment share one swap.
7. If the API still answers 401 (token not accepted), the client throws the token away, gets a new one, and retries the request once.

Any error from steps 3 or 4 fails the chat request before it is sent and comes out of `finalMessage()`, where `classifyError()` sorts it.

`src/main/claude/errors.ts`: what went wrong

Turns any error from a chat request into one of a fixed set of kinds, and each kind into a message for the chat. Only `ChatSession.runTurn()` uses it. The full error still goes to the log through `onError` (printed in the terminal in dev runs, see `index.ts`); the user sees only the plain English.

By the time an error gets here, the SDK has already retried temporary failures that happened before the reply started arriving (no connection, and statuses 408, 409, 429 and 5xx), twice by default.

ErrorKind

src/main/claude/errors.ts, lines 6–20

```
export type ErrorKind =  
  | 'aborted'  
  | 'signed-out'  
  | 'sign-in-unreachable'  
  | 'not-allowed'  
  | 'auth'  
  | 'permission'  
  | 'billing'  
  | 'not-found'  
  | 'rate-limit'  
  | 'overloaded'  
  | 'network'  
  | 'bad-request'  
  | 'server'  
  | 'unknown'
```

The kinds of failure the app tells apart. It's a union of string literals: a value of this type must be one of these strings, and TypeScript checks every use.

- `'aborted'`: Stop was pressed, or the conversation was cleared.
- `'signed-out'`, `'sign-in-unreachable'`, `'not-allowed'`: Getting a Claude token failed: nobody is signed in (or the sign-in just expired), JumpCloud couldn't be reached, or Anthropic refused the swap.
- `'auth'`, `'permission'`, `'billing'`, `'not-found'`: The Claude API turned the request down: token not accepted (401), no access (403), out of credit (402), model not available (404).
- `'rate-limit'`, `'overloaded'`, `'network'`, `'server'`: Temporary trouble: too many requests (429), Anthropic busy (529), no connection or a timeout, or another server error (5xx).
- `'bad-request'`: The API rejected the request itself (400).

classifyError()

src/main/claude/errors.ts, lines 22–46

```
/** Sorts an error from a chat request into what it means for the user. Most specific first. */
export function classifyError(error: unknown): ErrorKind {
  if (error instanceof Anthropic.APIUserAbortError) return 'aborted'
  // The SDK wraps errors it doesn't know (like the sign-in ones) and keeps them as the cause.
  const cause = error instanceof Anthropic.AnthropicError ? error.cause : undefined
  if (error instanceof SignInRequiredError || cause instanceof SignInRequiredError) {
    return 'signed-out'
  }
  if (error instanceof SignInUnavailableError || cause instanceof SignInUnavailableError) {
    return 'sign-in-unreachable'
  }
  if (error instanceof WorkloadIdentityError) return exchangeErrorKind(error.statusCode)
  if (error instanceof Anthropic.APIConnectionError) return 'network' // includes timeouts
  if (error instanceof Anthropic.AuthenticationError) return 'auth'
  if (error instanceof Anthropic.PermissionDeniedError) return 'permission'
  if (error instanceof Anthropic.NotFoundError) return 'not-found'
  if (error instanceof Anthropic.RateLimitError) return 'rate-limit'
  if (error instanceof Anthropic.BadRequestError) return 'bad-request'
  if (error instanceof Anthropic.APIError) {
    if (error.status === 402) return 'billing'
    if (error.status === 529) return 'overloaded'
    if (error.status !== undefined && error.status >= 500) return 'server'
  }
  return 'unknown'
}
```

Sorts an error into an `ErrorKind`. Order matters: SDK error classes inherit from each other (a cancelled request and a connection failure are both `APIError`s too), so the specific checks come first and the general `APIError` check comes last.

- `error instanceof Anthropic.APIUserAbortError`: The request was cancelled through its signal by Stop or New conversation.
- `const cause = error instanceof Anthropic.AnthropicError ? error.cause : undefined`: An error that isn't one of the SDK's own, such as the sign-in errors from `AuthManager.freshIdToken()`, comes out of the stream wrapped in a plain `AnthropicError`, with the original kept as `cause`. So the sign-in checks look at both the error and its cause.
- `error instanceof SignInRequiredError || cause instanceof SignInRequiredError`: Nobody is signed in, or JumpCloud just ended the sign-in. `AuthManager` has already switched the chat box to Sign in with JumpCloud.
- `error instanceof WorkloadIdentityError`: The swap at `/v1/oauth/token` failed. This class is an `AnthropicError` itself, so it arrives unwrapped; `exchangeErrorKind()` sorts it by status code.
- `error instanceof Anthropic.APIConnectionError`: No answer at all: offline, the address couldn't be found, or a timeout (the SDK's timeout error is a subclass of this one).
- `error instanceof Anthropic.AuthenticationError`: A 401 even after the SDK's retry with a new token.
- `if (error instanceof Anthropic.APIError) {`: Any other API error, sorted by HTTP status. 402 and 529 have no class of their own in the SDK; 529 is Anthropic's "overloaded" status.

exchangeErrorKind()

src/main/claude/errors.ts, lines 48–54

```
/** Anthropic refused (or couldn't do) the swap of the JumpCloud sign-in for Claude access. */
function exchangeErrorKind(status: number | null): ErrorKind {
  if (status === null) return 'network'
  if (status === 429) return 'rate-limit'
  if (status >= 500) return 'server'
  return 'not-allowed' // 400/401/403: the federation rule doesn't accept this user
}
```

Sorts a failed token swap by the HTTP status Anthropic returned.

- `if (status === null) return 'network'`: No status means the swap never got an answer (it's also how the SDK reports a problem it finds before sending, such as an address that isn't `https`).
- `return 'not-allowed'`: A 400, 401 or 403: Anthropic got the ID token and said no, usually because the federation rule doesn't accept this user. Only IT can fix that, so the message says to contact them.

isRetryable()

src/main/claude/errors.ts, lines 56–67

```
/** Problems worth simply trying again. */
export function isRetryable(kind: ErrorKind): boolean {
  return (
    kind === 'signed-out' || // after signing back in
    kind === 'sign-in-unreachable' ||
    kind === 'rate-limit' ||
    kind === 'overloaded' ||
    kind === 'network' ||
    kind === 'server' ||
    kind === 'unknown'
  )
}
```

Whether the chat offers Retry for a failed reply: yes for problems that may go away on their own, and for `'unknown'` to be safe. No for anything that needs IT or a different message (not-allowed, auth, permission, billing, not-found, bad-request). `ChatSession` also withholds Retry when part of the reply was kept.

- `kind === 'signed-out' || // after signing back in`: The Retry button stays on the failed reply. `ChatSession.retry()` does nothing until the user has signed in again, then resends.

MESSAGES, errorMessage()

src/main/claude/errors.ts, lines 69–88

```
const MESSAGES: Record<ErrorKind, string> = {
  aborted: 'Stopped.',
  'signed-out': 'Sign in with JumpCloud to keep chatting.',
  'sign-in-unreachable': "Couldn't reach JumpCloud to confirm your sign-in. Check your connection.",
  'not-allowed': "Your JumpCloud account isn't set up to use Claude yet. Contact IT.",
  auth: "Claude didn't accept your sign-in. Try again, or log out and sign in again.",
  permission: "Your account doesn't have access to Claude. Contact IT.",
  billing: "Your organisation's Claude account has run out of credit. Contact IT.",
  'not-found': `Your organisation can't use ${CLAUDE_MODEL_NAME}. Contact IT.`,
  'rate-limit': 'Claude is getting too many requests right now. Try again in a moment.',
  overloaded: 'Claude is busy right now. Try again in a moment.',
  network: "Couldn't reach Claude. Check your internet connection.",
  'bad-request': "Claude couldn't process that message.",
  server: 'Claude had a problem answering. Try again.',
  unknown: 'Something went wrong talking to Claude.',
}

export function errorMessage(kind: ErrorKind): string {
  return MESSAGES[kind]
}
```

The text shown under a failed reply, one per kind.

- `Record<ErrorKind, string>`: An object with exactly one string for every kind. If a kind is added to `ErrorKind` without a message, TypeScript refuses to compile.
- `aborted: 'Stopped.'`: Never actually shown: `ChatSession.runTurn()` handles Stop itself, with a notice that depends on whether any text arrived. It's here because the `Record` needs every kind.
- `'not-found'`: The model is the only thing a chat request names that could be missing, so the message names it (`CLAUDE_MODEL_NAME`).

src/main/claude/ChatSession.ts: one conversation

The conversation, and everything that happens during a reply: sending, streaming text to the page, the tool loop, Stop, errors and Retry. It knows nothing about Electron. `index.ts` creates it with the backend and callbacks (`onMessage` broadcasts `IPC assist:chat-message`, `onReset` broadcasts `assist:chat-reset`); the IPC handlers in `ipc.ts` call `send()`, `stop()`, `retry()` and `newConversation()`; and `getState()` in `index.ts` uses `list()` when a page loads. One `ChatSession` lasts for the whole run: New conversation empties it rather than replacing it.

It keeps two lists, because the API and the page need different things. `history` is exactly what is sent to the API: base64 image data and Claude's thinking blocks included. `messages` is what the chat shows: ids, statuses, notices and the attached screenshots' file names.

MessageParam, ContentBlock, OutgoingImage

src/main/claude/ChatSession.ts, lines 8–15

```
type MessageParam = Anthropic.Beta.BetaMessageParam

type ContentBlock = Anthropic.Beta.BetaContentBlock

/** A screenshot ready to send: base64 image data. */
export interface OutgoingImage {
  mediaType: 'image/jpeg' | 'image/png'
  data: string
}
```

- `type MessageParam = Anthropic.Beta.BetaMessageParam`: Short local names for two SDK types: one turn of the conversation as it is sent, and one content block of a reply (used by `replayContent()` and `runTools()`). The file imports only types from the SDK (`import type`); it never calls the SDK itself.
- `mediaType: 'image/jpeg' | 'image/png'`: The image format, which the API needs to know. `ScreenshotService.forClaude()` always produces JPEG, scaled so its longest side is at most 1,568 pixels.
- `data: string`: The image as base64 text, since the request body is JSON.

ChatTool

src/main/claude/ChatSession.ts, lines 17–21

```
/** A tool Claude may call. None are registered yet; the loop is ready for them. */
export interface ChatTool {
  definition: Anthropic.Beta.BetaTool
  run(input: unknown): Promise<string>
}
```

A tool Claude may call. **Tool use** lets Claude ask the app to do something, such as read a file, and then use the result. Each request lists the tools' definitions (name, description and input schema). If Claude wants one, its reply contains a `tool_use` block with the tool name and input and ends with `stop_reason: 'tool_use'`. The app runs the tool and sends the result back as a `tool_result` block in a new user turn, and Claude carries on from there. One answer can take several of these rounds. No tools are registered yet (`index.ts` passes none), so today every reply is a single round; `runTurn()` and `runTools()` are ready for them.

- `definition: Anthropic.Beta.BetaTool`: What's sent to Claude: name, description and input schema.
- `run(input: unknown): Promise<string>`: The tool's code. `input` is whatever Claude sent, so it is `unknown` and each tool must check it. The returned string becomes the `tool_result`.

ChatSessionDeps

src/main/claude/ChatSession.ts, lines 23–38

```
export interface ChatSessionDeps {
  backend: ChatBackend
  /** Whether someone is signed in, so a request can be made. */
  canChat(): boolean
  getEffort(): Effort
  /** Fixed for the session, so the cached prompt prefix stays valid. */
  system: string
  tools?: ChatTool[]
  /** A message was added or changed. */
  onMessage(message: ChatMessage): void
  onReset(): void
  /** A reply failed; gets the full error, for the log (the chat shows a plain-English message). */
  onError?(error: unknown): void
  /** Streamed text is sent to the UI at most this often. */
  emitIntervalMs?: number
}
```

What `index.ts` gives the session. Everything comes in through here rather than being imported, so the tests can drive the class with fakes.

- `canChat(): boolean`: `AuthManager.canChat` in the app: whether a sign-in exists.
- `getEffort(): Effort`: A function rather than a value, so each request uses the setting as it is at that moment.
- `system: string`: The prompt from `buildSystemPrompt()`. It never changes; see prompt caching there.
- `onMessage(message: ChatMessage): void`: Called with a copy whenever a message is added or changes, including every streaming update.
- `onError?(error: unknown): void`: `index.ts` logs it. The `?` makes it optional.
- `emitIntervalMs?: number`: How often streamed text may reach the page, 50 ms when not set. The tests set 0 to see every update.

SendStatus

src/main/claude/ChatSession.ts, line 40

```
export type SendStatus = 'sent' | 'busy' | 'empty' | 'signed-out'
```

What `send()` returns. The `assist:chat-send` handler in `ipc.ts` passes anything other than `'sent'` back to the page as the reason (`SendResult`), and only clears the draft on `'sent'`.

ChatSession

The class, with the comment that describes it:

```

/**
 * One conversation with Claude.
 *
 * It keeps two lists: `history`, exactly what is sent to the API, and `messages`, what the chat
 * shows. History is append-only: earlier turns are never edited or removed, because Claude's
 * thinking is tied to the exact conversation it was produced in. Every reply is appended with all
 * of its content blocks (thinking included) unchanged, except what the API says to drop after a
 * server-side fallback (`replayContent()`); a reply cut short by Stop or an error is appended as
 * plain text.
 */
export class ChatSession {
  // ...
}

```

Why thinking forces an append-only history: each thinking block's `signature` records the conversation it was produced in, meaning the system prompt, the tools and every earlier message. When the history comes back, the API checks that this conversation is unchanged, and if an earlier turn has been edited or removed it can reject the request. Append-only also keeps prompt caching working, because each request starts with exactly what the previous one sent.

Append-only means that once a turn is in the history, it is never changed or removed (New conversation starts again with an empty history instead). What goes in is decided once, as the turn is added. A complete reply normally goes in with all its content blocks unchanged; the one exception, as the comment says, is after a server-side fallback, when `replayContent()` first drops the blocks the API says must not be sent back. A reply cut short by Stop or an error goes in as plain text: just the text that had arrived.

The fields:

```

private history: MessageParam[] = []

private messages: ChatMessage[] = []

private controller: AbortController | null = null

private turn: Promise<void> = Promise.resolve()

/** Bumped by New conversation, so a reply still finishing can't touch the new one. */
private generation = 0

private readonly emitTimers = new Map<string, ReturnType<typeof setTimeout>>()

constructor(private readonly deps: ChatSessionDeps) {}

```

- `private history: MessageParam[] = []`: Exactly what is sent to the API.
- `private messages: ChatMessage[] = []`: What the chat shows. A reply's object is changed in place while it streams; the page always gets copies.
- `private controller: AbortController | null = null`: The running reply's cancel switch. It is set exactly while a reply is in progress, which is what `busy` checks.
- `private turn: Promise<void> = Promise.resolve()`: The running reply, for `idle()`. It starts as an already-finished promise.
- `private generation = 0`: A counter bumped by `newConversation()`. A reply remembers the value it started with, and stops touching anything once it no longer matches.

- `private readonly emitTimers = new Map<string, ReturnType<typeof setTimeout>>()`: At most one waiting page update per message id, for `scheduleEmit()`. `ReturnType<typeof setTimeout>` means "whatever `setTimeout` returns", which differs between Node.js and browsers.

ChatSession.busy and ChatSession.list()

src/main/claude/ChatSession.ts, lines 63–69

```
get busy(): boolean {
  return this.controller !== null
}

list(): ChatMessage[] {
  return this.messages.map((m) => ({ ...m }))
}
```

- `get busy(): boolean`: A getter, read as `chat.busy` without brackets. True while a reply is in progress. The `assist:chat-send` handler checks it before collecting screenshots.
- `this.messages.map((m) => ({ ...m }))`: Copies of the messages, for `getState()` when a page loads. Copies, so nothing outside can change the session's own objects.

ChatSession.send()

src/main/claude/ChatSession.ts, lines 71–87

```
/** Starts a reply and returns straight away; the reply streams in through `onMessage`. */
send(text: string, images: OutgoingImage[], attachments: Attachment[]): SendStatus {
  if (this.busy) return 'busy'
  if (!text.trim() && images.length === 0) return 'empty'
  if (!this.deps.canChat()) return 'signed-out'

  const content: Anthropic.Beta.BetaContentBlockParam[] = images.map((image) => ({
    type: 'image',
    source: { type: 'base64', media_type: image.mediaType, data: image.data },
  }))
  if (text.trim()) content.push({ type: 'text', text })
  this.history.push({ role: 'user', content })
  this.add({ role: 'user', text, attachments, status: 'done' })

  this.startTurn(this.add({ role: 'assistant', text: '', attachments: [], status: 'streaming' }))
  return 'sent'
}
```

Adds the user's message and starts a reply, then returns at once. The reply arrives through `onMessage`.

- `if (this.busy) return 'busy'`: One reply at a time.
- `if (!text.trim() && images.length === 0) return 'empty'`: A message needs some text or at least one screenshot.
- `if (!this.deps.canChat()) return 'signed-out'`: The panel shows the sign-in instead of the chat when nobody is signed in, so this is a safety net.
- `images.map((image) => ({`: Each screenshot becomes an `image` content block holding its base64 data. Images go before the text, the order Anthropic recommends.
- `if (text.trim()) content.push({ type: 'text', text })`: A blank text box with screenshots sends only the images. Text is sent as typed, not trimmed.

- `this.history.push({ role: 'user', content })`: If the previous reply failed before any text, the history already ends with a user turn. That's fine: the API joins consecutive user turns into one.
- `this.add({ role: 'user', text, attachments, status: 'done' })`: The chat's copy keeps the screenshot file details (`attachments`) for the chips under the message, not the image data.
- `this.startTurn(this.add({ role: 'assistant', text: '', attachments: [], status: 'streaming' })))`: An empty reply appears at once (the page shows "Thinking..." until text arrives), and `runTurn()` fills in that same object.

ChatSession.retry()

src/main/claude/ChatSession.ts, lines 89–96

```
/** Asks again after a failed reply (only if nothing from it was kept). */
retry(): void {
  const last = this.messages.at(-1)
  if (this.busy || !last?.retryable || !this.deps.canChat()) return
  Object.assign(last, { text: '', status: 'streaming', notice: undefined, retryable: undefined })
  this.emit(last)
  this.startTurn(last)
}
```

Asks again after a failed reply, from the Retry button (IPC `assist:chat-retry`).

- `const last = this.messages.at(-1)`: `at(-1)` is the last item. Only the latest reply can be retried; once the user sends something new, the old failure is no longer last.
- `!last?.retryable`: `runTurn()` sets `retryable` only when nothing from the failed request went into the history, so the history still ends with a user turn. `?.` copes with an empty chat.
- `!this.deps.canChat()`: After an expired sign-in, Retry does nothing until the user has signed back in.
- `Object.assign(last, {})`: Resets the failed reply in place, keeping its id, so the page replaces it rather than adding another.
- `this.startTurn(last)`: Nothing is added to the history: it still ends with a user turn, so this simply sends the same conversation again.

ChatSession.stop() and ChatSession.newConversation()

src/main/claude/ChatSession.ts, lines 98–111

```
stop(): void {
  this.controller?.abort()
}

newConversation(): void {
  this.generation++
  this.stop()
  this.controller = null
  for (const timer of this.emitTimers.values()) clearTimeout(timer)
  this.emitTimers.clear()
  this.history = []
  this.messages = []
  this.deps.onReset()
}
```

`stop()` is the Stop button (IPC `assist:chat-stop`), and `index.ts` also calls it when the app quits.
`newConversation()` is New conversation (IPC `assist:chat-new`), and `index.ts` also calls it on Log out. When the sign-in merely expires, the conversation is kept so the user can carry on.

- `this.controller?.abort()`: Cancels the request, if a reply is running. The stream then fails with `APIUserAbortError`, and `runTurn()` marks the reply stopped.
- `this.generation++`: Marks any reply still running as belonging to the old conversation; `runTurn()` checks this before touching anything.
- `this.controller = null`: Not busy any more, straight away, so a new message can be sent while the cancelled request is still winding down.
- `for (const timer of this.emitTimers.values()) clearTimeout(timer)`: Cancels page updates still waiting, so nothing from the old reply reaches the page after the chat has been cleared.
- `this.deps.onReset()`: `index.ts` broadcasts `assist:chat-reset`, and the page empties its list.

`ChatSession.idle()` and `ChatSession.startTurn()`

src/main/claude/ChatSession.ts, lines 113–124

```
/** Resolves when the current reply (if any) has finished. */
idle(): Promise<void> {
  return this.turn
}

private startTurn(reply: ChatMessage): void {
  const controller = new AbortController()
  this.controller = controller
  this.turn = this.runTurn(reply, controller).finally(() => {
    if (this.controller === controller) this.controller = null
  })
}
```

- `return this.turn`: For the tests: `await session.idle()` waits for the reply to finish. It never rejects, because `runTurn()` catches its own errors.
- `const controller = new AbortController()`: A new cancel switch for each reply. `send()` and `retry()` have already checked that no reply is running.
- `this.turn = this.runTurn(reply, controller).finally(() => {`: `runTurn()` is async, so calling it starts the reply in the background. The promise is stored, not awaited, which is why `send()` can return straight away.
- `if (this.controller === controller) this.controller = null`: Clears the controller only if it is still this reply's. After New conversation a newer reply may already be running with its own controller, and the old reply finishing must not make the session look idle.

`ChatSession.runTurn()`

src/main/claude/ChatSession.ts, lines 126–181

```
private async runTurn(reply: ChatMessage, controller: AbortController): Promise<void> {
  const generation = this.generation
  const current = () => generation === this.generation
  const tools = this.deps.tools ?? []
  let roundText = ''
  let keptPartial = false

  try {
```

```

for (let round = 0; round < MAX_TOOL_ROUNDS; round++) {
  roundText = ''
  const message = await this.deps.backend.reply({
    system: this.deps.system,
    messages: this.history,
    tools: tools.map((t) => t.definition),
    effort: this.deps.getEffort(),
    signal: controller.signal,
    onText: (delta) => {
      if (!current()) return
      roundText += delta
      reply.text += delta
      this.scheduleEmit(reply)
    },
  })
  if (!current()) return
  const content = replayContent(message.content)
  this.history.push({ role: 'assistant', content })

  if (message.stop_reason === 'tool_use') {
    this.history.push({ role: 'user', content: await runTools(content, tools) })
    continue
  }
  if (message.stop_reason === 'pause_turn') continue // the API asks us to send it back to resume

  return this.finish(reply, 'done', stopNotice(message.stop_reason))
}
this.finish(reply, 'done', 'Claude stopped after too many steps.')
} catch (error) {
  if (!current()) return
  // Keep what streamed before the failure, as plain text (the history stays append-only).
  if (roundText) {
    this.history.push({ role: 'assistant', content: [{ type: 'text', text: roundText }] })
    keptPartial = true
  }
  const kind = classifyError(error)
  if (kind === 'aborted') {
    return this.finish(
      reply,
      'stopped',
      reply.text ? 'Stopped.' : 'Stopped before Claude replied.',
    )
  }
  this.deps.onError?.(error)
  const retryable = !keptPartial && isRetryable(kind)
  this.finish(reply, 'error', errorMessage(kind), retryable)
}
}

```

One whole reply: one or more requests to Claude, the streamed text, and the outcome. It never throws; every failure becomes a status and a notice on the reply.

- `const current = () => generation === this.generation`: True while this reply's conversation is still the current one.
- `if (!current()) return`: (three places) New conversation was pressed while this reply was running, so it leaves everything alone: no text, no history, no error.
- `let roundText = ''`: The text streamed in the current round only, so a round that's cut off can keep its text (below). `reply.text` collects the text of all rounds.
- `for (let round = 0; round < MAX_TOOL_ROUNDS; round++) {`: The tool-use loop: each round is one request. Without tools, the first round is also the last.

- `messages: this.history, :` The whole conversation each time, ending with the user's turn or with tool results.
- `effort: this.deps.getEffort(), :` Read again each round, so a changed setting applies to the next request.
- `this.scheduleEmit(reply):` The new text goes to the page in batches (see `scheduleEmit()`).
- `const content = replayContent(message.content):` The reply as it must be sent back. Normally that is every block exactly as it came back: thinking blocks with their signatures, text, and any tool calls. Only after a server-side fallback partway through the reply does `replayContent()` drop some of the declining model's blocks.
- `this.history.push({ role: 'assistant', content }):` The reply goes into the history once and is never changed afterwards. This is what keeps the next request valid.
- `if (message.stop_reason === 'tool_use') {:` Claude asked for tools. `runTools()` runs them, the results are appended as a user turn, and `continue` starts the next round so Claude can carry on.
- `await runTools(content, tools):` Tools are run from the trimmed `content`, not the raw reply. A tool call the declining model made before a fallback isn't in the history, so it isn't run: a result for it would answer a call the API never sees.
- `if (message.stop_reason === 'pause_turn') continue:` The API can pause a long turn, which happens with tools that run on Anthropic's servers (this app doesn't use any yet). Sending the conversation back as it is lets Claude resume.
- `return this.finish(reply, 'done', stopNotice(message.stop_reason)):` Any other stop reason ends the reply: normally `end_turn` (Claude finished), or `refusal` or `max_tokens`, which get a notice.
- `this.finish(reply, 'done', 'Claude stopped after too many steps.')` Reached only when every round asked for more tools or paused.
- `if (roundText) {:` The round was cut off after some text had arrived. That text stays on screen, so it also goes into the history, as a plain text block, and Claude will see next time what the user saw. Nothing else from the unfinished round is kept.
- `keptPartial = true:` Then Retry isn't offered. The history now ends with Claude's partial reply instead of a user turn, so the same request can't simply be sent again (this model doesn't accept a request whose last turn is Claude's). The user can send a new message instead.
- `if (kind === 'aborted') {:` Stop isn't an error: the reply is marked `stopped`, without Retry, and the notice says whether anything had arrived.
- `this.deps.onError?.(error):` The full error goes to the log. `?.` calls it only if it was given.
- `const retryable = !keptPartial && isRetryable(kind):` Retry is offered only when nothing from this reply went into the history and the problem may be temporary.

ChatSession.add() and ChatSession.finish()

src/main/claude/ChatSession.ts, lines 183–198

```
private add(fields: Omit<ChatMessage, 'id'>): ChatMessage {
  const message: ChatMessage = { id: randomUUID(), ...fields }
  this.messages.push(message)
  this.emit(message)
  return message
}

private finish(
  message: ChatMessage,
  status: ChatMessage['status'],
  notice?: string,
  retryable?: boolean,
): void {
  Object.assign(message, { status, notice, retryable: retryable || undefined })
  this.emit(message)
}
```

- `Omit<ChatMessage, 'id'>`: A `ChatMessage` without its `id`; `add()` makes the `id` itself with `randomUUID()`. The page uses the `id` to replace a message when an update arrives.
- `return message`: The same object that is in `messages`, so `send()` can hand it to `runTurn()` to fill in.
- `retryable: retryable || undefined`: Stores nothing rather than `false`, so a message that can't be retried simply has no `retryable` field.
- `this.emit(message)`: New messages and final states go to the page at once, skipping the batching.

ChatSession.scheduleEmit() and ChatSession.emit()

src/main/claude/ChatSession.ts, lines 200–219

```
/** Streamed text arrives in many small pieces; batch them into fewer UI updates. */
private scheduleEmit(message: ChatMessage): void {
  const interval = this.deps.emitIntervalMs ?? 50
  if (interval <= 0) return this.emit(message)
  if (this.emitTimers.has(message.id)) return
  this.emitTimers.set(
    message.id,
    setTimeout(() => {
      this.emitTimers.delete(message.id)
      this.deps.onMessage({ ...message })
    }, interval),
  )
}

private emit(message: ChatMessage): void {
  const timer = this.emitTimers.get(message.id)
  if (timer) clearTimeout(timer)
  this.emitTimers.delete(message.id)
  this.deps.onMessage({ ...message })
}
```

A reply streams in as many small pieces. Sending each one over IPC, and re-rendering the Markdown each time, would be wasteful, so streaming updates are batched: at most one per message every 50 ms.

- `if (interval <= 0) return this.emit(message)`: With an interval of 0 (the tests) every piece is sent at once.

- `if (this.emitTimers.has(message.id)) return`: An update is already waiting. The message object is changed in place, so when that update fires it carries all the text received by then.
- `this.deps.onMessage({ ...message })`: Sends a copy, a snapshot of the message at that moment.
- `if (timer) clearTimeout(timer): emit()` sends straight away and cancels any waiting update, which would only repeat what this one already contains.

stopNotice()

src/main/claude/ChatSession.ts, lines 222–226

```
function stopNotice(stopReason: string | null): string | undefined {
  if (stopReason === 'refusal') return 'Claude declined to answer this.'
  if (stopReason === 'max_tokens') return 'This reply hit the length limit and was cut short.'
  return undefined
}
```

The notice for a reply that finished but not normally. Any other stop reason (usually `end_turn`) gets no notice.

- `if (stopReason === 'refusal')`: The safety classifiers declined, and either there was no fallback model for that kind of refusal or it declined too (see `FALLBACK_BETA`). Any text that had already streamed stays.
- `if (stopReason === 'max_tokens')`: The reply reached `MAX_TOKENS`.

DECLINED_REASONING, replayContent()

src/main/claude/ChatSession.ts, lines 228–250

```
/** Reasoning that belongs to a model that declined, so it isn't sent back after a fallback. */
const DECLINED_REASONING = new Set(['thinking', 'redacted_thinking', 'connector_text'])

/**
 * A reply's content as it must be sent back on the next turn: normally every block, unchanged.
 * After a server-side fallback (another model took over partway through), the API requires
 * dropping what the declining model produced before the last 'fallback' block: its reasoning,
 * its client-side tool calls, and server tool calls that got no result. The 'fallback' block
 * itself stays exactly where it was; the API checks the blocks around it.
 */
export function replayContent(content: ContentBlock[]): ContentBlock[] {
  const lastFallback = content.findLastIndex((block) => block.type === 'fallback')
  if (lastFallback === -1) return content
  const answered = new Set(
    content.flatMap((block) => ('tool_use_id' in block ? [block.tool_use_id] : [])),
  )
  return content.filter((block, index) => {
    if (index >= lastFallback) return true
    if (DECLINED_REASONING.has(block.type) || block.type === 'tool_use') return false
    if (block.type === 'server_tool_use') return answered.has(block.id)
    return true
  })
}
```

Prepares a reply for the history, following the rules on Anthropic's "Refusals and fallback" page. `runTurn()` passes every reply through it. Almost always it changes nothing. The exception is a **mid-reply fallback**: the safety classifiers declined after Claude Opus 5.5 had already started answering, and the fallback model carried on in the same stream. The reply then holds both models' output, separated by a `fallback` block. Some of what came before that block must not be sent back, because it belongs to a model that declined. A decline before any output puts the `fallback` block first, so nothing comes before it and nothing is dropped.

- `new Set(['thinking', 'redacted_thinking', 'connector_text'])`: The declining model's reasoning: thinking blocks, `redacted_thinking` (thinking that comes back encrypted instead of readable), and `connector_text` (narration some tool-using replies include between tool calls).
- `content.findLastIndex((block) => block.type === 'fallback')`: The position of the last `fallback` block. A reply can have several if more than one model declined; only the last one matters. `-1` means there is none.
- `if (lastFallback === -1) return content`: The normal case: the very same array comes back, untouched.
- `const answered = new Set()`: The ids of every tool call that got a result in this reply. A server tool's result block has a `tool_use_id` naming the call it answers. `'tool_use_id' in block` checks the property exists, and `flatMap` with `[]` or `[id]` collects only the blocks that have one.
- `if (index >= lastFallback) return true`: The `fallback` block itself and everything after it stay. The block must stay exactly where it was, because the API uses its position to check the thinking blocks around it.
- `if (DECLINED_REASONING.has(block.type) || block.type === 'tool_use') return false`: Before the boundary, the declining model's reasoning and its calls to the app's own tools are dropped.
- `if (block.type === 'server_tool_use') return answered.has(block.id)`: A call to a tool that Anthropic runs (such as web search) stays only if its result is in the reply too. The app uses no server tools yet.
- `return true`: Everything else before the boundary stays, notably the declining model's text. The user has already seen it, and it's what the fallback model continued from.
- `export function replayContent`: Exported so `tests/chatSession.test.ts` can check it directly.

runTools()

src/main/claude/ChatSession.ts, lines 252–274

```
/** Runs every tool call in a reply and returns the results to send back. */
async function runTools(
  content: ContentBlock[],
  tools: ChatTool[],
): Promise<Anthropic.Beta.BetaToolResultBlockParam[]> {
  const results: Anthropic.Beta.BetaToolResultBlockParam[] = []
  for (const block of content) {
    if (block.type !== 'tool_use') continue
    const tool = tools.find((t) => t.definition.name === block.name)
    try {
      if (!tool) throw new Error(`Unknown tool: ${block.name}`)
      results.push({
        type: 'tool_result',
        tool_use_id: block.id,
        content: await tool.run(block.input),
      })
    } catch (error) {
      const message = error instanceof Error ? error.message : String(error)
      results.push({ type: 'tool_result', tool_use_id: block.id, content: message, is_error: true })
    }
  }
  return results
}
```

Runs every tool call in one reply and builds the results to send back, as described under `ChatTool`. It gets the reply's content after `replayContent()`, so only calls that are in the history are run. The tools run one after another, in the order Claude asked for them.

- `if (block.type !== 'tool_use') continue`: The reply also holds thinking and text blocks; only tool calls are run.

- `tools.find((t) => t.definition.name === block.name)`: Finds the tool by the name Claude used.
- `if (!tool) throw new Error`: An unknown name is reported to Claude as a failed result, through the `catch` below, instead of breaking the reply.
- `tool_use_id: block.id`: Each result names the call it answers. Claude can ask for several tools in one reply, and all the results go back together in one user turn.
- `is_error: true`: A tool that throws is reported to Claude as a failed result with the error message, so Claude can try something else or explain the problem.

The bubble: windows, layout, bounce and the state machine

The bubble is the always-on-top logo in a corner of the screen; clicking it opens the panel (the chat card and the action icons). Each lives in its own transparent, frameless *overlay window*, created in `windows.ts`.

`BubbleController` decides what the bubble is doing and moves, shows and hides the two windows to match, using the pure maths in `layout.ts` (where things go) and `bounce.ts` (how they move).

The controller is always in exactly one *mode* (the `Mode` type in `src/shared/types.ts`). Every change is broadcast to both pages, which draw themselves to match:

- `collapsed`: only the bubble shows, resting in its corner. The app starts here.
- `expanded`: the panel is open beside the bubble.
- `dragging`: the bubble follows the mouse pointer.
- `bouncing`: the bubble flies around its display (the Bounce action).
- `returning`: the bubble glides back into its corner after a drag or a bounce.
- `capturing`: both windows are hidden while a screenshot is taken.

What moves the controller from one mode to another:

- `collapsed` → `expanded`: a click on the bubble (`clickBubble()`), or the tray icon, launching the app a second time, or finishing sign-in in the browser (all `expand()`).
- `expanded` → `collapsed`: a click on the bubble, Esc in the panel (`collapse()`), or clicking somewhere else so the panel loses focus (`panelBlurred()`).
- `collapsed` or `expanded` → `dragging` → `returning`: pressing and moving the bubble (`startDrag()`), then letting go (`endDrag()`).
- `collapsed` or `expanded` → `bouncing` → `returning`: the Bounce action (`startBounce()`), then a click on the bubble or anything that calls `expand()`.
- `returning` → `collapsed`: the glide finishes (inside `glideHome()`). If something asked to open the panel during the glide, it carries straight on to `expanded`.
- `expanded` → `capturing` → `expanded`: the screenshot action (`whileHidden()`).

`src/main/bubble/BubbleController.ts`: the state machine

This file holds the `BubbleController` class and the small interfaces it is built from. The controller owns three pieces of state: the mode, the bubble's position, and its *anchor* (which display, and which corner of it, the bubble rests in). Each public method is a request such as "the bubble was clicked" or "a drag started", and the controller decides what that means in the current mode.

Who calls it:

- `src/main/index.ts` creates the one controller in `start()` and wires Electron events to it: the panel window's `blur` event to `panelBlurred()`, screen changes to `displayChanged()`, the tray icon and a second launch to `expand()`, and quitting to `dispose()`.

- `src/main/ipc.ts` forwards the pages' requests: the bubble's click and drag (`clickBubble()`, `startDrag()`, `endDrag()`) and the panel's Esc (`collapse()`).
- `src/main/actions.ts` calls `startBounce()` for the Bounce icon and `whileHidden()` for the screenshot icon.

The controller never imports Electron. It drives windows through the `Surface` interface and asks about screens through the `Displays` interface. `index.ts` passes real ones: the adapters from `windows.ts`, and `electronDisplays`, a thin wrapper around Electron's `screen` module. `tests/bubbleController.test.ts` passes a `FakeSurface` that just records its bounds and whether it's visible, a pretend pair of monitors, and Vitest's fake timers. That way a whole drag or bounce can be checked in milliseconds without opening a window.

Surface, PanelSurface

`src/main/bubble/BubbleController.ts`, lines 12–21

```
/** The parts of a window the controller drives. Electron windows are adapted in windows.ts. */
export interface Surface {
  setBounds(bounds: Rect): void
  show(): void
  hide(): void
}

export interface PanelSurface extends Surface {
  focus(): void
}
```

What the controller needs from a window, and nothing more.

- `setBounds(bounds: Rect)`: moves and sizes the window in one call. `Rect` (from `src/shared/geometry.ts`) is `x`, `y`, `width` and `height`, in DIPs (see `DisplayArea` below).
- `show(): void`: what "show" does differs per window. In `windows.ts`, the bubble appears without taking focus, and the panel resets its click-through first.
- `interface PanelSurface extends Surface`: a `PanelSurface` has everything a `Surface` has, plus `focus()`, so you can type as soon as the panel opens.

DisplayArea, Displays

`src/main/bubble/BubbleController.ts`, lines 23–37

```
/** One display: its id and work area (the screen minus the taskbar). */
export interface DisplayArea {
  id: number
  workArea: Rect
}

/** What the controller needs to know about the screens. Electron's `screen` is adapted in index.ts. */
export interface Displays {
  primary(): DisplayArea
  byId(id: number): DisplayArea | undefined
  /** The display containing `point`, or the closest one. */
  nearest(point: Point): DisplayArea
  /** Where the mouse pointer is. */
  cursor(): Point
}
```

The controller's view of the monitors. Each display has an id (a number Electron assigns) and a *work area*: the part of the screen that windows may use, which is the screen minus the taskbar. The bubble rests inside the work area, so it never sits on top of the taskbar.

All coordinates are DIPs (device-independent pixels). Windows scales the desktop up on high-DPI screens: at 150 % scaling, a 1920×1080 monitor is 1280×720 DIPs. Electron's window and screen functions all use DIPs, so the bubble is the same visual size on every monitor and none of this code needs to know the scale factor. With several monitors, all displays share one coordinate space: the main display's top-left is (0, 0), and a monitor to its left or above it has negative coordinates. That is why a work area has its own `x` and `y`, and why the layout code always measures from them.

- `primary()`: the main display. It's the fallback when the bubble's display has gone.
- `byId(id: number): DisplayArea | undefined: undefined` when no connected display has that id, for example after a monitor was unplugged.
- `nearest(point: Point):` the display containing a point, or the closest one if the point is in a gap between monitors. Used to decide which display the bubble was dropped on.
- `cursor()`: where the mouse pointer is, in the same DIP coordinates. Dragging reads it every frame.

In `index.ts`, `electronDisplays` implements these four with Electron's `screen` module (`getPrimaryDisplay()`, `getAllDisplays()`, `getDisplayNearestPoint()` and `getCursorScreenPoint()`).

BubbleAnchor

src/main/bubble/BubbleController.ts, lines 39–43

```
/** Where the bubble rests: a corner of a particular display. */
export interface BubbleAnchor {
  displayId: number
  corner: Corner
}
```

Where the bubble rests between moves. `Corner` (from `geometry.ts`) is one of `'top-left'`, `'top-right'`, `'bottom-left'` or `'bottom-right'`. Storing a display and a corner, rather than a pixel position, means the bubble still lands in the right place if that display's resolution or taskbar has changed since. Whenever the anchor changes, `index.ts` saves it to `preferences.json` (`SettingsService.setBubbleAnchor()`), and it passes the saved one back in at the next start.

BubbleControllerDeps

src/main/bubble/BubbleController.ts, lines 45–57

```
export interface BubbleControllerDeps {
  bubble: Surface
  panel: PanelSurface
  displays: Displays
  actionCount: number
  /** Where the bubble was last time, if saved. */
  initialAnchor?: BubbleAnchor | null
  onModeChange(mode: Mode): void
  /** The bubble settled in a new corner or on a new display. */
  onAnchorChange(anchor: BubbleAnchor): void
  now?: () => number
  random?: () => number
}
```

Everything the constructor needs, passed as one object (its *dependencies*).

- `actionCount: number`: how many action icons the tenant has. The panel window's size depends on it (`panelWindowSize()` in `geometry.ts`), and so does its position next to the bubble.
- `initialAnchor?: BubbleAnchor | null`: the `?` means the field may be left out; `null` means "nothing saved yet". The constructor treats both the same.
- `onModeChange(mode: Mode): void`: `index.ts` broadcasts the new mode to both pages (`IPC.modeChanged`). That's how the panel knows to animate open or closed, and the bubble knows to show its ring.
- `onAnchorChange(anchor: BubbleAnchor): void`: `index.ts` broadcasts the new corner (`IPC.cornerChanged`) so the panel can lay itself out to open toward the middle of the screen, and saves the anchor.
- `now?: () => number`: a clock, and `random?: () => number` a random-number source. The app leaves them out and gets the real ones; tests pass their own so timings and bounce angles are predictable.

Constants

`src/main/bubble/BubbleController.ts`, lines 59–69

```
const FRAME_MS = 16

const DEFAULT_CORNER: Corner = 'bottom-right'

/** Matches the panel's fade-out in the renderer, so the window hides once it's invisible. */
export const PANEL_FADE_MS = 150

/**
 * Clicking the bubble while the panel is open can blur the panel just before the click lands.
 * A click this soon after a blur-collapse belongs to the same gesture and must not reopen it.
 */
export const BLUR_CLICK_GRACE_MS = 300

/** Lets Windows repaint the area under our windows before a screenshot. */
export const HIDE_SETTLE_MS = 150
```

- `FRAME_MS = 16`: how often the animation timer fires. $1000 / 16$ is about 62 times a second, which this guide calls the 60 fps ticker (see `runTicker()`).
- `DEFAULT_CORNER`: the corner used when nothing was saved.
- `PANEL_FADE_MS = 150`: when the panel closes, its page fades its contents out. The window is hidden only after this delay, otherwise the fade would be cut off (see `collapse()`). It matches the page's fade exactly: the renderer's transitions are also 150 ms (`duration-150`).
- `BLUR_CLICK_GRACE_MS = 300`: see `clickBubble()`.
- `HIDE_SETTLE_MS = 150`: after the windows are hidden, Windows needs a moment to redraw what was behind them. A screenshot taken sooner could still show them.
- `export const`: the last three are exported so the tests can wait exactly the right time.

BubbleController

The class, with the comment that describes it:

src/main/bubble/BubbleController.ts, lines 71–80

```
/**
 * The bubble's state machine. Owns the mode, the bubble's position and its anchor (the display
 * and corner it rests in), and moves/shows/hides the two windows to match:
 *
 * collapsed ⇌ expanded                click, Esc, blur
 * collapsed/expanded → dragging → returning → collapsed    drag, release (snaps to a corner)
 * * → bouncing → returning → collapsed    Bounce action, then a click
 * expanded → capturing → expanded        screenshot
 */
export class BubbleController {
  // ...
}
```

The comment sums up the state machine in four lines; the lists at the top of this page are the long version. These are the class's fields:

src/main/bubble/BubbleController.ts, lines 81–90

```
private mode: Mode = 'collapsed'

private position: Point = { x: 0, y: 0 }

private anchor: BubbleAnchor

private motion: Motion | null = null

private ticker: ReturnType<typeof setInterval> | null = null

private hideTimer: ReturnType<typeof setTimeout> | null = null

private lastBlurCollapse = Number.NEGATIVE_INFINITY

private expandAfterReturn = false

private readonly now: () => number

private readonly random: () => number
```

- `private mode: Mode = 'collapsed'`: `private` means only this class's own methods can read or change the field (TypeScript checks this when compiling). Everything starts at rest.
- `private position: Point = { x: 0, y: 0 }`: the top-left corner of the bubble *circle* (not of its window), in DIPs. A placeholder until `start()` sets it.
- `private anchor: BubbleAnchor`: no starting value here; the constructor sets it. With the project's `strict` settings, TypeScript refuses to compile if the constructor could finish without assigning it.
- `private motion: Motion | null = null`: the bounce's current position and velocity (see `bounce.ts`), or `null` when not bouncing. `| null` makes TypeScript insist on a check before it's used.
- `ReturnType<typeof setInterval>`: "whatever type `setInterval` returns". It saves caring whether that's Node's `Timeout` object or a browser's number. `ticker` is the running animation timer, if any.
- `private hideTimer`: the pending "hide the panel window after the fade" timer from `collapse()`.
- `private lastBlurCollapse = Number.NEGATIVE_INFINITY`: when the panel last closed because it lost focus. Starting at minus infinity means "never", so the first click can't fall inside the grace period.
- `private expandAfterReturn = false`: set when something asks to open the panel while the bubble is still on its way home; `glideHome()` opens it on arrival.

- `private readonly now: () => number: readonly` means it's set once, in the constructor, and never reassigned. The same goes for `random`.

BubbleController.constructor()

src/main/bubble/BubbleController.ts, lines 92–102

```
constructor(private readonly deps: BubbleControllerDeps) {
  this.now = deps.now ?? (() => performance.now())
  this.random = deps.random ?? Math.random
  // Set now, not in start(): the pages ask for the corner as soon as they load, which can be
  // before start() runs.
  const saved = deps.initialAnchor
  this.anchor =
    saved && deps.displays.byId(saved.displayId)
    ? { ...saved }
    : { displayId: deps.displays.primary().id, corner: saved?.corner ?? DEFAULT_CORNER }
}
```

Stores the dependencies, picks the clock and random source, and decides the anchor. Nothing appears on screen until `start()`.

- `constructor(private readonly deps: BubbleControllerDeps)`: TypeScript shorthand. Marking a constructor parameter `private readonly` declares a field called `deps` and stores the argument in it.
- `deps.now ?? (() => performance.now())`: `??` uses the right-hand side only when the left is `null` or `undefined`. `performance.now()` is a millisecond clock that only moves forward; unlike `Date.now()`, it doesn't jump when the PC's clock is adjusted, which would make an animation skip.
- `// Set now, not in start(): index.ts` registers the IPC handlers (including `getState()`, which reads `corner`) before it waits for the pages to load, and only calls `start()` after both have drawn. A page can ask for the corner in between. Choosing the anchor here means that answer is already right, so the panel lays itself out for the correct corner from the start.
- `saved && deps.displays.byId(saved.displayId)`: the saved anchor is used only if its display is still connected. Otherwise the bubble goes to the main display, keeping the saved corner if there was one.
- `saved?.corner ?? DEFAULT_CORNER: ?`. gives `undefined` instead of an error when `saved` is `null`, and `??` then falls back to bottom-right.
- `{ ...saved }`: a copy, so the controller's anchor isn't the same object the settings hold.
- `this.anchor =`: sets the field directly rather than through `setAnchor()`, so `onAnchorChange` doesn't fire and a fallback anchor isn't saved. The pages get the corner from `getState()`.

BubbleController.currentMode, bubblePosition, corner

src/main/bubble/BubbleController.ts, lines 104–114

```
get currentMode(): Mode {
    return this.mode
}

get bubblePosition(): Point {
    return { ...this.position }
}

get corner(): Corner {
    return this.anchor.corner
}
```

`get` defines a read-only property: other code writes `controller.currentMode` (no brackets) but can't assign to it. `index.ts` reads `currentMode` and `corner` in its `getState()`, which is how a page that has just loaded learns the current state.

- `return { ...this.position }`: returns a copy, so a caller can't move the bubble by editing the object it got back. Nothing outside the class reads `bubblePosition` at the moment.

BubbleController.start()

src/main/bubble/BubbleController.ts, lines 116–121

```
/** Puts the bubble in its saved corner (or bottom-right of the main display) and shows it. */
start(): void {
    this.position = this.home()
    this.placeWindows()
    this.deps.bubble.show()
}
```

Called once by `index.ts`, after both pages have drawn for the first time (their windows' `ready-to-show` event), so the bubble doesn't appear before its page has anything to show. The anchor was already chosen in the constructor; this just puts the windows there.

- `this.position = this.home()`: the resting position in the anchor's corner (see `home()`).
- `this.placeWindows()`: positions both windows, although only the bubble is shown. The panel is then already in the right place when it first opens.
- `this.deps.bubble.show()`: for the real window this is `showInactive()` (see `bubbleSurface()`), so the bubble appears without stealing focus from whatever you're doing.

BubbleController.clickBubble()

src/main/bubble/BubbleController.ts, lines 123–139

```
clickBubble(): void {
  switch (this.mode) {
    case 'collapsed':
      if (this.now() - this.lastBlurCollapse >= BLUR_CLICK_GRACE_MS) this.expand()
      return
    case 'expanded':
      this.collapse()
      return
    case 'bouncing':
      this.glideHome()
      return
    case 'dragging':
    case 'returning':
    case 'capturing':
      return
  }
}
```

What a click on the bubble means depends on the mode. The bubble page just sends `IPC.bubbleClick` and leaves the decision to the main process, which knows the mode and the blur timing.

- `if (this.now() - this.lastBlurCollapse >= BLUR_CLICK_GRACE_MS) this.expand()`: with the panel open, pressing on the bubble can take focus away from the panel just before the click itself arrives. The blur closes the panel, and without this check the click would open it straight back up, so the panel would seem to ignore you. A click within 300 ms of a blur-close counts as part of the same gesture. (`windows.ts` also makes the bubble window unable to take focus, which normally prevents that blur; this check covers the times it still happens.)
- `case 'bouncing':`: clicking a bouncing bubble catches it and sends it home, without opening the panel.
- `case 'capturing':`: the dragging, returning and capturing modes ignore clicks. Listing them explicitly makes it clear none was forgotten. (A drag ends with a click event in the page too, but `BubbleView` swallows that one before it's sent.)

BubbleController.expand()

src/main/bubble/BubbleController.ts, lines 141–164

```
/** Opens the panel. While bouncing, glides home first and then opens. */
expand(): void {
  switch (this.mode) {
    case 'expanded':
      this.deps.panel.focus()
      return
    case 'bouncing':
      this.expandAfterReturn = true
      this.glideHome()
      return
    case 'returning':
      this.expandAfterReturn = true
      return
    case 'dragging':
    case 'capturing':
      return
    case 'collapsed':
      this.cancelHide()
      this.setMode('expanded')
      this.deps.panel.show()
      this.deps.panel.focus()
      return
  }
}
```

Opens the panel. Unlike `clickBubble()` it never closes it, so it's what "open" requests from outside use: the tray icon, launching the app again while it's running, and finishing sign-in in the browser (all in `index.ts`).

- `this.deps.panel.focus()`: already open, so just give it the keyboard again.
- `this.expandAfterReturn = true`: the panel opens beside the bubble's corner, so it can't open while the bubble is away from it. The request is remembered, and `glideHome()` opens the panel once the bubble has arrived. A bouncing bubble is sent home first; a returning one is already on its way.
- `case 'dragging':`: ignored while you're holding the bubble, and while capturing (the panel reopens by itself when the screenshot is done).
- `this.cancelHide()`: if the panel is still fading out from a moment ago, its pending "hide" timer is cancelled. Otherwise the window would vanish 150 ms after reopening.
- `this.setMode('expanded')`: the pages hear the new mode (through `onModeChange`) and animate open; the window is then shown and focused so you can type at once.

BubbleController.collapse()

src/main/bubble/BubbleController.ts, lines 166–174

```
collapse(): void {
  if (this.mode !== 'expanded') return
  this.setMode('collapsed')
  this.cancelHide()
  this.hideTimer = setTimeout(() => {
    this.hideTimer = null
    this.deps.panel.hide()
  }, PANEL_FADE_MS)
}
```

Closes the panel in two steps: the mode changes now, so the page starts its fade-out, and the window itself is hidden `PANEL_FADE_MS` later. Hiding the window at once would cut the animation off. Called by `clickBubble()`, `panelBlurred()` and, for Esc in the panel, by `ipc.ts`.

- `if (this.mode !== 'expanded') return`: closing only makes sense when open. It also makes a repeated request harmless.
- `this.cancelHide()`: never leaves two hide timers running.
- `this.hideTimer = null`: inside the timer, marks that nothing is pending any more.

BubbleController.panelBlurred()

src/main/bubble/BubbleController.ts, lines 176–180

```
panelBlurred(): void {
  if (this.mode !== 'expanded') return
  this.lastBlurCollapse = this.now()
  this.collapse()
}
```

`index.ts` calls this on the panel window's `blur` event, which Electron fires when the window loses keyboard focus, for example because you clicked another app or the desktop. The panel behaves like a popup and closes.

- `if (this.mode !== 'expanded') return`: blurs in any other mode are ignored, such as the one caused when `whileHidden()` hides the panel for a screenshot.
- `this.lastBlurCollapse = this.now()`: noted for the grace period in `clickBubble()`.

BubbleController.startDrag()

src/main/bubble/BubbleController.ts, lines 182–194

```
/** The mouse pressed on the bubble and moved: the bubble follows the pointer until released. */
startDrag(): void {
  if (this.mode !== 'collapsed' && this.mode !== 'expanded') return
  this.cancelHide()
  this.deps.panel.hide()
  const cursor = this.deps.displays.cursor()
  const grab = { x: cursor.x - this.position.x, y: cursor.y - this.position.y }
  this.setMode('dragging')
  this.runTicker(() => {
    const now = this.deps.displays.cursor()
    this.moveBubble({ x: now.x - grab.x, y: now.y - grab.y })
  })
}
```

The bubble page decides a press has become a drag once the pointer moves more than 5 pixels (`DRAG_THRESHOLD` in `BubbleView.tsx`), and sends `IPC.bubbleDragStart`. From then on the main process moves the window and the page just waits for the release.

- `if (this.mode !== 'collapsed' && this.mode !== 'expanded') return`: a drag can only start from rest, not mid-bounce, mid-glide or during a screenshot.
- `this.deps.panel.hide()`: the panel can't follow the bubble, so it's hidden at once, without the fade. `cancelHide()` drops any fade timer that was still pending.
- `const grab`: the offset from the bubble's top-left to the pointer. Keeping that offset fixed means the spot you grabbed stays under the pointer, instead of the bubble jumping so its corner is there.

- `this.runTicker()` => {}: every frame, read where the pointer is now and move the bubble to match. The callback ignores the frame time, since following the pointer only needs its current position.

Reading the real pointer position in the main process keeps the drag smooth, even across monitors with different scaling. The page's own coordinates would keep shifting as its window moves under it.

BubbleController.endDrag()

src/main/bubble/BubbleController.ts, lines 196–210

```
/** Released: snap to the nearest corner of the display the bubble was dropped on. */
endDrag(): void {
  if (this.mode !== 'dragging') return
  this.stopTicker()
  const centre = {
    x: Math.round(this.position.x + UI.bubbleSize / 2),
    y: Math.round(this.position.y + UI.bubbleSize / 2),
  }
  const display = this.deps.displays.nearest(centre)
  this.setAnchor({
    displayId: display.id,
    corner: nearestCorner(display.workArea, this.position),
  })
  this.glideHome()
}
```

On release, `BubbleView` sends `IPC.bubbleDragEnd`. The bubble snaps to the nearest corner of whichever display it was dropped on.

- `if (this.mode !== 'dragging') return`: a release that doesn't end a drag (say, a drag that was refused because the bubble was bouncing) does nothing.
- `const centre`: the display is picked by the bubble's centre, not its top-left, so a bubble straddling two monitors goes to the one its centre is on. It's rounded because Electron's display lookup works in whole DIPs.
- `nearestCorner(display.workArea, this.position)`: the corner of the quarter of that display the bubble is in (`layout.ts`).
- `this.setAnchor({`: saves and broadcasts the new anchor, but only if it actually changed.
- `this.glideHome()`: the bubble glides into its new corner; the mode goes to `returning`, then `collapsed`.

BubbleController.startBounce()

src/main/bubble/BubbleController.ts, lines 212–223

```
startBounce(): void {
  if (this.mode !== 'collapsed' && this.mode !== 'expanded') return
  this.cancelHide()
  this.deps.panel.hide()
  this.motion = launch(this.position, this.random, this.anchor.corner)
  this.setMode('bouncing')
  this.runTicker((dtMs) => {
    if (!this.motion) return
    this.motion = step(this.motion, dtMs, travelBox(this.display().workArea))
    this.moveBubble({ x: this.motion.x, y: this.motion.y })
  })
}
```

The Bounce action: the bubble flies around its display, bouncing off the edges, until clicked. `actions.ts` calls this for the Bounce icon.

- `if (this.mode !== 'collapsed' && this.mode !== 'expanded') return`: the same rule as for dragging. In practice it's `expanded`, because the Bounce icon is in the panel.
- `launch(this.position, this.random, this.anchor.corner)`: the starting position and velocity, heading away from the bubble's corner into the screen (`bounce.ts`).
- `if (!this.motion) return`: mostly for TypeScript. `motion` is typed `Motion | null`, and inside the callback TypeScript can't assume it's still set.
- `travelBox(this.display().workArea)`: worked out again every frame from the anchor's display. If the resolution or taskbar changes mid-bounce, the bubble stays within the new area (`displayChanged()` deliberately leaves a bounce alone).
- `step(this.motion, dtMs, :` moves the bubble by the real time since the last frame.
- `this.moveBubble()`: moves only the bubble window. The panel is hidden, and is put back next to the bubble when it settles.

BubbleController.whileHidden()

src/main/bubble/BubbleController.ts, lines 225–240

```
/** Hides both windows, runs `task` (a screenshot), then reopens the panel. */
async whileHidden<T>(task: () => Promise<T>): Promise<T> {
  if (this.mode !== 'expanded') throw new Error(`Can't hide the windows while ${this.mode}`)
  this.setMode('capturing')
  this.deps.panel.hide()
  this.deps.bubble.hide()
  try {
    await new Promise((resolve) => setTimeout(resolve, HIDE_SETTLE_MS))
    return await task()
  } finally {
    this.deps.bubble.show()
    this.setMode('expanded')
    this.deps.panel.show()
    this.deps.panel.focus()
  }
}
```

Used by the screenshot action in `actions.ts`, which passes a task that calls `ScreenshotService.capture()`. The overlay windows mustn't appear in your screenshot, so they're hidden first and brought back afterwards.

- `async whileHidden<T>(task: () => Promise<T>): Promise<T>: <T>` makes it *generic*: it returns whatever `task` returns, with the same type.
- `throw new Error()`: only allowed from `expanded`, since the camera icon is in the panel. `actions.ts` catches the error and shows "Couldn't take a screenshot".
- `this.setMode('capturing')`: comes before hiding the windows. Hiding the focused panel makes it lose focus, and `panelBlurred()` ignores that blur only because the mode is no longer `expanded`.
- `await new Promise((resolve) => setTimeout(resolve, HIDE_SETTLE_MS))`: a pause written as a promise that resolves after 150 ms.
- `return await task()`: the `await` matters. Without it, the `finally` block would run as soon as the task started, bringing the windows back before the screenshot was taken.
- `finally {}`: runs whether the task succeeded or threw, so a failed capture never leaves the app invisible. The panel comes back open and focused, as it was before.

BubbleController.displayChanged()

src/main/bubble/BubbleController.ts, lines 242–253

```
/**
 * The resolution, taskbar or monitors changed. If the bubble's display was unplugged, it moves
 * to the same corner of the main display. Moving bubbles re-read the display every frame.
 */
displayChanged(): void {
  if (!this.deps.displays.byId(this.anchor.displayId)) {
    this.setAnchor({ displayId: this.deps.displays.primary().id, corner: this.anchor.corner })
  }
  if (this.mode === 'bouncing' || this.mode === 'returning' || this.mode === 'dragging') return
  this.position = this.home()
  this.placeWindows()
}
```

`index.ts` calls this on Electron's `screen` events `display-added`, `display-removed` and `display-metrics-changed`. The last covers resolution, scaling and rotation changes, and the taskbar moving or resizing (which changes the work area).

- `if (!this.deps.displays.byId(this.anchor.displayId))`: the bubble's display is gone, for example unplugged or a laptop undocked. The anchor moves to the same corner of the main display, through `setAnchor()`, so the change is saved and broadcast.
- `this.mode === 'bouncing' || this.mode === 'returning'`: moving bubbles adapt by themselves. The bounce and the glide re-read the display every frame, and a drag follows the pointer. Snapping the windows here would make the bubble jump.
- `this.position = this.home()`: in every other mode, both windows go back to the corner of the (possibly resized) work area. That includes `expanded`, so an open panel moves with the bubble, and `capturing`, where the windows are hidden but will come back in the right place.

If a bouncing bubble's display is unplugged, the next frame's `step()` pulls it into the main display's travel box, so it reappears there.

BubbleController.dispose()

src/main/bubble/BubbleController.ts, lines 255–258

```
dispose(): void {
  this.stopTicker()
  this.cancelHide()
}
```

Stops both timers. `index.ts` calls it while quitting (in `before-quit`), so no timer fires against a window that is being destroyed. The tests call it after each test.

BubbleController.display(), home()

src/main/bubble/BubbleController.ts, lines 260–267

```
/** The anchor's display, or the main display if it's gone. */
private display(): DisplayArea {
    return this.deps.displays.byId(this.anchor.displayId) ?? this.deps.displays.primary()
}

private home(): Point {
    return homePosition(this.display().workArea, this.anchor.corner)
}
```

- `?? this.deps.displays.primary()`: if the anchor's display has gone and `displayChanged()` hasn't run yet, use the main display rather than fail.
- `homePosition(this.display().workArea, this.anchor.corner)`: the resting position for the current anchor (`layout.ts`). It's worked out fresh on every call, from the display's current work area, so it's never stale.

BubbleController.glideHome()

src/main/bubble/BubbleController.ts, lines 269–293

```
/** Glides the bubble into its anchor corner, then switches to collapsed. */
private glideHome(): void {
    this.motion = null
    const from = { ...this.position }
    const duration = glideDuration(from, this.home())
    const startedAt = this.now()
    this.setMode('returning')
    this.runTicker(() => {
        // Re-read home every frame in case the display changes mid-glide.
        const home = this.home()
        const t = (this.now() - startedAt) / duration
        if (t < 1) {
            this.moveBubble(glidePosition(from, home, t))
            return
        }
        this.stopTicker()
        this.position = home
        this.placeWindows()
        this.setMode('collapsed')
        if (this.expandAfterReturn) {
            this.expandAfterReturn = false
            this.expand()
        }
    })
}
```

Animates the bubble from wherever it is into its anchor corner, then switches to `collapsed`. Used after a drag (`endDrag()`) and to end a bounce (`clickBubble()`, `expand()`).

- `this.motion = null`: any bounce is over.
- `glideDuration(from, this.home())`: longer trips take longer, between 0.3 and 0.9 s (`bounce.ts`). The duration is fixed when the glide starts.
- `const home = this.home()`: inside the frame callback, so it's re-read every frame. If the display changes mid-glide, the bubble still ends up in the right place.

- `const t = (this.now() - startedAt) / duration`: progress from 0 to 1, based on real elapsed time, so a late frame doesn't slow the glide down.
- `this.moveBubble(glidePosition(from, home, t))`: only the bubble window moves during the glide.
- `this.position = home`: when `t` reaches 1, the timer stops and the bubble is put exactly on its home (the last frame was probably just short). `placeWindows()` moves the hidden panel next to the bubble's corner, ready to open.
- `if (this.expandAfterReturn)`: something asked to open the panel during the trip (see `expand()`). The flag is cleared, then `expand()` runs from `collapsed` as normal.

BubbleController.runTicker(), stopTicker()

src/main/bubble/BubbleController.ts, lines 295–308

```
private runTicker(onFrame: (dtMs: number) => void): void {
  this.stopTicker()
  let last = this.now()
  this.ticker = setInterval(() => {
    const now = this.now()
    onFrame(now - last)
    last = now
  }, FRAME_MS)
}

private stopTicker(): void {
  if (this.ticker) clearInterval(this.ticker)
  this.ticker = null
}
```

The 60 fps ticker. It drives all three animations (drag, bounce and glide), but only one runs at a time.

- `this.stopTicker()`: starting a new animation always replaces the old one. That's how a click on a bouncing bubble switches from the bounce to the glide.
- `this.ticker = setInterval()`: calls the frame function about every 16 ms (`FRAME_MS`).
- `onFrame(now - last)`: Node's timers aren't exact, and can run late when the main process is busy, so each frame is told how many milliseconds really passed. The bounce uses this to move the right distance.
- `if (this.ticker) clearInterval(this.ticker)`: safe to call when nothing is running.

BubbleController.moveBubble(), placeWindows()

src/main/bubble/BubbleController.ts, lines 310–320

```
private moveBubble(position: Point): void {
  this.position = position
  this.deps.bubble.setBounds(bubbleWindowBounds(position))
}

private placeWindows(): void {
  this.deps.bubble.setBounds(bubbleWindowBounds(this.position))
  this.deps.panel.setBounds(
    panelWindowBounds(this.position, this.deps.actionCount, this.anchor.corner),
  )
}
```

- `moveBubble`: changes the position and moves only the bubble window. Animations call it every frame; the panel is hidden during them, so moving it too would be wasted work.

- `placeWindows`: positions both windows for the current position and anchor. The panel's place depends on the corner, because it always opens toward the middle of the screen.
- `bubbleWindowBounds(this.position)`: turns the bubble's position (the circle's top-left) into its window's rectangle, which includes the transparent padding around the circle (`layout.ts`).
- `panelWindowBounds(this.position, this.deps.actionCount, this.anchor.corner)`: the panel's rectangle (`layout.ts`).

`BubbleController.setAnchor(), cancelHide(), setMode()`

`src/main/bubble/BubbleController.ts`, lines 322–337

```
private setAnchor(anchor: BubbleAnchor): void {
  if (anchor.displayId === this.anchor.displayId && anchor.corner === this.anchor.corner) return
  this.anchor = anchor
  this.deps.onAnchorChange({ ...anchor })
}

private cancelHide(): void {
  if (this.hideTimer) clearTimeout(this.hideTimer)
  this.hideTimer = null
}

private setMode(mode: Mode): void {
  if (mode === this.mode) return
  this.mode = mode
  this.deps.onModeChange(mode)
}
```

Small helpers.

- `if (anchor.displayId === this.anchor.displayId)`: dropping the bubble back in its own corner changes nothing, so nothing is saved or broadcast.
- `this.deps.onAnchorChange({ ...anchor })`: passes a copy, so the receiver can't change the controller's own anchor.
- `cancelHide`: cancels the panel hide that `collapse()` scheduled, if there is one.
- `if (mode === this.mode) return`: `onModeChange` (a broadcast to both pages) fires only for real changes.

`src/main/bubble/layout.ts`: where the windows go

Pure maths, no Electron. Given a work area and a corner, it works out where the bubble rests, which corner a dropped bubble belongs to, where it may bounce, and the exact rectangles of the two windows.

`BubbleController` is its only user in the app; `tests/layout.test.ts` checks each function on its own.

Key ideas:

- A position always means the top-left corner of the bubble *circle* (56×56 DIPs, `UI.bubbleSize`), not of its window. The bubble window is bigger, `BUBBLE_BOX` = 72×72, because it has 8 DIPs of transparent padding (`UI.bubblePad`) on every side for the shadow and the ring. Only `bubbleWindowBounds()` deals with the difference.
- Everything is measured from the work area's own `x` and `y`, never from (0, 0), so the same code works on a second monitor or with the taskbar at the top or left.

- All sizes come from `UI` in `src/shared/geometry.ts`, which the pages use too, so the window sizes and the page layout can't drift apart.

TravelBox

`src/main/bubble/layout.ts`, lines 12–19

```
// Positions are the top-left corner of the bubble circle itself, not of its window.

export interface TravelBox {
  minX: number
  minY: number
  maxX: number
  maxY: number
}
```

The range of positions a bouncing bubble may be in: lowest and highest `x`, lowest and highest `y`. Made by `travelBox()` below and used by `step()` in `bounce.ts`, which compares a position with each limit directly. The comment above it applies to the whole file.

homePosition()

`src/main/bubble/layout.ts`, lines 21–31

```
/** The bubble's resting place in 'corner' of the work area (the screen minus the taskbar). */
export function homePosition(workArea: Rect, corner: Corner = 'bottom-right'): Point {
  return {
    x: isLeftCorner(corner)
      ? workArea.x + UI.edgeMargin
      : workArea.x + workArea.width - UI.edgeMargin - UI.bubbleSize,
    y: isTopCorner(corner)
      ? workArea.y + UI.edgeMargin
      : workArea.y + workArea.height - UI.edgeMargin - UI.bubbleSize,
  }
}
```

The bubble's resting position in a corner: `UI.edgeMargin` (16 DIPs) in from both edges of the work area.

- `isLeftCorner(corner)`: from `geometry.ts`; true for `top-left` and `bottom-left`. `isTopCorner()` works the same way for the top.
- `workArea.x + workArea.width - UI.edgeMargin - UI.bubbleSize`: on the right, the bubble's own width is subtracted too, because the position is its left edge.
- `corner: Corner = 'bottom-right'`: a default value, used if the caller leaves the argument out.

nearestCorner()

`src/main/bubble/layout.ts`, lines 33–40

```
/** The corner nearest the bubble: whichever quarter of the work area its centre is in. */
export function nearestCorner(workArea: Rect, bubble: Point): Corner {
  const centreX = bubble.x + UI.bubbleSize / 2
  const centreY = bubble.y + UI.bubbleSize / 2
  const top = centreY < workArea.y + workArea.height / 2
  const left = centreX < workArea.x + workArea.width / 2
  return `${top ? 'top' : 'bottom'}-${left ? 'left' : 'right'}`
}
```

Used by `endDrag()`. It splits the work area into four quarters and returns the corner of the quarter the bubble's centre is in.

- `bubble.x + UI.bubbleSize / 2`: the centre, not the top-left, so a bubble across the middle line goes to the side most of it is on.
- ``${top ? 'top' : 'bottom'}-${left ? 'left' : 'right'}``: builds the corner's name. TypeScript works out that this can only produce the four `Corner` values, so it needs no cast.

`travelBox()`

`src/main/bubble/layout.ts`, lines 42–50

```
/** Every position where the bubble is fully inside the work area. */
export function travelBox(workArea: Rect): TravelBox {
  return {
    minX: workArea.x,
    minY: workArea.y,
    maxX: workArea.x + workArea.width - UI.bubbleSize,
    maxY: workArea.y + workArea.height - UI.bubbleSize,
  }
}
```

Every position where the whole circle is inside the work area. Unlike the resting position there's no 16 DIP margin, so a bouncing bubble touches the edges, but it never goes under the taskbar or off the screen.

- `workArea.x + workArea.width - UI.bubbleSize`: the highest `x` is a bubble's width short of the right edge, because the position is the circle's left edge. If the work area were narrower than the bubble, the maximum would be less than the minimum; `reflect()` in `bounce.ts` handles that.

`bubbleWindowBounds()`

`src/main/bubble/layout.ts`, lines 52–59

```
export function bubbleWindowBounds(bubble: Point): Rect {
  return {
    x: Math.round(bubble.x) - UI.bubblePad,
    y: Math.round(bubble.y) - UI.bubblePad,
    width: BUBBLE_BOX,
    height: BUBBLE_BOX,
  }
}
```

The bubble window's rectangle: 8 DIPs up and to the left of the circle, 72×72.

- `Math.round(bubble.x)`: the bounce and the glide produce fractional positions, but a window can only be placed on whole pixels.

panelWindowBounds()

src/main/bubble/layout.ts, lines 61–77

```
/**
 * The panel window shares the bubble window's corner that points into the screen's `corner`, so
 * it extends toward the middle of the screen (left and up from bottom-right, and so on).
 */
export function panelWindowBounds(
  bubble: Point,
  actionCount: number,
  corner: Corner = 'bottom-right',
): Rect {
  const box = bubbleWindowBounds(bubble)
  const size = panelWindowSize(actionCount)
  return {
    x: isLeftCorner(corner) ? box.x : box.x + box.width - size.width,
    y: isTopCorner(corner) ? box.y : box.y + box.height - size.height,
    ...size,
  }
}
```

The panel window's rectangle. Its size comes from `panelWindowSize()` in `geometry.ts` (big enough for the chat card at full height, a toast, and the action icons), and it's placed so it shares one corner with the bubble window: the corner that points into the screen's corner. With the bubble bottom-right, the two windows' bottom-right corners coincide and the panel extends left and up, toward the middle of the screen.

- `isLeftCorner(corner) ? box.x : box.x + box.width - size.width`: in a left corner the left edges line up; in a right corner the right edges do. The `y` line does the same for top and bottom.
- `...size`: copies `width` and `height` into the result.

The two windows overlap. The panel's page lays out its content from the shared corner (`PANEL_LAYOUT` in `geometry.ts`) and leaves the bubble's square empty. Empty parts of the panel are click-through, so the bubble still gets its clicks.

src/main/bubble/bounce.ts: bounce physics

Pure maths for the two animations: the bounce (the bubble moves at a constant speed and reflects off the edges of the work area, like an old DVD screensaver) and the glide home (a smooth slow-down into the corner). Each function takes a state and returns a new one; none keeps anything between calls or touches a window.

`BubbleController` calls them from its frame timer (`startBounce()` and `glideHome()`), and `tests/bounce.test.ts` tests them directly.

Motion

src/main/bubble/bounce.ts, lines 4–10

```
/** Position (DIP) and velocity (DIP per second) of the bouncing bubble. */
export interface Motion {
  x: number
  y: number
  vx: number
  vy: number
}
```

A position plus a velocity: how many DIPs per second the bubble moves in each direction. A positive `vx` means moving right; a positive `vy` means moving down, because screen coordinates grow downward.

BOUNCE_SPEED, MAX_STEP_MS

src/main/bubble/bounce.ts, lines 12–14

```
export const BOUNCE_SPEED = 280

/** Longer gaps (e.g. the machine was asleep) are treated as one short step. */
const MAX_STEP_MS = 100
```

- `BOUNCE_SPEED = 280`: DIPs per second, about 4.5 DIPs per frame.
- `MAX_STEP_MS = 100`: if the timer stalls (the PC slept, or the main process was busy), the next frame reports a long gap. Moving the full distance would make the bubble teleport; capping the step means it just carries on from where it was.

launch()

src/main/bubble/bounce.ts, lines 16–32

```
/** Sets off away from the bubble's corner, into the screen, at a random 25°–65° angle. */
export function launch(
  from: Point,
  random: () => number = Math.random,
  corner: Corner = 'bottom-right',
  speed = BOUNCE_SPEED,
): Motion {
  const angle = ((25 + random() * 40) * Math.PI) / 180
  const dx = isLeftCorner(corner) ? 1 : -1
  const dy = isTopCorner(corner) ? 1 : -1
  return {
    x: from.x,
    y: from.y,
    vx: dx * speed * Math.cos(angle),
    vy: dy * speed * Math.sin(angle),
  }
}
```

Sets the bubble off: away from the corner it rests in, into the screen, at a random angle between 25° and 65° from horizontal. The range keeps it from running flat along an edge or straight up.

- `((25 + random() * 40) * Math.PI) / 180`: `random()` gives a number from 0 to 1, so the angle is 25° to 65°. It's converted to radians, which `Math.cos` and `Math.sin` expect.
- `const dx = isLeftCorner(corner) ? 1 : -1`: from a left corner go right (+1); from a right corner go left (-1). `dy` does the same vertically: from a bottom corner, -1 means up.
- `vx: dx * speed * Math.cos(angle)`: splits the speed into its horizontal and vertical parts, so the overall speed is always `speed`, whatever the angle.
- `random: () => number = Math.random`: a default, like `speed = BOUNCE_SPEED`. The controller passes its own `random` so the tests can fix the angle.

step()

src/main/bubble/bounce.ts, lines 34–40

```
/** Advances the motion by `dtMs`, reflecting off the edges of `box`. */
export function step(motion: Motion, dtMs: number, box: TravelBox): Motion {
  const dt = Math.min(Math.max(dtMs, 0), MAX_STEP_MS) / 1000
  const [x, vx] = reflect(motion.x + motion.vx * dt, motion.vx, box.minX, box.maxX)
  const [y, vy] = reflect(motion.y + motion.vy * dt, motion.vy, box.minY, box.maxY)
  return { x, y, vx, vy }
}
```

Advances one frame: moves by velocity × time on each axis, then bounces off any edge it crossed. `startBounce()` calls it every frame with the real milliseconds since the last one and the current travel box.

- `Math.min(Math.max(dtMs, 0), MAX_STEP_MS) / 1000`: clamps the elapsed time to 0–100 ms (it should never be negative, but that costs nothing to rule out), then converts it to seconds to match the velocity.
- `const [x, vx] = reflect(: reflect()` returns a pair, and this *destructures* it into two variables. The two axes are handled separately, which is how a bounce off a wall works: hitting a side wall flips only `vx`.
- `return { x, y, vx, vy }`: a new `Motion`; the old one is left unchanged.

reflect()

src/main/bubble/bounce.ts, lines 42–47

```
function reflect(position: number, velocity: number, min: number, max: number): [number, number] {
  if (max <= min) return [min, velocity]
  if (position < min) return [Math.min(min + (min - position), max), Math.abs(velocity)]
  if (position > max) return [Math.max(max - (position - max), min), -Math.abs(velocity)]
  return [position, velocity]
}
```

One axis of a bounce. It isn't exported: only `step()` uses it.

- `if (max <= min) return [min, velocity]`: the work area is too small for the bubble to move in this direction, so it's pinned at the edge rather than jittering back and forth.
- `if (position < min)`: the bubble went past the low edge by `min - position`. It's mirrored back inside by the same amount, as if it had bounced exactly at the edge.
- `Math.min(min + (min - position), max)`: the cap stops a huge overshoot from bouncing the bubble out past the far edge. So whatever happens, the result is inside the box.
- `Math.abs(velocity)`: forces the velocity to point inward rather than just flipping it. If the bubble is outside the box because the work area shrank mid-bounce, it may already be heading back in; flipping would send it out again.

glideDuration()

src/main/bubble/bounce.ts, lines 49–53

```
/** Glide length scales with distance, kept between 0.3 s and 0.9 s. */
export function glideDuration(from: Point, to: Point): number {
  const distance = Math.hypot(to.x - from.x, to.y - from.y)
  return Math.min(900, Math.max(300, distance / 1.5))
}
```

How long the glide home takes, in milliseconds: the straight-line distance (`Math.hypot`) divided by 1.5, so roughly 1.5 DIPs per millisecond, but never less than 300 ms or more than 900 ms. A short hop still looks deliberate, and a long trip across the screen doesn't drag on.

`glidePosition()`

`src/main/bubble/bounce.ts`, lines 55–60

```
/** Position at progress `t` (0–1) along the glide home, easing out as it settles. */
export function glidePosition(from: Point, to: Point, t: number): Point {
  const progress = Math.min(Math.max(t, 0), 1)
  const eased = 1 - (1 - progress) ** 3
  return { x: from.x + (to.x - from.x) * eased, y: from.y + (to.y - from.y) * eased }
}
```

Where the bubble is at progress `t` along the glide (0 at the start, 1 at home).

- `Math.min(Math.max(t, 0), 1)`: clamps `t`, so the bubble can never overshoot home.
- `1 - (1 - progress) ** 3`: a cubic *ease-out* curve (`**` means "to the power of"). It climbs quickly at first and flattens out toward 1, so the bubble moves fast at the start and slows as it settles into the corner.
- `from.x + (to.x - from.x) * eased`: moves that fraction of the way from the start to home, on each axis.

`src/main/bubble/windows.ts`: creating the Electron windows

The only file in this folder that uses Electron. It creates the two overlay windows, locks them down, and adapts them to the controller's `Surface` interface. `index.ts` calls `createOverlayWindow()` twice at startup, once per view, and wraps the results with `bubbleSurface()` and `panelSurface()` before handing them to `BubbleController`.

Key ideas:

- **Two windows, one page.** Both windows load the same renderer page; `?view=bubble` or `?view=panel` in its address tells `main.tsx` which UI to draw. The bubble has its own tiny window so it never resizes when the panel opens, and only a 72×72 window moves during a drag or a bounce.
- **Click-through.** The panel window is a large rectangle (sized for the chat card at its tallest), but most of it is transparent. `setIgnoreMouseEvents(true, { forward: true })` makes Windows pass clicks on it to whatever is underneath, while still telling the page where the pointer is. While the pointer is over a piece of real UI (an element marked `data-hit`), the page's `useClickThrough` hook asks the main process to take mouse input again (`IPC.setInteractive`, handled in `ipc.ts`). The bubble window is never click-through: it's barely larger than the bubble, and relying on Windows' mouse forwarding there made clicks occasionally fall through the bubble to the window behind it.

View

`src/main/bubble/windows.ts`, line 7

```
export type View = 'bubble' | 'panel'
```

A *union type*: a `View` can only be the string `'bubble'` or `'panel'`, and TypeScript rejects anything else.

createOverlayWindow()

src/main/bubble/windows.ts, lines 9–59

```
/**
 * Creates one of the two overlay windows. Both are frameless, transparent, always on top and
 * hidden from the taskbar and Alt+Tab.
 *
 * The panel window is mostly see-through, so it's click-through: clicks on its empty areas go to
 * whatever is underneath, and the renderer turns mouse input back on while the pointer is over
 * real UI (see useClickThrough). The bubble window is never click-through: it's barely larger
 * than the bubble, and relying on Windows' mouse forwarding there made clicks occasionally fall
 * through the bubble to the window behind it.
 */
export function createOverlayWindow(view: View, size: Size): BrowserWindow {
  const win = new BrowserWindow({
    ...size,
    show: false,
    frame: false,
    transparent: true,
    backgroundColor: '#00000000',
    hasShadow: false,
    thickFrame: false,
    resizable: false,
    minimizable: false,
    maximizable: false,
    fullscreenable: false,
    skipTaskbar: true,
    alwaysOnTop: true,
    type: 'toolbar',
    // The bubble never takes keyboard focus, so clicking it doesn't blur the panel.
    focusable: view === 'panel',
    webPreferences: {
      preload: join(__dirname, '../preload/index.js'),
      sandbox: true,
      contextIsolation: true,
      nodeIntegration: false,
      backgroundThrottling: false,
      spellcheck: view === 'panel',
    },
  })
  lockDown(win, view)
  if (view === 'panel') {
    win.setIgnoreMouseEvents(true, { forward: true })
    addEditContextMenu(win)
  }

  const devServer = process.env['ELECTRON_RENDERER_URL']
  if (!app.isPackaged && devServer) {
    void win.loadURL(`${devServer}?view=${view}`)
  } else {
    void win.loadFile(join(__dirname, '../renderer/index.html'), { query: { view } })
  }
  return win
}
```

Creates one overlay window. `new BrowserWindow({...})` makes a native window with a Chromium page inside; the options say what kind of window.

- `...size`: the window's `width` and `height`: `BUBBLE_BOX` square for the bubble, `panelWindowSize(actionCount)` for the panel. Neither window changes size later, only position.
- `show: false`: created hidden. The controller decides when each one appears.

- `transparent: true`: together with `frame: false` (no title bar or border) and a fully transparent `backgroundColor` (in `'#00000000'` the last two hex digits are the alpha), only what the page draws is visible.
- `thickFrame: false`: Windows-only. The standard resizable frame would bring the system shadow and the minimise and maximise animations. With `hasShadow: false`, Windows adds no shadow; the page draws its own.
- `type: 'toolbar'`: on Windows, a toolbar window is left out of Alt+Tab. `skipTaskbar: true` keeps it off the taskbar too.
- `focusable: view === 'panel'`: the bubble window can never take keyboard focus, so clicking it doesn't take focus from the panel (which would close it, see `panelBlurred()`) or from the app you're working in. The panel must be focusable, since you type in it.
- `preload: join(__dirname, '../preload/index.js')`: the preload script, which gives the page `window.assist`. `__dirname` is the folder of the built main-process file (`out/main`), so this points at the built preload in `out/preload`.
- `sandbox: true`: with `contextIsolation: true` and `nodeIntegration: false`, the page runs like an ordinary web page: no Node.js, no access to the preload's internals, only what the preload hands it. Even hostile code in the page couldn't reach files.
- `backgroundThrottling: false`: Chromium normally slows timers and animation frames in windows it thinks are hidden or in the background. These windows are hidden often and must respond at once when shown.
- `spellcheck: view === 'panel'`: spell checking only where there's a text box.
- `win.setIgnoreMouseEvents(true, { forward: true })`: the panel starts click-through. The `forward` option (Windows and macOS only) keeps sending mouse moves to the page, which is what lets `useClickThrough` notice the pointer arriving over real UI.
- `process.env['ELECTRON_RENDERER_URL']`: set by `electron-vite` during `npm run dev` to the address of its dev server, which serves the page with live reloading. `!app.isPackaged` makes sure an installed app always loads the page bundled with it.
- `{ query: { view } }`: `loadFile()` adds `?view=bubble` or `?view=panel` to the built page's address.
- `void win.loadURL()`: loading returns a promise that isn't awaited; `void` marks that as deliberate.

`bubbleSurface()`, `panelSurface()`

`src/main/bubble/windows.ts`, lines 61–79

```
export function bubbleSurface(win: BrowserWindow): Surface {
  return {
    setBounds: (bounds) => win.setBounds(bounds),
    show: () => win.showInactive(),
    hide: () => win.hide(),
  }
}

export function panelSurface(win: BrowserWindow): PanelSurface {
  return {
    setBounds: (bounds) => win.setBounds(bounds),
    show: () => {
      resetClickThrough(win)
      win.show()
    },
    hide: () => win.hide(),
    focus: () => win.focus(),
  }
}
```

Adapters: each wraps a real `BrowserWindow` in the small object the controller expects, and forwards each call to the matching Electron method.

- `show: () => win.showInactive()`: shows the bubble without activating it, so it never steals focus.
- `resetClickThrough(win)`: the panel's click-through is reset each time, just before it's shown.
- `win.show()`: unlike `showInactive()`, this shows and activates the panel.
- `focus: () => win.focus()`: gives the panel keyboard focus, so you can type straight away.
- `hide: () => win.hide()`: hides the window; its page keeps running.

`resetClickThrough()`

`src/main/bubble/windows.ts`, lines 81–93

```
/**
 * Starts the panel's click-through afresh each time it's shown. While hidden, the page can't
 * follow the pointer, so its idea of whether the pointer is over real UI is out of date (and
 * Windows may have dropped the mouse forwarding). Switching forwarding off and on re-installs it,
 * and the page is told exactly where the pointer is now, so it can decide straight away.
 */
function resetClickThrough(win: BrowserWindow): void {
  win.setIgnoreMouseEvents(false)
  win.setIgnoreMouseEvents(true, { forward: true })
  const cursor = screen.getCursorScreenPoint()
  const bounds = win.getBounds()
  win.webContents.send(IPC.clickThroughReset, { x: cursor.x - bounds.x, y: cursor.y - bounds.y })
}
```

Starts the panel's click-through afresh each time it's shown. While the panel is hidden, its page can't follow the pointer, so its idea of whether the pointer is over real UI is out of date, and Windows may have dropped the mouse forwarding. This fixes clicks going astray after the panel has been hidden, for example for a screenshot.

- `win.setIgnoreMouseEvents(false)`: switching off and then on again installs the forwarding again, and leaves the window click-through.
- `screen.getCursorScreenPoint()`: with `win.getBounds()`, both in DIPs, so subtracting gives the pointer's position inside the window. That's what the page calls `clientX` and `clientY`, since a frameless page fills its window exactly.
- `win.webContents.send(IPC.clickThroughReset)`: tells the page. `useClickThrough` checks what's under that point on the next animation frame and takes mouse input straight away if it's over real UI.

`lockDown()`

`src/main/bubble/windows.ts`, lines 95–104

```
function lockDown(win: BrowserWindow, view: View): void {
  const contents = win.webContents
  contents.setWindowOpenHandler(() => ({ action: 'deny' }))
  contents.on('will-navigate', (event) => event.preventDefault())
  contents.on('render-process-gone', () => {
    // A crashed panel can't report the pointer, so let clicks through until it has reloaded.
    if (view === 'panel') win.setIgnoreMouseEvents(true, { forward: true })
    contents.reload()
  })
}
```

Basic safety and crash recovery for both pages.

- `setWindowOpenHandler(() => ({ action: 'deny' })))`: the page may never open a new Electron window (`window.open`, or a link with `target="_blank"`). Links open in your browser through `IPC.openExternal` instead.
- `contents.on('will-navigate', (event) => event.preventDefault())`: the page can never be replaced by another page, for example by following a link or by a file dropped onto the window.
- `render-process-gone`: Electron fires this when the page's process crashes or is killed. The page is reloaded.
- `if (view === 'panel') win.setIgnoreMouseEvents(true, { forward: true })`: a crashed panel can't report where the pointer is, and might be left taking the mouse over its whole invisible rectangle. Making it click-through until the page has reloaded avoids a dead patch of screen. The reloaded `useClickThrough` then starts from "not interactive" again.

addEditContextMenu()

src/main/bubble/windows.ts, lines 106–127

```
/** Right-click menu for the text box: spelling suggestions plus the usual edit commands. */
function addEditContextMenu(win: BrowserWindow): void {
  win.webContents.on('context-menu', (_event, params) => {
    if (!params.isEditable) return
    const suggestions: MenuItemConstructorOptions[] = params.dictionarySuggestions
      .slice(0, 5)
      .map((word) => ({ label: word, click: () => win.webContents.replaceMisspelling(word) }))
    const template: MenuItemConstructorOptions[] = [
      ...suggestions,
      ...(suggestions.length > 0 ? [{ type: 'separator' as const }] : []),
      { role: 'undo' },
      { role: 'redo' },
      { type: 'separator' },
      { role: 'cut' },
      { role: 'copy' },
      { role: 'paste' },
      { type: 'separator' },
      { role: 'selectAll' },
    ]
    Menu.buildFromTemplate(template).popup({ window: win })
  })
}
```

The right-click menu for the panel's text box. Electron shows no context menu by default, so without this you couldn't cut, paste or fix spelling with the mouse.

- `if (!params.isEditable) return`: only in editable fields, not on the rest of the panel.
- `params.dictionarySuggestions`: Chromium's spelling suggestions for the word under the pointer (none if it's spelt correctly). At most five are shown; choosing one calls `replaceMisspelling(word)`.
- `{ type: 'separator' as const }`: a separator line, added only when there are suggestions. `as const` tells TypeScript the type is exactly `'separator'`, not just any string, which the menu item type requires.
- `{ role: 'undo' }`: a `role` gives Electron's built-in item, with its standard label, shortcut and behaviour. The same goes for redo, cut, copy, paste and select all.
- `Menu.buildFromTemplate(template).popup({ window: win })`: builds the menu and shows it at the pointer.

The draft, screenshots and safe files

This part covers the main-process code that keeps things on disk. `NotesStore` saves the unsent draft so it survives a restart; `ScreenshotService` and its helpers take, preview and shrink screenshots, and make sure the pages can only touch files in the screenshots folder. Underneath both, `jsonFile.ts` reads saved files with a validity check and writes them so that a crash can't leave a half-written file. All of this runs in the main process (Electron's Node.js side, which can use the disk); the pages reach it only through the IPC handlers in `src/main/ipc.ts`.

`src/main/notes/NotesStore.ts`: saving the draft

The draft is the text typed into the chat box but not yet sent, plus any screenshots attached to it. `NotesStore` holds it in memory and saves it to `notes.json` in the app's data folder (`app.getPath('userData')`, under `%APPDATA%`), so a half-written message is still there after a restart or a crash. `src/main/index.ts` creates one store at startup, loads it, and flushes it when the app quits. `src/main/ipc.ts` calls it whenever the page changes the draft, and clears it when a message is sent.

Key ideas:

- **Debounced autosave.** The page sends the whole text on every keystroke. Writing a file each time would be wasteful, so every change restarts a 500 ms timer, and the file is written only once typing pauses. To "debounce" means to wait until calls stop coming for a moment, then act once.
- **Serialised writes.** Saves wait in a queue (a chain of promises), so two writes to the file never run at the same time.
- **Overtaken writes are dropped.** Each save is numbered. A save that would land after a newer one has already reached the disk throws itself away instead of putting older text back.
- **Validated loading.** The file is checked against a zod schema when it's read. A damaged file is renamed out of the way instead of crashing the app or being silently overwritten.

`MAX_NOTE_LENGTH` and `NotesFileSchema`

`src/main/notes/NotesStore.ts`, lines 8–17

```
export const MAX_NOTE_LENGTH = 200_000

const NotesFileSchema = z.object({
  version: z.literal(1),
  text: z.string(),
  attachments: z.array(
    z.object({ id: z.string(), path: z.string(), fileName: z.string(), addedAt: z.string() }),
  ),
  updatedAt: z.string().nullable(),
})
```

`MAX_NOTE_LENGTH` is a safety limit on the draft's size. `NotesFileSchema` describes exactly what a valid `notes.json` looks like.

- `200_000`: the underscore is only a digit separator, for readability (200,000 characters). `ipc.ts` uses the same constant to reject over-long text in its `notesSetText` and `chatSend` handlers.
- `z.object({`: this is **zod**, a validation library. A schema describes the expected shape; `schema.parse(data)` returns the data if it matches and throws if it doesn't. TypeScript's types disappear when the code runs, so a file

read from disk (which a crash could damage, or anyone could edit) must be checked at run time.

`readJsonFile()` in `jsonFile.ts` does the checking. Unknown extra fields are dropped by `parse`.

- `version: z.literal(1)`: the file records its format version, and only `1` is accepted, so a future format can be told apart. Today a file with any other version counts as invalid and is set aside like a damaged one (see `load()`).
- `updatedAt: z.string().nullable()`: a timestamp string, or `null` if nothing has been typed yet.

emptyNotes and NotesStoreOptions

src/main/notes/NotesStore.ts, lines 19–27

```
export const emptyNotes = (): Notes => ({ text: '', attachments: [], updatedAt: null })

export interface NotesStoreOptions {
  debounceMs?: number
  /** Called when everything changed so far has reached the disk. */
  onSave?: () => void
  onError?: (error: unknown) => void
  now?: () => Date
}
```

`emptyNotes()` makes a blank draft. `NotesStoreOptions` are the optional settings the constructor accepts.

- `emptyNotes = (): Notes =>`: a function rather than one shared constant, so every caller gets its own fresh `attachments` array. `Notes` is the draft type from `src/shared/types.ts`.
- `debounceMs?:` the `?` makes the option optional. It's the pause before saving, 500 ms when left out; the tests set it very short or very long.
- `onSave`: only the tests use it, to know when a save has finished.
- `onError`: `index.ts` passes a function that logs "Saving the text box failed".
- `now`: a way to supply the current time, for testing. When it's left out, as in the app, the real clock is used.

class NotesStore

The class, with the comment that describes it:

src/main/notes/NotesStore.ts, lines 29–33

```
/**
 * The text box's contents. Changes are kept in memory and written to disk shortly after the
 * last edit; writes are serialised, and a write overtaken by a newer one is discarded.
 */
export class NotesStore {
  // ...
}
```

NotesStore fields and constructor

src/main/notes/NotesStore.ts, lines 34–44

```
private notes: Notes = emptyNotes()

private dirty = false

private timer: ReturnType<typeof setTimeout> | null = null

private writes: Promise<void> = Promise.resolve()

private seq = 0

private committedSeq = 0

constructor(
  private readonly filePath: string,
  private readonly options: NotesStoreOptions = {},
) {}
```

The in-memory `notes` is the real draft while the app runs; the file is only a copy. The other fields are bookkeeping for saving.

- `dirty`: true when there are changes that haven't been handed to a write yet.
- `ReturnType<typeof setTimeout>`: "whatever `setTimeout` returns". That's an object in Node and a number in browsers, and this type fits both. `timer` is the pending autosave, or `null`.
- `writes`: the end of the write queue. It starts as an already-finished promise, and each new write is chained on with `.then`, so writes happen strictly one after another.
- `seq`: counts the snapshots handed to a write. `committedSeq` is the number of the newest snapshot actually on disk. Comparing the two is how an overtaken write knows to give up (see `write()`).
- `private readonly filePath: string`: in a constructor, putting `private` or `readonly` before a parameter is TypeScript shorthand that declares a field and stores the argument in it. That's why the constructor body is empty yet the class has `this.filePath` and `this.options`.
- `options: NotesStoreOptions = {}`: `{}` is the default when no options are passed.

NotesStore.load()

src/main/notes/NotesStore.ts, lines 46–56

```
async load(): Promise<Notes> {
  const result = await readJsonFile(this.filePath, NotesFileSchema)
  if (result.status === 'ok') {
    const { text, attachments, updatedAt } = result.value
    this.notes = { text, attachments, updatedAt }
  } else {
    if (result.status === 'invalid') await this.setAsideCorruptFile()
    this.notes = emptyNotes()
  }
  return this.get()
}
```

Reads `notes.json` and makes it the current draft. `index.ts` calls it once at startup, before it registers the IPC handlers.

- `readJsonFile(this.filePath, NotesFileSchema)`: returns one of three results: `ok` (with the checked value), `missing` (no file yet, e.g. on the first run) or `invalid` (not JSON, or the wrong shape). Any other read error, such as permission denied, is thrown to the caller.
- `const { text, attachments, updatedAt } = result.value`: keeps just the draft's fields, leaving the file's `version` behind.
- `this.setAsideCorruptFile()`: a damaged file is renamed, not deleted, so its contents can still be recovered by hand and the next save doesn't overwrite it. A missing file needs nothing; both cases start from an empty draft.

NotesStore.get()

src/main/notes/NotesStore.ts, lines 58–60

```
get(): Notes {
  return { ...this.notes, attachments: this.notes.attachments.map((a) => ({ ...a })) }
}
```

Returns a copy of the draft. `ipc.ts` uses it when sending a message, and `index.ts` includes it in the app state the pages ask for when they load.

- `attachments: this.notes.attachments.map((a) => ({ ...a })): { ...x }` (the spread syntax) copies an object's fields into a new object. Spreading `this.notes` alone would copy only the top level and share the attachments array, so each attachment is copied too. Callers can then change what they got without changing the store (a test checks this).

NotesStore.setText()

src/main/notes/NotesStore.ts, lines 62–65

```
setText(text: string): void {
  const next = text.slice(0, MAX_NOTE_LENGTH)
  if (next !== this.notes.text) this.update({ text: next })
}
```

Replaces the draft's text. `ipc.ts` calls it for the `notesSetText` message, which the page sends on every change in the text box.

- `text.slice(0, MAX_NOTE_LENGTH)`: cuts over-long text. `ipc.ts` already ignores a message longer than this, so this is a second line of defence for any other caller.
- `if (next !== this.notes.text)`: unchanged text does nothing, so it doesn't move `updatedAt` or trigger a pointless save.

NotesStore.addAttachment()

src/main/notes/NotesStore.ts, lines 67–78

```
/** Adds a screenshot to the text box. Returns null if that file is already attached. */
addAttachment(path: string): Notes | null {
  if (this.notes.attachments.some((a) => samePath(a.path, path))) return null
  const attachment = {
    id: randomUUID(),
    path,
    fileName: basename(path),
    addedAt: this.now().toISOString(),
  }
  this.update({ attachments: [...this.notes.attachments, attachment] })
  return this.get()
}
```

Attaches a screenshot file to the draft. `ipc.ts` calls it from the "Attach latest screenshot" handler (`notesAttachLatest`) with the path from `ScreenshotService.latest()`, so the page never supplies the path.

- `samePath(a.path, path)`: refuses a file that's already attached. The `null` lets `ipc.ts` answer `already-attached`, and the page shows "That screenshot is already attached".
- `randomUUID()`: a random unique ID from Node's `crypto` module, used later to remove this attachment.
- `basename(path)`: the file name without its folder, e.g. `Screenshot 2026-10-02 140311.png`. The page turns it into the chip's label.
- `this.now().toISOString()`: a timestamp such as `2026-10-02T03:03:11.000Z` (always in UTC).

NotesStore.removeAttachment() and NotesStore.clear()

src/main/notes/NotesStore.ts, lines 80–88

```
removeAttachment(id: string): Notes {
  this.update({ attachments: this.notes.attachments.filter((a) => a.id !== id) })
  return this.get()
}

clear(): Notes {
  this.update({ text: '', attachments: [] })
  return this.get()
}
```

`removeAttachment()` drops one screenshot by its ID (the × on a chip). `clear()` empties the draft: `ipc.ts` calls it after a message has been handed to `ChatSession.send()`, and returns the empty draft to the page. Both go through the same debounced autosave as any other change, so the saved file is emptied too shortly after a message is sent.

- `filter((a) => a.id !== id)`: an unknown ID removes nothing, but it still counts as a change and schedules a save.

NotesStore.flush()

src/main/notes/NotesStore.ts, lines 90–100

```
/** Writes any pending change now and waits for all writes to finish. */
flush(): Promise<void> {
  this.clearTimer()
  if (this.dirty) {
    this.dirty = false
    const seq = ++this.seq
    const snapshot = this.toFile()
    this.writes = this.writes.then(() => this.write(snapshot, seq))
  }
  return this.writes
}
```

Saves any pending change now instead of waiting for the timer, and returns a promise that settles when every queued write has finished. The autosave timer calls it, and so does `index.ts` when the app is quitting (giving it at most 2 seconds before quitting anyway).

- `this.clearTimer()`: a pending autosave is no longer needed.
- `const seq = ++this.seq`: numbers this snapshot. `++x` adds one first, then gives the new value.
- `const snapshot = this.toFile()`: copies the draft as it is now. Edits made while this write waits in the queue don't change what it saves; they mark the store dirty again and get a write of their own.
- `this.writes = this.writes.then(() => this.write(snapshot, seq))`: adds the write to the end of the queue. It starts only after every earlier write has finished, so writes never overlap.
- `return this.writes`: even when nothing was dirty, the caller waits for any write already in progress. That's what makes awaiting `flush()` on quit mean "everything is on disk".

NotesStore.flushSync()

src/main/notes/NotesStore.ts, lines 102–110

```
/** Blocking write for when the OS is ending the session and async work may never finish. */
flushSync(): void {
  this.clearTimer()
  if (!this.dirty) return
  this.dirty = false
  const seq = ++this.seq
  writeJsonFileSync(this.filePath, this.toFile())
  this.committedSeq = seq
}
```

The emergency version of `flush()`. When Windows logs off or shuts down, Electron fires `session-end` on the window, and `index.ts` calls this. Windows may end the process before any async work completes, so this can't use the queue.

- `writeJsonFileSync(this.filePath, this.toFile())`: a blocking write. The event handler doesn't return until the file is saved.
- `this.committedSeq = seq`: records that this newer snapshot is on disk. An async write still in the queue has a smaller number, so when it reaches its commit check it discards itself instead of putting older text back (the "overtaken write" test checks exactly this).

Unlike `write()`, it doesn't catch errors: a failure is thrown from the event handler.

NotesStore.update()

src/main/notes/NotesStore.ts, lines 112–117

```
private update(patch: Partial<Notes>): void {
  this.notes = { ...this.notes, ...patch, updatedAt: this.now().toISOString() }
  this.dirty = true
  this.clearTimer()
  this.timer = setTimeout(() => void this.flush(), this.options.debounceMs ?? 500)
}
```

The private helper every change goes through. It applies the change, stamps the time and restarts the autosave timer; this is where the debounce happens.

- `Partial<Notes>`: TypeScript for "a `Notes` with every field optional", so callers pass only what changed.
- `{ ...this.notes, ...patch, updatedAt: this.now().toISOString() }`: builds a new draft object. Later fields win, so the values in `patch` replace the old ones.
- `this.clearTimer()`: each change cancels the previous timer before starting a new one. While you keep typing the save keeps being pushed back; 500 ms after the last keystroke the timer finally fires and saves once.
- `() => void this.flush()`: `void` marks the promise as deliberately not awaited. Nothing here could act on the result, and errors are handled inside `write()`.
- `this.options.debounceMs ?? 500`: `??` uses the right-hand value only when the left is `null` or `undefined`, so a caller could still pass `0`.

NotesStore.write()

src/main/notes/NotesStore.ts, lines 119–128

```
private async write(snapshot: object, seq: number): Promise<void> {
  try {
    const committed = await writeJsonFile(this.filePath, snapshot, () => seq > this.committedSeq)
    if (committed) this.committedSeq = seq
    if (!this.dirty) this.options.onSaved?.()
  } catch (error) {
    this.dirty = true // try again on the next change or flush
    this.options.onError?.(error)
  }
}
```

Performs one queued save, through `writeJsonFile()` in `jsonFile.ts`.

That call is an **atomic write**: it doesn't write `notes.json` directly. It writes the whole new content to a temporary file next to it, then renames the temporary file over the real one. A rename is all-or-nothing, so if the app crashes mid-save, `notes.json` holds either the old draft or the new one, never a mix. See `writeFileAtomic()` below.

- `() => seq > this.committedSeq`: the commit check, which `writeJsonFile` calls just before the rename. If a newer snapshot has already reached the disk (from `flushSync()`), this one is out of date and is thrown away.
- `if (committed) this.committedSeq = seq`: records the newest snapshot on disk.
- `if (!this.dirty) this.options.onSaved?.()`: reports "saved" only when no newer change is waiting. `?.()` calls the function only if one was provided.
- `this.dirty = true`: if the save failed (say, the disk is full), the draft is marked unsaved again, so the next change or flush retries with the latest text. No new timer is started.

- `catch (error)`: catching also protects the queue. Because `write()` never rejects, one failed save can't break the promise chain in `writes`; a rejected promise in the chain would make every later `.then` step skip its write.

NotesStore.toFile()

src/main/notes/NotesStore.ts, lines 130–132

```
private toFile() {
  return { version: 1 as const, ...this.notes }
}
```

The object that gets written to disk: the draft plus `version: 1`.

- `1 as const`: tells TypeScript the type is exactly `1`, not any number, matching `z.literal(1)` in `NotesFileSchema`.
- `...this.notes`: only the top level is copied. That's safe because every change replaces `notes` and its attachments array with new values rather than editing them in place, so a snapshot can't change after it's taken.

NotesStore.setAsideCorruptFile()

src/main/notes/NotesStore.ts, lines 134–141

```
private async setAsideCorruptFile(): Promise<void> {
  const backup = `${this.filePath}.corrupt-${this.now().getTime()}`
  try {
    await fs.rename(this.filePath, backup)
  } catch (error) {
    this.options.onError?.(error)
  }
}
```

Renames a damaged `notes.json` to `notes.json.corrupt-<time>`, so it's kept for inspection while the app starts with an empty draft.

- `this.now().getTime()`: the time in milliseconds since 1970, which makes each backup name unique.
- `catch (error)`: if even the rename fails, the error is reported through `onError` and startup carries on. The next save will then overwrite the damaged file.

NotesStore.clearTimer() and NotesStore.now()

src/main/notes/NotesStore.ts, lines 143–150

```
private clearTimer(): void {
  if (this.timer) clearTimeout(this.timer)
  this.timer = null
}

private now(): Date {
  return this.options.now?.() ?? new Date()
}
```

Two small helpers. `clearTimer()` cancels any pending autosave. `now()` takes the time from the `now` option when there is one, and otherwise from the real clock.

samePath

src/main/notes/NotesStore.ts, lines 153–156

```
function samePath(a: string, b: string): boolean {
  const norm = (p: string) => resolve(p).toLowerCase()
  return norm(a) === norm(b)
}
```

Compares two file paths the way Windows does, for `addAttachment()`.

- `resolve(p)`: turns a path into a full, normalised absolute path (working out `.` and `..` parts and using one kind of slash).
- `.toLowerCase()`: Windows file names ignore case, so `C:\Pics\Shot.png` and `c:\pics\shot.png` are the same file.

src/main/screenshots/ScreenshotService.ts: capturing and serving screenshots

Everything the app does with screenshot images goes through this class: taking one, finding the newest, making small previews, preparing one for Claude, and opening one in another app. `index.ts` creates a single instance with the screenshots folder (`Pictures\<appName>`, for example `Pictures\Desktop Assist`) and a function that returns the display the bubble is on. `actions.ts` calls `capture()` for the camera button, and `ipc.ts` exposes the rest to the pages.

Key ideas:

- **Electron's image tools.** `desktopCapturer` grabs the screen, and `nativeImage` loads, resizes and encodes images, so no extra image library is needed. Both are explained where they're first used below.
- **The inside-the-folder check.** Several methods take a file path from a page. The main process doesn't trust what pages send (`ipc.ts` validates every argument for the same reason): if a page were ever compromised, it mustn't be able to make the main process preview, read or open any file on the PC. Opening an `.exe` this way would even run it. So every method that takes a path first checks, with `owns()`, that the file is inside the screenshots folder. How the check works is in `isInsideDir()` in `files.ts`.

Constants

src/main/screenshots/ScreenshotService.ts, lines 7–9

```
const THUMBNAIL_HEIGHT = 112

const MAX_CACHED_THUMBNAILS = 50

const JPEG_QUALITY = 85
```

- `THUMBNAIL_HEIGHT = 112`: a chip's preview is drawn 56 pixels tall. Making the image twice that keeps it sharp on high-DPI screens, where Windows scales the display up.
- `MAX_CACHED_THUMBNAILS = 50`: how many previews to keep in memory.
- `JPEG_QUALITY = 85`: the JPEG quality (0 to 100) for images sent to Claude, a common balance between file size and sharpness.

ScreenshotService fields and constructor

src/main/screenshots/ScreenshotService.ts, lines 12–18

```
private readonly thumbnails = new Map<string, string>()

constructor(
  readonly dir: string,
  /** The display to capture: the one the bubble is on. */
  private readonly targetDisplay: () => Display,
) {}
```

- `new Map<string, string>()`: the preview cache. Each key is a path plus the file's modified time; each value is a preview as a data URL.
- `readonly dir: string`: the screenshots folder. Without `private` it's a public, read-only field (the same constructor shorthand as in `NotesStore`).
- `targetDisplay: () => Display`: a function rather than a fixed value, because the bubble can be dragged to another monitor. `index.ts` passes `screen.getDisplayMatching(bubbleWindow.getBounds())`, the display that most of the bubble window is on. A `Display` is Electron's description of a monitor: its ID, size, position and scale factor.

ScreenshotService.capture()

src/main/screenshots/ScreenshotService.ts, lines 20–33

```
/** Captures the target display at full resolution and saves it. Returns the file path. */
async capture(): Promise<string> {
  const display = this.targetDisplay()
  const thumbnailSize = {
    width: Math.round(display.size.width * display.scaleFactor),
    height: Math.round(display.size.height * display.scaleFactor),
  }
  const sources = await desktopCapturer.getSources({ types: ['screen'], thumbnailSize })
  const source =
    sources.find((s) => s.display_id === String(display.id)) ??
    (sources.length === 1 ? sources[0] : undefined)
  if (!source || source.thumbnail.isEmpty()) throw new Error('Could not capture the screen')
  return saveScreenshot(this.dir, source.thumbnail.toPNG(), new Date())
}
```

Takes a picture of the whole display the bubble is on, saves it as a PNG, and returns the new file's path. The camera button's handler in `actions.ts` runs it inside `BubbleController.isHidden()`, which hides the bubble and the panel first so they aren't in the picture.

desktopCapturer is Electron's way to list the screens and windows that can be captured. It's mostly used to start screen sharing, but every source it returns comes with a still `thumbnail` image. Asking for a thumbnail as big as the screen turns that "thumbnail" into a full-resolution screenshot.

- `display.size.width * display.scaleFactor`: Electron measures displays in device-independent pixels (DIPs), which ignore Windows' display scaling. Multiplying by the scale factor gives the real pixel size: a 3840×2160 screen at 150% scaling is 2560×1440 DIPs with a scale factor of 1.5. `Math.round` keeps the result a whole number.
- `types: ['screen']`: whole screens only, not individual windows.
- `s.display_id === String(display.id)`: picks the source for the bubble's display. A capture source names its display with a string, while `Display.id` is a number.

- `(sources.length === 1 ? sources[0] : undefined)`: Electron documents that `display_id` can be empty when it isn't available. With only one screen there's no doubt which one it is, so it's used anyway; with several, a guess could capture the wrong screen, so it fails instead.
- `source.thumbnail.isEmpty()`: an empty image means the capture itself failed.
- `throw new Error('Could not capture the screen')`: `actions.ts` catches this, logs it, and the page shows "Couldn't take a screenshot".
- `source.thumbnail.toPNG()`: encodes the image as PNG bytes (a Node `Buffer`). PNG is lossless, so the file is exactly what was on screen.
- `saveScreenshot(this.dir, source.thumbnail.toPNG(), new Date())`: from `files.ts`. It names the file after the current local time and never overwrites an existing one.

ScreenshotService.latest()

src/main/screenshots/ScreenshotService.ts, lines 35–37

```
latest(): Promise<string | null> {
  return findLatestScreenshot(this.dir)
}
```

The newest screenshot in the folder, or `null` if there isn't one. The "Attach latest screenshot" handler in `ipc.ts` calls it and passes the result to `NotesStore.addAttachment()`. The work is done by `findLatestScreenshot()` in `files.ts`.

ScreenshotService.thumbnail()

src/main/screenshots/ScreenshotService.ts, lines 39–59

```
/** A small preview as a data URL, or null if the file is missing or not one of ours. */
async thumbnail(path: string): Promise<string | null> {
  if (!this.owns(path)) return null
  let mtimeMs: number
  try {
    mtimeMs = (await fs.stat(path)).mtimeMs
  } catch {
    return null
  }
  const key = `${path}|${mtimeMs}`
  const cached = this.thumbnails.get(key)
  if (cached) return cached
  const image = nativeImage.createFromPath(path)
  if (image.isEmpty()) return null
  const url = image.resize({ height: THUMBNAIL_HEIGHT, quality: 'good' }).toDataURL()
  this.thumbnails.set(key, url)
  if (this.thumbnails.size > MAX_CACHED_THUMBNAILS) {
    this.thumbnails.delete(this.thumbnails.keys().next().value!)
  }
  return url
}
```

Makes a small preview of a screenshot for the page to show, on the chips in the text box and on sent messages. The page asks through `ipc.ts` (`screenshotThumbnail`), from its `useThumbnail` hook.

`nativeImage` is Electron's image class. It can load a PNG or JPEG file, report its size, resize it, and encode it as PNG, JPEG or a data URL.

- `if (!this.owns(path)) return null`: the inside-the-folder check from the overview.
- `(await fs.stat(path)).mtimeMs`: the file's last-modified time. If the file is gone, the method returns `null`, and a draft chip says "File missing" instead of showing a preview.
- `this.thumbnails.get(key)`: the cache key is the path plus that modified time. If the screenshot is edited (cropped in Paint, say), the key changes and a fresh preview is made.
- `nativeImage.createFromPath(path)`: loads and decodes the file. It doesn't throw for a file it can't read; it returns an empty image, hence the `isEmpty()` check.
- `image.resize({ height: THUMBNAIL_HEIGHT, quality: 'good' })`: when only the height is given, the width is scaled to keep the image's shape. 'good' is the fastest of Electron's three quality settings (`good`, `better`, `best`), plenty for a tiny preview.
- `.toDataURL()`: encodes the preview as a PNG inside a `data:image/png;base64,...` string. The page can put that straight into an image's `src`, so it never needs access to the file itself.
- `this.thumbnails.keys().next().value!`: once the cache holds more than 50 previews, the oldest is removed. A `Map` remembers the order entries were added, so its first key is the oldest (first in, first out: reading an entry doesn't move it to the back). The `!` tells TypeScript the value can't be `undefined` here, because the map isn't empty.

ScreenshotService.forClaude()

src/main/screenshots/ScreenshotService.ts, lines 61–74

```
/**
 * A screenshot prepared for Claude: scaled down to a size the model reads well and re-encoded
 * as JPEG, which keeps requests small. Null if the file is missing or not one of ours.
 */
forClaude(path: string): OutgoingImage | null {
  if (!this.owns(path)) return null
  const image = nativeImage.createFromPath(path)
  if (image.isEmpty()) return null
  const original = image.getSize()
  const size = fitWithin(original)
  const scaled =
    size.width === original.width ? image : image.resize({ ...size, quality: 'best' })
  return { mediaType: 'image/jpeg', data: scaled.toJPEG(JPEG_QUALITY).toString('base64') }
}
```

Prepares a screenshot to send to Claude. The `chatSend` handler in `ipc.ts` calls it for each attached screenshot; if any returns `null`, nothing is sent and the page is told `missing-screenshot`. Unlike most methods here it isn't async, because `nativeImage` does its work straight away; the main process waits while each image is decoded and re-encoded.

The image is **resized and re-encoded as JPEG**. A full-screen PNG can be several megabytes, and pixels beyond what Claude uses only make the request bigger and slower (see `imageSize.ts`).

- `image.getSize()`: the image's width and height in pixels.
- `fitWithin(original)`: from `imageSize.ts`. The size to scale to, so the longest side is at most 1,568 pixels.
- `size.width === original.width ? image` :: skips the resize when the image is already small enough. Comparing the width is enough for screen-shaped images, where shrinking always changes both sides.
- `quality: 'best'`: the slowest, sharpest resize, because Claude needs to read small text in the screenshot.
- `scaled.toJPEG(JPEG_QUALITY)`: encodes as JPEG at quality 85. JPEG throws away fine detail the eye barely notices, which keeps the request small.

- `.toString('base64')`: turns the bytes into text. The Claude API takes images as base64 text inside the JSON request; `ChatSession.send()` puts this into an `image` block with the `mediaType` given here.

`ScreenshotService.open()` and `ScreenshotService.openFolder()`

`src/main/screenshots/ScreenshotService.ts`, lines 76–90

```
/** Opens a screenshot in the default image viewer. Returns false if it no longer exists. */
async open(path: string): Promise<boolean> {
  if (!this.owns(path)) return false
  try {
    await fs.access(path)
  } catch {
    return false
  }
  return (await shell.openPath(path)) === ''
}

async openFolder(): Promise<void> {
  await fs.mkdir(this.dir, { recursive: true })
  await shell.openPath(this.dir)
}
```

`open()` shows a screenshot in Windows' default image viewer, when a chip or a sent screenshot is clicked.

`openFolder()` opens the screenshots folder in File Explorer, from the settings menu.

- `fs.access(path)`: checks the file exists, and throws if it doesn't. Returning `false` lets the page show "That screenshot can't be found".
- `shell.openPath(path)`: Electron's "open this file with its default app", like double-clicking it. It resolves to an empty string on success and to an error message otherwise, hence the `=== ''`.
- `fs.mkdir(this.dir, { recursive: true })`: creates the folder if it doesn't exist yet (before the first screenshot), so there's always something to open. `recursive: true` also creates missing parent folders and doesn't fail if the folder already exists.

`openFolder()` takes no path from the page, so it needs no `owns()` check.

`ScreenshotService.owns()`

`src/main/screenshots/ScreenshotService.ts`, lines 92–95

```
/** Renderers may only ask about files inside the screenshots folder. */
private owns(path: string): boolean {
  return isInsideDir(this.dir, path)
}
```

The inside-the-folder check used by `thumbnail()`, `forClaude()` and `open()`. It's a one-line wrapper around `isInsideDir()` in `files.ts`, which explains how the check works.

`src/main/screenshots/files.ts`: naming, saving and finding screenshot files

Plain file helpers with no Electron in them, so they can be tested with ordinary files

(`tests/screenshotFiles.test.ts`). `ScreenshotService` uses them to name and save captures, to find the newest one, and to check that a path is inside the screenshots folder.

screenshotFileName and isScreenshotFileName

src/main/screenshots/files.ts, lines 5–19

```
const SCREENSHOT_NAME = /^Screenshot \d{4}-\d{2}-\d{2} \d{6}(?: \(\d+\))?.png$/

const pad = (n: number) => String(n).padStart(2, '0')

/** "Screenshot 2026-10-02 140311.png", or "... (2).png" for a second capture in the same second. */
export function screenshotFileName(date: Date, copy = 1): string {
  const stamp =
    `${date.getFullYear()}-${pad(date.getMonth() + 1)}-${pad(date.getDate())} ` +
    `${pad(date.getHours())}${pad(date.getMinutes())}${pad(date.getSeconds())}`
  return copy > 1 ? `Screenshot ${stamp} (${copy}).png` : `Screenshot ${stamp}.png`
}

export function isScreenshotFileName(name: string): boolean {
  return SCREENSHOT_NAME.test(name)
}
```

screenshotFileName() builds a name such as Screenshot 2026-10-02 140311.png from a date, in local time. The time is written 140311 rather than 14:03:11 because Windows doesn't allow : in file names.

isScreenshotFileName() checks whether a name has exactly this pattern.

- SCREENSHOT_NAME: a regular expression for our names. \d{4} means four digits, (?: \(\d+\))? an optional copy number such as (2) after a space, and ^ and \$ pin it to the whole name.
- pad: turns 7 into 07, so every date part has a fixed width and the names sort in time order.
- date.getMonth() + 1: JavaScript numbers months from 0.
- copy > 1?: the second capture in the same second becomes ... (2).png, and so on.

saveScreenshot

src/main/screenshots/files.ts, lines 21–34

```
/** Saves a PNG into `dir` without ever overwriting an existing file. Returns the new path. */
export async function saveScreenshot(dir: string, png: Buffer, date: Date): Promise<string> {
  await fs.mkdir(dir, { recursive: true })
  for (let copy = 1; copy <= 100; copy++) {
    const path = join(dir, screenshotFileName(date, copy))
    try {
      await fs.writeFile(path, png, { flag: 'wx' })
      return path
    } catch (err) {
      if (!isErrno(err, 'EEXIST')) throw err
    }
  }
  throw new Error('Too many screenshots with the same timestamp')
}
```

Writes the PNG into the folder under a new name, never replacing an existing file. This isn't an atomic write (see jsonFile.ts): it never replaces a file, so at worst a crash leaves one incomplete new image.

- fs.mkdir(dir, { recursive: true }): creates the folder on the first capture.
- { flag: 'wx' }: "write, but fail if the file already exists". The check and the creation happen in one step in the operating system, so two captures can't both claim the same name.
- isErrno(err, 'EEXIST'): the name is taken, so the loop tries (2), (3) and so on. Any other error (disk full, no permission) is thrown at once.

- `copy <= 100`: a limit, so the loop can't run forever.

findLatestScreenshot

src/main/screenshots/files.ts, lines 36–56

```
/** The most recently written screenshot in `dir`, ignoring files we didn't name. */
export async function findLatestScreenshot(dir: string): Promise<string | null> {
  let names: string[]
  try {
    names = await fs.readdir(dir)
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return null
    throw err
  }
  let latest: { path: string; mtimeMs: number } | null = null
  for (const name of names.filter(isScreenshotFileName)) {
    const path = join(dir, name)
    try {
      const { mtimeMs } = await fs.stat(path)
      if (!latest || mtimeMs > latest.mtimeMs) latest = { path, mtimeMs }
    } catch {
      // Deleted between readdir and stat.
    }
  }
  return latest?.path ?? null
}
```

Returns the path of the most recently written screenshot in the folder, or `null` if there are none.

- `isErrno(err, 'ENOENT')`: `ENOENT` means "no such file or directory". Before the first screenshot the folder doesn't exist, which just means there are no screenshots.
- `names.filter(isScreenshotFileName)`: only files with our names count, so other pictures dropped into the folder are never attached by accident.
- `mtimeMs > latest.mtimeMs`: picks by the time each file was last written, not by its name, so the file written last wins even if its name says otherwise (a test checks this).
- `// Deleted between readdir and stat.`: a file can disappear between listing the folder and looking at it; it's simply skipped.
- `latest?.path ?? null`: `?.` reads `path` only if `latest` isn't `null`.

isInsideDir

src/main/screenshots/files.ts, lines 58–62

```
/** True if `file` is inside `dir` (not `dir` itself). Case-insensitive on Windows. */
export function isInsideDir(dir: string, file: string): boolean {
  const rel = relative(resolve(dir), resolve(file))
  return rel !== '' && rel.split(/[\\\/]/)[0] !== '..' && !isAbsolute(rel)
}
```

The heart of the inside-the-folder check: is `file` really inside `dir` (or one of its subfolders), and not `dir` itself?

A simple test such as "does the file's path start with the folder's path?" isn't safe. `C:\Users\me\Pictures\Desktop Assist\...\Documents\taxes.pdf` starts with the folder's path but points outside it, and `C:\Users\me\Pictures\Desktop Assist Copy\x.png` starts with the same text but is a different folder. This function compares the real path structure instead. It works on the path text alone and doesn't look at the disk.

- `relative(resolve(dir), resolve(file))`: `resolve` turns both into full, normalised paths, so any `..` parts are worked out first. `relative` then gives the route from the folder to the file. On Windows it ignores case, so `C:\USERS\...` still matches.
- `rel !== ''`: an empty route means `file` is the folder itself.
- `rel.split(/[\\\/])/[0] !== '..'`: a route that starts by going up a level leads outside the folder. Only the first part of the route is compared, so a file genuinely named `..notes.png` inside the folder is still allowed.
- `!isAbsolute(rel)`: on Windows a file on another drive has no relative route, so `relative` returns its full path (such as `D:\elsewhere\shot.png`). An absolute result means "not inside".

src/main/screenshots/imageSize.ts: sizing images for Claude

One small calculation, kept apart from `ScreenshotService` so it can be tested without Electron (in `tests/claudeHelpers.test.ts`). `ScreenshotService.forClaude()` uses it to choose the size to scale a screenshot to.

src/main/screenshots/imageSize.ts, lines 1–15

```
import type { Size } from '@shared/geometry'

/** Screenshots sent to Claude are scaled so their longest side is at most this many pixels. */
export const MAX_IMAGE_EDGE = 1568

/** Scales `size` down (never up) so its longest side fits within `maxEdge`, keeping the shape. */
export function fitWithin(size: Size, maxEdge: number = MAX_IMAGE_EDGE): Size {
  const longest = Math.max(size.width, size.height)
  if (longest <= maxEdge) return { ...size }
  const scale = maxEdge / longest
  return {
    width: Math.max(1, Math.round(size.width * scale)),
    height: Math.max(1, Math.round(size.height * scale)),
  }
}
```

- `MAX_IMAGE_EDGE = 1568`: the long-edge limit in Anthropic's guidance for images. Bigger images are scaled down by the API before Claude looks at them, so sending more pixels only makes requests bigger and slower.
- `size: Size`: `Size` is `{ width, height }`, from `src/shared/geometry.ts`.
- `maxEdge: number = MAX_IMAGE_EDGE`: a default parameter. The app always uses the default; the tests pass a value explicitly.
- `return { ...size }`: already small enough, so the size stays the same (returned as a copy). It never scales up.
- `const scale = maxEdge / longest`: one factor for both sides keeps the shape. For example, 1920×1080 becomes 1568×882 .
- `Math.max(1, Math.round(size.width * scale))`: rounding could make a very thin image's short side 0 pixels; this keeps every side at least 1.

src/main/storage/jsonFile.ts: safe files

The low-level helpers for files the app saves: reading a JSON file and checking it with a schema, and writing files atomically. `NotesStore` (the draft), `SettingsService` in `settings.ts` (`preferences.json`) and `SecretStore` (the encrypted sign-in, through `writeFileAtomic`) all use them, and `isErrno` is used across the main process.

Key ideas:

- **Reading never trusts the file.** The text must parse as JSON and match a zod schema, and the caller gets a clear result: `ok`, `missing` or `invalid`.
- **Atomic writes.** Never write over the real file directly. Write the whole content to a temporary file in the same folder, then rename it over the real one. A rename within one drive replaces the file in a single step, so anyone reading the file, or the app after a crash, sees either the complete old version or the complete new one.

ReadResult and readJsonFile

src/main/storage/jsonFile.ts, lines 5–21

```
export type ReadResult<T> =
  { status: 'ok'; value: T } | { status: 'missing' } | { status: 'invalid'; error: unknown }

export async function readJsonFile<T>(path: string, schema: ZodType<T>): Promise<ReadResult<T>> {
  let raw: string
  try {
    raw = await fs.readFile(path, 'utf8')
  } catch (err) {
    if (isErrno(err, 'ENOENT')) return { status: 'missing' }
    throw err
  }
  try {
    return { status: 'ok', value: schema.parse(JSON.parse(raw)) }
  } catch (error) {
    return { status: 'invalid', error }
  }
}
```

`readJsonFile()` reads a JSON file and checks it against a zod schema. `ReadResult` is what it returns.

- `{ status: 'ok'; value: T }`: one of three possible shapes, told apart by `status` (a "discriminated union"). Once the caller has checked `result.status === 'ok'`, TypeScript knows `result.value` exists.
- `ZodType<T>: T` is a generic type parameter: whatever type the schema describes. For `NotesFileSchema`, the value comes back typed as the notes file's shape, with no casts needed.
- `fs.readFile(path, 'utf8')`: reads the whole file as text.
- `isErrno(err, 'ENOENT')`: no file is normal (the first run), so it's `missing`, not an error. Any other error, such as no permission, is thrown.
- `schema.parse(JSON.parse(raw))`: `JSON.parse` throws on text that isn't JSON, and `schema.parse` throws if the JSON has the wrong shape. Both end up as `invalid`, with the error attached. Callers decide what that means: `NotesStore.load()` sets the file aside, and `SettingsService.init()` falls back to default preferences.

tmpPath

src/main/storage/jsonFile.ts, lines 23–24

```
let tmpCounter = 0

const tmpPath = (path: string) => `${path}.${process.pid}.${++tmpCounter}.tmp`
```

Makes a unique temporary file name next to the target, such as `notes.json.12345.3.tmp`.

- `process.pid`: the process ID, so two running copies of the app can't pick the same name.

- `++tmpCounter`: a counter, so two writes in progress at once in this process can't either. That can happen: `SettingsService` doesn't queue its saves the way `NotesStore` does.

The temporary file is in the same folder as the target on purpose: a rename only swaps the file in one step when both names are on the same drive.

writeJsonFile

src/main/storage/jsonFile.ts, lines 26–37

```
/**
 * Writes via a temp file and a rename, so a crash mid-write can't leave a half-written file.
 * 'shouldCommit' runs just before the rename; returning false discards the write (used to drop
 * a write that a newer one has already overtaken). Resolves to whether the file was replaced.
 */
export function writeJsonFile(
  path: string,
  value: unknown,
  shouldCommit: () => boolean = () => true,
): Promise<boolean> {
  return writeFileAtomic(path, JSON.stringify(value, null, 2), shouldCommit)
}
```

Saves a value as JSON, atomically. It's a thin wrapper around `writeFileAtomic()`, and resolves to `true` if the file was replaced or `false` if `shouldCommit` said no.

- `JSON.stringify(value, null, 2)`: turns the value into JSON text, indented by 2 spaces so a person can read the file.
- `shouldCommit: () => boolean = () => true`: an optional last-moment check, passed on to `writeFileAtomic()`. By default every write is kept; `NotesStore.write()` passes one that drops overtaken writes.

writeFileAtomic

src/main/storage/jsonFile.ts, lines 39–54

```
/** The same temp-file-and-rename write for any content (text or binary). */
export async function writeFileAtomic(
  path: string,
  data: string | Buffer,
  shouldCommit: () => boolean = () => true,
): Promise<boolean> {
  await fs.mkdir(dirname(path), { recursive: true })
  const tmp = tmpPath(path)
  await fs.writeFile(tmp, data)
  if (!shouldCommit()) {
    await fs.rm(tmp, { force: true })
    return false
  }
  await renameWithRetry(tmp, path)
  return true
}
```

The atomic write itself, for any content: JSON text or binary data (`SecretStore.save()` writes encrypted bytes with it).

- `fs.mkdir(dirname(path), { recursive: true })`: makes sure the folder exists first.

- `await fs.writeFile(tmp, data)`: the slow part, writing all the bytes, happens on the temporary file. The real file is untouched meanwhile.
- `if (!shouldCommit())`: asked after the temporary file is written and just before the rename, so it sees the latest state. If the answer is no, the temporary file is deleted and the real file stays as it was.
- `fs.rm(tmp, { force: true })`: `force` means "don't fail if it's already gone".
- `renameWithRetry(tmp, path)`: moves the new file into place. Node's `fs.rename` replaces the target if it exists.

If writing or renaming fails, the error goes to the caller and the real file is unchanged, though the temporary file may be left behind.

writeJsonFileSync

src/main/storage/jsonFile.ts, lines 56–61

```
export function writeJsonFileSync(path: string, value: unknown): void {
  mkdirSync(dirname(path), { recursive: true })
  const tmp = tmpPath(path)
  writeFileSync(tmp, JSON.stringify(value, null, 2), 'utf8')
  renameSync(tmp, path)
}
```

The same temp-file-and-rename write, but blocking: each step finishes before the next line runs, and the function returns only when the file is in place. `NotesStore.flushSync()` uses it when Windows is ending the session. It has no `shouldCommit` check and doesn't retry a failed rename.

- `mkdirSync`, `writeFileSync` and `renameSync`: Node's blocking versions of `fs.mkdir`, `fs.writeFile` and `fs.rename`.

renameWithRetry

src/main/storage/jsonFile.ts, lines 63–75

```
// On Windows, antivirus and the search indexer can hold the target open for a moment.
async function renameWithRetry(from: string, to: string, attempts = 5): Promise<void> {
  for (let attempt = 1; ; attempt++) {
    try {
      await fs.rename(from, to)
      return
    } catch (err) {
      const transient = ['EPERM', 'EBUSY', 'EACCES'].some((code) => isErrno(err, code))
      if (!transient || attempt >= attempts) throw err
      await new Promise((resolve) => setTimeout(resolve, 25 * attempt))
    }
  }
}
```

Renames a file, trying again a few times if Windows briefly refuses. As the comment above it says, antivirus software and the search indexer can hold a file open for a moment, and a rename onto it then fails.

- `['EPERM', 'EBUSY', 'EACCES']`: the error codes Windows gives in that case (operation not permitted, busy, access denied). Any other error is thrown at once.
- `for (let attempt = 1; ; attempt++)`: a loop with no end condition. It leaves by `return` (success) or `throw` (giving up after the fifth attempt).

- `new Promise((resolve) => setTimeout(resolve, 25 * attempt))`: the usual way to `await` a pause in JavaScript. The waits grow: 25, 50, 75 and then 100 ms between the five attempts, at most a quarter of a second in all.

isErrno

src/main/storage/jsonFile.ts, lines 77–79

```
export function isErrno(err: unknown, code: string): boolean {
  return typeof err === 'object' && err !== null && (err as NodeJS.ErrnoException).code === code
}
```

Checks whether an error is a Node file-system error with a given code, such as `ENOENT` (not found) or `EEXIST` (already exists). It's used by `readJsonFile()` and `renameWithRetry()` here, by `saveScreenshot()` and `findLatestScreenshot()` in `files.ts`, and by `SecretStore.load()`.

- `err: unknown`: with the project's `strict` TypeScript settings, a caught error has the type `unknown`, because any value can be thrown. It has to be checked before reading `.code`.
- `typeof err === 'object' && err !== null`: `typeof null` is also `'object'`, hence the second test.
- `(err as NodeJS.ErrnoException).code`: once it's known to be an object, it's treated as Node's error type so its `code` can be read. If there's no `code`, the comparison is simply false.

The pages: entry points, styles, hooks and views

This part covers the renderer's own files: the HTML page both windows load, the startup script, the stylesheet, two small helpers, the hooks that connect React to the main process, and the two views (the bubble and the panel). The building blocks the views use are covered in [the components part](#). If React is new to you: a React *component* is a function that returns a description of what to show, written in JSX (HTML-like tags inside TypeScript, in `.tsx` files), and whenever the data it uses changes, React calls it again and updates only the parts of the page that changed.

`src/renderer/index.html`: the page both windows load

The bubble window and the panel window both load this one page. It's almost empty: a `<div>` for React to fill and a script tag that starts the app. The interesting part is the security policy.

`src/renderer/index.html`, lines 1–15

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      http-equiv="Content-Security-Policy"
      content="default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self'
data:; connect-src 'self'; object-src 'none'; base-uri 'none'; form-action 'none'"
    />
    <title>Desktop Assist</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="./src/main.tsx"></script>
  </body>
</html>
```

- `http-equiv="Content-Security-Policy"`: a Content-Security-Policy (CSP) is a set of rules the browser enforces about what the page may load and run. Even if something managed to slip markup into the page (say, a crafted reply from Claude), the browser would refuse to run scripts from anywhere but the app itself. It's a second lock behind the sandbox and context isolation set in `createOverlayWindow` (`src/main/bubble/windows.ts`).
- `default-src 'self'`: anything without its own rule below may only come from the app's own origin: the built files, or the Vite dev server during `npm run dev`.
- `script-src 'self'`: scripts only from the app's own files. Inline `<script>` blocks and `eval` are blocked.
- `style-src 'self' 'unsafe-inline'`: the app's stylesheets plus inline styles. During `npm run dev`, Vite injects the CSS as `<style>` tags, which needs `'unsafe-inline'`.
- `img-src 'self' data:;` images from the app (the tenant's logo) or `data:` URLs, which is how screenshot thumbnails arrive (see `useThumbnail` below).
- `connect-src 'self'`: the page can't `fetch` anything from the internet. Only the main process talks to Claude and JumpCloud.
- `object-src 'none'; base-uri 'none'; form-action 'none'`: no plugins (`<object>`, `<embed>`), no `<base>` tag that could redirect relative links, and no form submissions anywhere.
- `<div id="root"></div>`: the empty element React takes over (see `main.tsx`).

- `<script type="module" src="./src/main.tsx"></script>`: the entry point. Vite compiles `main.tsx` and everything it imports into plain JavaScript and, in the build, rewrites this tag to point at the bundled file.

src/renderer/src/main.tsx: choosing bubble or panel

The first code that runs in each window. It works out which window it's in and renders the matching view.

`BubbleView` and `PanelView` are React components: functions whose names start with a capital letter and return JSX. In JSX, `<BubbleView />` means "render the `BubbleView` component here".

src/renderer/src/main.tsx, lines 1–12

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles.css'
import { BubbleView } from './views/BubbleView'
import { PanelView } from './views/PanelView'

// Both overlay windows load this page; the query string says which one this is.
const view = new URLSearchParams(window.location.search).get('view')

createRoot(document.getElementById('root')!).render(
  <StrictMode>{view === 'bubble' ? <BubbleView /> : <PanelView />}</StrictMode>,
)
```

- `import './styles.css'`: importing a CSS file from code is a Vite feature; it adds the stylesheet (after Tailwind has processed it) to the page.
- `new URLSearchParams(window.location.search).get('view')`: reads the `?view=bubble` or `?view=panel` that `createOverlayWindow` (src/main/bubble/windows.ts) adds when it loads the page. Anything other than `bubble` gets the panel.
- `document.getElementById('root')!`: the `!` tells TypeScript "this is not null"; the div is in `index.html`.
- `createRoot()`: hands that div to React, which from now on owns everything inside it. `render` draws the first screen.
- `<StrictMode>`: a development-only checker. In `npm run dev` it calls each component twice and runs each effect, cleans it up, then runs it again, to expose code that doesn't clean up after itself. That's why every hook below has a careful cleanup. It does nothing in the installed app.
- `{view === 'bubble' ? <BubbleView /> : <PanelView />}`: curly braces switch from JSX back to ordinary TypeScript, here a choice between the two views.

src/renderer/src/env.d.ts: telling TypeScript about window.assist

The preload script puts an `assist` object on `window`, but TypeScript only knows the browser's own `Window` type. This file adds the field, so `window.assist.getState()` and the rest are type-checked in every renderer file. It contains types only and produces no JavaScript.

src/renderer/src/env.d.ts, lines 1–10

```
import type { AssistApi } from '@shared/ipc'

declare global {
  interface Window {
    /** Exposed by the preload script (src/preload/index.ts). */
    assist: AssistApi
  }
}

export {}
```

- `declare global`: adds to the global types instead of declaring something new in this file. `interface Window` merges with the browser's own `Window` interface.
- `assist: AssistApi`: the type is defined in `src/shared/ipc.ts`. The object itself is created by `src/preload/index.ts`, which calls `contextBridge.exposeInMainWorld('assist', api)`.
- `export {}`: makes sure the file counts as a module, which `declare global` requires. The `import type` line already does that, so this is a safety net.

src/renderer/src/styles.css: Tailwind setup and the few hand-written styles

Tailwind in brief. Tailwind CSS provides small, single-purpose class names: `flex`, `rounded-full`, `px-3` (horizontal padding), `bg-white`, `text-xs`. Components style themselves by listing these in `className` instead of writing a separate stylesheet. A prefix applies a class only in some situation: `hover`: while the pointer is over it, `dark`: when Windows is in dark mode, `motion-reduce`: when Windows animations are turned off, and `group-data-open`: when a parent marked `group` has a `data-open` attribute (used by the panel). At build time Tailwind scans the source for the class names actually used and generates CSS for just those. This file sets Tailwind up and holds the styles that classes can't express.

src/renderer/src/styles.css, lines 1–231

```
@import 'tailwindcss';

@theme {
  /* Overridden at runtime with the tenant's colour (see useAccentColor). Every highlight uses it. */
  --color-accent: #9333ea;
  /* A lighter accent, for accent-coloured text on dark backgrounds. */
  --color-accent-soft: color-mix(in srgb, var(--color-accent) 60%, white);
  --font-sans: 'Segoe UI Variable Text', 'Segoe UI', system-ui, sans-serif;
}

@layer base {
  html,
  body,
  #root {
    width: 100%;
    height: 100%;
    margin: 0;
    overflow: hidden;
    background: transparent;
  }

  body {
    font-family: var(--font-sans);
    user-select: none;
    -webkit-font-smoothing: antialiased;
  }
}
```

```

}

textarea {
  user-select: text;
}

/* Selected text uses the accent instead of the system blue. */
::selection {
  background-color: color-mix(in srgb, var(--color-accent) 25%, transparent);
}

/* Keyboard focus too; Chromium's default outline follows the Windows accent colour. */
:focus-visible {
  outline: 2px solid var(--color-accent);
  outline-offset: 2px;
}

/* Slim scrollbars suit the small panel. */
* {
  scrollbar-width: thin;
  scrollbar-color: rgb(0 0 0 / 0.2) transparent;
}
@media (prefers-color-scheme: dark) {
  * {
    scrollbar-color: rgb(255 255 255 / 0.2) transparent;
  }
}
}

/* Claude's replies (rendered by components/Markdown.tsx). Sized for a narrow chat panel. */
@layer components {
  .markdown {
    user-select: text;
    line-height: 1.55;
    overflow-wrap: anywhere;
  }
  .markdown > :first-child {
    margin-top: 0;
  }
  .markdown > :last-child {
    margin-bottom: 0;
  }
  .markdown :where(p, ul, ol, pre, table, blockquote) {
    margin: 0.5em 0;
  }
  .markdown :where(h1, h2, h3, h4) {
    margin: 0.8em 0 0.3em;
    font-weight: 600;
    line-height: 1.3;
  }
  .markdown h1 {
    font-size: 1.15em;
  }
  .markdown h2 {
    font-size: 1.08em;
  }
  .markdown :where(h3, h4) {
    font-size: 1em;
  }
  .markdown ul {
    list-style: disc;
    padding-left: 1.25em;
  }
  .markdown ol {
    list-style: decimal;

```

```

padding-left: 1.4em;
}
.markdown li + li {
margin-top: 0.15em;
}
.markdown a {
color: var(--color-accent);
text-decoration: underline;
cursor: pointer;
}
.markdown blockquote {
border-left: 3px solid var(--color-zinc-300);
padding-left: 0.75em;
color: var(--color-zinc-500);
}
.markdown :not(pre) > code {
border-radius: 4px;
background: var(--color-zinc-100);
padding: 0.1em 0.3em;
font-size: 0.9em;
}
.markdown pre {
overflow-x: auto;
border-radius: 8px;
background: var(--color-zinc-100);
padding: 0.6em 0.75em;
font-size: 0.85em;
line-height: 1.45;
}
.markdown code {
font-family: 'Cascadia Code', Consolas, monospace;
}
.markdown table {
display: block;
overflow-x: auto;
border-collapse: collapse;
font-size: 0.9em;
}
.markdown :where(th, td) {
border: 1px solid var(--color-zinc-200);
padding: 0.25em 0.5em;
text-align: left;
}
.markdown hr {
margin: 0.8em 0;
border-color: var(--color-zinc-200);
}

@media (prefers-color-scheme: dark) {
.markdown a {
color: var(--color-accent-soft);
}
.markdown blockquote {
border-color: var(--color-zinc-600);
color: var(--color-zinc-400);
}
.markdown :not(pre) > code,
.markdown pre {
background: var(--color-zinc-800);
}
.markdown :where(th, td),
.markdown hr {
border-color: var(--color-zinc-700);
}
}

```

```

/* Code highlighting (highlight.js classes), light then dark. Purple in place of the usual
   blues, with names in orange so every kind of token still stands apart. */
.hljs-comment,
.hljs-quote {
  color: #6e7781;
  font-style: italic;
}
.hljs-keyword,
.hljs-selector-tag,
.hljs-built_in,
.hljs-literal {
  color: #cf222e;
}
.hljs-string,
.hljs-regexp,
.hljs-addition {
  color: #b21a8b;
}
.hljs-number,
.hljs-attr,
.hljs-attribute,
.hljs-variable {
  color: #9333ea;
}
.hljs-title,
.hljs-section,
.hljs-function {
  color: #953800;
}
.hljs-type,
.hljs-name,
.hljs-tag {
  color: #116329;
}
.hljs-deletion {
  color: #82071e;
}

@media (prefers-color-scheme: dark) {
  .hljs-comment,
  .hljs-quote {
    color: #8b949e;
  }
  .hljs-keyword,
  .hljs-selector-tag,
  .hljs-built_in,
  .hljs-literal {
    color: #ff7b72;
  }
  .hljs-string,
  .hljs-regexp,
  .hljs-addition {
    color: #e9d5ff;
  }
  .hljs-number,
  .hljs-attr,
  .hljs-attribute,
  .hljs-variable {
    color: #c084fc;
  }
  .hljs-title,
  .hljs-section,
  .hljs-function {
    color: #ffa657;
  }
}

```

```

    }
    .hljs-type,
    .hljs-name,
    .hljs-tag {
      color: #7ee787;
    }
    .hljs-deletion {
      color: #ffa198;
    }
  }
}

```

- `@import 'tailwindcss'`: brings in Tailwind: its reset of browser default styles ("Preflight", which among other things removes list bullets, margins and heading sizes), its default theme (colours such as `zinc`, spacing, fonts) and the generated utility classes.
- `@theme`: adds to Tailwind's theme. Each `--color-*` variable becomes a colour every utility can use, so `--color-accent` gives `bg-accent`, `text-accent`, `ring-accent` and so on. Those classes read the CSS variable when the page is drawn, not at build time.
- `--color-accent: #9333ea`: the default purple. `useAccentColor` (below) overwrites this variable with the tenant's colour once the app state arrives, which recolours every accent use at once.
- `--color-accent-soft: color-mix(in srgb, var(--color-accent) 60%, white)`: 60% accent mixed with white, for accent-coloured text on dark backgrounds (`dark:text-accent-soft`). Because it refers to `--color-accent`, it follows the tenant's colour too.
- `--font-sans`: the Windows 11 system font, falling back to older Segoe UI and then any system font.
- `@layer base`: page-wide defaults. Tailwind orders its CSS in layers (theme, base, components, utilities) and a later layer wins, so a utility class on an element always beats these rules.
- `background: transparent`: both windows are transparent (`transparent: true` in `createOverlayWindow`), so the page must not paint a background, or the see-through areas would be white. `overflow: hidden` stops the page itself ever showing scrollbars.
- `user-select: none`: the UI shouldn't highlight like a web page when you click or drag around. `textarea` and `.markdown` turn selection back on, so the text box and Claude's replies can still be selected and copied.
- `::selection`: selected text gets a 25% tint of the accent instead of the Windows blue.
- `:focus-visible`: the outline drawn around whatever has keyboard focus, in the accent colour. `:focus-visible` only matches focus that came from the keyboard, so mouse clicks don't draw it.
- `scrollbar-width: thin`: slim scrollbars (the conversation scrolls), with a lighter thumb in dark mode.
- `@layer components`: the styles for Claude's replies, which `Markdown` wraps in `<div className="markdown">` (see [the components part](#)). The Markdown library produces plain `<p>`, `<ul>`, `<pre>` and similar elements that can't carry Tailwind classes, so they're styled here by element.
- `overflow-wrap: anywhere`: long words and URLs break instead of overflowing the 400-pixel card.
- `.markdown > :first-child`: with the matching `:last-child` rule, removes the gap above the first block and below the last, so a reply has no extra space at its top and bottom.
- `.markdown :where(p, ul, ol, pre, table, blockquote): :where()` groups selectors without adding specificity, so these rules stay easy to override.
- `list-style: disc`: puts back the bullets (and, for `ol`, the numbers) that Preflight removes.
- `.markdown :not(pre) > code`: inline code gets a small grey pill. Code blocks (`.markdown pre`) get a grey box that scrolls sideways instead of wrapping.
- `.markdown table: display: block` with `overflow-x: auto` lets a wide table scroll inside the card.

- `@media (prefers-color-scheme: dark)`: the dark-mode colours for the same elements. This media query follows the Windows light/dark setting.
- `.hljs-comment`: the start of the code-highlighting colours. `rehype-highlight` (used by `Markdown`) wraps each token of a code block in a `<span>` with a highlight.js class such as `hljs-keyword` or `hljs-string`; these rules colour them. The palette is GitHub's, light then dark, with purple in place of the usual blues.

src/renderer/src/lib/cn.ts: joining class names

cn

src/renderer/src/lib/cn.ts, lines 1–4

```
/** Joins class names, skipping falsy ones. */
export function cn(...classes: Array<string | false | null | undefined>): string {
  return classes.filter(Boolean).join(' ')
}
```

Builds a `className` string from several pieces, some of them conditional: `cn('rounded-full', open && 'ring-accent')` gives `'rounded-full ring-accent'` when `open` is true and `'rounded-full'` otherwise. Many React projects use the `clsx` library for this; four lines do the job here.

- `...classes: Array<string | false | null | undefined>`: a rest parameter collects any number of arguments into an array. `false` is allowed so that `condition && 'class'` can be passed directly.
- `classes.filter(Boolean)`: keeps only truthy values, dropping `false`, `null`, `undefined` and empty strings.

src/renderer/src/lib/anchor.ts: positioning from the bubble's corner

The panel window is placed so that one of its corners sits on the bubble's corner of the screen (`panelWindowBounds` in `src/main/bubble/layout.ts`), and everything inside it is positioned from that corner. These helpers turn "this far from the bubble's corner" into CSS for whichever corner the bubble is in. That's what makes the whole panel mirror itself when the bubble is dragged to another corner.

anchored

src/renderer/src/lib/anchor.ts, lines 4–14

```
/**
 * Positions an element inside the panel window measured from the bubble's corner: `x` across
 * and `y` away from the screen edge. In the bottom-right corner that's `right`/`bottom`; in the
 * top-left it's `left`/`top`, and so on, so the panel mirrors itself to open into the screen.
 */
export function anchored(corner: Corner, x: number, y: number): CSSProperties {
  return {
    [isLeftCorner(corner) ? 'left' : 'right']: x,
    [isTopCorner(corner) ? 'top' : 'bottom']: y,
  }
}
```

Returns a `style` object that places an element `x` pixels across and `y` pixels away from the screen edge, measured from the bubble's corner. `ChatBox`, `ActionStack` and `SettingsMenu` call it with the offsets in `PANEL_LAYOUT` (`src/shared/geometry.ts`), the same numbers the main process uses to size the window.

- `CSSProperties`: React's type for the `style` prop. Plain numbers in it mean pixels.
- `[isLeftCorner(corner) ? 'left' : 'right']`: `x`: a computed property name, where the key is chosen at runtime. In a left corner the element is placed `x` pixels from the left edge, otherwise from the right.
- `[isTopCorner(corner) ? 'top' : 'bottom']`: `y`: the same, vertically. In a bottom corner, `y` is measured up from the bottom.

originClass

src/renderer/src/lib/anchor.ts, lines 16–24

```
/** Tailwind transform-origin class for animations that should grow out of the bubble's corner. */
export function originClass(corner: Corner): string {
  return {
    'top-left': 'origin-top-left',
    'top-right': 'origin-top-right',
    'bottom-left': 'origin-bottom-left',
    'bottom-right': 'origin-bottom-right',
  }[corner]
}
```

The Tailwind `origin-*` class that sets `transform-origin` to the bubble's corner. `ChatBox` opens by scaling from 95% to 100% size; with the origin at the bubble's corner, it appears to grow out of the bubble.

- `'top-left': 'origin-top-left'`: a lookup table instead of building the name as `'origin-' + corner`. Tailwind finds classes by scanning the source text, so every class name must appear in full somewhere or its CSS is never generated.
- `{[corner]}`: picks the entry for this corner. TypeScript checks the table has all four corners.

src/renderer/src/hooks/useAssistState.ts: the app state, kept live

Hooks in brief. Functions whose names start with `use` are *hooks*. A component calls them at the top of its body, in the same order on every render.

- **State:** `useState(initial)` gives a value that survives between renders, and a setter. Calling the setter stores a new value and makes React render the component again.
- **Effects:** `useEffect(fn, deps)` runs `fn` after React has updated the page. It's the place for work outside React, such as subscribing to IPC events or adding a `window` listener. If `fn` returns a function, React calls it to clean up before running the effect again and when the component goes away. `deps` lists the values the effect uses: it re-runs when one of them changes, and `[]` means "only when the component first appears".
- **Custom hooks:** an ordinary function that calls other hooks, so several components can share the same stateful logic. Each component that calls it gets its own separate copy of the state.

This file's hooks are used by both views. Each window is a separate page, so each keeps its own copy of the state.

useAssistState

src/renderer/src/hooks/useAssistState.ts, lines 4–38

```
/**
 * The app state, kept live for the parts the main process changes on its own: the mode, the
 * bubble's corner, the sign-in status and the chat. (Notes and settings only change through this
 * window's own requests, so the panel tracks their results itself.)
 *
 * Subscriptions start before the state is fetched. Events that arrive before the reply are
 * already reflected in it; events after it are applied on top.
 */
export function useAssistState(): AppState | null {
  const [state, setState] = useState<AppState | null>(null)

  useEffect(() => {
    let alive = true
    const update = (change: (s: AppState) => AppState) =>
      setState((current) => (current ? change(current) : current))
    const unsubscribe = [
      window.assist.onModeChanged((mode) => update((s) => ({ ...s, mode }))),
      window.assist.onCornerChanged((corner) => update((s) => ({ ...s, corner }))),
      window.assist.onAuthStatus((auth) => update((s) => ({ ...s, auth }))),
      window.assist.onChatMessage((message) =>
        update((s) => ({ ...s, chat: upsert(s.chat, message) })),
      ),
      window.assist.onChatReset(() => update((s) => ({ ...s, chat: [] }))),
    ]
    void window.assist.getState().then((initial) => {
      if (alive) setState(initial)
    })
    return () => {
      alive = false
      for (const off of unsubscribe) off()
    }
  }, [])

  return state
}
```

Fetches the whole app state (`AppState` in `src/shared/types.ts`: mode, corner, draft, settings, branding, version, sign-in status and chat) when the page loads, then applies the changes the main process broadcasts. It returns `null` until the first reply arrives.

The order matters. The listeners are attached before `getState` is called, and the main process builds the reply in one go. Messages from the main process reach a window in the order they were sent, so a change broadcast before the reply was built is already included in the reply, and any later one arrives after it. Nothing is missed and nothing is applied twice.

- `useState<AppState | null>(null)`: the `<AppState | null>` tells TypeScript what the state can hold.
- `let alive = true`: set to `false` by the cleanup, so a reply that arrives after the component has gone is ignored. (In development, `StrictMode` unmounts and remounts once, so this really happens.)
- `setState((current) => (current ? change(current) : current))`: passing a function to the setter means "compute the new state from the latest one". The effect runs only once, so the `state` variable it could see would be the `null` from the first render forever. A change that arrives before the first state is dropped (`current` is still `null`), which is safe for the reason above.
- `({ ...s, mode })`: a copy of the state with one field replaced. React notices a change by getting a new object, so state is always replaced, never edited in place.

- `window.assist.onModeChanged()`: each `on...` function listens for one main-process broadcast (sent to both windows by `broadcast` in `src/main/index.ts`) and returns a function that stops listening (`unsubscribe` in `src/preload/index.ts`).
- `upsert(s.chat, message)`: a streaming reply arrives many times with the same id and growing text (from `ChatSession`, at most every 50 ms), so it replaces the old copy rather than being added again.
- `window.assist.onChatReset(() => update((s) => ({ ...s, chat: [] })))`: New conversation, or Log out, cleared the chat.
- `void window.assist.getState().then()`: asks for the full state, which `getState` in `src/main/index.ts` assembles. `void` marks a promise the code deliberately doesn't wait for.
- `return () => {}`: the cleanup: ignore a late reply and unsubscribe everything.

The draft (`notes`) and `settings` aren't kept live here: only this window's own requests change them, so `Panel` tracks their results itself.

upsert

`src/renderer/src/hooks/useAssistState.ts`, lines 40–46

```
function upsert(messages: ChatMessage[], message: ChatMessage): ChatMessage[] {
  const index = messages.findIndex((m) => m.id === message.id)
  if (index === -1) return [...messages, message]
  const next = messages.slice()
  next[index] = message
  return next
}
```

Replaces the message with the same `id`, or adds it at the end if it's new. `messages.slice()` copies the array first, so the old array React is holding is left unchanged.

useAccentColor

`src/renderer/src/hooks/useAssistState.ts`, lines 48–53

```
/** Applies the tenant's accent colour to the `accent` Tailwind colour. */
export function useAccentColor(color: string | undefined): void {
  useEffect(() => {
    if (color) document.documentElement.style.setProperty('--color-accent', color)
  }, [color])
}
```

Applies the tenant's accent colour (`accentColor` in `tenant.json`) to the page by overwriting the `--color-accent` variable from `styles.css`.

- `document.documentElement.style.setProperty('--color-accent', color)`: sets the variable as an inline style on `<html>`, the element Tailwind declares its theme variables on, and an inline style beats the stylesheet. `--color-accent-soft` is declared on the same element and recomputes from the new value.
- `if (color)`: while the state is loading, `color` is `undefined` and the default purple stays.
- `[color]`: the effect re-runs only if the colour changes.

src/renderer/src/hooks/useClickThrough.ts: the panel's half of click-through

The panel window is a large rectangle that's mostly transparent. So that clicks on its empty areas reach whatever is underneath, the main process makes the window ignore the mouse, while still passing pointer movements to the page. This hook watches those movements and asks the main process to switch mouse input on only while the pointer is over real UI, which is any element marked `data-hit` (the chat card, each action icon and the settings menu). See [Code Guide section 1](#) for the background. Only `PanelView` uses it; the bubble window always takes the mouse.

useClickThrough

src/renderer/src/hooks/useClickThrough.ts, lines 3–61

```
/**
 * Makes the panel window's see-through areas pass clicks to whatever is underneath. The main
 * process keeps the window ignoring the mouse (while still forwarding pointer moves to it), and
 * this hook switches mouse input on only while the pointer is over an element marked `data-hit`.
 *
 * There's deliberately no "pointer left the window" handling: switching mouse input on makes
 * Windows send a stale leave message, and reacting to it left the window click-through while the
 * pointer sat on real UI. A window with input on gets a real move as soon as the pointer is over
 * an empty area again, which switches it back off.
 */
export function useClickThrough(): void {
  useEffect(() => {
    // null = unknown (after a reset): the next decision is always sent.
    let interactive: boolean | null = false
    let pointer: { x: number; y: number } | null = null

    const set = (next: boolean) => {
      if (next === interactive) return
      interactive = next
      window.assist.setInteractive(next)
    }

    const hitAt = (x: number, y: number) =>
      document.elementFromPoint(x, y)?.closest('[data-hit]') != null

    const onMove = (event: MouseEvent) => {
      pointer = { x: event.clientX, y: event.clientY }
      // While a button is held (selecting text, say) keep mouse input on, so the window doesn't
      // lose the release when the pointer briefly runs outside it.
      if (event.buttons !== 0) return
      set(hitAt(pointer.x, pointer.y))
    }

    // Content can change under a still pointer (a menu closes, the chat grows).
    const recheck = () =>
      requestAnimationFrame(() => {
        if (pointer) set(hitAt(pointer.x, pointer.y))
      })

    // The panel was just shown: the main process has made it click-through and says where the
    // pointer is. Decide once the page has rendered its open state.
    const offReset = window.assist.onClickThroughReset((point) => {
      interactive = null
      pointer = point
      requestAnimationFrame(() => {
        if (pointer) set(hitAt(pointer.x, pointer.y))
      })
    })

    // After a reload the main process may still have input switched on; resync.
  })
}
```

```

window.assist.setInteractive(false)
window.addEventListener('mousemove', onMove)
window.addEventListener('mouseup', recheck)
return () => {
  offReset()
  window.removeEventListener('mousemove', onMove)
  window.removeEventListener('mouseup', recheck)
  set(false)
}
}, [])
}

```

- `let interactive: boolean | null = false`: what the page last told the main process. This and `pointer` are plain variables inside the effect, not React state: they change on every mouse move and must not cause re-renders. They live as long as the effect does.
- `if (next === interactive) return`: sends a message only when the answer changes, not on every mouse move.
- `window.assist.setInteractive(next)`: the main process's handler (`src/main/ipc.ts`) calls `setIgnoreMouseEvents(!interactive, { forward: true })` on this window. `forward: true` keeps mouse moves coming to the page even while clicks pass through, which is what lets this hook keep watching.
- `document.elementFromPoint(x, y)?.closest('[data-hit]') !== null`: finds the topmost element under the pointer, then checks it and its ancestors for `data-hit`. `elementFromPoint` skips elements with `pointer-events: none`, so while the panel is closed (`Panel` adds `pointer-events-none`) nothing counts as a hit.
- `if (event.buttons !== 0) return`: while a mouse button is held (selecting text, say), input stays on, so the release isn't lost if the pointer briefly runs outside the UI.
- `const recheck = () =>`: runs after each mouse button release. The click may have changed what's under a pointer that hasn't moved (a menu closed, the chat grew), so the answer is worked out again.
- `requestAnimationFrame()`: runs the callback just before the browser draws the next frame, by which time React has applied the change the click caused.
- `window.assist.onClickThroughReset((point) => {`: each time the panel is shown, `resetClickThrough` (`src/main/bubble/windows.ts`) makes the window ignore the mouse again and sends where the pointer is inside the window. The new mode is broadcast just before the window is shown (`BubbleController.expand`), so waiting a frame lets the page draw its open state first.
- `interactive = null`: the main process has just switched input off, so what the page last sent may no longer be true. `null` equals neither `true` nor `false`, so the next `set` always sends.
- `window.assist.setInteractive(false)`: after a reload (the page crashed and `lockDown` in `src/main/bubble/windows.ts` reloaded it, or a development reload) the main process may still have input switched on for the old page.
- `set(false)`: in the cleanup, leaves the window click-through when the hook goes away.

The doc comment explains the one thing deliberately missing: a "pointer left the window" handler.

`src/renderer/src/hooks/useToast.ts`: short messages

A toast is a small message that appears briefly and goes away by itself, such as "Copied" or "Screenshot saved". `Panel` owns the toast; `ChatBox` draws it just outside the chat card.

ToastMessage, TOAST_MS

src/renderer/src/hooks/useToast.ts, lines 3–9

```
export interface ToastMessage {
  id: number
  text: string
  tone: 'info' | 'error'
}

const TOAST_MS = 2600
```

One toast, and how long it stays (2.6 seconds). `tone` picks its colours in `ChatBox`: a dark pill for `info`, red for `error`.

useToast

src/renderer/src/hooks/useToast.ts, lines 11–29

```
/** One short-lived message at a time; a new one replaces the old. */
export function useToast(): [
  ToastMessage | null,
  (text: string, tone?: ToastMessage['tone']) => void,
] {
  const [toast, setToast] = useState<ToastMessage | null>(null)

  useEffect(() => {
    if (!toast) return
    const timer = setTimeout(() => setToast(null), TOAST_MS)
    return () => clearTimeout(timer)
  }, [toast])

  const show = useCallback((text: string, tone: ToastMessage['tone'] = 'info') => {
    setToast({ id: Date.now(), text, tone })
  }, [])

  return [toast, show]
}
```

Returns the current toast (or `null`) and a function to show a new one, as a pair, the same shape `useState` returns. A new toast replaces the old one.

- `ToastMessage['tone']`: "the type of `ToastMessage`'s `tone` field", so `'info' | 'error'`.
- `if (!toast) return`: nothing to time when no toast is showing.
- `return () => clearTimeout(timer)`: when a new toast replaces the old one, React runs this cleanup before starting the new timer. The old timer can't clear the new toast early, and the new one gets its full 2.6 seconds.
- `useCallback`: returns the same function on every render, since its dependency list `[]` never changes. Without it, `show` would be a new function each time, which matters when it's passed to other components or listed in an effect's dependencies.
- `id: Date.now()`: every call creates a new object, even for the same text, so the effect restarts the timer. `ChatBox` uses the `id` as the toast's React `key`, so a repeated message is drawn as a fresh element.

src/renderer/src/hooks/useThumbnail.ts: loading a screenshot preview

useThumbnail

src/renderer/src/hooks/useThumbnail.ts, lines 3–18

```
/** A screenshot preview: undefined while loading, null if the file is missing. */
export function useThumbnail(path: string): string | null | undefined {
  const [result, setResult] = useState<{ path: string; url: string | null } | null>(null)

  useEffect(() => {
    let alive = true
    void window.assist.screenshots.thumbnail(path).then((url) => {
      if (alive) setResult({ path, url })
    })
    return () => {
      alive = false
    }
  }, [path])

  return result?.path === path ? result.url : undefined
}
```

Asks the main process for a small preview of a screenshot. It returns three kinds of value: `undefined` while loading, `null` if the file is missing, or the image as a `data: URL`. `AttachmentChip` uses the difference to show "File missing" only when the file really is gone; the sent-screenshot thumbnails in `MessageList` use it too (see [the components part](#)).

- `window.assist.screenshots.thumbnail(path)`: `ScreenshotService.thumbnail` in the main process returns a cached, shrunk copy of the image, or `null` if the file is gone or isn't inside the screenshots folder. `data:` images are allowed by the `img-src` rule in `index.html`.
- `useState<{ path: string; url: string | null } | null>(null)`: the result remembers which path it belongs to.
- `[path]`: fetch again whenever the path changes.
- `if (alive) setResult({ path, url })`: ignore a reply for a path that's no longer wanted (the component moved on to another screenshot, or went away).
- `return result?.path === path ? result.url : undefined`: while a new path loads, the previous picture isn't shown for it. Comparing paths here means the effect doesn't have to clear the old result itself.

src/renderer/src/views/BubbleView.tsx: the logo bubble

The page shown in the small bubble window: one round button with the tenant's logo. A click asks the main process what to do; pressing and moving drags the bubble.

logos, logoUrl

src/renderers/src/views/BubbleView.tsx, lines 7–14

```
// The tenant's logo: logo.png, or logo.svg (if a tenant has both, the PNG wins).
const logos = import.meta.glob<string>('@tenant/logo.{png,svg}', {
  eager: true,
  query: '?url',
  import: 'default',
})

const logoUrl =
  Object.entries(logos).find(([path]) => path.endsWith('.png'))?.[1] ?? Object.values(logos)[0]
```

Finds the tenant's logo at build time.

- `import.meta.glob<string>('@tenant/logo.{png,svg}', {`: a Vite feature that imports every file matching a pattern. `@tenant` is the build's tenant folder (set up in `electron.vite.config.ts`), so this matches `logo.png` and/or `logo.svg`. The result is an object from file path to import.
- `eager: true`: import the files straight away, rather than as functions that load them later.
- `query: '?url'`: import each file as its URL (Vite copies the image into the build).
- `import: 'default'`: take each module's default export, which is that URL.
- `Object.entries(logos).find(([path]) => path.endsWith('.png'))?.[1]`: the PNG's URL if there is one.
- `?? Object.values(logos)[0]`: otherwise the only other match, the SVG. Every tenant must have a logo; `tests/tenants.test.ts` checks.

ACTION_LABEL, DRAG_THRESHOLD

src/renderers/src/views/BubbleView.tsx, lines 16–26

```
const ACTION_LABEL: Record<Mode, string> = {
  collapsed: 'Open',
  expanded: 'Close',
  dragging: 'Moving',
  bouncing: 'Stop bouncing',
  returning: 'Returning',
  capturing: 'Taking screenshot',
}

/** Moving the pointer further than this while pressed turns a click into a drag. */
const DRAG_THRESHOLD = 5
```

- `Record<Mode, string>`: a label for every `Mode` (`src/shared/types.ts`). TypeScript reports an error if a mode is ever added without one. The label becomes the button's accessible name for screen readers, such as "Open Desktop Assist".
- `DRAG_THRESHOLD = 5`: a press that moves no more than 5 pixels is still a click, so a slightly shaky click doesn't start a drag.

<BubbleView>

src/renderers/src/views/BubbleView.tsx, lines 28–104

```
/**
 * The always-on-top logo bubble. A click goes to the main process, which decides what it means.
 * Press and move to drag it: the main process makes it follow the mouse, and on release snaps it
```

```

* to the nearest corner of whichever display it's on.
*/
export function BubbleView() {
  // No useClickThrough here: the bubble window always takes the mouse (see bubble/windows.ts).
  const state = useAssistState()
  useAccentColor(state?.branding.accentColor)
  const mode = state?.mode ?? 'collapsed'
  const appName = state?.branding.appName ?? 'Desktop Assist'

  const press = useRef<{ x: number; y: number; dragging: boolean } | null>(null)
  // A drag ends with a click event on the bubble; this stops it counting as a click.
  const swallowClick = useRef(false)

  function onPointerDown(event: PointerEvent<HTMLButtonElement>) {
    if (event.button !== 0) return
    swallowClick.current = false
    event.currentTarget.setPointerCapture(event.pointerId)
    press.current = { x: event.screenX, y: event.screenY, dragging: false }
  }

  function onPointerMove(event: PointerEvent<HTMLButtonElement>) {
    const current = press.current
    if (!current || current.dragging) return
    // Screen coordinates, because the window itself moves once the drag starts.
    if (Math.hypot(event.screenX - current.x, event.screenY - current.y) > DRAG_THRESHOLD) {
      current.dragging = true
      window.assist.bubbleDragStart()
    }
  }

  function endPress() {
    const current = press.current
    press.current = null
    if (current?.dragging) {
      swallowClick.current = true
      window.assist.bubbleDragEnd()
    }
  }

  return (
    <div className="flex h-full w-full items-center justify-center">
      <button
        data-hit
        type="button"
        aria-label={`${ACTION_LABEL[mode]} ${appName}`}
        title={mode === 'collapsed' ? 'Click to open, drag to move' : undefined}
        onPointerDown={onPointerDown}
        onPointerMove={onPointerMove}
        onPointerUp={endPress}
        onLostPointerCapture={endPress}
        onClick={() => {
          if (swallowClick.current) swallowClick.current = false
          else window.assist.bubbleClick()
        }}
        style={{ width: UI.bubbleSize, height: UI.bubbleSize }}
        className={cn(
          'overflow-hidden rounded-full shadow-[0_2px_6px_rgba(0,0,0,0.35)] outline-none',
          'transition-[transform,box-shadow] duration-150 ease-out motion-reduce:transition-none',
          mode === 'dragging' ? 'scale-105' : 'hover:scale-105 active:scale-95',
          // Always a 2px white border; while the panel is open, a purple one outside it.
          'ring-2',
          mode === 'expanded' ? 'ring-accent ring-offset-2 ring-offset-white' : 'ring-white',
        )}
      >
        <img

```

```

        src={logoUrl}
        alt=""
        draggable={false}
        className="pointer-events-none size-full object-contain"
      />
    </button>
  </div>
)
}

```

The bubble itself. It decides nothing on its own: clicks and drags are sent to `BubbleController` in the main process, which owns the bubble's mode and position. In JSX, attributes are *props*, the inputs a component receives. On built-in elements like `<button>` they become HTML attributes and event handlers (`onClick`, `onPointerDown`); on our own components they arrive as the function's argument (see `Panel` below).

- `const state = useAssistState()`: the bubble needs only the mode (for its look and label) and the branding. Until the state arrives, it falls back to `'collapsed'` and `'Desktop Assist'`.
- `useAccentColor(state?.branding.accentColor)`: `?.` because `state` is `null` at first.
- `const press = useRef<{ x: number; y: number; dragging: boolean } | null>(null)`: `useRef` gives a box (`.current`) that keeps its contents between renders, but unlike `state`, changing it doesn't re-render. It suits values that only event handlers need: here, where the press started and whether it has become a drag.
- `const swallowClick = useRef(false)`: releasing after a drag still fires a `click` on the button. This flag makes that one click be ignored.
- `if (event.button !== 0) return`: only the main (left) mouse button.
- `event.currentTarget.setPointerCapture(event.pointerId)`: sends all further events from this pointer to the button, even when the pointer is outside it, until it's released. The window is only 72 pixels square and gets moved during a drag, so without this the release could be lost.
- `if (!current || current.dragging) return`: only needed until the drag starts. From then on the main process moves the window with the mouse itself (`BubbleController.startDrag`), so the page just waits for the release.
- `Math.hypot(event.screenX - current.x, event.screenY - current.y) > DRAG_THRESHOLD`: the distance moved since the press, in screen coordinates, because the window itself moves once the drag starts.
- `window.assist.bubbleDragStart()`: tells the main process to start following the mouse.
- `function endPress() {`: runs on release. If the press had become a drag, it arms `swallowClick` and calls `bubbleDragEnd()`, and `BubbleController.endDrag` snaps the bubble to the nearest corner. If it was a plain click, it does nothing and the `click` event that follows handles it.
- `onLostPointerCapture={endPress}`: capture can also end without a release (the system can take it away), so this ends the press too. After a normal release both events fire; the second call finds `press.current` already `null` and does nothing.
- `onClick={() => {`: a real click calls `bubbleClick()`, and `BubbleController.clickBubble` decides what it means: open, close, or glide home from a bounce.
- `data-hit`: the marker `useClickThrough` looks for. The bubble window doesn't use click-through, so here it has no effect.
- `ACTION_LABEL[mode]`: the `aria-label` reads, for example, "Close Desktop Assist" while the panel is open.
- `style={{ width: UI.bubbleSize, height: UI.bubbleSize }}`: 56 pixels, the same number the main process uses to size the window (`BUBBLE_BOX`, 56 plus 8 on each side).
- `<div className="flex h-full w-full items-center justify-center">`: centres the button in the window. The 8-pixel margin around it leaves room for the shadow and ring.

- `shadow-[0_2px_6px_rgba(0,0,0,0.35)]`: square brackets give a Tailwind class a custom value (the underscores stand for spaces).
- `motion-reduce:transition-none`: no animation if animations are turned off in Windows.
- `mode === 'dragging' ? 'scale-105' : 'hover:scale-105 active:scale-95'`: slightly larger while dragged or hovered, slightly smaller while pressed.
- `ring-accent ring-offset-2 ring-offset-white`: while the panel is open, a 2-pixel white gap and then an accent-coloured ring. Otherwise the ring itself is white. Tailwind rings are drawn with `box-shadow`, which is why `box-shadow` is in the transition list.
- `draggable={false}`: stops the browser's own drag-the-image behaviour, which would take over the pointer.
- `pointer-events-none size-full object-contain`: the image ignores the mouse, so every event goes to the button; it fills the button without distortion.

`src/renderers/src/views/PanelView.tsx`: everything around the bubble

The page shown in the panel window: the chat card, the action icons and the settings menu. All the panel's shared state and handlers live in `Panel`; the components it renders (`ChatBox`, `ActionStack`, `SettingsMenu`, `SignInPanel`, `Banner`) get their data and callbacks from it as props. Those are covered in [the components part](#).

`<PanelView>`

`src/renderers/src/views/PanelView.tsx`, lines 14–20

```
/** Everything that appears around the bubble when it's clicked. */
export function PanelView() {
  useClickThrough()
  const state = useAssistState()
  useAccentColor(state?.branding.accentColor)
  return state ? <Panel state={state} /> : null
}
```

Sets up click-through and the live state, then renders `Panel` once the state has arrived.

- `useClickThrough()`: the panel window is the click-through one (see above).
- `return state ? <Panel state={state} /> : null`: returning `null` renders nothing. `Panel` is only created once the state exists because its `useState` calls take their starting values from it, and hooks can't be called conditionally. Splitting into two components is the usual way to wait for data before those hooks run.
- `state={state}`: passes the state to `Panel` as a prop. Each time `useAssistState` produces a new state, `PanelView` re-renders and `Panel` gets the new one.

`<Panel>`

`Panel` is long, so it's shown whole first, with notes on its state, effects and the JSX it returns. The sections after it take its values and functions one at a time.

`src/renderers/src/views/PanelView.tsx`, lines 22–217

```
function Panel({ state }: { state: AppState }) {
  const open = state.mode === 'expanded'
  const { branding, auth } = state

  // The draft is edited here and mirrored to the main process, which saves it.
```

```

const [draft, setDraft] = useState(state.notes.text)
const [attachments, setAttachments] = useState(state.notes.attachments)
const [settings, setSettings] = useState(state.settings)
const [settingsOpen, setSettingsOpen] = useState(false)
const [toast, showToast] = useToast()

useEffect(
  () =>
    window.assist.onModeChanged((mode) => {
      if (mode !== 'expanded') setSettingsOpen(false)
    }),
  [],
)
useEffect(() => {
  const onKeyDown = (event: KeyboardEvent) => {
    if (event.key !== 'Escape') return
    if (settingsOpen) setSettingsOpen(false)
    else window.assist.collapse()
  }
  window.addEventListener('keydown', onKeyDown)
  return () => window.removeEventListener('keydown', onKeyDown)
}, [settingsOpen])

// Not signed in: the chat box shows the sign-in instead of the chat. While a saved sign-in is
// being renewed, or JumpCloud can't be reached, the chat stays (sending tries JumpCloud again).
const signedIn = auth.state === 'signed-in' || auth.state === 'offline'
const busy = state.chat.some((message) => message.status === 'streaming')
const canSend = signedIn && !busy && (draft.trim() !== '' || attachments.length > 0)

async function runAction(id: ActionId) {
  if (id === 'settings') {
    setSettingsOpen(!settingsOpen)
    // Refresh: "Start with Windows" can also be changed in Windows Settings.
    if (!settingsOpen) setSettings(await window.assist.settings.get())
    return
  }
  setSettingsOpen(false)
  const result = await window.assist.invokeAction(id)
  if (result.message) showToast(result.message, result.ok ? 'info' : 'error')
}

function changeDraft(next: string) {
  setDraft(next)
  window.assist.notes.setText(next)
}

async function send() {
  if (!canSend) return
  const result = await window.assist.chat.send(draft)
  if (result.ok) {
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason === 'missing-screenshot') {
    showToast('An attached screenshot is missing. Remove it and try again.', 'error')
  } else if (result.reason === 'signed-out') {
    showToast('Sign in with JumpCloud first.', 'error')
  }
}

async function attachLatest() {
  const result = await window.assist.notes.attachLatestScreenshot()
  if (result.ok) setAttachments(result.notes.attachments)
  else if (result.reason === 'no-screenshots') {
    showToast('No screenshots yet. Take one with the camera button.')
  } else showToast('That screenshot is already attached')
}

```

```

}

async function removeAttachment(attachment: Attachment) {
  const notes = await window.assist.notes.removeAttachment(attachment.id)
  setAttachments(notes.attachments)
}

async function openAttachment(attachment: Attachment) {
  const opened = await window.assist.screenshots.open(attachment.path)
  if (!opened) showToast("That screenshot can't be found", 'error')
}

async function copy(text: string) {
  await window.assist.copyText(text)
  showToast('Copied')
}

async function signOut() {
  setSettingsOpen(false)
  await window.assist.auth.signOut()
  showToast('Logged out')
}

async function newConversation() {
  await window.assist.chat.newConversation()
  setSettingsOpen(false)
  showToast('Started a new conversation')
}

async function setEffort(effort: Effort) {
  setSettings(await window.assist.settings.setEffort(effort))
}

// Clicking anywhere else in the panel closes the settings menu.
function closeSettingsOnOutsideClick(event: PointerEvent) {
  if (!settingsOpen || !(event.target instanceof Element)) return
  if (!event.target.closest('[data-settings-menu], [data-action="settings"]')) {
    setSettingsOpen(false)
  }
}

const signInPrompt =
  auth.state === 'unconfigured' || auth.state === 'signed-out' || auth.state === 'signing-in' ? (
    <SignInPanel
      open={open}
      status={auth}
      appName={branding.appName}
      onSignIn={() => void window.assist.auth.signIn()}
      onCancel={() => void window.assist.auth.cancel()}
    />
  ) : null

const banner =
  auth.state === 'checking' ? (
    <Banner spinner>Checking your JumpCloud sign-in...</Banner>
  ) : auth.state === 'offline' ? (
    <Banner
      action={
        <button
          type="button"
          onClick={() => void window.assist.auth.retry()}
          className="rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100 dark:text-accent-soft dark:hover:bg-zinc-800"
        >
          Retry

```

```

        </button>
      }
    >
    {auth.message}
  </Banner>
) : null

const settingsIndex = branding.actions.indexOf('settings')

return (
  <div
    className={cn('group relative h-full w-full', !open && 'pointer-events-none')}
    data-open={open ? '' : undefined}
    onPointerDown={closeSettingsOnOutsideClick}
  >
    <ChatBox
      corner={state.corner}
      open={open}
      toast={toast}
      signInPrompt={signInPrompt}
      banner={banner}
      messages={state.chat}
      busy={busy}
      canSend={canSend}
      draft={draft}
      attachments={attachments}
      onDraftChange={changeDraft}
      onSend={() => void send()}
      onStop={() => void window.assist.chat.stop()}
      onRetry={() => void window.assist.chat.retry()}
      onCopy={(text) => void copy(text)}
      onAttachLatest={() => void attachLatest()}
      onOpenAttachment={(attachment) => void openAttachment(attachment)}
      onRemoveAttachment={(attachment) => void removeAttachment(attachment)}
    />
    <ActionStack
      corner={state.corner}
      actions={branding.actions}
      open={open}
      activeId={settingsOpen ? 'settings' : null}
      onAction={(id) => void runAction(id)}
    />
    {open && settingsOpen && settingsIndex >= 0 && (
      <SettingsMenu
        corner={state.corner}
        offset={actionOffset(settingsIndex)}
        settings={settings}
        branding={branding}
        version={state.version}
        user={signedIn ? auth.user : null}
        onToggleAutoStart={async () =>
          setSettings(await window.assist.settings.setAutoStart(!settings.autoStart))
        }
        onSetEffort={(effort) => void setEffort(effort)}
        onSignOut={() => void signOut()}
        onOpenScreenshotsFolder={() => void window.assist.screenshots.openFolder()}
        onNewConversation={() => void newConversation()}
      />
    )}
  </div>
)
}

```

Props and state:

- `function Panel({ state }: { state: AppState }):` a component receives its props as one object. This takes `state` out of it, and `{ state: AppState }` is the type of the props.
- `const open = state.mode === 'expanded':` the panel is open only in `expanded`. In every other mode (collapsed, dragging, bouncing and so on) it's closed or hidden.
- `const [draft, setDraft] = useState(state.notes.text):` the text box's contents. Like `attachments` and `settings`, it starts from the app state but is then owned here: the panel updates it as you type and from the main process's replies.
- `const [settingsOpen, setSettingsOpen] = useState(false):` whether the settings menu is showing.
- `const [toast, showToast] = useToast():` the current toast and the function to show one.

Effects:

- `window.assist.onModeChanged((mode) => {`: closes the settings menu whenever the panel closes, so it isn't still open the next time. The arrow returns what `onModeChanged` returns, the unsubscribe function, which React uses as the cleanup.
- `if (event.key !== 'Escape') return:` Esc closes the settings menu if it's open, otherwise asks the main process to close the panel (`BubbleController.collapse`). Keys reach the panel because the main process focuses it when it opens.
- `}, [settingsOpen]):` the listener reads `settingsOpen`, and a function only sees the values from the render it was created in. Listing it here makes React remove the old listener and add a fresh one whenever the menu opens or closes.

The JSX it returns:

- `className={cn('group relative h-full w-full', !open && 'pointer-events-none')}`: the root fills the window. `relative` makes it the reference box for the children's `absolute` positions (from `anchored`). `group` lets children style themselves from this element's attributes.
- `pointer-events-none`: while closed, nothing in the panel can be clicked, and `useClickThrough` finds no `data-hit` element, so the window stays click-through. This covers the moment the panel is fading out before the main process hides the window.
- `data-open={open ? '' : undefined}`: the attribute is present while open and left out while closed (React omits attributes set to `undefined`). Children use `group-data-open` classes, such as `group-data-open:opacity-100`, so the open and close animations are plain CSS transitions.
- `onPointerDown={closeSettingsOnOutsideClick}`: see below.
- `<ChatBox`: the card. It gets what to draw (corner, toast, sign-in prompt, banner, messages, draft and attachments) and a callback for each thing you can do there.
- `onSend={() => void send()}:` the handlers are `async`, so they return promises. Each is wrapped in an arrow with `void`, which marks the promise as deliberately not awaited and makes the callback return nothing.
- `activeId={settingsOpen ? 'settings' : null}`: the gear stays highlighted while its menu is open.
- `{open && settingsOpen && settingsIndex >= 0 && (`: JSX's way of saying "only if": when any part is false, nothing is rendered. Removing the menu also resets its own state (its "click again to confirm" on New conversation).
- `offset={actionOffset(settingsIndex)}`: how far the gear is from the bubble, so the menu lines up with it.
- `user={signedIn ? auth.user : null}`: TypeScript allows `auth.user` here because it remembers that `signedIn` being true means `auth` is a state that has a user.
- `onToggleAutoStart={async () =>}`: flips Start with Windows and shows the settings the main process returns.

signedIn, busy, canSend

src/renderer/src/views/PanelView.tsx, lines 50–54

```
// Not signed in: the chat box shows the sign-in instead of the chat. While a saved sign-in is
// being renewed, or JumpCloud can't be reached, the chat stays (sending tries JumpCloud again).
const signedIn = auth.state === 'signed-in' || auth.state === 'offline'

const busy = state.chat.some((message) => message.status === 'streaming')

const canSend = signedIn && !busy && (draft.trim() !== '' || attachments.length > 0)
```

Values worked out on every render from the current state.

- `auth.state === 'signed-in' || auth.state === 'offline'`: `offline` counts as signed in: the sign-in is kept while JumpCloud can't be reached, and sending a message tries JumpCloud again.
- `state.chat.some((message) => message.status === 'streaming')`: Claude is still replying.
- `draft.trim() !== '' || attachments.length > 0`: there must be some text or at least one screenshot. `ChatBox` disables Send, and Enter does nothing, unless `canSend` is true.

runAction

src/renderer/src/views/PanelView.tsx, lines 56–66

```
async function runAction(id: ActionId) {
  if (id === 'settings') {
    setSettingsOpen(!settingsOpen)
    // Refresh: "Start with Windows" can also be changed in Windows Settings.
    if (!settingsOpen) setSettings(await window.assist.settings.get())
    return
  }
  setSettingsOpen(false)
  const result = await window.assist.invokeAction(id)
  if (result.message) showToast(result.message, result.ok ? 'info' : 'error')
}
```

Runs when an action icon is clicked (`ActionStack`'s `onAction`).

- `if (id === 'settings') {`: Settings is a "popover" action (src/shared/actions.ts): it opens UI inside the panel and never reaches the main process.
- `setSettingsOpen(!settingsOpen)`: setting state doesn't change the variable in this run of the function. `settingsOpen` still holds the value from before the toggle, so the next line's `if (!settingsOpen)` means "the menu is being opened now".
- `setSettings(await window.assist.settings.get())`: refreshes the settings as the menu opens, because Start with Windows can also be changed in Windows Settings.
- `window.assist.invokeAction(id)`: after the `settings` branch returns, TypeScript knows `id` is `screenshot`, `bounce` or `close`, which is exactly the `CommandActionId` type `invokeAction` accepts. The main process runs the matching handler from src/main/actions.ts.
- `if (result.message) showToast(result.message, result.ok ? 'info' : 'error')`: for example "Screenshot saved", or "Couldn't take a screenshot" in red. Bounce and close return no message.

changeDraft

src/renderer/src/views/PanelView.tsx, lines 68–71

```
function changeDraft(next: string) {
  setDraft(next)
  window.assist.notes.setText(next)
}
```

Runs on every keystroke in the text box. It updates the box, then sends the text to the main process (`notes.setText`, which doesn't wait for a reply). There, `NotesStore.setText` keeps it and writes `notes.json` about half a second after typing stops, so an unsent message survives a restart.

send

src/renderer/src/views/PanelView.tsx, lines 73–84

```
async function send() {
  if (!canSend) return
  const result = await window.assist.chat.send(draft)
  if (result.ok) {
    setDraft(result.notes.text)
    setAttachments(result.notes.attachments)
  } else if (result.reason === 'missing-screenshot') {
    showToast('An attached screenshot is missing. Remove it and try again.', 'error')
  } else if (result.reason === 'signed-out') {
    showToast('Sign in with JumpCloud first.', 'error')
  }
}
```

Sends the draft to Claude.

- `if (!canSend) return`: a second guard; `ChatBox` checks too.
- `window.assist.chat.send(draft)`: only the text is sent. The main process takes the attachments from its own copy of the draft, prepares the screenshots and starts the reply (the `chatSend` handler in `src/main/ipc.ts`). Claude's reply doesn't come back here: it streams in through `onChatMessage` in `useAssistState`.
- `setDraft(result.notes.text)`: the main process cleared the draft after sending; the box adopts the cleared version, as do the attachments.
- `result.reason === 'missing-screenshot'`: an attached file was deleted after it was attached.
- `result.reason === 'signed-out'`: the sign-in ran out just before sending.

The other two reasons, `busy` and `empty`, are ignored: `canSend` already rules them out.

attachLatest, removeAttachment, openAttachment

src/renderer/src/views/PanelView.tsx, lines 86–102

```
async function attachLatest() {
  const result = await window.assist.notes.attachLatestScreenshot()
  if (result.ok) setAttachments(result.notes.attachments)
  else if (result.reason === 'no-screenshots') {
    showToast('No screenshots yet. Take one with the camera button.')
  } else showToast('That screenshot is already attached')
}

async function removeAttachment(attachment: Attachment) {
  const notes = await window.assist.notes.removeAttachment(attachment.id)
  setAttachments(notes.attachments)
}

async function openAttachment(attachment: Attachment) {
  const opened = await window.assist.screenshots.open(attachment.path)
  if (!opened) showToast("That screenshot can't be found", 'error')
}
```

The screenshot chips on the draft. Each asks the main process, which owns the draft, and shows the result.

- `window.assist.notes.attachLatestScreenshot()`: the main process finds the newest file in the screenshots folder and adds it to the draft.
- `else showToast('That screenshot is already attached')`: the only other failure, `already-attached`.
- `window.assist.notes.removeAttachment(attachment.id)`: returns the updated draft, whose attachment list the panel adopts.
- `window.assist.screenshots.open(attachment.path)`: opens the file in the default image viewer; `false` means the file is gone.

copy

src/renderer/src/views/PanelView.tsx, lines 104–107

```
async function copy(text: string) {
  await window.assist.copyText(text)
  showToast('Copied')
}
```

The Copy button on Claude's replies. The main process writes the text to the clipboard (`clipboard.writeText` in `src/main/ipc.ts`).

signOut, newConversation, setEffort

src/renderer/src/views/PanelView.tsx, lines 109–123

```
async function signOut() {
  setSettingsOpen(false)
  await window.assist.auth.signOut()
  showToast('Logged out')
}

async function newConversation() {
  await window.assist.chat.newConversation()
  setSettingsOpen(false)
  showToast('Started a new conversation')
}

async function setEffort(effort: Effort) {
  setSettings(await window.assist.settings.setEffort(effort))
}
```

Settings menu actions.

- `await window.assist.auth.signOut()`: `AuthManager.signOut` forgets the saved sign-in. Because this is a log-out, the main process also starts a new conversation (`onSignedOut` in `src/main/index.ts`). The new `signed-out` status and the chat reset arrive through `useAssistState`, and the chat card switches to the sign-in.
- `await window.assist.chat.newConversation()`: `ChatSession` forgets the conversation and broadcasts a reset, which empties the chat in `useAssistState`.
- `setSettings(await window.assist.settings.setEffort(effort))`: saves the response style (Fast, Balanced or Thorough) and shows the settings the main process returns.

closeSettingsOnOutsideClick

src/renderer/src/views/PanelView.tsx, lines 125–131

```
// Clicking anywhere else in the panel closes the settings menu.
function closeSettingsOnOutsideClick(event: PointerEvent) {
  if (!settingsOpen || !(event.target instanceof Element)) return
  if (!event.target.closest('[data-settings-menu], [data-action="settings"]')) {
    setSettingsOpen(false)
  }
}
```

Closes the settings menu when you press anywhere else in the panel. The page only receives the press where the window takes the mouse, so "elsewhere" means the chat card or another icon. Clicking outside the panel altogether takes focus away from the window, the main process closes the panel (`BubbleController.panelBlurred`), and the mode effect above closes the menu.

- `event: PointerEvent`: React's own event type, imported from `react`, not the browser's.
- `!(event.target instanceof Element)`: `target` is typed as a general `EventTarget`; this check lets TypeScript allow `closest`.
- `closest('[data-settings-menu], [data-action="settings"]')`: presses inside the menu or on the gear don't count. Without the gear exception, pressing the gear would close the menu on the way down and its click would open it again straight away.

signInPrompt, banner: sign-in or chat

src/renderer/src/views/PanelView.tsx, lines 133-161

```
const signInPrompt =
  auth.state === 'unconfigured' || auth.state === 'signed-out' || auth.state === 'signing-in' ? (
    <SignInPanel
      open={open}
      status={auth}
      appName={branding.appName}
      onSignIn={() => void window.assist.auth.signIn()}
      onCancel={() => void window.assist.auth.cancel()}
    />
  ) : null

const banner =
  auth.state === 'checking' ? (
    <Banner spinner>Checking your JumpCloud sign-in...</Banner>
  ) : auth.state === 'offline' ? (
    <Banner
      action={
        <button
          type="button"
          onClick={() => void window.assist.auth.retry()}
          className="rounded-md px-1.5 py-0.5 font-medium text-accent hover:bg-zinc-100 dark:text-accent-soft dark:hover:bg-zinc-800"
        >
          Retry
        </button>
      }
    >
      {auth.message}
    </Banner>
  ) : null
```

This is how the panel decides what the chat card shows. `signInPrompt` is a `SignInPanel` element or `null`, and `banner` is a `Banner` element or `null`. `ChatBox` shows `signInPrompt` in place of the conversation when it isn't `null`; otherwise it shows the messages, then the banner, then the text box. The sign-in status comes from `AuthManager` in the main process, through `onAuthStatus` in `useAssistState`.

auth.state	The chat card shows
unconfigured	Sign-in: "Sign-in isn't set up yet"
signed-out	Sign-in: "Sign in with JumpCloud"
signing-in	Sign-in: "Finish signing in in your browser", Cancel
checking	The chat, "Checking..." banner; sending is off
offline	The chat, the reason and Retry; sending works
signed-in	The chat

- `auth.state === 'unconfigured' || auth.state === 'signed-out' || auth.state === 'signing-in'`: inside the `? (` branch, TypeScript has narrowed `auth` to those three states, which is exactly the type `SignInPanel`'s `status` prop accepts.
- `onSignIn={() => void window.assist.auth.signIn()}:` `AuthManager.signIn` opens JumpCloud in the browser and returns at once. Progress arrives as status changes (`signing-in`, then `signed-in`), and the main process reopens the panel when it's done.

- `onCancel={() => void window.assist.auth.cancel()}: stops waiting for the browser.`
- `<Banner spinner>`: shown while a saved sign-in is being renewed, at startup or after Retry. The chat is visible, but `signedIn` is false, so nothing can be sent yet.
- `onClick={() => void window.assist.auth.retry()}: AuthManager.retry` tries again to renew the saved sign-in.
- `{auth.message}`: the offline reason, written by `AuthManager`.

settingsIndex

src/renderer/src/views/PanelView.tsx, line 163

```
const settingsIndex = branding.actions.indexOf('settings')
```

Where the gear is in the tenant's list of icons (`actions` in `tenant.json`), counting from the bubble. It's `-1` if the tenant doesn't show Settings, and then the menu is never drawn.

The UI components

These seven files draw everything that appears around the bubble when you click it. `Panel` in `PanelView.tsx` (see [The pages](#)) renders three of them directly: the `ActionStack` of round icons in a column beside the bubble, the `ChatBox` card next to the bubble on the side facing the middle of the screen, and the `SettingsMenu`, which opens beside the gear icon. Everything else lives inside the chat card: `SignInPanel` replaces the chat while nobody is signed in, `MessageList` shows the conversation (using `Markdown` for Claude's replies), and an `AttachmentChip` above the text box stands for each attached screenshot.

`src/renderer/src/components/ChatBox.tsx`: the chat card

The card beside the bubble, with the text box at its bottom. `Panel` renders one `ChatBox` and passes it everything it shows: the bubble's `corner`, whether the panel is `open`, the current `toast`, the conversation (`messages`), the `draft` and its `attachments`, whether Claude is replying (`busy`) and whether Send is allowed (`canSend`). `Panel` also builds the two optional pieces that go inside it, the sign-in (`signInPrompt`) and a status line (`banner`). The `on...` props are what the buttons and the text box call. The file has three components: `ChatBox` (the card), `Composer` (the text box, private to this file) and `Banner` (exported, used by `Panel`).

ChatBox

`src/renderer/src/components/ChatBox.tsx`, lines 11–90

```
/**
 * The card beside the bubble, on the side facing the middle of the screen. It's anchored at the
 * bubble's edge (bottom in a bottom corner, top in a top corner), so it starts as just the text
 * box and grows away from that edge as the conversation gets longer, then scrolls. When nobody is
 * signed in, `signInPrompt` is shown instead of the chat.
 */
export function ChatBox(props: {
  corner: Corner
  open: boolean
  toast: ToastMessage | null
  /** Replaces the chat (the JumpCloud sign-in), or null to show the chat. */
  signInPrompt: ReactNode
  /** A line above the text box, e.g. "Checking your sign-in...". */
  banner: ReactNode
  messages: ChatMessage[]
  busy: boolean
  canSend: boolean
  draft: string
  attachments: Attachment[]
  onDraftChange: (text: string) => void
  onSend: () => void
  onStop: () => void
  onRetry: () => void
  onCopy: (text: string) => void
  onAttachLatest: () => void
  onOpenAttachment: (attachment: Attachment) => void
  onRemoveAttachment: (attachment: Attachment) => void
}) {
  return (
    <div
      data-hit
      className={cn(
        'absolute flex flex-col rounded-2xl border shadow-xl',
```

```

100',
    'border-black/10 bg-white text-zinc-900 dark:border-white/10 dark:bg-zinc-900 dark:text-zinc-900',
    originClass(props.corner),
    'scale-95 opacity-0 transition duration-150 ease-out',
    'group-data-open:scale-100 group-data-open:opacity-100',
    'motion-reduce:transition-none',
  )}
  style={{
    ...anchored(props.corner, PANEL_LAYOUT.chatX, PANEL_LAYOUT.chatY),
    width: UI.panelWidth,
    maxHeight: UI.panelMaxHeight,
  }}
  >
  {props.toast && (
    <div
      key={props.toast.id}
      role="status"
      className={cn(
        'absolute rounded-full px-3 py-1.5 text-xs font-medium shadow-lg',
        // Just outside the card, on the side away from the screen edge.
        isTopCorner(props.corner) ? 'top-full mt-2' : 'bottom-full mb-2',
        isLeftCorner(props.corner) ? 'left-0' : 'right-0',
        props.toast.tone === 'error'
          ? 'bg-red-600 text-white'
          : 'bg-zinc-900 text-white dark:bg-white dark:text-zinc-900',
      )}
    >
      {props.toast.text}
    </div>
  )}

  {props.signInPrompt ?? (
    <>
      {props.messages.length > 0 && (
        <MessageList
          messages={props.messages}
          onRetry={props.onRetry}
          onCopy={props.onCopy}
          onOpenScreenshot={props.onOpenAttachment}
        />
      )}
      {props.banner}
      <Composer {...props} />
    </>
  )}
</div>
)
}

```

`ChatBox` draws the card and places it in the panel window. Inside it shows a toast (if there is one), then either the sign-in or the chat: the conversation, the banner and the text box. It needs a few ideas that the rest of this page relies on.

Components, props and JSX. A React component is a function that returns a description of the UI. React calls `ChatBox(props)` again whenever its inputs change. The HTML-like `<div ...>` it returns is JSX: the build turns each tag into a function call that creates a lightweight description, and React then updates the real page (the DOM) to match. Lowercase tags (`div`, `button`) are HTML elements; capitalised ones (`MessageList`, `Composer`) are other components. The attributes written on a component's tag become its *props*, the single object it receives; the type after `props:` lists every prop it accepts. Inside JSX, curly braces `{...}` switch back to ordinary TypeScript. `className` is JSX's name for HTML's `class`.

Callbacks. The `on...` props are functions. The card never talks to the main process itself: when you press Send it calls `props.onSend()`, and `Panel` does the work (see `send()` in [The pages](#)). That keeps every decision in one place.

Tailwind classes and `cn()`. These components have almost no CSS of their own. Each element lists small *utility classes* that each do one thing: `absolute` is `position: absolute`, `rounded-2xl` rounds the corners, `px-3` sets the left and right padding, `text-xs` is small text. Tailwind generates CSS for just the classes the code uses. A prefix makes a class conditional: `hover:` applies while the pointer is over the element, `dark:` while Windows is in dark mode, `disabled:` while a button is disabled. `bg-accent` and `text-accent` use the tenant's colour, which `styles.css` defines and `useAccentColor` sets at runtime (see [The pages](#)). `cn()` (from `lib/cn.ts`) joins class strings and drops `false`, `null` and `undefined`, which is what lets you write `condition && 'some-class'` inside it.

`data-hit`. The panel window is a large transparent rectangle that normally lets clicks fall through to whatever is underneath. `useClickThrough` (see [The pages](#)) switches mouse input on only while the pointer is over an element that is, or is inside, one marked `data-hit`. So every piece of real UI must carry it. Here it's the whole card, which covers everything inside it; in `SettingsMenu` it's the menu, and in `ActionStack` each icon.

How the layout mirrors for each corner. The bubble can rest in any corner of the screen, and the panel always opens toward the middle. The panel window shares its outer corner with the bubble's window (`panelWindowBounds` in [The bubble](#)), so the components measure their positions from that corner. `PANEL_LAYOUT` in `src/shared/geometry.ts` gives each piece's distance from it, `X` across and `Y` away from the screen edge (see [Shared code and the preload bridge](#)). `anchored(corner, x, y)` from `lib/anchor.ts` turns those two numbers into CSS: `right` and `bottom` in the bottom-right corner, `left` and `top` in the top-left, and so on. So the same numbers work in all four corners, and a component only has to check the corner where something must change direction (the toast here, the order of the icons in `ActionStack`). In the bottom-right corner it looks roughly like this; the other corners are mirror images:

```
+-----+
| MessageList |
| Banner (only sometimes) |
| Composer    |
+-----+

[close]
[bounce]
[settings]    <- SettingsMenu opens leftward, over the card
[screenshot]

( bubble )
```

The parts of `ChatBox` itself:

- `originClass(props.corner)`: sets the CSS `transform-origin` to the bubble's corner (for example `origin-bottom-right`), so the zoom-in animation grows out of the bubble.
- `'scale-95 opacity-0 transition duration-150 ease-out'`: the card is always rendered, even while the panel is closed. It's just invisible and slightly shrunk.
- `'group-data-open:scale-100 group-data-open:opacity-100'`: `Panel`'s outer `<div>` has the class `group`, and a `data-open` attribute while the panel is open. `group-data-open:` means "when an ancestor marked `group` has `data-open`". So opening animates the card in over 150 ms, and closing animates it out; the main process waits 140 ms (`PANEL_FADE_MS` in `BubbleController.ts`) before hiding the window, so the fade-out can be seen.
- `'motion-reduce:transition-none'`: when Windows is set to show fewer animations (the system's reduced-motion setting), the card appears and disappears instantly.
- `...anchored(props.corner, PANEL_LAYOUT.chatX, PANEL_LAYOUT.chatY)`: `style` takes an object of CSS properties, and plain numbers mean pixels. `chatX` is 76 (the 8 px padding around the bubble, the 56 px bubble and a 12 px gap), so the card starts just past the bubble. `chatY` is 8, so the card's near edge is level with the bubble's.

- `maxHeight: UI.panelMaxHeight`: nothing sets a height. The card is pinned at the bubble's edge and grows away from it as content is added, up to 520 px. After that the message list scrolls.
- `{props.toast && (}`: the part after `&&` is rendered only when the left side is truthy, so with no toast nothing is drawn. Toasts come from `useToast` in `Panel` and vanish after 2.6 seconds.
- `key={props.toast.id}`: a `key` tells React which element is which. Each toast has a new id, so React puts in a fresh element for each new message instead of editing the old one's text.
- `role="status"`: screen readers read the toast out without moving the keyboard focus.
- `isTopCorner(props.corner) ? 'top-full mt-2' : 'bottom-full mb-2'`: the toast sits just outside the card, on the side facing the middle of the screen: above the card in a bottom corner, below it in a top corner. The panel window has 40 px spare beyond the card's full height (`UI.toastRoom`), so the toast always fits.
- `isLeftCorner(props.corner) ? 'left-0' : 'right-0'`: lines the toast up with the card's edge nearest the bubble.
- `{props.signInPrompt ?? (`: `??` uses the left side unless it's `null` or `undefined`. `Panel` passes a `SignInPanel` when nobody is signed in and `null` otherwise, so the card shows either the sign-in or the chat, never both.
- `<>`: a *fragment*. It groups several elements without adding an extra `<div>` to the page.
- `props.messages.length > 0 &&`: before the first message there's no list, so the card is just the text box.
- `{props.banner}`: whatever `Banner Panel` built (or `null`), between the conversation and the text box.
- `<Composer {...props} />`: `{...props}` passes every one of `ChatBox`'s props on to `Composer` unchanged.

Composer

src/renderer/src/components/ChatBox.tsx, lines 92–184

```
/** The text box: attached screenshots, the text, and the attach / send / stop buttons. */
function Composer(props: Parameters<typeof ChatBox>[0]) {
  const textarea = useRef<HTMLTextAreaElement>(null)
  const placedCaret = useRef(false)

  // Focus the text box whenever the panel opens; the first time, put the caret at the end.
  useEffect(() => {
    const el = textarea.current
    if (!props.open || !el) return
    el.focus()
    if (!placedCaret.current) {
      placedCaret.current = true
      el.setSelectionRange(el.value.length, el.value.length)
    }
  }, [props.open])

  // Enter sends; Shift+Enter starts a new line.
  function onKeyDown(event: KeyboardEvent<HTMLTextAreaElement>) {
    if (event.key === 'Enter' && !event.shiftKey && !event.nativeEvent.isComposing) {
      event.preventDefault()
      if (props.canSend) props.onSend()
    }
  }

  return (
    <div
      className={cn(
        'shrink-0',
        props.messages.length > 0 && 'border-t border-black/5 dark:border-white/10',
      )}
    >
```

```

{props.attachments.length > 0 && (
  <div className="flex gap-2 overflow-x-auto px-3 pt-3">
    {props.attachments.map((attachment) => (
      <AttachmentChip
        key={attachment.id}
        attachment={attachment}
        onOpen={() => props.onOpenAttachment(attachment)}
        onRemove={() => props.onRemoveAttachment(attachment)}
      />
    ))}
  </div>
)}

<textarea
  ref={textarea}
  value={props.draft}
  onChange={(event) => props.onDraftChange(event.target.value)}
  onKeyDown={onKeyDown}
  placeholder="Ask Claude anything..."
  aria-label="Message Claude"
  spellCheck
  className={cn(
    'block max-h-40 min-h-11 w-full resize-none overflow-y-auto bg-transparent field-sizing-
content',
    'px-3.5 pt-3 pb-1 text-sm leading-relaxed outline-none placeholder:text-zinc-400',
  )}
/>

<div className="flex items-center justify-between gap-2 px-2 pb-2">
  <button
    type="button"
    onClick={props.onAttachLatest}
    className="inline-flex items-center gap-1.5 rounded-lg px-2 py-1 text-xs font-medium text-zinc-
600 hover:bg-zinc-100 hover:text-zinc-900 dark:text-zinc-300 dark:hover:bg-zinc-800 dark:hover:text-
white"
  >
    <Paperclip size={14} aria-hidden />
    Attach latest screenshot
  </button>
  {props.busy ? (
    <button
      type="button"
      onClick={props.onStop}
      aria-label="Stop"
      title="Stop"
      className="grid size-8 place-items-center rounded-full bg-zinc-900 text-white hover:bg-zinc-
700 dark:bg-white dark:text-zinc-900"
    >
      <Square size={12} fill="currentColor" aria-hidden />
    </button>
  ) : (
    <button
      type="button"
      onClick={props.onSend}
      disabled={!props.canSend}
      aria-label="Send"
      title="Send (Enter)"
      className="grid size-8 place-items-center rounded-full bg-accent text-white disabled:opacity-
40"
    >
      <ArrowUp size={16} aria-hidden />
    </button>
  )}
</div>
</div>

```

```
)  
}
```

The text box at the bottom of the card: the attached screenshots, the text itself, and the buttons under it. It's a separate component so it can have its own refs and effect; only `ChatBox` uses it.

Refs. React runs a component's function again on every render, so its ordinary local variables start afresh each time. `useRef(initial)` gives a small box, `.current`, that keeps its value between renders. Changing it doesn't redraw anything. `Composer` has two: `textarea` is attached to the `<textarea>` element with `ref={textarea}`, so React puts the real DOM element in `textarea.current` (needed to call `.focus()`); `placedCaret` is just a remembered flag.

Effects. `useEffect(fn, [deps])` runs `fn` after React has updated the page, and again only when a value in the dependency list `[deps]` has changed. It also runs once when the component first appears. Effects are for work outside drawing the UI, such as moving the keyboard focus.

- `function Composer(props: Parameters<typeof ChatBox>[0]): Parameters<typeof ChatBox>` is TypeScript for "the types of `ChatBox`'s arguments", and `[0]` is its props object. So `Composer` takes exactly the same props without repeating the long list.
- `}, [props.open]]`: the effect runs when the panel opens or closes; the early `return` skips the close. On open it focuses the text box so you can type straight away. Because it also runs when `Composer` first appears, the box is focused after you sign in with the panel open.
- `el.setSelectionRange(el.value.length, el.value.length)`: only the first time, puts the caret at the end, since the draft may have been restored from the last session. After that the caret stays wherever you left it.
- `!event.nativeEvent.isComposing`: while typing with an input method (for example for Japanese or Chinese), Enter confirms the chosen characters. This check stops that Enter from sending.
- `event.preventDefault()`: stops Enter adding a new line. Enter never adds a line, even when the message can't be sent yet (for example while Claude is replying); Shift+Enter skips this whole `if` and adds one.
- `{props.attachments.map((attachment) => (`: `.map` turns each attachment into an `AttachmentChip`. The arrow functions passed as `onOpen` and `onRemove` remember which attachment the chip is for.
- `key={attachment.id}`: every item in a list needs a `key` that stays the same between renders, so React can tell the items apart when one is removed.
- `overflow-x-auto`: with many attachments, the row of chips scrolls sideways.
- `value={props.draft}`: a *controlled* text box. What it shows always comes from the `draft` prop, not from the element's own memory.
- `onChange={(event) => props.onDraftChange(event.target.value)}`: React's `onChange` fires on every keystroke. It calls `Panel`'s `changeDraft`, which updates `draft` and sends the text to the main process to be saved (`NotesStore.setText`; see [The draft, screenshots and safe files](#)).
- `field-sizing-content`: CSS `field-sizing: content` makes the text box grow with its text, from `min-h-11` (44 px) to `max-h-40` (160 px), after which it scrolls.
- `spellCheck`: turns on Chromium's spell checker. Right-clicking a misspelt word shows suggestions in the menu built by `addEditContextMenu` in `src/main/bubble/windows.ts`.
- `<Paperclip size={14} aria-hidden />`: the icons come from `lucide-react`, where every icon is a small React component that draws an SVG. `size` sets its width and height in pixels. `aria-hidden` hides it from screen readers, since the button's text or `aria-label` already says what it does.
- `onClick={props.onAttachLatest}`: `Panel.attachLatest` adds the newest screenshot to the draft, or shows a toast saying why it couldn't.

- `props.busy ?` (: while Claude is replying, Send is replaced by Stop. `Panel` sets `busy` when any message is still streaming.
- `fill="currentColor"`: lucide icons are outlines; this fills the square with the text colour so it reads as a solid "stop" symbol.
- `disabled={!props.canSend}`: `Panel` allows sending only when you're signed in, Claude isn't busy, and there's text or a screenshot. `disabled:opacity-40` fades the button out.

Banner

`src/renderersrc/components/ChatBox.tsx`, lines 186–195

```
/** A one-line status above the text box, with an optional action. */
export function Banner(props: { spinner?: boolean; children: ReactNode; action?: ReactNode }) {
  return (
    <div className="flex shrink-0 items-center gap-2 border-t border-black/5 px-3.5 py-2 text-xs text-zinc-500 dark:border-white/10">
      {props.spinner && <LoaderCircle size={12} className="animate-spin" aria-hidden />}
      <span className="flex-1">{props.children}</span>
      {props.action}
    </div>
  )
}
```

A one-line status strip between the conversation and the text box. `Panel` uses it in two cases: "Checking your JumpCloud sign-in..." with a spinner while a saved sign-in is renewed at startup, and the "couldn't reach JumpCloud" message with a Retry button (see [JumpCloud sign-in](#)).

- `spinner?: boolean`: the `?` makes the prop optional. Writing `<Banner spinner>` with no value means `spinner={true}`.
- `children: ReactNode`: `children` is a special prop holding whatever is written between `<Banner>` and `</Banner>`. `ReactNode` is the type for anything React can display: text, elements, or `null`.
- `action?: ReactNode`: an optional element at the right-hand end, such as `Panel`'s Retry button.
- `animate-spin`: Tailwind's spinning animation, applied to lucide's `LoaderCircle` icon.
- `flex-1`: the text takes all the spare width, which pushes the action to the right.

`src/renderersrc/components/MessageList.tsx`: the conversation

The scrolling list of messages at the top of the chat card. `ChatBox` renders it once there's at least one message. Its props are `messages` (the chat from the app state, kept up to date by `useAssistState`; see [The pages](#)), and `onRetry` and `onCopy` from `Panel`. Your messages and Claude's are drawn by different components.

MessageList

`src/renderersrc/components/MessageList.tsx`, lines 8–66

```
/** Within this many pixels of the bottom counts as "at the bottom". */
const STICK_THRESHOLD = 40

/**
 * The conversation. It follows new text as it streams in, unless you've scrolled up to read
 * something earlier. Sending a message always brings it back to the bottom.
 */
```

```

export function MessageList(props: {
  messages: ChatMessage[]
  onRetry: () => void
  onCopy: (text: string) => void
  onOpenScreenshot: (attachment: Attachment) => void
}) {
  const scroller = useRef<HTMLDivElement>(null)
  const stickToBottom = useRef(true)
  const lastSentId = useRef<string | undefined>(undefined)

  useEffect(() => {
    const el = scroller.current
    if (!el) return
    // A message you've just sent: follow it and the reply, even if you'd scrolled up.
    const sentId = props.messages.findLast((m) => m.role === 'user')?.id
    if (sentId !== lastSentId.current) {
      lastSentId.current = sentId
      stickToBottom.current = true
    }
    if (stickToBottom.current) el.scrollTop = el.scrollHeight
  }, [props.messages])

  return (
    <div
      ref={scroller}
      onScroll={(event) => {
        const el = event.currentTarget
        stickToBottom.current = el.scrollHeight - el.scrollTop - el.clientHeight < STICK_THRESHOLD
      }}
      className="min-h-0 flex-1 space-y-3 overflow-y-auto px-3.5 pt-3.5 pb-1"
      aria-live="polite"
    >
      {props.messages.map((message, index) =>
        message.role === 'user' ? (
          <UserMessage
            key={message.id}
            message={message}
            onOpenScreenshot={props.onOpenScreenshot}
          />
        ) : (
          <AssistantMessage
            key={message.id}
            message={message}
            canRetry={index === props.messages.length - 1}
            onRetry={props.onRetry}
            onCopy={props.onCopy}
          />
        ),
      )}
    </div>
  )
}

```

Draws each message, and keeps the view scrolled to the bottom as a reply streams in. If you scroll up to read something earlier it stops following; scrolling back to the bottom, or sending a message, turns following on again.

- `const stickToBottom = useRef(true)`: whether to follow new text. It's a ref rather than state because changing it doesn't need anything redrawn.
- `const lastSentId = useRef<string | undefined>(undefined)`: the id of the newest message you sent, as of the last update, so the effect can tell when you've just sent another.

- `useLayoutEffect(() => {})`: like `useEffect`, but it runs after React has changed the page and before the browser paints it. Scrolling there means you never see a frame with the new text hidden below the visible area.
- `}, [props.messages])`: runs whenever the list changes. `useAssistState` makes a new array for every update, including each batch of streamed text (at most one every 50 ms), so this runs as the reply grows.
- `const sentId = props.messages.findLast((m) => m.role === 'user')?.id`: the newest message you sent. When it differs from last time you've just sent one, so following is switched back on: your message and Claude's reply come into view even if you'd scrolled up to read something.
- `el.scrollTop = el.scrollHeight`: scrolls to the bottom. The browser clamps the value to the furthest it can actually scroll.
- `onScroll={event => {}}`: on every scroll, works out how far the view is from the bottom (`scrollHeight - scrollTop - clientHeight`) and follows only if that's under `STICK_THRESHOLD` (40 px). The effect's own scrolling lands at 0, so following stays on.
- `min-h-0 flex-1 overflow-y-auto`: the list takes whatever height the card has left after the text box, and scrolls. `min-h-0` matters because an item in a flex column normally refuses to shrink below its content's height, which would push the text box out of the card.
- `aria-live="polite"`: screen readers announce new text in the list when they're not busy.
- `message.role === 'user' ? (`: chooses the component by who sent the message.
- `key={message.id}`: each message's id from `ChatSession` is its list key.
- `canRetry={index === props.messages.length - 1}`: only the last message may offer Retry, since `ChatSession.retry()` asks again for the latest reply (see [Talking to Claude](#)).

UserMessage

src/renderer/src/components/MessageList.tsx, lines 68–95

```
function UserMessage({
  message,
  onOpenScreenshot,
}): {
  message: ChatMessage
  onOpenScreenshot: (attachment: Attachment) => void
} {
  return (
    <div className="ml-auto flex max-w-[85%] flex-col items-end gap-1.5">
      {message.attachments.length > 0 && (
        <div className="flex flex-wrap justify-end gap-1.5">
          {message.attachments.map((attachment) => (
            <SentScreenshot
              key={attachment.id}
              attachment={attachment}
              onOpen={() => onOpenScreenshot(attachment)}
            />
          ))}
        </div>
      )}
      {message.text && (
        <div className="rounded-2xl rounded-br-md bg-accent px-3 py-2 text-sm whitespace-pre-wrap text-white select-text">
          {message.text}
        </div>
      )}
    </div>
  )
}
```

Your own message, on the right: any screenshots, then the text in an accent-coloured bubble.

- `function UserMessage({`: the braces in the parameter list pull `message` and `onOpenScreenshot` straight out of the props object (*destructuring*).
- `ml-auto`: with `items-end`, pushes the message to the right. `max-w-[85%]` is Tailwind's syntax for a one-off value; it keeps the message from spanning the whole card.
- `{message.text} && (`: a message can be screenshots only, with no text.
- `whitespace-pre-wrap`: keeps the line breaks you typed with Shift+Enter. Your text is shown exactly as typed, not as Markdown.
- `select-text: styles.css` turns text selection off for the whole page (so the UI doesn't feel like a web page); this turns it back on so you can select and copy your own message.
- `rounded-br-md`: a smaller bottom-right corner, the usual chat-bubble "tail".

SentScreenshot

src/renderers/src/components/MessageList.tsx, lines 97–110

```
/** A sent screenshot's thumbnail. Opening it goes through the panel, which says if it's gone. */
function SentScreenshot({ attachment, onOpen }: { attachment: Attachment; onOpen: () => void }) {
  const thumbnail = useThumbnail(attachment.path)
  return (
    <button
      type="button"
      title={attachment.fileName}
      onClick={onOpen}
      className="h-12 w-20 overflow-hidden rounded-lg border border-black/10 bg-zinc-200 dark:border-
white/10 dark:bg-zinc-700"
    >
      {thumbnail && <img src={thumbnail} alt="" className="size-full object-cover" />}
    </button>
  )
}
```

A small thumbnail of a screenshot sent with a message. Clicking it opens the file in your image viewer.

- `useThumbnail(attachment.path)`: a hook (see [The pages](#)) that asks the main process for a small preview. It returns `undefined` while loading, `null` if the file is gone, or a `data:` URL to use as the image. Until there's an image the button is an empty grey box.
- `onClick={onOpen}`: opening goes through the panel. `ChatBox` passes the same handler the draft chips use, `Panel.openAttachment` (see [The pages](#)), which asks the main process to open the file and shows "That screenshot can't be found" if it has been deleted.
- `alt=""`: the image is decorative; the button's `title` tooltip shows the file name.
- `object-cover`: fills the 80 × 48 px box, cropping the screenshot rather than squashing it.

AssistantMessage

src/renderers/src/components/MessageList.tsx, lines 112–160

```
function AssistantMessage(props: {
  message: ChatMessage
  canRetry: boolean
  onRetry: () => void
  onCopy: (text: string) => void
}) {
```

```

const { message } = props
const waiting = message.status === 'streaming' && !message.text

return (
  <div className="group/message max-w-full text-sm text-zinc-800 dark:text-zinc-100">
    {waiting ? <Thinking /> : message.text && <Markdown text={message.text} />}

    {(message.notice || message.status === 'done') && (
      <div className="mt-1 flex items-center gap-2 text-xs">
        {message.notice && (
          <span
            className={cn(
              message.status === 'error' ? 'text-red-600 dark:text-red-400' : 'text-zinc-500',
            )}
          >
            {message.notice}
          </span>
        )}
        {message.retryable && props.canRetry && (
          <button
            type="button"
            onClick={props.onRetry}
            className="inline-flex items-center gap-1 rounded-md px-1.5 py-0.5 font-medium text-accent
            hover:bg-zinc-100 dark:text-accent-soft dark:hover:bg-zinc-800"
          >
            <RotateCcw size={12} aria-hidden /> Retry
          </button>
        )}
        {message.text && message.status !== 'streaming' && (
          <button
            type="button"
            aria-label="Copy reply"
            title="Copy reply"
            onClick={() => props.onCopy(message.text)}
            className="rounded-md p-1 text-zinc-400 opacity-0 group-hover/message:opacity-100 hover:bg-
            zinc-100 hover:text-zinc-700 focus-visible:opacity-100 dark:hover:bg-zinc-800 dark:hover:text-zinc-200"
          >
            <Copy size={12} aria-hidden />
          </button>
        )}
      </div>
    )}
  </div>
)
}

```

Claude's reply: full width, with no bubble, rendered as Markdown. Below it is a small row with any notice, a Retry button for a failed reply, and Copy.

- `const { message } = props`: destructuring again, so the code can say `message` rather than `props.message`.
- `const waiting = message.status === 'streaming' && !message.text`: the reply has started but no text has arrived yet (Claude may think for a while first).
- `{waiting ? <Thinking /> : message.text && <Markdown text={message.text} />}`: the `? :` picks one of the two. As each chunk of text arrives the whole reply is rendered as Markdown again.
- `(message.notice || message.status === 'done') &&`: the footer row appears once the reply has finished (`done`), or when there's a notice. `ChatSession` gives every stopped or failed reply a notice ("Stopped.", or the error message), so in practice the row shows for every finished reply and never while text is streaming.
- `message.status === 'error' ? 'text-red-600 dark:text-red-400' : 'text-zinc-500'`: errors in red; other notices (stopped, refused, cut off at the length limit) in grey.

- `message.retryable` && `props.canRetry`: Retry only for a failed reply that `ChatSession` marked retryable (a temporary error, with no text kept from it), and only on the last message. It calls `window.assist.chat.retry()` through `Panel`.
- `message.text` && `message.status !== 'streaming'`: Copy appears once there's a finished text to copy.
- `group/message`: a *named group*. `group-hover/message:opacity-100` shows the Copy button (normally `opacity-0`) only while the pointer is over this message. The name keeps it separate from `Panel`'s `group`, which wraps the whole panel. `focus-visible:opacity-100` shows it when you reach it with Tab.
- `onClick={() => props.onCopy(message.text)}`: copies the reply's Markdown source, not the formatted text. `Panel.copy` puts it on the clipboard (through the main process) and shows a "Copied" toast.

Thinking

`src/renderer/src/components/MessageList.tsx`, lines 162–177

```
function Thinking() {
  return (
    <div className="flex items-center gap-1.5 py-1 text-xs text-zinc-500" role="status">
      <span className="flex gap-0.5" aria-hidden>
        {[0, 150, 300].map((delay) => (
          <span
            key={delay}
            className="size-1.5 animate-bounce rounded-full bg-zinc-400"
            style={{ animationDelay: `${delay}ms` }}
          />
        ))}
      </span>
      Thinking...
    </div>
  )
}
```

Three bouncing dots and the word "Thinking...", shown until the first text of a reply arrives.

- `[0, 150, 300].map((delay) => (`: three dots, each starting its `animate-bounce` 150 ms after the one before, so they ripple.
- `animationDelay`: set as an inline style because Tailwind only generates classes it finds written out in full in the source, so it can't build one from a variable.
- `role="status"`: screen readers announce "Thinking..."; the dots are `aria-hidden`, so they're skipped.

`src/renderer/src/components/Markdown.tsx`: rendering Claude's replies

Turns a reply's Markdown into formatted text: headings, lists, tables, links and highlighted code. Only `AssistantMessage` uses it, with one prop, `text`. The look (spacing, code backgrounds, the highlight colours) comes from the `.markdown` and `.hljs-...` rules in `styles.css`.

components

src/renderer/src/components/Markdown.tsx, lines 5–18

```
// Links open in the default browser instead of inside the panel.
const components: Components = {
  a: ({ href, children }) => (
    <a
      href={href}
      onClick={(event) => {
        event.preventDefault()
        if (href) void window.assist.openExternal(href)
      }}
    >
      {children}
    </a>
  ),
}
```

Tells react-markdown how to draw particular HTML elements. Only links are changed.

- `const components: Components = {`: declared outside the component, so the object is made once rather than on every render.
- `a: ({ href, children }) => (`: react-markdown calls this for every link instead of making a plain `<a>`, passing the props it would have used. `children` is the link text.
- `event.preventDefault()`: stops the click navigating the panel's own page to the link. The main process would refuse that navigation anyway (`lockDown` in `src/main/bubble/windows.ts`).
- `if (href) void window.assist.openExternal(href)`: asks the main process to open the link in your default browser, or an email link (`mailto:`) in your email app. Anything else, such as `file:` or `javascript:`, is refused (`ExternalUrl` in `src/main/ipc.ts`).

Markdown

src/renderer/src/components/Markdown.tsx, lines 20–36

```
/**
 * Renders Claude's reply: GitHub-flavoured Markdown (tables, task lists) with highlighted code.
 * Raw HTML in the text is never rendered, so a reply can't inject markup.
 */
export function Markdown({ text }: { text: string }) {
  return (
    <div className="markdown">
      <ReactMarkdown
        remarkPlugins={[remarkGfm]}
        rehypePlugins=[[rehypeHighlight, { detect: false }]]
        components={components}
      >
        {text}
      </ReactMarkdown>
    </div>
  )
}
```

react-markdown parses the text and builds React elements from it directly, so no HTML string is ever inserted into the page. Raw HTML written in a reply is shown as plain text rather than rendered (react-markdown does this unless a plugin such as `rehype-raw` is added, and none is), and its default link check blanks out unsafe addresses such as `javascript:`. So a reply can't inject markup or scripts into the panel.

- `className="markdown"`: hooks into the `.markdown` styles in `styles.css`, sized for the narrow panel. They also turn text selection back on, so replies can be selected and copied.
- `remarkPlugins=[remarkGfm]`: *remark* plugins work on the Markdown before it becomes HTML. *remark-gfm* adds GitHub's extensions: tables, task lists (`- [ ]`), `~~strikethrough~~`, footnotes, and bare web addresses turned into links.
- `rehypePlugins=[[rehypeHighlight, { detect: false }]]`: *rehype* plugins work on the HTML tree afterwards. *rehype-highlight* colours code blocks with *highlight.js* (through the *lowlight* library), adding the `hljs-...` classes that `styles.css` colours. The inner array is the plugin followed by its options. `detect: false` means only a code block that names its language (a fence like ```ts`) is highlighted; unlabelled blocks stay plain rather than being guessed at. It knows *highlight.js*'s common languages; a block in any other language is left plain.
- `{text}`: the Markdown goes in as the component's children.

src/renderer/src/components/SignInPanel.tsx: the sign-in prompts

Shown in the chat card instead of the conversation while nobody is signed in. `Panel` builds it and passes it to `ChatBox` as `signInPrompt` when the sign-in status (`AuthStatus`, kept by `AuthManager`; see [JumpCloud sign-in](#)) is `unconfigured`, `signed-out` or `signing-in`. Its props are `open`, that `status`, the app's name, and `onSignIn` and `onCancel`, which `Panel` wires to `window.assist.auth.signIn()` and `window.assist.auth.cancel()`.

SignInPanel

src/renderer/src/components/SignInPanel.tsx, lines 5–88

```
/**
 * Shown in place of the chat while nobody is signed in. Signing in happens in the browser, on
 * JumpCloud's own page; if the user is already signed in to JumpCloud there, it finishes by itself
 * and the chat appears.
 */
export function SignInPanel(props: {
  open: boolean
  status: Extract<AuthStatus, { state: 'unconfigured' | 'signed-out' | 'signing-in' }>
  appName: string
  onSignIn: () => void
  onCancel: () => void
}) {
  const { status } = props
  const button = useRef<HTMLButtonElement>(null)

  useEffect(() => {
    if (props.open) button.current?.focus()
  }, [props.open, status.state])

  if (status.state === 'unconfigured') {
    return (
      <Card icon={<Settings2 size={16} aria-hidden />} title="Sign-in isn't set up yet">
        <p className="text-xs text-zinc-500">
          {props.appName} still needs to be connected to JumpCloud and Claude. Your IT team can do
          this by following docs/JUMPCLOUD_SETUP.md. Missing settings:
        </p>
        <ul className="list-inside list-disc text-[11px] text-zinc-500">
          {status.missing.map((name) => (
            <li key={name}>
              <code>{name}</code>
            </li>
          ))}
        </ul>
      </Card>
    )
  }
}
```

```

    ))}
  </ul>
</Card>
)
}

if (status.state === 'signing-in') {
  return (
    <Card
      icon={<LoaderCircle size={16} className="animate-spin" aria-hidden />}
      title="Finish signing in in your browser"
    >
      <p className="text-xs text-zinc-500">
        JumpCloud opened in your browser. Once you're signed in there, come back here.
      </p>
      <div className="flex justify-end">
        <button
          ref={button}
          type="button"
          onClick={props.onCancel}
          className="rounded-md px-2 py-1 text-xs text-zinc-600 hover:bg-zinc-100 dark:text-zinc-300
dark: hover:bg-zinc-800"
        >
          Cancel
        </button>
      </div>
    </Card>
  )
}

return (
  <Card icon={<LogIn size={16} aria-hidden />} title="Sign in to start chatting">
    {status.message ? (
      <p role="alert" className="text-xs text-amber-700 dark:text-amber-400">
        {status.message}
      </p>
    ) : (
      <p className="text-xs text-zinc-500">Use your work JumpCloud account.</p>
    )}
    <button
      ref={button}
      type="button"
      onClick={props.onSignIn}
      className="w-full rounded-lg bg-accent px-3 py-2 text-sm font-medium text-white hover:bg-
accent/90"
    >
      Sign in with JumpCloud
    </button>
    <p className="text-[11px] leading-relaxed text-zinc-500">
      This opens JumpCloud in your browser. If you're already signed in there, it finishes on its
      own, and you'll stay signed in here.
    </p>
  </Card>
)
}

```

Three versions, one for each state: "Sign-in isn't set up yet", "Finish signing in in your browser" with Cancel, and "Sign in to start chatting" with the Sign in button.

- `status`: `Extract<AuthStatus, { state: 'unconfigured' | 'signed-out' | 'signing-in' }>`: `AuthStatus` is a *union* of six shapes, one per `state`. `Extract` keeps just the three that match, so TypeScript won't let anyone pass a signed-in status here.

- `const button = useRef<HTMLButtonElement>(null)`: the same ref is attached (`ref={button}`) to whichever button is showing, Sign in or Cancel.
- `if (props.open) button.current?.focus()`: moves the keyboard focus to that button, so Enter or Space presses it. With no text box on screen, nothing else would have focus. In the `unconfigured` version there's no button; `?.` skips the call when `button.current` is `null`.
- `}, [props.open, status.state]`: focuses again when the panel opens and when the state changes, because a different button appears.
- `if (status.state === 'unconfigured') {`: checking `state` *narrows* the type. Inside this branch TypeScript knows `status` has `missing`; after the two early `returns` it knows only `signed-out` is left, which has the optional `message`.
- `status.missing.map((name) => (`: lists the tenant.json settings that are still empty (from `missingSettings()` in `src/main/tenant.ts`), each name also serving as its `key`.
- `if (status.state === 'signing-in') {`: while `AuthManager.signIn()` waits for the browser to come back from JumpCloud.
- `onClick={props.onCancel}`: `AuthManager.cancelSignIn()` stops waiting, and the status goes back to `signed-out` with no message.
- `{status.message ? (`: after a sign-in expires or fails, `AuthManager` sets a message saying why. It's shown in amber with `role="alert"`, which screen readers read out at once. Otherwise there's a short hint.
- `onClick={props.onSignIn}`: starts the browser sign-in (`AuthManager.signIn()`).

Card

`src/renderer/src/components/SignInPanel.tsx`, lines 90–102

```
function Card(props: { icon: ReactNode; title: string; children: ReactNode }) {
  return (
    <div className="space-y-3 p-4">
      <div className="flex items-center gap-2.5">
        <span className="grid size-8 shrink-0 place-items-center rounded-full bg-accent/10 text-accent dark:text-accent-soft">
          {props.icon}
        </span>
        <p className="text-sm font-semibold">{props.title}</p>
      </div>
      {props.children}
    </div>
  )
}
```

The layout the three versions share: a round icon badge and a bold title, then whatever content is passed as `children`.

- `bg-accent/10`: the `/10` makes the accent colour 10% opaque, a pale tint behind the icon.
- `dark:text-accent-soft`: a lighter accent (`--color-accent-soft` in `styles.css`) that's easier to read on a dark background.

src/renderers/src/components/SettingsMenu.tsx: the settings menu

The small menu that opens beside the gear icon. `Panel` renders it only while the panel is open, the gear has toggled it on, and the tenant's list of actions includes `settings`. Its props are the `corner` and `offset` for positioning; the current `settings`, `branding` (app and company name), `version` and signed-in `user` to display; and one callback per menu item. `Panel` closes it on Esc, on a click anywhere else in the panel, and when the panel closes.

CONFIRM_MS, EFFORTS

src/renderers/src/components/SettingsMenu.tsx, lines 8–14

```
const CONFIRM_MS = 3000

const EFFORTS: { value: Effort; label: string; hint: string }[] = [
  { value: 'low', label: 'Fast', hint: 'Quick answers' },
  { value: 'medium', label: 'Balanced', hint: 'Thinks a little longer' },
  { value: 'high', label: 'Thorough', hint: 'Thinks hardest; slower' },
]
```

- `const CONFIRM_MS = 3000`: how long "New conversation" waits for its second click.
- `EFFORTS: { value: Effort; label: string; hint: string }[]`: the three response styles. Each maps an `Effort` value (what's sent to the Claude API) to the label on its button and a tooltip. Typing `value` as `Effort` means TypeScript rejects a misspelt value. The API side is `AnthropicBackend.reply` in [Talking to Claude](#).

SettingsMenu

src/renderers/src/components/SettingsMenu.tsx, lines 16–139

```
/** The menu that opens beside the gear icon, on the side facing the middle of the screen. */
export function SettingsMenu(props: {
  corner: Corner
  /** Distance from the bubble's edge of the window to the gear, so the menu lines up with it. */
  offset: number
  settings: Settings
  branding: Branding
  version: string
  /** Who's signed in, or null when nobody is. */
  user: SignedInUser | null
  onToggleAutoStart: () => void
  onSetEffort: (effort: Effort) => void
  onSignOut: () => void
  onOpenScreenshotsFolder: () => void
  onNewConversation: () => void
}) {
  // "New conversation" needs a second click within a few seconds.
  const [confirmingNew, setConfirmingNew] = useState(false)
  useEffect(() => {
    if (!confirmingNew) return
    const timer = setTimeout(() => setConfirmingNew(false), CONFIRM_MS)
    return () => clearTimeout(timer)
  }, [confirmingNew])

  const { settings, branding } = props

  return (
    <div
      data-hit
```

```

data-settings-menu
role="menu"
aria-label="Settings"
className={cn(
  'absolute rounded-xl border p-1.5 shadow-xl',
  'border-black/10 bg-white text-zinc-800 dark:border-white/10 dark:bg-zinc-900 dark:text-zinc-
100',
)}
style={{
  ...anchored(props.corner, PANEL_LAYOUT.settingsX, props.offset),
  width: UI.settingsMenuWidth,
}}
>
<button
  type="button"
  role="menuitemcheckbox"
  aria-checked={settings.autoStart}
  disabled={!settings.autoStartAvailable}
  onClick={props.onToggleAutoStart}
  className={cn(
    'flex w-full items-center justify-between gap-3 rounded-lg px-2.5 py-2 text-left text-sm',
    settings.autoStartAvailable
      ? 'hover:bg-zinc-100 dark:hover:bg-zinc-800'
      : 'cursor-default opacity-60',
  )}
>
  <span>
    Start with Windows
    {!settings.autoStartAvailable && (
      <span className="block text-[11px] text-zinc-500">Only in the installed app</span>
    )}
  </span>
  <Switch on={settings.autoStart} />
</button>

<div className="px-2.5 pt-1.5 pb-2">
  <p className="mb-1.5 text-sm">Response style</p>
  <div
    role="radiogroup"
    aria-label="Response style"
    className="flex rounded-lg bg-zinc-100 p-0.5 dark:bg-zinc-800"
  >
    {EFFORTS.map((option) => (
      <button
        key={option.value}
        type="button"
        role="radio"
        aria-checked={settings.effort === option.value}
        title={option.hint}
        onClick={() => props.onSetEffort(option.value)}
        className={cn(
          'flex-1 rounded-md py-1 text-xs font-medium',
          settings.effort === option.value
            ? 'bg-white text-zinc-900 shadow-sm dark:bg-zinc-600 dark:text-white'
            : 'text-zinc-500 hover:text-zinc-800 dark:hover:text-zinc-200',
        )}
      >
        {option.label}
      </button>
    )}
  </div>
</div>

<MenuItem icon={FolderOpen} onClick={props.onOpenScreenshotsFolder}>
  Open screenshots folder

```

```

    </MenuItem>

    <MenuItem
      icon={MessageSquarePlus}
      danger={confirmingNew}
      onClick={() => {
        if (!confirmingNew) return setConfirmingNew(true)
        setConfirmingNew(false)
        props.onNewConversation()
      }}
    >
      {confirmingNew ? 'Click again to clear the chat' : 'New conversation'}
    </MenuItem>

    {props.user && (
      <div className="mt-1 border-t border-black/10 pt-1 dark:border-white/10">
        <p className="truncate px-2.5 pt-1 text-[11px] text-zinc-500" title={props.user.email}>
          Signed in as {props.user.name ?? props.user.email ?? 'you'}
        </p>
        <MenuItem icon={Logout} onClick={props.onSignOut}>
          Log out
        </MenuItem>
      </div>
    )}

    <div className="mt-1 border-t border-black/10 px-2.5 pt-1.5 pb-0.5 text-[11px] text-zinc-500
dark:border-white/10">
      {branding.appName} {props.version} · {branding.companyName}
    </div>
  </div>
)
}

```

Draws the menu's rows: Start with Windows, Response style, Open screenshots folder, New conversation, who's signed in with Log out, and the version. It keeps one piece of its own state.

State. `useState(initial)` returns the current value and a function that changes it. Calling that function makes React run the component again with the new value, which redraws it. Use a ref for values that don't affect what's shown, and state for values that do. Here `confirmingNew` switches the "New conversation" row into its "Click again" form.

- `offset: number`: `Panel` passes `actionOffset()` of the gear's place in the icon stack.
- `...anchored(props.corner, PANEL_LAYOUT.settingsX, props.offset)`: `settingsX` (64 px) puts the menu 8 px past the action icons, on the side facing the middle of the screen. `offset` makes its near edge level with the gear's: in a bottom corner its bottom lines up with the gear's bottom, so it opens upward; in a top corner it opens downward. It overlaps the chat card and is drawn on top because `Panel` renders it after `ChatBox`.
- `data-settings-menu`: lets `Panel`'s `closeSettingsOnOutsideClick` recognise a click inside the menu, so it doesn't close it.
- `role="menu"`: this and `role="menuItemcheckbox"`, `role="radiogroup"` and the rest tell screen readers what each control is (and `aria-checked` whether it's on), since they're all styled buttons rather than real checkboxes and radio buttons.
- `const [confirmingNew, setConfirmingNew] = useState(false)`: the square brackets unpack the two things `useState` returns: the value and its setter.
- `return () => clearTimeout(timer)`: when `confirmingNew` turns on, the effect starts a 3-second timer that turns it off again. The function an effect returns is its *cleanup*: React calls it before running the effect again and

when the component disappears. So the timer is cancelled after a confirming second click, or when the menu closes.

- `const { settings, branding } = props`: shorthand for two props used often below.
- `disabled={!settings.autoStartAvailable}`: Start with Windows only works in the installed app, not in `npm run dev`; there the row is greyed out with a note. `onToggleAutoStart` asks the main process to flip it, and `Panel` keeps the settings it returns (see `settings.ts` in [Main process: startup, IPC and app plumbing](#)).
- `<Switch on={settings.autoStart} />`: the on/off graphic at the end of the row.
- `aria-checked={settings.effort === option.value}`: the response styles work like radio buttons. The current one is white with a shadow; the others are plain text on the grey track.
- `onClick={() => props.onSetEffort(option.value)}`: `Panel.setEffort` saves the choice in the main process. `ChatSession` reads it at the start of each request, so it applies from the next message.
- `if (!confirmingNew) return setConfirmingNew(true)`: the first click on "New conversation" only arms it (the `return` just ends the function early). A second click within 3 seconds turns it off and calls `onNewConversation`, which clears the chat (`ChatSession.newConversation()`).
- `danger={confirmingNew}`: the row turns red while it's armed.
- `{props.user && (: only when someone is signed in; Panel passes null otherwise.`
- `props.user.name ?? props.user.email ?? 'you'`: the first of the name or the email that JumpCloud provided. `truncate` cuts a long one short with "...", and `title={props.user.email}` shows the email as a tooltip.
- `{branding.appName} {props.version} · {branding.companyName}`: the footer, for example "Desktop Assist 0.1.0 · Morse Micro".

MenuItem

`src/renderer/src/components/SettingsMenu.tsx`, lines 141–162

```
function MenuItem(props: {
  icon: LucideIcon
  danger?: boolean
  onClick: () => void
  children: ReactNode
}) {
  const Icon = props.icon
  return (
    <button
      type="button"
      role="menuitem"
      onClick={props.onClick}
      className={cn(
        'flex w-full items-center gap-2.5 rounded-lg px-2.5 py-2 text-left text-sm',
        props.danger ? 'bg-red-600 text-white' : 'hover:bg-zinc-100 dark:hover:bg-zinc-800',
      )}
    >
      <Icon size={16} aria-hidden />
      {props.children}
    </button>
  )
}
```

One row of the menu: an icon and a label, as a button.

- `icon: LucideIcon`: the prop is the icon component itself (`FolderOpen`), not a drawn icon (`<FolderOpen />`), so `MenuItem` decides the size.

- `const Icon = props.icon`: JSX treats a lowercase tag as an HTML element, so the component has to be in a capitalised variable before it can be written as `<Icon size={16} aria-hidden />`.
- `props.danger ? 'bg-red-600 text-white'`: the red version, used for the armed "New conversation".

Switch

`src/renderers/src/components/SettingsMenu.tsx`, lines 164–181

```
function Switch({ on }: { on: boolean }) {
  return (
    <span
      aria-hidden
      className={cn(
        'relative inline-flex h-5 w-9 shrink-0 rounded-full transition-colors',
        on ? 'bg-accent' : 'bg-zinc-300 dark:bg-zinc-600',
      )}
    >
      <span
        className={cn(
          'absolute top-0.5 size-4 rounded-full bg-white shadow transition-transform',
          on ? 'translate-x-4.5' : 'translate-x-0.5',
        )}
      />
    </span>
  )
}
```

A pill-shaped track with a white knob. It's purely a picture: the `menuItemcheckbox` button around it tells screen readers whether it's on, so the switch itself is `aria-hidden`.

- `on ? 'bg-accent' : 'bg-zinc-300 dark:bg-zinc-600'`: the track is the accent colour when on.
- `on ? 'translate-x-4.5' : 'translate-x-0.5'`: the 16 px knob sits 2 px from the left of the 36 px track when off and slides to 18 px when on; `transition-transform` animates the slide.

`src/renderers/src/components/ActionStack.tsx`: the action icons

The column of round icon buttons that pops out of the bubble when the panel opens: screenshot, settings, bounce and close, in the order the tenant's `tenant.json` lists them, nearest the bubble first. `Panel` always renders it; while closed it's simply invisible. Its props are the `corner`, the tenant's `actions`, `open`, `activeId` (`settings` while the settings menu is open, otherwise `null`) and `onAction`, which is `Panel.runAction`.

ICONS

`src/renderers/src/components/ActionStack.tsx`, lines 7–12

```
const ICONS: Record<ActionId, LucideIcon> = {
  screenshot: Camera,
  settings: Settings,
  bounce: Volleyball,
  close: Power,
}
```

Which lucide icon each action shows.

- `Record<ActionId, LucideIcon>`: an object with exactly one entry for every action id. If a new id is added to `ACTION_IDS` in `src/shared/actions.ts` without an icon here, TypeScript reports an error.

ActionStack

src/renderer/src/components/ActionStack.tsx, lines 14-75

```

/**
 * The icons that pop out of the bubble, nearest-first in the tenant's order: stacked above it in
 * a bottom corner, below it in a top corner.
 */
export function ActionStack(props: {
  corner: Corner
  actions: ActionId[]
  open: boolean
  activeId: ActionId | null
  onAction: (id: ActionId) => void
}) {
  const top = isTopCorner(props.corner)
  return (
    <div
      className={cn('absolute flex', top ? 'flex-col' : 'flex-col-reverse')}
      style={{
        ...anchored(props.corner, PANEL_LAYOUT.actionsX, PANEL_LAYOUT.actionsY),
        gap: UI.actionGap,
      }}
    >
      {props.actions.map((id, index) => {
        const Icon = ICONS[id]
        const { label } = ACTIONS[id]
        const active = id === props.activeId
        return (
          <button
            key={id}
            data-hit
            data-action={id}
            type="button"
            title={label}
            aria-label={label}
            aria-expanded={ACTIONS[id].kind === 'popover' ? active : undefined}
            onClick={() => props.onAction(id)}
            style={{
              width: UI.actionSize,
              height: UI.actionSize,
              // Pop out one after another, nearest the bubble first.
              transitionDelay: props.open ? `${index * 30}ms` : '0ms',
            }}
            className={cn(
              'grid place-items-center rounded-full border shadow-md outline-none',
              'transition duration-150 ease-out motion-reduce:transition-none',
              'focus-visible:ring-2 focus-visible:ring-accent',
              // Hidden icons sit tucked toward the bubble, then slide out.
              top ? '-translate-y-3' : 'translate-y-3',
              'scale-75 opacity-0',
              'group-data-open:translate-y-0 group-data-open:scale-100 group-data-open:opacity-100',
              active
                ? 'border-transparent bg-accent text-white'
                : id === 'close'
                ? 'border-black/10 bg-white text-zinc-700 hover:border-transparent hover:bg-red-600'
                : 'border-black/10 bg-white text-zinc-700 hover:bg-zinc-100 dark:border-white/10'
                : 'border-black/10 bg-white text-zinc-700 hover:bg-zinc-100 dark:border-white/10'
                : 'border-black/10 bg-white text-zinc-700 hover:bg-zinc-100 dark:border-white/10'
            )}
          </button>
        )
      })}
    </div>
  )
}

```

```

    >
    <Icon size={18} strokeWidth={2} aria-hidden />
  </button>
)
}}
</div>
)
}

```

Lays the icons out in a column beside the bubble and animates them in and out.

- `top ? 'flex-col' : 'flex-col-reverse'`: the first action must be nearest the bubble. In a top corner the bubble is above the stack, so it's a normal top-to-bottom column. In a bottom corner the bubble is below, so the column is reversed and the first action sits at the bottom.
- `...anchored(props.corner, PANEL_LAYOUT.actionsX, PANEL_LAYOUT.actionsY)`: `actionsX` (16 px) centres the 40 px icons on the 56 px bubble. `actionsY` (76 px) starts the stack 12 px past the bubble.
- `gap: UI.actionGap`: 8 px between icons. The main process sizes the panel window from the same numbers (`panelWindowSize` in `src/shared/geometry.ts`), so the stack always fits.
- `const { label } = ACTIONS[id]`: the tooltip and screen-reader name, from the action registry.
- `data-hit`: on each button, not on the column, so the gaps between icons stay click-through.
- `data-action={id}`: `Panel.closeSettingsOnOutsideClick` looks for `[data-action="settings"]`, so clicking the gear toggles the menu instead of counting as a click outside it.
- `aria-expanded={ACTIONS[id].kind === 'popover' ? active : undefined}`: for a popover action (only Settings) it tells screen readers whether its menu is open. `undefined` leaves the attribute off the others.
- `onClick={() => props.onAction(id)}`: `Panel.runAction` toggles the menu for Settings, and asks the main process to run any other action (see `src/main/actions.ts` in [Main process: startup, IPC and app plumbing](#)).
- `transitionDelay: props.open ? `${index * 30}ms` : '0ms'`: each icon starts its animation 30 ms after the one before, so they pop out one after another. Closing has no delay, so they all go together.
- `top ? '-translate-y-3' : 'translate-y-3'`: while closed, each icon sits 12 px toward the bubble, and `'scale-75 opacity-0'` shrinks it to 75% and hides it. The `group-data-open` classes (explained under `ChatBox`) move it to its place at full size when the panel opens.
- `outline-none`: together with `'focus-visible:ring-2 focus-visible:ring-accent'`, swaps the usual square focus outline for an accent ring that follows the round button.
- `active`: the active action (the gear while its menu is open) is filled with the accent colour. Otherwise `id === 'close'` turns red on hover, as a warning that it quits the app, and the rest get a grey hover.

[src/renderer/src/components/AttachmentChip.tsx](#): a screenshot in the draft

A screenshot attached to the draft, shown in the row above the text box. `Composer` renders one per attachment. Its props are the `attachment`, `onOpen` (`Panel.openAttachment`, which shows a toast if the file can't be found) and `onRemove` (`Panel.removeAttachment`).

AttachmentChip

src/renderer/src/components/AttachmentChip.tsx, lines 6–43

```
/** A screenshot attached to the text box: click to open it, × to remove it. */
export function AttachmentChip(props: {
  attachment: Attachment
  onOpen: () => void
  onRemove: () => void
}) {
  const thumbnail = useThumbnail(props.attachment.path)
  const missing = thumbnail === null
  const label = missing ? 'File missing' : screenshotLabel(props.attachment.fileName)

  return (
    <div className="relative flex shrink-0 items-center rounded-lg border border-black/10 bg-zinc-50
dark:border-white/10 dark:bg-zinc-800">
      <button
        type="button"
        title={props.attachment.fileName}
        onClick={props.onOpen}
        className="flex items-center gap-2 rounded-lg py-1 pr-8 pl-1 text-left hover:bg-zinc-100
dark:hover:bg-zinc-700"
      >
        <span className="grid h-10 w-14 place-items-center overflow-hidden rounded-md bg-zinc-200 text-
zinc-500 dark:bg-zinc-700">
          {thumbnail ? (
            <img src={thumbnail} alt="" draggable={false} className="size-full object-cover" />
          ) : (
            missing && <ImageOff size={16} aria-hidden />
          )}
        </span>
        <span className="text-xs whitespace-nowrap">{label}</span>
      </button>
      <button
        type="button"
        aria-label={`Remove ${label}`}
        onClick={props.onRemove}
        className="absolute top-1/2 right-1 -translate-y-1/2 rounded-full p-1 text-zinc-500 hover:bg-
black/10 hover:text-zinc-900 dark:hover:bg-white/10 dark:hover:text-white"
      >
        <X size={14} aria-hidden />
      </button>
    </div>
  )
}
```

A thumbnail and a short label. Clicking the chip opens the screenshot; the × removes it from the draft (the file itself stays in the screenshots folder).

- `const thumbnail = useThumbnail(props.attachment.path)`: the preview, as for `SentScreenshot`.
- `const missing = thumbnail === null`: `useThumbnail` returns `null` only when the main process can't make a preview (for example the file was deleted or moved out of the screenshots folder), and `undefined` while it's still loading. `===` keeps the two apart, so a chip that's still loading doesn't say "File missing".
- `screenshotLabel(props.attachment.fileName)`: from `src/shared/format.ts`, gives "Screenshot 14:03" for one taken today and "Screenshot 2 Oct 14:03" for an older one.
- `draggable={false}`: stops the browser letting you drag the image out of the chip.
- `missing && <ImageOff size={16} aria-hidden />`: a "no image" icon for a missing file; while loading, the box is simply empty.

- `whitespace-nowrap`: keeps the label on one line.
- `pr-8`: the chip is two separate buttons, because HTML doesn't allow a button inside another button. The open button leaves 32 px of padding on its right for the remove button.
- `absolute top-1/2 right-1 -translate-y-1/2`: places the remove button over the right end of the chip, centred vertically (its top at half the height, then moved up by half its own height).
- `aria-label={`Remove ${label}`}`: the button shows only an `×`, so this gives it a name for screen readers, such as "Remove Screenshot 14:03".
- `onClick={props.onRemove}`: `Panel.removeAttachment` asks the main process to drop it from the saved draft and shows the updated list.